

Complete study notes

Core lessons and supplementary references. Independent study material, not official SEBI content. Policy summaries may become dated: verify linked primary sources before relying on current rules or numerical thresholds.

Cloze Test, Fillers and Sentence Completion

Core preparation lesson

Cloze test

A passage with blanks; the correct word must fit **grammar, meaning and tone** simultaneously.

Method:

1. **Read the whole passage first** for the theme and tone before filling anything. Later blanks often clarify earlier ones.
2. Fill the **easy blanks first** — they narrow the context for the hard ones.
3. For each blank ask: what **part of speech** is needed? Then, does the sentence need a **contrast, continuation, cause or example**?
4. Re-read the completed passage at the end. A wrong word usually *sounds* wrong in the flow.

Connector logic — the highest-return skill here

| Relationship | Signals |

|---|---|

| **Contrast** | but, however, although, though, yet, nevertheless, whereas, despite, in spite of, on the contrary, conversely |

| **Continuation / addition** | and, moreover, furthermore, in addition, besides, likewise, similarly |

| **Cause** | because, since, as, due to, owing to, on account of |

| **Effect** | therefore, hence, thus, consequently, as a result, accordingly |

| **Example** | for instance, for example, such as, namely |

| **Condition** | if, unless, provided that, in case |

| **Concession** | granted that, even if, admittedly |

| **Sequence** | first, subsequently, finally, meanwhile |

Most cloze and filler questions reduce to identifying which of these relationships the sentence needs. Get the relationship right and usually only one option survives.

Double fillers

Both blanks must work. Strategy: **find the blank you are more certain about**, eliminate on that one alone, then check the survivors against the second blank. Do not try to solve both at once.

Sentence completion / improvement

Common error patterns to check in order:

1. **Subject-verb agreement**

2. **Tense consistency**
3. **Preposition** (the most common single error in Indian competitive English)
4. **Parallelism** in lists and comparisons
5. **Misplaced modifier**
6. **Redundancy** — "return back", "repeat again", "advance planning", "free gift", "cancel out"
7. **Article usage** — a/an/the

Word-pair sense

Many options are near-synonyms with different registers or connotations. Distinguish:

- **affect** (verb, to influence) vs **effect** (noun, result; verb, to bring about)
- **comprise** (the whole comprises the parts — never "comprised of") vs **compose**
- **imply** (speaker) vs **infer** (listener)
- **principal** (main, or head) vs **principle** (rule)
- **ensure** (make certain) vs **insure** (insurance) vs **assure** (tell someone confidently)
- **eminent** (distinguished) vs **imminent** (about to happen)
- **stationary** (not moving) vs **stationery** (paper)
- **complement** (completes) vs **compliment** (praise)

Exam pointers

1. **Read the whole cloze passage before filling any blank.**
2. **Connector logic solves most of these questions** — learn the table.
3. In double fillers, eliminate on the blank you are **surer** about.
4. "Comprised of" is **always wrong**.
5. Redundancy is the trap examiners most enjoy: "return back", "reason is because", "more better".

Reading Comprehension — Method

Core preparation lesson

The reality

RC is the largest and most reliably scoring block in the English section, and the one most candidates handle badly by reading too slowly and re-reading too often.

The method

1. **Read the questions first — only the question stems, not the options.** 20 seconds. You now know what to look for.
2. **Read the passage once, at a normal pace, without stopping to re-read.** Mark (mentally or with a light pencil in your rough sheet) where each idea sits: paragraph 1 = problem, paragraph 2 = counter-argument, paragraph 3 = conclusion.
3. **Answer the fact-based questions first** by returning to the relevant paragraph. These are free.
4. **Then the inference and tone questions**, which need the whole passage.
5. **Never bring outside knowledge.** The correct answer is always supported by the text, even if you believe the text is wrong.

Question types and how to attack each

| Type | Approach |

---|---

| **Fact / detail** | Locate and match. The answer is stated, not implied |

| **Main idea / title** | Must cover the **whole** passage, not one paragraph. Eliminate options that are too narrow or too broad |

| **Inference** | Must follow **necessarily** from the text. Reject anything that needs an extra assumption |

| **Tone / attitude** | Look at adjectives and adverbs. Common answers: critical, appreciative, analytical, objective, sceptical, sarcastic, cautionary |

| **Vocabulary in context** | Substitute each option into the sentence; the word's dictionary meaning is secondary to its use here |

| **Author's purpose** | Why was this written — to argue, describe, criticise, or explain? |

| **Strengthen / weaken** | Find the option that affects the **link** between evidence and conclusion |

Elimination patterns

Wrong options are usually wrong in predictable ways:

- **Too extreme** — contains "always", "never", "all", "only", "must". Passages rarely support absolutes.
- **Half right** — the first half matches, the second half adds something not in the text. Read the whole option.

- **Out of scope** — true in the world, absent from the passage.
- **Reversed** — states the opposite relationship.
- **Too narrow** — correct about one paragraph, offered as the main idea.

Speed

Aim for **7–9 minutes per passage with 5 questions** in Phase I. If a passage is dense and unfamiliar, and there is a second passage available, **do the easier one first**. Attempt rate matters more than perfection.

Practice discipline

One passage a day from an editorial page (The Hindu, Indian Express, Mint) — not from a question bank. Summarise it in **two sentences** after reading. That one habit builds comprehension speed, essay material for Phase II, and current affairs, all at once.

Exam pointers

1. **Question stems first**, options later.
2. **Inference must follow necessarily** — no outside knowledge, no extra assumptions.
3. **Extreme words usually mark a wrong option.**
4. **Half-right options are the main trap** — read every option fully.
5. The main idea must cover the **entire** passage.

Grammar — The Rules That Actually Get Tested

Core preparation lesson

1. Subject-verb agreement

- **The verb agrees with the subject, not with the nearest noun.** *The list of items **is** on the desk.*
- Phrases like *along with, as well as, together with, in addition to, besides, accompanied by* do **not** change the number of the subject. *The manager, along with his staff, **is** attending.*
- **Either/or, neither/nor** — the verb agrees with the **nearer** subject. *Neither the director nor the officers **were** informed.*
- **Each, every, everyone, everybody, someone, nobody, anyone** are **singular**. *Each of the candidates **has** submitted.*
- **Collective nouns** take a singular verb when acting as a unit, plural when the members act individually.
- **"A number of"** is plural; **"the number of"** is singular.
- Uncountable nouns (information, furniture, advice, equipment, luggage, machinery, news, scenery) are **singular** and have no plural form. *News **is** good.*
- Nouns plural in form but singular in meaning: mathematics, physics, economics, politics, statistics (as a subject).
- Sums of money, distances and periods of time take a **singular** verb. *Ten lakh rupees **is** a large amount.*

2. Tenses

| Use | Tense |

|---|---|

| Habitual action, universal truth | Simple present |

| Action in progress now | Present continuous |

| Past action with present relevance; "since"/"for" | **Present perfect** |

| Action completed before another past action | **Past perfect** |

| Definite past time (yesterday, in 2019, ago) | **Simple past — never present perfect** |

- **Since** + a point in time; **for** + a duration. **Since 2020 / for three years.**
- In a conditional: **If** + past perfect, ... **would have** + past participle (third conditional). **If he had studied, he would have passed.**
- After **"It is time", "I wish", "as if"** → use the **past subjunctive**. **I wish I were there.**

3. Prepositions — the highest-yield list

| Verb/adjective | Preposition |

|---|---|

| agree **to** a proposal / agree **with** a person | |

| comply **with**, compliance **with** | |

| conform **to** | |

| differ **from** (things) / differ **with** (people) | |

| consist **of** | |

| **comprise** (no preposition) | |

| discuss (no *about*) | |

| married **to** | |

| superior/inferior **to** (never *than*) | |

| prefer X **to** Y | |

| capable **of**, incapable **of** | |

| deprive **of**, accuse **of**, suspect **of** | |

| depend **on**, rely **on**, insist **on** | |

| result **in** (outcome) / result **from** (cause) | |

| die **of** a disease / die **from** an injury | |

4. Parallelism

Items joined by *and*, *or*, *both...and*, *either...or*, *neither...nor*, *not only...but also* must be the **same grammatical form**.

☐ *He likes reading, to write and jogging.* ☐ *He likes reading, writing and jogging.*

With **not only...but also**, whatever follows *not only* must be matched in form after *but also*.

5. Modifiers

A modifier must sit next to what it modifies.

☐ *Walking to the station, the rain started.* (the rain was not walking)

☐ *Walking to the station, I was caught in the rain.*

This is a **dangling modifier**, and it is a favourite in error-spotting sets.

6. Pronouns

Clear antecedent, correct case (*between you and **me***, not *I*), agreement in number (*Every student must submit **his** or **her** form*, or recast as plural), and *who* (subject) vs *whom* (object).

7. Articles

A before consonant **sounds** (*a university, a one-rupee coin*); *an* before vowel **sounds** (*an hour, an MBA, an honest man*). **The** for specific or unique things, superlatives, ordinals, rivers, ranges, oceans, and plural country names.

8. Common redundancies

return back · repeat again · revert back · advance planning · free gift · new innovation · final conclusion · basic fundamentals · cancel out · join together · sufficient enough · reason is because · more superior.

Exam pointers

1. **The verb agrees with the subject, not the nearest noun.**
2. *Either/or, neither/nor* → verb agrees with the **nearer** subject.
3. **Superior/inferior/prefer take "to", never "than".**
4. **Definite past time cannot take the present perfect.**
5. Check **parallelism and dangling modifiers** in every error-spotting question — they are planted constantly.

Para Jumbles and Sentence Rearrangement

Core preparation lesson

The method

1. **Find the opening sentence.** It introduces the subject by **full name or noun**, contains no pronoun without an antecedent, and no connector like *however, therefore, moreover, this, these, such, also*.
2. **Find mandatory pairs.** Two sentences that must be adjacent because one supplies a noun the other refers to by pronoun, or because a connector demands an immediate predecessor.
3. **Use transition words** to order the middle: *first... then... finally*; *however* must follow a contrasting statement; *for example* must follow a general claim; *therefore* must follow a reason.
4. **Find the closing sentence.** It concludes, summarises, or states a consequence — often containing *thus, hence, finally, in conclusion*, or a general statement after specifics.
5. **Check the options.** Often you need only one confident pair to eliminate three of four options. Do not construct the whole sequence if elimination is faster.

Chronology and logic markers

- **Time:** dates, years, *initially, later, subsequently, eventually, then*.
- **Cause before effect** — always.
- **General before specific** — the claim, then the example.
- **Problem → cause → solution → result** is the commonest structure in these passages.
- **Article shift:** the **first** mention of a noun usually takes "a/an"; later mentions take "the" or a pronoun. *A committee was formed... The committee recommended...* This alone often fixes the order of two sentences.

Pronoun chains

Pronouns (*it, they, this, these, such, he, its*) **cannot appear before their antecedent**. Scanning for the sentence that introduces the noun, and then for the sentence using the pronoun, produces a forced pair more often than any other technique.

Common traps

- Two sentences that both look like openers — check which one the other depends on.
- A connector at the start of a sentence that seems to fit two places — check which preceding sentence it genuinely contrasts with.
- Options designed so that the first sentence is correct in three of them; in that case the discrimination is at the end, so **work backwards from the closing sentence**.

Practice discipline

Do 10 sets a day for two weeks and the pattern recognition becomes automatic. Time yourself: **60–75 seconds per set** is the target; if you exceed two minutes, mark and move on.

Exam pointers

1. **Openers have no pronouns and no connectors.**
2. **A pronoun can never precede its antecedent** — this creates the mandatory pairs.
3. The **a/an → the** article shift orders first and later mentions.
4. **Eliminate using options**; do not always build the full sequence.
5. If the openers are identical across options, **solve from the end**.

Vocabulary, Idioms and Word Usage

Core preparation lesson

How to build vocabulary efficiently

Do not memorise word lists alphabetically. Instead:

1. **Harvest words from your daily reading** — the editorial you already read for current affairs.
2. Record them as **word + the sentence you found it in + one synonym + one antonym**.
3. Put them in the **Flashcards** section of this app and let spaced repetition do the work.
4. Learn **roots and affixes** — they let you guess unfamiliar words in the exam.

High-value roots

| Root | Meaning | Examples |

|---|---|---|

| bene / bon | good | benevolent, benign, bonafide |

| mal | bad | malign, malicious, malfeasance |

| dict | say | verdict, edict, contradict |

| cred | believe | credible, credential, incredulous |

| ject | throw | reject, project, conjecture |

| spec / spic | look | inspect, perspicacious, conspicuous |

| ten / tain | hold | tenacious, retain, tenable |

| vert / vers | turn | avert, versatile, inadvertent |

| lucid / luc | light | elucidate, lucid, translucent |

| greg | group | gregarious, egregious, congregate |

| pac | peace | pacify, pacific, appease |

| ambi / amphi | both | ambivalent, ambiguous, amphibious |

Prefixes: a-/an- (without), anti- (against), circum- (around), de- (down), dis- (not), ex- (out), in-/im- (not or into), inter- (between), intra- (within), mis- (wrong), pre- (before), pro- (forward), re- (again), sub- (under), super- (above), trans- (across), un- (not).

Confusable pairs that appear repeatedly

adverse/averse · affect/effect · allusion/illusion · amiable/amicable · appraise/apprise · censure/censor · complement/compliment · continual/continuous · credible/credulous · deprecate/depreciate · disinterested/uninterested · eminent/imminent · flaunt/flout · ingenious/ingenuous · judicial/judicious · luxuriant/luxurious · practicable/practical · precede/proceed · prescribe/proscribe · stationary/stationery · temerity/timidity · venal/venial.

Idioms worth knowing (finance and formal register)

| Idiom | Meaning |

|---|---|

| a blessing in disguise | something good that first seemed bad |

| beat about the bush | avoid the main point |

| bite the bullet | accept something unpleasant and get on with it |

| call it a day | stop working on something |

| cut corners | do something badly to save time or money |

| get the ball rolling | start |

| in the red / in the black | making a loss / making a profit |

| keep something at bay | keep it away |

| let the cat out of the bag | reveal a secret |

| on the back burner | postponed |

| pass the buck | shift responsibility |

| put all eggs in one basket | concentrate risk |

| read between the lines | infer the unstated |

| ring a bell | sound familiar |

| take with a pinch of salt | treat sceptically |

| the tip of the iceberg | a small visible part of a large problem |

| turn a blind eye | deliberately ignore |

| under the table | secretly, corruptly |

Register

For the descriptive paper, prefer ****precise formal vocabulary over ornate vocabulary****. **Mitigate** is better than **ameliorate** if you are not certain of the nuance. A misused impressive word costs more than a correctly used ordinary one.

Exam pointers

1. ****Harvest words from reading****, not from alphabetical lists.
2. Learn ****roots**** — they let you attack unseen words.
3. **Disinterested** = impartial; **uninterested** = bored. This pair appears constantly.
4. **Comprise**, **flout/flaunt**, **prescribe/proscribe** are perennial favourites.
5. In descriptive writing, ****precision beats decoration****.

Descriptive comprehension

Core preparation lesson

Read the questions before the passage to identify what evidence you need. Distinguish the author's claim, examples, qualifications and conclusion. Answer in the requested form; a question asking for two reasons needs two distinct reasons.

Original passage and answer

Passage: A monitoring tool flags unusual transactions. A flag indicates a need for review, not proof of wrongdoing. Analysts must check data quality and legitimate explanations before escalating a case.

Question: Why is human review necessary? Answer: A statistical anomaly may have a legitimate cause or arise from faulty data, so analysts must assess the evidence before escalation.

Practice

Explain the distinction between anomaly and proof in one sentence. Identify the passage's tone: cautious and analytical, not dismissive of technology. Mark unsupported inferences explicitly. In a timed session reserve the final minutes to check grammar, completeness and whether you have accidentally copied a large part of the passage.

Essay prompts and weekly writing plan

Core preparation lesson

Rotate these original prompts: financial literacy in a digital economy; responsible use of AI in supervision; privacy and useful data collection; cyber resilience in capital markets; investor education for first-time savers; inclusion and accessible technology; balancing innovation with oversight; human judgement in automated decisions.

Weekly routine

Monday: plan two prompts. Tuesday: type one essay. Wednesday: rewrite its introduction and transitions.

Thursday: write a counterargument. Friday: type another essay under a timer. Saturday: review both using the five-part rubric. Sunday: save three language errors to your revision notebook.

Evidence discipline

Use named, dated primary sources for policy examples. Separate an existing rule from your recommendation. If you cannot verify a statistic, use a precise qualitative statement instead. Sample opening: Digital access can lower barriers to participation, but access alone does not ensure informed decisions. Investor education and usable safeguards must develop alongside distribution.

Essay writing under a timer

Core preparation lesson

Use a clear thesis, two developed reasons, a limitation or counterargument, and a practical conclusion. Each paragraph should make one point supported by an explanation or accurate example. Avoid memorised statistics you cannot verify.

30-minute practice

Spend 4 minutes planning, 22 drafting and 4 editing. Use the word limit given in the actual question; 250-300 words is a useful practice range, not a declared exam requirement. Type practice answers to develop speed and editing control.

Example outline: technology and investor protection

Thesis: technology can widen access while requiring stronger safeguards. Paragraph 1: digital onboarding reduces effort and travel. Paragraph 2: monitoring can detect unusual activity, but models need validation and human review. Counterargument: convenience can expose inexperienced investors to impersonation and unsuitable products. Conclusion: combine accessible tools, effective complaints handling and education.

Self-review rubric

Score relevance, structure, reasoning, language and editing from 0 to 2 each. Rewrite the weakest paragraph. Save the dated draft in Your notes. Do not treat an unreviewed word count as mastery.

Formal correspondence and short reports — extra writing practice

Core preparation lesson

Correspondence is supplementary drafting practice; consult the English handout for the actual task types. Use an informative subject, a clear purpose, necessary context and an explicit requested action. Keep tone respectful and specific.

Original example

Subject: Request for correction of account contact details

Dear Sir or Madam,

I request a correction to the contact details recorded for my account. The current entry contains a typographical error. Please advise which documents are required and the expected processing time. I will submit the information through your approved secure channel.

Yours faithfully,

Applicant

Reports

Organise an incident report as summary, observed facts, impact, immediate actions, open questions and next steps. Separate confirmed evidence from hypotheses. Avoid including unnecessary personal details. Rewrite the example as a 100-word service report; assess clarity and whether the recipient knows what to do next.

Precis writing with a worked example

Core preparation lesson

A precis preserves the main argument in fewer words, in your own language, without opinion or new examples. Read the passage twice, identify its central claim and supporting chain, remove repetition, then draft. Obey the supplied word limit; one-third length is a practice convention when none is supplied.

Original practice passage

Digital tools let investors access information quickly and transact without visiting an office. Yet easy access does not ensure that users understand costs or risks. Clear disclosures help only when people can find and understand them. Education and accessible complaint channels therefore remain essential companions to technology.

Sample precis

Title: Beyond digital access. Digital investing needs understandable disclosures, education and accessible complaint channels so convenience supports informed decisions.

Check

The compressed version retains access, its limitation, and the proposed response. It omits supporting detail without adding an opinion. Practise with a longer editorial paragraph and compare each sentence of your precis with the original meaning.

Current Affairs — The Method, Not the Material

Core preparation lesson

> Most candidates fail current affairs not from lack of reading but from lack of a system. This note is the system. Follow it for 180 days and you will out-perform people who read three times as much.

1. The window

SEBI questions cluster in the ****6 months before the exam****, with a heavier weight on the ****last 3 months****. Anything older matters only if it is still live policy. If the notification is out, that fixes your window immediately.

2. What actually gets asked

In descending order of return:

1. ****SEBI itself**** — new regulations and amendments, consultation papers, board meeting outcomes, big enforcement orders, new frameworks, committee reports, leadership.
2. ****Securities market events**** — index milestones, new instruments, settlement cycle changes, MII developments, large IPOs, FPI/DII flows.
3. ****RBI and banking**** — policy rate decisions, new frameworks, payment system developments.
4. ****Economy**** — budget numbers, GDP/inflation prints, major schemes.
5. ****Technology in finance**** — CERT-In directions, DPDP rules, AI/blockchain in the market, cyber incidents at regulated entities. *For an IT-stream candidate this rises to number 2.*
6. Generic national/international affairs, awards, sports, appointments — lowest return per hour.

3. The daily 20-minute routine

5 min SEBI website: Press Releases + latest Circulars (headlines only)
7 min One business paper: markets + economy pages
5 min Log 3-5 items in the Current Affairs page of this app
3 min Skim yesterday's log

Weekly, on your revision day: ****compile the week into one page**** and delete anything that did not survive the week.

4. The note format that works

For each item, write exactly four lines:

WHAT: One sentence on what happened.
WHO: Which regulator/body/company, and under which regulation or Act.
WHY: The problem it solves.
SO WHAT: The one line you would say in an interview.

Four lines force you to process rather than copy. An item you cannot compress to four lines is one you did not understand.

5. Sources, ranked

****Primary (read these):**** sebi.gov.in press releases, circulars, orders, board meeting outcomes, annual report; rbi.org.in press releases and the Financial Stability Report; PIB; CERT-In advisories; MeitY.

****Secondary (skim these):**** Mint, Business Standard, Economic Times markets pages.

****Tertiary (revision only):**** monthly current-affairs PDFs from any coaching source. Never read these first — they strip out the reasoning you need for the interview.

6. The interview multiplier

Phase III is largely a current-affairs conversation with your CV attached. A candidate who can say **“SEBI's CSCRf, issued in August 2024, consolidated the earlier cyber circulars and moved regulated entities to a graded, outcome-based model”** is in a different category from one who says **“SEBI has cyber rules”**. Your daily log is what produces the first sentence.

Keep a separate short list of **10 SEBI developments** you can discuss for two minutes each. Rehearse them aloud. That list, plus your own projects, is 60% of the interview.

Exam pointers

1. Six months back, with the last three months weighted heaviest.
2. SEBI's own website beats every compilation — it is the source those compilations copy.
3. Four lines per item: WHAT / WHO / WHY / SO WHAT.
4. Compile weekly; revise monthly; never leave compilation to the last fortnight.
5. For the IT stream, treat CERT-In, DPDP and SEBI's technology circulars as core current affairs, not as optional extras.

Depositories Act, 1996 — Dematerialisation and Settlement

Core preparation lesson

> India dematerialised faster than most markets, and it is a genuine success story. For an IT-stream candidate this note is doubly important: a depository is, in the end, a very large, very well-audited database.

1. Why the Act exists

Before 1996, shares were **physical certificates**. That meant forged transfers, lost certificates, bad deliveries, stamp duty on every transfer and settlement cycles measured in weeks. The **Depositories Act, 1996** created the legal basis for holding securities in **electronic (dematerialised) form** and transferring them by **book entry**.

2. The two depositories

| | NSDL | CDSL |

|---|---|---|

| Established | 1996 | 1999 |

| Promoters | IDBI, UTI, NSE | BSE and banks |

| Status | First depository in India | First listed depository (listed 2017) |

Both are **Market Infrastructure Institutions** and are regulated by SEBI under the Depositories Act and the **SEBI (Depositories and Participants) Regulations, 2018**.

3. Key concepts

- **Depository Participant (DP)** — the agent through which an investor accesses the depository (banks, brokers, financial institutions). You open a **demat account** with a DP, not with the depository directly.
- **Section 9 — Fungibility**: securities held in a depository are **fungible**; they bear no distinctive numbers. This is the legal foundation of electronic settlement.
- **Section 10 — Registered owner vs beneficial owner**: the **depository is the registered owner** in the issuer's register, but has **no voting or economic rights**. The **investor is the beneficial owner** and holds all rights.
- **Section 7 — Issuer's obligation** to give the depository details of the allotment.
- **Section 16 — Liability** for loss due to depository or DP negligence; the depository indemnifies the beneficial owner.
- **Section 19** onwards — penalties; appeals go to **SAT**.

4. Dematerialisation flow

Investor -> DP: Demat Request Form (DRF) + physical certificates

DP -> Depository: electronic Demat Request Number (DRN)

Depository -> Issuer / RTA: confirm

RTA verifies and confirms -> Depository credits the investor's demat account

Rematerialisation is the reverse — converting electronic holdings back to physical certificates (rare, and largely blocked for transfers since 2019).

5. Milestones you should be able to quote

- **2019:** transfer of securities of listed companies in **physical form was disallowed** — transfers must be in demat form (transmission and transposition remain exceptions).
- **T+2 → T+1:** India moved to a **T+1 rolling settlement** cycle in phases, completed in **January 2023**, making it one of the first major markets to do so.
- **T+0 (optional, same-day) settlement** was introduced in a **beta phase in March 2024** for a limited set of scrips, with a roadmap towards optional instant settlement. Verify the current status before the exam — this is an actively evolving area and a very likely interview question.
- **ASBA-like facility for the secondary market (UPI block mechanism)** — funds remain in the investor's bank account, blocked, until settlement.

6. Clearing and settlement — who does what

Trade on exchange

- > Clearing Corporation (NSE Clearing / Indian Clearing Corporation)
 - novation: becomes the counterparty to both sides (CCP)
 - computes obligations, collects margins, manages the SGF
- > Depository (NSDL/CDSL): securities leg by book entry
- > Clearing banks: funds leg
- > Settlement complete

Key terms: **novation**, **central counterparty (CCP)**, **netting**, **Settlement Guarantee Fund (SGF)**, **core SGF**, **auction for short delivery**.

7. Investor-facing services worth knowing

- **CAS** — Consolidated Account Statement, showing all holdings across both depositories and mutual funds.
- **e-DIS / DDPI** — electronic delivery instructions replacing paper slips and the old Power of Attorney model.
- **SPEED-e (NSDL)** and **easiest (CDSL)** — internet-based instruction platforms.
- **Nomination** — mandatory to nominate or explicitly opt out for demat accounts.
- **Demat account freezing** and the **Investor Protection Fund** maintained by depositories.

Exam pointers

1. NSDL **1996**, CDSL **1999**; CDSL is the **listed** one.
2. **S.9 fungibility · S.10 registered vs beneficial owner · S.16 liability**.
3. The depository is the **registered owner with no voting rights**; the investor is the **beneficial owner**.
4. You open a demat account with a **DP**, never directly with the depository.
5. **T+1 completed January 2023**; optional **T+0 beta from March 2024**.

Derivatives — Futures, Options and Regulation

Core preparation lesson

> Derivatives are ~99% of Indian equity market turnover by notional value. SEBI's recent regulatory attention to the segment makes this a very likely interview topic.

1. What a derivative is

A contract whose value is **derived** from an underlying asset (equity, index, currency, commodity, interest rate). Under SCRA **2(ac)** derivatives are securities. Four types: **forwards, futures, options, swaps**.

| | Forward | Future |

|---|---|---|

| Traded | OTC, bilateral | Exchange |

| Terms | Customised | Standardised |

| Counterparty risk | Yes | No — CCP novation |

| Settlement | On maturity | Daily mark-to-market |

2. Options — the core vocabulary

- **Call option** — right (not obligation) to **buy** at the strike price.

- **Put option** — right to **sell** at the strike price.

- **Buyer** pays the **premium** and has limited loss / unlimited (or large) profit. **Writer/seller** receives the premium and carries the large risk — which is why only sellers pay margin.

- **European** options can be exercised only at expiry; **American** any time. **Indian index and stock options** are European style; futures are cash or physically settled depending on the contract (stock derivatives are **physically settled**).

Payoffs at expiry (S = spot, K = strike, P = premium):

| Position | Payoff |

|---|---|

| Long call | $\max(S - K, 0) - P$ |

| Short call | $P - \max(S - K, 0)$ |

| Long put | $\max(K - S, 0) - P$ |

| Short put | $P - \max(K - S, 0)$ |

Moneyness: a call is **ITM** if $S > K$, **ATM** if $S = K$, **OTM** if $S < K$ (reverse for a put).

Option premium = intrinsic value + time value. Intrinsic value can never be negative.

The Greeks: **Delta** (sensitivity to the underlying), **Gamma** (rate of change of delta), **Theta** (time decay), **Vega** (sensitivity to volatility), **Rho** (interest rate).

3. Margins and risk management

SPAN + Exposure margin is the framework; add **mark-to-market**, **peak margin** reporting and **upfront margin collection**. Positions are subject to **position limits** at client, member and market level. The **Market Wide Position Limit (MWPL)** applies to stock derivatives, with a ban period when open interest crosses 95%.

4. Regulatory developments to have an answer on

- **Physical settlement** of stock derivatives (from 2018-19) to link the cash and derivatives markets.
 - **Peak margin norms** and **upfront margin collection** to curb excessive intraday leverage.
 - SEBI's studies showing that a large majority of individual traders in equity F&O make **net losses**, and the consequent **tightening measures** — higher contract sizes, rationalised weekly expiries, upfront premium collection, intraday position monitoring. Know the *direction* of policy: reduce retail speculation, preserve genuine hedging.
 - **Investor risk disclosure** requirements on brokers' platforms.
- > ☐ Specific numerical limits in this area changed during 2024-25. Quote the *policy direction* confidently and the exact figures only if you have verified them recently.

5. Uses

Hedging (reduce existing risk), **speculation** (take a view), **arbitrage** (exploit price differences — cash-futures basis, calendar spread). SEBI's concern is not derivatives themselves but retail speculation mis-sold as investment.

Exam pointers

1. Indian **index and stock options** are **European**; stock F&O is **physically settled**.
2. Only **option sellers** pay margin; buyers pay premium.
3. Intrinsic value is never negative; premium = intrinsic + time value.
4. **SPAN + exposure margin**; **MWPL** triggers a ban period at 95% of open interest.
5. Derivatives are **securities** under SCRA 2(ac) — they are SEBI's jurisdiction, not RBI's (except OTC rupee derivatives).

Indian Economy — The Part That Gets Asked

Core preparation lesson

1. National income

- **GDP** = $C + I + G + (X - M)$. **GVA** = GDP – product taxes + product subsidies. India's official releases use both.
- **Nominal vs real GDP**; the **GDP deflator** = $\text{nominal} \div \text{real} \times 100$.
- **Base year** for the current GDP series: **2011-12** (a revision to a newer base year has been under discussion — verify the current position).
- **NSO / MoSPI** publishes GDP; advance, provisional and revised estimates.

2. Inflation

| Index | Published by | Base year | Notes |

---|---|---|---

| **CPI (Combined)** | NSO | 2012 | The **monetary policy target** index |

| **WPI** | Office of the Economic Adviser, DPIIT | 2011-12 | No services; wholesale transactions |

Types: demand-pull, cost-push, core (excluding food and fuel), headline, imported. **Phillips curve** — the short-run inflation-unemployment trade-off. **Stagflation** = high inflation with stagnant growth.

3. Fiscal side

- **Budget documents:** Annual Financial Statement (Art. 112), Demands for Grants, Finance Bill, **FRBM** statements, **Expenditure Budget**, **Receipt Budget**.
- **Fiscal deficit** = total expenditure – total receipts excluding borrowings. **Revenue deficit** = revenue expenditure – revenue receipts. **Primary deficit** = fiscal deficit – interest payments. **Effective revenue deficit** excludes grants for capital asset creation.
- **FRBM Act, 2003** — targets and the escape clause. **Finance Commission** (Art. 280) — the 15th FC recommended a **41% share** of the divisible pool for states.
- **GST** — Article 246A, **GST Council** (Art. 279A) with the Union having one-third weight and states two-thirds; slabs 0/5/12/18/28 plus cess. **Compensation cess**.
- Taxes: direct (income, corporate) vs indirect (GST, customs); **tax buoyancy**, **tax-to-GDP ratio**, **laffer curve**.

4. External sector

- **Balance of Payments** = **Current account** (trade + invisibles) + **Capital/Financial account** + errors and omissions + reserve movements.
- **CAD**, **FDI vs FPI**, **ECB**, **NRI deposits**, **forex reserves** (FCA + gold + SDR + reserve tranche position).

- **Exchange rate regimes**: India runs a **managed float**. **NEER/REER**. **Depreciation vs devaluation**.
- **FEMA, 1999** replaced FERA; **FDI policy** — automatic vs government route; sectoral caps.

5. Growth, poverty and employment

- **PLFS** for employment (unemployment rate, LFPR, WPR). **MPCE survey** for consumption.
- **HDI** (UNDP), **Multidimensional Poverty Index** (NITI Aayog's national MPI), **Gini coefficient**, **Lorenz curve**.
- **NITI Aayog** (2015, replaced the Planning Commission), **Economic Survey** (published before the Budget by the CEA).

6. Financial-sector economics worth a line in the interview

Financialisation of household savings (the shift from bank deposits and gold into mutual funds and equities), the rise of **SIP flows**, **retail participation** post-2020, **demat account growth**, and the policy question of **household leverage in derivatives**. These are the exact themes SEBI cares about, and they connect the economy section to your stream.

Exam pointers

1. **CPI**, not **WPI**, is the monetary policy target.
2. Fiscal deficit excludes borrowings from receipts; **primary deficit = fiscal deficit – interest**.
3. GST Council: **Centre one-third, states two-thirds**; decisions need a **three-fourths majority**.
4. **Forex reserves** = FCA + gold + SDRs + reserve tranche position.
5. India runs a **managed float**, not a fixed or free float.

Indian Financial System — Structure & Regulators

Core preparation lesson

> Why this matters: almost every General Awareness question, and the opening minute of most interviews, assumes you can place SEBI correctly inside the wider system.

1. What a financial system does

A financial system moves surplus funds from **savers** to **users of capital**, and in doing so performs four jobs: mobilising savings, allocating capital, managing risk, and providing a payment mechanism. It has four components — **institutions, markets, instruments, services**.

2. The four pillars of Indian regulation

| Regulator | Established | Regulates |

---|---|---

| RBI | 1 April 1935 (nationalised 1949) | Monetary policy, banks, NBFCs, payment systems, forex, G-sec market |

| SEBI | 12 April 1988; statutory 30 Jan 1992 | Securities markets, intermediaries, listed companies' disclosures |

| IRDAI | 1999 (IRDA Act 1999) | Insurance companies and intermediaries |

| PFRDA | 2003; statutory 2013 | NPS, pension funds, Atal Pension Yojana |

Two more you must know: **IBBI** (Insolvency and Bankruptcy Board of India, 2016, under the IBC) and **IFSCA** (International Financial Services Centres Authority, 2020, the unified regulator for GIFT City).

FSDC — Financial Stability and Development Council (2010, non-statutory) — is chaired by the **Union Finance Minister**, with all regulator heads as members. It is the coordination forum for systemic risk; its sub-committee is chaired by the **RBI Governor**.

3. Money market vs capital market

| | Money market | Capital market |

---|---|---

| Tenure | Up to 1 year | Over 1 year |

| Regulator | RBI | SEBI (RBI for G-sec) |

| Instruments | T-bills, CP, CD, call money, repo, TREPS | Equity, debentures, bonds, derivatives |

| Purpose | Liquidity management | Long-term capital formation |

Money market instruments: Treasury Bills (91/182/364 day, issued at a discount, no default risk), Commercial Paper (unsecured promissory note issued by corporates, 7 days to 1 year), Certificate of Deposit (issued by banks/FIs), Call money (1 day, inter-bank), Notice money (2–14 days), Term money (15 days–1 year), Repo and Reverse Repo, TREPS (Triparty Repo, which replaced CBLO in 2018).

4. Structure of the capital market

Capital Market

- Primary market (new issues) -> IPO, FPO, rights, preferential, QIP, private placement
- Secondary market (trading) -> stock exchanges, OTC
 - Equity segment
 - Debt segment
 - Derivatives (equity F&O, currency, commodity)
 - SLB (securities lending and borrowing)

5. Market participants and intermediaries

- **Issuers** — companies, governments, REITs/InvITs raising money.
- **Investors** — retail, HNI, DII (mutual funds, insurers, pension funds, banks), FPI.
- **SEBI-registered intermediaries** — stock brokers, merchant bankers, registrars to an issue and share transfer agents (RTA), depository participants, underwriters, bankers to an issue, debenture trustees, credit rating agencies, custodians, investment advisers, research analysts, portfolio managers, AIF managers, mutual funds.
- **Market Infrastructure Institutions (MIIs)** — **stock exchanges, clearing corporations, depositories**. Systemically important, and held to the strictest governance, technology and cyber standards. *For the IT stream, MIIs are your home turf.*

6. Banking system at a glance

Scheduled banks (listed in the Second Schedule of the RBI Act) split into commercial and cooperative banks. Commercial banks: public sector (12 after the 2020 mergers), private sector, foreign, regional rural banks, small finance banks, and payments banks (which cannot lend; deposit cap raised to ₹2 lakh in 2021).

NBFCs are regulated by the RBI, cannot accept demand deposits, offer no cheque facility and carry no deposit insurance. Since October 2022 they sit in a **scale-based regulation** pyramid: Base, Middle, Upper and Top Layer.

7. Financial inclusion and development institutions

PMJDY (2014), PMJJBY / PMSBY / APY (2015), MUDRA, Stand-Up India. Development finance: **NABARD** (1982, rural), **SIDBI** (1990, MSME), **EXIM Bank** (1982), **NHB** (1988), **NaBFID** (2021, the infrastructure DFI).

Exam pointers

1. SEBI is **statutory** since 1992 but was **set up** in 1988. Both dates get asked.
2. FSDC is **non-statutory**, chaired by the **Finance Minister**; its sub-committee by the **RBI Governor**.
3. The G-sec market is regulated by the **RBI**, not SEBI — even though G-secs trade on exchanges (retail access through **RBI Retail Direct**).
4. "MII" is a category, not an institution: exchanges + clearing corporations + depositories.
5. Do not confuse **IRDAI** (insurance) with **IBBI** (insolvency) — the options deliberately swap them.

Market Instruments — Equity, Debt and Hybrids

Core preparation lesson

1. Equity instruments

- **Equity shares** — ownership, residual claim, voting rights, dividend not guaranteed.
- **Preference shares** — fixed dividend, priority over equity in dividend and winding up, usually no voting rights. Types: cumulative/non-cumulative, participating/non-participating, convertible/non-convertible, redeemable (must be redeemed within **20 years** under the Companies Act; 30 years for infrastructure projects).
- **DVR — Differential Voting Rights** shares; SEBI permits **superior voting rights (SR) shares** for founders of tech companies subject to strict conditions and sunset clauses.
- **Sweat equity, ESOP, warrants.**
- **Depository receipts:** **ADR** (US), **GDR** (other markets), **IDR** (foreign company raising in India). The underlying shares are held by a custodian; the receipt trades abroad.

2. Debt instruments

| Instrument | Feature |

|---|

| **Debenture** | Corporate debt; secured or unsecured; NCDs are the listed form |

| **Bond** | Generic debt security, usually with a coupon |

| **G-sec** | Sovereign, issued by RBI on behalf of the government; dated securities and T-bills |

| **SDL** | State Development Loans |

| **Masala bond** | Rupee-denominated bond issued **outside India** |

| **Municipal bond** | Issued by urban local bodies; SEBI framework |

| **Green / ESG bond** | Proceeds earmarked for environmental projects |

| **Perpetual / AT1 bond** | No maturity; issued by banks for Basel III capital; can be written down |

Bond mathematics you must know:

- Price and yield move in **opposite** directions.
- **Current yield** = $\text{annual coupon} \div \text{market price}$. **YTM** = the discount rate equating price to the present value of all cash flows.
- **Duration** measures interest-rate sensitivity; **modified duration** estimates the % price change for a 1% yield change. **Convexity** corrects duration for large moves.
- **Credit rating** scale: AAA (highest safety) down through AA, A, BBB (lowest investment grade), BB and below (speculative), D (default).

3. Hybrid and structured instruments

- **Convertible debentures** — FCD (fully), PCD (partly), OCD (optionally).

- **REITs** — Real Estate Investment Trusts. Must distribute at least **90%** of net distributable cash flows; at least **80%** of assets in completed, income-generating property. **SM REITs** (small and medium REITs) were introduced by SEBI in 2024.
- **InvITs** — Infrastructure Investment Trusts; similar structure for infrastructure assets.
- **Securitised debt instruments / pass-through certificates**; **security receipts** under SARFAESI.
- **Zero Coupon Zero Principal (ZCZP)** instruments on the **Social Stock Exchange**.

4. Structure of REIT/InvIT (frequently asked)

Sponsor -> sets up and holds a minimum stake
 Trustee -> independent, holds assets for unit holders (SEBI-registered debenture trustee)
 Manager -> runs operations and investment decisions
 Unit holders -> hold listed units, receive distributions

Exam pointers

1. Masala bonds are **rupee-denominated but issued offshore** — currency risk sits with the investor.
2. AT1 bonds are **perpetual, loss-absorbing** and were at the centre of a well-known Indian banking episode — know the risk story.
3. REIT: **90% distribution, 80% completed assets**.
4. Price and yield move **inversely**; duration measures the sensitivity.
5. IDR = foreign company raising in India; ADR/GDR = Indian company raising abroad.

Investor Protection, Grievance Redress and ODR

Core preparation lesson

1. Why this is SEBI's first mandate

The preamble puts **protection of investors** before development and regulation. Every mechanism below exists because retail investors are structurally disadvantaged in information and bargaining power.

2. SCORES — SEBI Complaints Redress System

- A **web-based centralised grievance portal**; complaints are lodged online and routed to the entity concerned, with defined turnaround times and escalation.
- **SCORES 2.0** (launched 2024) added **auto-routing, auto-escalation, monitoring by designated bodies** and reduced timelines. Complaints can also be filed through the **SEBI SCORES mobile app**.
- Flow: investor first approaches the entity → if unresolved, lodges on SCORES → the entity responds through the portal → review/escalation → if still unresolved, the **ODR** route.

3. ODR — Online Dispute Resolution (SMART ODR)

SEBI operationalised a **common ODR portal** for the securities market (2023). It provides **conciliation followed by arbitration**, conducted online through empanelled ODR institutions, with timelines and cost caps. It sits **after** the direct-with-entity and SCORES steps.

4. Funds that protect investors

- **IPEF — Investor Protection and Education Fund** (SEBI): funded by penalties, disgorgement, unclaimed amounts, grants; used for investor education and, in specified circumstances, restitution.
- **IPF at exchanges and depositories** — compensates investors for a **defaulting member** up to a specified limit.
- **Core SGF** at clearing corporations — protects the settlement system, not individual investors.
- **IEPF** (Ministry of Corporate Affairs, under the Companies Act) — unclaimed dividends and shares; a different fund from SEBI's IPEF. Do not confuse them.

5. Key protective mechanisms

- **KYC through KRAs**, **PAN as the unique identifier**, **nomination mandatory or explicit opt-out**.
- **Segregation of client funds and securities**; **upstreaming** of client funds to clearing corporations; **client-level margin reporting**.
- **Running account settlement** — brokers must settle client funds periodically.
- **Direct payout of securities** to the client demat account.
- **Investor charter** for every intermediary category, disclosing rights, services and timelines.
- **Risk disclosure** requirements, including for equity F&O.

- **Unified KYC (CKYC)**, **e-KYC**, and the **KYC Registration Agencies** regime.
- **Investor awareness programmes**, **SEBI Investor website (investor.sebi.gov.in)**, and the **"Saaṛēthi"** investor education mobile app.

6. Frauds investors must be warned about

Ponzi and multi-level marketing schemes disguised as investments; **unregistered advisers** promising assured returns; **dabba trading** (off-exchange trading); social-media "tips" groups and **front-running of recommendations**; **deepfake and impersonation scams** using the names of genuine market participants. SEBI's standard advice: **check the registration number**, and remember that **no registered entity may promise assured returns**.

Exam pointers

1. **IPEF** is SEBI's; **IEPF** is MCA's. They are different funds.
2. Order of escalation: **entity** → **SCORES** → **ODR** (conciliation then arbitration).
3. **IPF** at exchanges compensates for **member default**, not for market losses.
4. Assured-return promises are the single clearest marker of an illegal scheme.
5. **SCORES 2.0** and the **SMART ODR** portal are recent — quote them by name.

Mutual Funds, AIFs and Portfolio Management

Core preparation lesson

1. Structure of an Indian mutual fund (three-tier)

Sponsor (sets up the fund; min 40% of AMC net worth; track record of 5 years)
|
Trust / Trustees (hold assets for unit holders; at least 2/3 independent)
|
AMC (Asset Management Company – manages schemes; min net worth Rs 50 crore)
+-- Custodian (holds securities, SEBI-registered, independent of sponsor)
+-- RTA (registrar and transfer agent)

Governed by the **SEBI (Mutual Funds) Regulations, 1996**. Industry body: **AMFI** (an SRO-like association, not a statutory regulator).

2. Key concepts

- **NAV** = (Assets – Liabilities) ÷ Units outstanding. Computed daily for open-ended schemes.
- **Open-ended** — continuous subscription/redemption at NAV. **Close-ended** — fixed tenure, listed on an exchange. **Interval** — hybrid.
- **TER** — Total Expense Ratio: capped on a slab basis by AUM; **direct plans** have a lower TER than **regular plans** because they carry no distributor commission.
- **Exit load**, **lock-in** (ELSS: 3 years), **SIP / SWP / STP**.
- **Riskometer** — six levels from Low to Very High; scheme risk must be disclosed and updated monthly.
- **Scheme categorisation (2017-18)**: SEBI standardised categories so that one AMC can have only one scheme per category — Equity (11 categories including large cap, mid cap, small cap, flexi cap, ELSS), Debt (16), Hybrid (6), Solution-oriented (2), Other (index funds, FoF).

3. Types worth knowing

- **Index funds and ETFs** — passive, track an index; ETFs trade on the exchange and need a demat account. **Tracking error** is the key quality metric.
- **Liquid, overnight, money market, ultra-short, corporate bond, credit risk, gilt** — debt categories by maturity/credit.
- **Fund of Funds**, **International funds**, **Gold/Silver ETFs**, **Target maturity funds**.

4. Alternative Investment Funds (AIF) — Regulations, 2012

| Category | What it invests in | Examples |

|---|---|---|

| **Category I** | Socially/economically desirable sectors | Venture capital, SME, infrastructure, social venture, angel funds |

| **Category II** | Funds that are neither I nor III and do not use leverage except for operations | Private equity, debt funds, real estate funds |

| **Category III** | Complex/diverse strategies, may use leverage | Hedge funds, PIPE funds |

- Minimum investment by an investor: **₹1 crore** (₹25 lakh for employees/directors of the manager).
- Minimum corpus: **₹20 crore** (₹10 crore for angel funds).
- Maximum **1,000 investors** per scheme (angel funds: 200 in earlier norms — verify).

5. Portfolio Management Services (PMS)

- Minimum investment **₹50 lakh** (raised from ₹25 lakh in 2020).
- **Discretionary** (manager decides), **non-discretionary** (client decides), **advisory**.
- Securities are held in the client's own demat account — a key difference from mutual funds.

6. Investment advisers vs research analysts vs distributors

| | Investment Adviser | Research Analyst | MF Distributor |

|---|---|---|---|

| Regulation | IA Regulations, 2013 | RA Regulations, 2014 | AMFI registration (ARN) |

| Earns from | Fee from the client | Research subscription/employer | Commission from the AMC |

| Duty | **Fiduciary**, must act in the client's best interest | Disclosure of conflicts | Suitability, no advice for a fee |

The separation of **advice** from **distribution** — you cannot charge a fee for advice and also earn a commission on the same client — is a core SEBI principle.

Exam pointers

1. Three-tier: **Sponsor → Trustee → AMC**; the **trust** structure is what protects unit holders.
2. AMC minimum net worth **₹50 crore**; sponsor must contribute at least **40%** of it.
3. **AIF**: Cat I socially desirable, Cat II PE/debt, Cat III hedge-like with leverage. Minimum ticket **₹1 crore**.
4. **PMS** minimum **₹50 lakh**; securities held in the client's own name.
5. ELSS lock-in **3 years**; direct plans always have a lower TER than regular plans.

Other Regulators and Institutions

Core preparation lesson

IRDAI

- **Insurance Regulatory and Development Authority of India**, IRDA Act **1999**; headquarters **Hyderabad**.
- Regulates insurers, intermediaries, products and solvency. Key terms: **solvency margin (1.5x)**, **IRDAI Ombudsman**, **free-look period (15/30 days)**, **Bima Sugam / Bima Trinity** initiatives, **FDI in insurance up to 74%** (with a policy proposal to go higher — verify).

PFRDA

- **Pension Fund Regulatory and Development Authority**; set up 2003, **statutory in 2013** (PFRDA Act).
- Regulates **NPS** (Tier I and Tier II accounts, PRAN, Point of Presence, Central Recordkeeping Agency — **Protean/NSDL e-Gov, KFinTech, CAMS**), **APY** and the **NPS Trust**.

IBBI

- **Insolvency and Bankruptcy Board of India**, 2016, under the **IBC**. Regulates insolvency professionals, IPAs, information utilities. **CIRP** timeline 180 + 90 days, outer limit **330 days** including litigation. **Committee of Creditors**, **resolution professional**, **liquidation waterfall (S.53)**.

IFSCA

- **International Financial Services Centres Authority**, 2020, **Gandhinagar**. A **unified regulator** for banking, capital markets, insurance and funds in **GIFT IFSC** — the only Indian regulator combining all four mandates. Know **NSE IFSC, India INX**, **bullion exchange (IIBX)**, and the aircraft/ship leasing and fund-management activities there.

Other bodies

| Body | Role |

|---|---|

| **FSDC** | Systemic risk coordination; chaired by the **FM** |

| **NABARD / SIDBI / EXIM / NHB / NaBFID** | Development finance |

| **NPCI** | Retail payment systems |

| **CCIL** | Clearing Corporation of India — G-sec, forex, derivatives clearing |

| **SEBI-recognised SROs** | e.g. AMFI-like bodies; SEBI has been building an SRO framework for intermediaries |

| **FIU-IND** | Financial Intelligence Unit — PMLA reporting (STRs, CTRs) |

| **NFRA** | National Financial Reporting Authority — audit quality for large companies |

| **CCI** | Competition Commission — combinations and anti-competitive conduct |

| **ICAI / ICSI / ICMAI** | Professional institutes |

| **IBA** | Indian Banks' Association — industry body, not a regulator |

PMLA, 2002 — because every intermediary is a reporting entity

- Reporting entities must maintain records, do **KYC**, and report **STRs / CTRs** to **FIU-IND**.
- Record retention **five years**; **designated director** and **principal officer** must be appointed.
- SEBI issues its own **master circular on AML/CFT** for market intermediaries.

Exam pointers

1. IRDAI **Hyderabad**, IFSCA **Gandhinagar**, IBBI and SEBI **Delhi/Mumbai** respectively — headquarters get asked.
2. IFSCA is the **only unified** financial regulator in India.
3. SAT hears appeals from **SEBI, IRDAI, PFRDA and IBBI**.
4. CCIL clears **G-sec and forex**; clearing corporations of exchanges clear equities and derivatives.
5. PMLA reports go to **FIU-IND**, not to SEBI.

Primary Market — IPOs, Book Building, ASBA and UPI

Core preparation lesson

> The primary market is where capital formation actually happens. Expect at least one question on the IPO process, and one on ASBA/UIP mechanics.

1. Ways a company raises capital

| Route | What it is |

|---|---|

| **IPO** | First sale of shares to the public; company becomes listed |

| **FPO** | Further public offer by an already-listed company |

| **Rights issue** | Shares offered to existing shareholders in a fixed ratio |

| **Bonus issue** | Free shares from reserves — no fresh capital raised |

| **Preferential allotment** | Shares to selected persons, priced per ICDR formula |

| **QIP** | Qualified Institutions Placement — quick route for listed companies to raise from QIBs |

| **Private placement** | Offer to fewer than 200 persons in a financial year (Companies Act) |

| **OFS** | Offer for Sale — promoters sell existing shares through the exchange mechanism |

In an IPO, a **fresh issue** brings new money into the company; an **offer for sale (OFS) component** only transfers money to selling shareholders. Know the difference — it is a favourite question.

2. The IPO process end to end

1. Board and shareholder approval
2. Appoint merchant banker (BRLM), registrar, legal counsel, auditors
3. Due diligence; file Draft Red Herring Prospectus (DRHP) with SEBI + exchanges
4. SEBI observations ("comments"); revise
5. File Red Herring Prospectus (RHP) with the RoC
6. Price band announced; anchor investor allocation (1 day before)
7. Issue opens: minimum 3 working days, maximum 10 working days
8. Bidding via ASBA (UPI for retail); basis of allotment finalised with the exchange
9. Allotment, refund/unblocking, credit to demat
10. Listing and trading

Documents: DRHP → RHP → Prospectus. A **red herring** prospectus omits the final price or number of shares.

3. Pricing methods

- **Fixed price issue** — price printed in the prospectus; demand known only after the issue closes; 50% of the issue reserved for applications up to ₹2 lakh.

- **Book building** — a price band (cap must not exceed 120% of the floor) within which investors bid; the **cut-off price** option is available to **retail individual investors** only. Demand is visible on the exchange website during the issue.

Anchor investors: QIBs who are allotted shares one day before the issue opens, with a lock-in (a portion for 30 days and a portion for 90 days under the revised norms). They signal quality and stabilise the book.

4. Investor categories and reservation (book-built issue)

| Category | Typical reservation |

|---|---|

| **QIB** (Qualified Institutional Buyer) | Not more than 50% |

| **NII / HNI** (Non-Institutional Investor, above ₹2 lakh) | Not less than 15% |

| **RI** (Retail Individual Investor, up to ₹2 lakh) | Not less than 35% |

If the issuer does not meet the profitability route under ICDR, at least **75%** must be allotted to QIBs — remember this contrast.

Minimum subscription: the issue must receive at least **90% subscription** of the fresh issue portion, failing which the money is refunded.

5. ASBA and UPI — the plumbing

ASBA — Application Supported by Blocked Amount. Your money stays in **your** bank account and is merely **blocked**; it is debited only on allotment. You keep earning interest, and refunds are instant unblocking rather than cheques. ASBA is **mandatory** for public issues.

UPI mechanism for retail: retail investors applying through an intermediary use their **UPI ID** as the payment instrument; the block happens through a UPI mandate. The limit for UPI-based IPO applications was raised to **₹5 lakh**. Verify the current cap.

Key participants: **SCSB** (Self Certified Syndicate Bank), **Sponsor Bank** (routes the UPI mandate), **Registrar (RTA)**.

6. Timelines

- **T+3 listing:** SEBI reduced the IPO listing timeline from T+6 to **T+3 working days** from the issue closing date — voluntary from September 2023 and mandatory for issues opening on or after 1 December 2023. Verify the current position.
- **Issue open period:** 3 to 10 working days.
- **Price band announcement:** at least 2 working days before opening.

7. Other primary-market instruments

- **SME platforms** — BSE SME and NSE Emerge, with lighter norms and a migration path to the main board.
- **Social Stock Exchange (SSE)** — a segment for not-for-profit organisations and social enterprises; **Zero Coupon Zero Principal (ZCZP) instruments** are issued there.
- **REITs and InvITs** — trusts that hold income-generating real estate / infrastructure assets and list their units.
- **Green bonds and ESG debt** — SEBI framework for green debt securities.

Exam pointers

1. **DRHP → RHP → Prospectus**; the red herring omits price or quantity.
2. **$QIB \leq 50\%$, $NII \geq 15\%$, $RII \geq 35\%$** ; **75% to QIBs** if the issuer uses the non-profitability route.
3. **Cut-off price is only for retail** investors.
4. ASBA blocks funds in your own account; UPI is the retail payment rail.
5. **Minimum subscription 90%**; listing in **T+3** working days.

RBI, Monetary Policy and the Banking System

Core preparation lesson

1. RBI basics

- Established **1 April 1935** under the RBI Act, 1934, on the recommendation of the **Hilton Young Commission**; **nationalised 1 January 1949**. Headquarters: **Mumbai** (moved from Kolkata in 1937).
- **Central Board**: Governor + up to 4 Deputy Governors + government-nominated directors.
- Functions: note issue (**Minimum Reserve System** since 1957 — ₹200 crore of assets, of which ₹115 crore in gold), banker to the government, banker to banks, regulator and supervisor, manager of forex (FEMA), manager of the public debt, developmental roles, and operator of payment systems.

2. Monetary policy framework

- **Flexible Inflation Targeting** since 2016 (amendment to the RBI Act). Target: **CPI inflation of 4%**, with a band of **+/- 2%**.
- **Monetary Policy Committee (MPC)** — **six members**: 3 from RBI (Governor as chair, a Deputy Governor, one officer) and **3 external members** appointed by the government for 4 years. Decisions by majority; the **Governor has a casting vote**. Meets at least **four times a year** (in practice, six bi-monthly meetings).
- **Failure** to maintain the target for **three consecutive quarters** requires a report to the government.

3. Policy instruments

| Instrument | Meaning |

|---|

| **Repo rate** | Rate at which RBI lends to banks against securities |

| **Reverse repo** | Rate at which RBI absorbs liquidity (largely replaced by SDF) |

| **SDF** | Standing Deposit Facility — absorbs liquidity **without collateral** (introduced 2022) |

| **MSF** | Marginal Standing Facility — emergency borrowing, above the repo rate |

| **Bank rate** | Long-term rate; aligned with MSF; penal rate for CRR/SLR shortfalls |

| **CRR** | Cash Reserve Ratio — % of NDTL as cash with RBI; **no interest paid** |

| **SLR** | Statutory Liquidity Ratio — % of NDTL in gold, cash or approved securities |

| **OMO** | Open market operations — purchase/sale of government securities |

| **LAF corridor** | MSF (ceiling) — repo (policy rate) — SDF (floor) |

Qualitative tools: margin requirements, moral suasion, direct action, selective credit control.

4. Banking terms you must know

- **NPA** — an asset overdue for **more than 90 days**. Classification: Standard → **Sub-standard** (up to 12 months as NPA) → **Doubtful** → **Loss**. **SMA-0/1/2** are early warning categories before NPA.

- **Provisioning Coverage Ratio**, **CRAR / Capital Adequacy** under **Basel III** (minimum 9% in India plus CCB), **Tier 1 and Tier 2 capital**, **leverage ratio**, **LCR** and **NSFR**.
- **PCA — Prompt Corrective Action**: triggered by breaches of capital, asset quality (net NPA) and leverage thresholds.
- **SARFAESI Act, 2002** — enforcement of security interest without court intervention; **ARCs**, **DRT**, and the **IBC, 2016** for resolution (270/330-day timelines).
- **Priority Sector Lending** — 40% of ANBC for commercial banks, with sub-targets for agriculture (18%), weaker sections (12%) and micro enterprises (7.5%). **PSLCs** allow trading of the obligation.
- **Deposit insurance** by **DICGC** — up to **₹5 lakh** per depositor per bank (raised from ₹1 lakh in 2020).

5. Payment systems (directly relevant to IT)

NPCI (2008, a not-for-profit under the Payment and Settlement Systems Act, 2007) runs **UPI**, **IMPS**, **NACH**, **AePS**, **RuPay**, **NETC FASTag**, **BBPS**, **NFS ATM network**. **RTGS** (real time, gross, high value, **24x7** since December 2020), **NEFT** (half-hourly batches, **24x7** since December 2019). **CBDC (e-Rupee)** pilots in retail and wholesale since 2022. **Account Aggregator** framework for consented financial data sharing; **OCEN** for credit rails.

Exam pointers

- CPI target 4% +/- 2%**; MPC has **6 members**, Governor has the **casting vote**.
- SDF** is collateral-free and forms the floor of the LAF corridor.
- NPA at **90 days**; DICGC insurance **₹5 lakh**.
- CRR earns **no interest**; SLR is held in approved securities and does earn returns.
- NPCI operates UPI; **RBI** regulates payment systems under the **PSS Act, 2007**.

Securities Contracts (Regulation) Act, 1956 (SCRA)

Core preparation lesson

> The oldest of the three securities statutes. It answers two questions: *what is a security*, and *who may run a stock exchange*.

1. Purpose

Enacted to **prevent undesirable transactions in securities** by regulating the business of dealing in them. Before SEBI existed it was administered by the Central Government; today SEBI administers most of it by delegation.

2. Section 2(h) — definition of "securities"

Securities **include**:

1. **shares, scrips, stocks, bonds, debentures, debenture stock** or other marketable securities of a like nature in or of any incorporated company or **other body corporate**;
 - (ia) **derivatives**;
 - (ib) **units or any other instrument issued by any collective investment scheme**;
 - (ic) **security receipts** (as defined in the SARFAESI Act);
 - (id) units issued to investors under any **mutual fund** scheme;
 - (ie) units or instruments issued by an **InvIT / REIT**;
2. **Government securities**;
3. such other instruments as the **Central Government** declares to be securities;
4. **rights or interest in securities**.

Not securities: bank fixed deposits, insurance policies, and physical commodities — though **commodity derivatives** became securities when the **Forward Markets Commission merged into SEBI on 28 September 2015**.

3. Other key definitions

- **2(i) spot delivery contract** — delivery and payment on the same day or the next day. Spot delivery sits **outside** most SCRA prohibitions, which is why off-market transfers work.
- **2(ac) derivative** — a security derived from a debt instrument, share, loan, risk instrument or contract for differences; includes commodity derivatives.
- **2(j) stock exchange** — a body of individuals or a **corporatised and demutualised** entity assisting in buying, selling and dealing in securities.

4. Recognition and governance of exchanges

| Section | Content |

|---|---|

| ****3**** | Application for recognition |

| ****4**** | ****Grant of recognition**** |

| ****4A / 4B**** | ****Corporatisation and demutualisation**** |

| ****5**** | Withdrawal of recognition |

| ****9**** | Power of exchanges to make ****bye-laws**** |

| ****11**** | Power to ****supersede**** an exchange's governing body |

| ****12**** | Power to ****suspend business**** of an exchange |

| ****13**** | Contracts in ****notified areas**** — only between members |

| ****16**** | Power to prohibit contracts in certain cases |

| ****18**** | Exemptions — spot delivery contracts |

| ****21**** | ****Conditions for listing**** |

| ****21A**** | ****Delisting**** by a stock exchange |

| ****23**** | ****Offences and penalties**** |

| ****23A–23H**** | Monetary penalties for specific defaults |

| ****23L**** | ****Appeal to SAT**** |

| ****30**** | Rule-making power → ****SCRR, 1957**** |

****Demutualisation**** (S.4A/4B) separates ****ownership, management and trading rights****. Before it, brokers owned and ran the exchange they traded on — a structural conflict of interest. Post-demutualisation, at least ****51% of an exchange's equity must be held by the public**** (non-trading members) and exchanges became for-profit corporate entities.

5. SCRR, 1957 — the operational rules

****Rule 19(2)(b) and Rule 19A — Minimum Public Shareholding (MPS)****

- Every listed company must maintain ****at least 25% public shareholding****.
- Certain categories (including some PSUs) have had extended timelines; the minimum offer at IPO varies by post-issue market capitalisation, with a timeline to reach 25%.
- Verify the current slabs before the exam — they have changed more than once.

6. How the three Acts fit together

| Act | Answers |

|---|---|

| ****SCRA 1956**** | What is a security? Who may run an exchange? What are listing obligations? |

| ****SEBI Act 1992**** | Who regulates? What powers, penalties and appeals exist? |

| ****Depositories Act 1996**** | How are securities held and transferred electronically? |

The **Companies Act, 2013** sits alongside for issue of shares, prospectus and corporate governance — which is why LODR and the Companies Act sometimes overlap.

Exam pointers

1. Derivatives, CIS units, MF units, security receipts and REIT/InvIT units **are** securities under 2(h).
2. **Spot delivery** = same day or next day.
3. **4A/4B** = demutualisation · 21 = listing · 23L = appeal to SAT.
4. **MPS = 25%** under SCRR Rule 19A.
5. FMC merged into SEBI on **28 September 2015**; the Forward Contracts (Regulation) Act, 1952 was repealed.

SEBI Act, 1992 — Sections, Penalties and Appeals

Core preparation lesson

> Learn the section numbers. In Phase I they are one-mark questions; in the interview, saying "Section 11B directions" instead of "SEBI told them to stop" changes how you are read.

1. Sections you must know cold

| Section | Content |

| ---|---|

| **3** | Establishment of the Board |

| **4** | Management — Chairman + 8 members |

| **11** | Functions of the Board |

| **11(2A)** | Power to call for information and conduct inspections |

| **11(3)** | Civil court powers — discovery, summons, records |

| **11(4)** | Interim measures — suspend trading, restrain persons, impound proceeds, attach accounts |

| **11A** | Powers over issue of capital, transfer of securities, listing |

| **11AA** | Definition of a **collective investment scheme** |

| **11B** | Power to issue **directions**, including **disgorgement** |

| **11C** | **Investigation** — investigating authority, seizure of books |

| **11D** | Cease and desist proceedings |

| **12** | **Registration** of intermediaries |

| **12A** | Prohibition of manipulative and deceptive devices, insider trading, substantial acquisition |

| **15-I** | **Adjudicating Officer** — inquiry and penalty |

| **15J** | Factors for quantifying penalty — disproportionate gain, loss to investors, repetitive default |

| **15JB** | **Settlement (consent) of proceedings** |

| **15K–15M** | Establishment and composition of **SAT** |

| **15T** | **Appeal to SAT** — within 45 days |

| **15Z** | **Appeal to the Supreme Court on a question of law** — within 60 days |

| **24** | **Offences** — imprisonment up to 10 years and/or fine up to ₹25 crore |

| **26A–26E** | **Special Courts**, attachment and recovery, SEBI's first charge |

| **28A** | **Recovery** as arrears of land revenue |

| **30** | Power to **make regulations** |

2. Penalty sections (Chapter VI-A)

| Section | Default | Penalty as commonly cited |

|---|---|---|

| 15A | Failure to furnish information/returns | ₹1 lakh per day, up to ₹1 crore |

| 15B | Failure to redress investor grievances | ₹1 lakh/day, up to ₹1 crore |

| 15D | Defaults by a mutual fund | ₹1 lakh/day, up to ₹1 crore |

| 15E | AMC failing to observe investment norms | up to ₹1 crore |

| 15F | Broker defaults | up to ₹1 crore |

| **15G** | **Insider trading** | **₹10 lakh minimum**, up to **₹25 crore or 3× the profit, whichever is higher** |

| **15HA** | **Fraudulent and unfair trade practices** | **₹5 lakh minimum**, up to **₹25 crore or 3× the profit, whichever is higher** |

| 15HB | Residual penalty | ₹1 lakh minimum, up to ₹1 crore |

> □ These amounts have been revised by amendment more than once (notably the Securities Laws (Amendment) Act, 2014 and the Finance Act, 2018). Confirm current figures from the **bare Act on sebi.gov.in** before the exam.

Settlement (S.15JB): proceedings may be settled on payment of a settlement amount under the **SEBI (Settlement Proceedings) Regulations, 2018**. Settlement is **without admission of guilt**, is discretionary, and cannot be claimed as a right.

3. Securities Appellate Tribunal (SAT)

- Established under **Section 15K**; principal bench at **Mumbai**.
- **Presiding Officer** — a sitting or retired Supreme Court judge or Chief Justice of a High Court — plus judicial and technical members.
- Hears appeals against orders of **SEBI, IRDAI, PFRDA and IBBI** — a commonly missed fact.
- **45 days** to appeal from receipt of the order; delay condonable for sufficient cause.
- Not bound by the **CPC**; follows natural justice and regulates its own procedure.
- Further appeal: **Supreme Court on a question of law within 60 days** (extendable by 60 days).

4. How an enforcement case actually moves

Surveillance alert / complaint / reference

- > Preliminary examination
- > Investigation (S.11C)
- > Show-cause notice
- > Adjudication (S.15-I) OR Directions (S.11B / 11(4)) OR Prosecution (S.24, Special Court)
-)
- > Order: penalty, disgorgement, debarment, suspension or cancellation of registration
- > Appeal: SAT (45 days) -> Supreme Court on a question of law (60 days)

A parallel route — **settlement under S.15JB** — is available at most stages.

Exam pointers

1. **45 days** to SAT, **60 days** to the Supreme Court.

2. SAT hears **SEBI + IRDAI + PFRDA + IBBI** appeals.
3. **11B** = directions · **11C** = investigation · **15-I** = adjudication · **15JB** = settlement · **28A** = recovery.
4. Disgorgement is **not** a penalty — it removes unlawful gain and can be ordered **in addition to** a penalty.
5. S.24 offences: up to **10 years** imprisonment and/or ₹25 crore fine, tried by a **Special Court**.

The SEBI Regulations You Must Know By Name

Core preparation lesson

> You will not be asked to recite regulations, but you will be expected to know which one governs what. Learn the five-letter shortlist perfectly; recognise the rest.

1. LODR — Listing Obligations and Disclosure Requirements Regulations, 2015

The corporate governance and disclosure rulebook for listed companies.

- **Regulation 17** — Board of Directors: at least **6 directors**; at least **one woman director** (and an **independent woman director** for the top 1000 listed entities by market cap); at least **one-third independent directors**, and **half** if the chairperson is an executive or a promoter.
- **Regulation 18** — **Audit Committee**: minimum 3 directors, **two-thirds independent**, all financially literate; chaired by an independent director.
- **Regulation 19** — Nomination and Remuneration Committee. **Regulation 20** — Stakeholders Relationship Committee. **Regulation 21** — Risk Management Committee (top 1000 entities).
- **Regulation 23** — **Related Party Transactions**: prior approval of the audit committee; material RPTs need shareholder approval, and **related parties cannot vote**.
- **Regulation 24A** — annual **secretarial audit**.
- **Regulation 30** — **disclosure of material events**, with a **quantitative materiality threshold** and strict timelines (12/24 hours depending on the event); the 2023 amendments tightened this considerably and added **verification of market rumours** for large listed entities.
- **Regulation 33** — quarterly **financial results**. **Regulation 31** — shareholding pattern. **Regulation 34** — annual report, including the **Business Responsibility and Sustainability Report (BRSR)** for the top listed entities, with **BRSR Core** assurance.
- **Regulation 44** — **e-voting** for shareholder resolutions.

2. ICDR — Issue of Capital and Disclosure Requirements Regulations, 2018

Governs public issues, rights issues, preferential allotment, QIP, bonus, IPO eligibility (the profitability route vs the 75%-to-QIB route), promoter contribution (**20% minimum, locked in**), pricing formulas and disclosure standards in the offer document.

3. PIT — Prohibition of Insider Trading Regulations, 2015

- **UPSI** — Unpublished Price Sensitive Information: information not generally available which, on becoming available, is likely to **materially affect the price** of securities. Illustrative list: financial results, dividends, change in capital structure, mergers/acquisitions/delisting, changes in key managerial personnel.
- **Insider** — a **connected person**, or anyone **in possession of or having access to UPSI**. Note that possession is enough; you need not be an employee.
- **Trading Plans**, **Code of Fair Disclosure**, **Structured Digital Database (SDD)** — every listed entity must maintain a database of persons with whom UPSI is shared, with time stamps and **non-tamperable audit trails**.
This is an IT-relevant requirement: know it.

- **Contra-trade restriction** (six months), **trading window closure**, and **legitimate purpose** exceptions.

4. SAST — Substantial Acquisition of Shares and Takeovers Regulations, 2011

- **25% trigger** — acquiring 25% or more of voting rights requires an **open offer**.
- **Creeping acquisition** — a holder between 25% and the maximum permissible limit may acquire up to **5%** in a financial year without an open offer.
- **Open offer size**: minimum **26%** of the target's shares.
- **Indirect acquisition, competing offers, exemptions** (inter-se promoter transfers, rights issues) and the role of the **manager to the offer**.

5. PFUTP — Prohibition of Fraudulent and Unfair Trade Practices Regulations, 2003

Prohibits manipulation, misleading statements, and market abuse. Classic patterns: **circular trading**, **synchronised trades**, **pump and dump**, **front running**, **spoofing/layering**, **wash trades**, **misleading corporate announcements**. Penalty under **S.15HA** of the SEBI Act.

6. The rest — recognise these

| Regulation | Subject |

|---|---|

| Mutual Funds, 1996 | Mutual funds |

| AIF, 2012 | Alternative investment funds |

| Investment Advisers, 2013 · Research Analysts, 2014 | Advice and research |

| Portfolio Managers, 2020 | PMS |

| Depositories and Participants, 2018 | Depositories, DPs |

| Stock Brokers, 1992 | Brokers |

| SECC, 2018 | **Stock Exchanges and Clearing Corporations** — MII governance |

| Credit Rating Agencies, 1999 | CRAs |

| Delisting of Equity Shares, 2021 | Delisting |

| Buy-back of Securities, 2018 | Buybacks |

| REIT, 2014 · InvIT, 2014 | Trusts |

| Intermediaries, 2008 | Common intermediary norms |

| Settlement Proceedings, 2018 | Consent settlement |

| Informant Mechanism (under PIT) | Whistleblower rewards for insider trading information |

SECC Regulations matter most for the IT stream: they carry the governance, **technology**, **system audit**, **business continuity** and **cyber requirements** for exchanges and clearing corporations.

Exam pointers

1. **LODR 17/18/23/30/33** are the regulation numbers most often asked.
2. PIT's **Structured Digital Database** is a technology obligation — a natural IT-stream interview question.
3. SAST: **25% trigger, 5% creeping, 26% open offer**.
4. PFUTP → penalty under **S.15HA**; insider trading → **S.15G**.
5. **SECC 2018** governs MIIIs — know it by name.

SEBI — Establishment, Structure, Powers and Functions

Core preparation lesson

> Assume you will be asked "What does SEBI do?" in the interview. A generic answer is a wasted opportunity. Answer with the **three-fold preamble**, then give one live example of each mandate.

1. Establishment

- Set up as a **non-statutory body** on 12 April 1988 by a government resolution.
- Given **statutory status** by the SEBI Act, 1992, brought into force from 30 January 1992 (ordinance route; the Act received assent on 4 April 1992 with retrospective effect).
- Trigger: the 1991–92 securities scam and the repeal of the **Capital Issues (Control) Act, 1947**, which had made the Controller of Capital Issues the pricing authority. With the CCI gone, issuers got **free pricing** — and the market needed a disclosure-based regulator.
- **Headquarters:** Bandra Kurla Complex, Mumbai. **Regional offices:** Northern (New Delhi), Eastern (Kolkata), Southern (Chennai), Western (Ahmedabad), plus local offices.

2. The preamble — memorise it

> "...to protect the interests of investors in securities and to promote the development of, and to regulate, the securities market and for matters connected therewith or incidental thereto."

Three mandates, deliberately in tension:

| Mandate | Example |

|---|---|

| **Protection** | Grievance redress (SCORES), disclosure norms, action on mis-selling |

| **Development** | REITs/InvITs, T+1 settlement, UPI in IPOs, Social Stock Exchange |

| **Regulation** | Registration, inspection, investigation, penalties, debarment |

3. Composition of the Board (Section 4)

1. A **Chairman**;
2. **Two members** from the Ministries dealing with **Finance** and the administration of the **Companies Act**;
3. **One member** nominated by the **RBI**;
4. **Five other members**, of whom **at least three are whole-time members**.

Total strength **nine**; appointments are made by the **Central Government**.

4. Functions (Section 11)

Section 11(1) states the duty; **Section 11(2)** lists the measures:

- regulating the **business in stock exchanges** and any securities market;
- **registering and regulating intermediaries** — brokers, share transfer agents, bankers to an issue, trustees of trust deeds, registrars, merchant bankers, underwriters, portfolio managers, investment advisers, depositories, custodians, FPIs, credit rating agencies;
- registering and regulating **collective investment schemes** including **mutual funds**;
- promoting and regulating **self-regulatory organisations**;
- **prohibiting fraudulent and unfair trade practices**;
- **prohibiting insider trading**;
- regulating **substantial acquisition of shares and takeovers**;
- promoting **investor education** and training of intermediaries;
- **inspection, inquiries and audits** of exchanges, intermediaries and SROs;
- calling for information from banks and authorities;
- levying **fees and other charges**;
- conducting **research**.

5. Powers — the three hats

(a) Quasi-legislative — it **makes** rules. SEBI frames regulations under **Section 30** (LODR, ICDR, PIT, SAST, PFUTP, Mutual Funds, AIF and so on). Regulations are laid before Parliament.

(b) Quasi-executive — it **enforces**: registration (S.12), inspection and **investigation** (S.11C), search and seizure, **directions** (S.11B) including **disgorgement**, and civil-court powers under **S.11(3)**.

(c) Quasi-judicial — it **adjudicates**: Adjudicating Officers under **S.15-I** impose monetary penalties; the Board debars persons from the market. Appeals lie to **SAT**.

Because SEBI wears all three hats, its orders are appealable and its regulations reviewable — a standard discussion question.

6. Key departments (useful in the interview)

Market Regulation (MRD) · Market Intermediaries Regulation and Supervision (MIRSD) · Corporation Finance (CFD) · Investment Management (IMD — MFs, AIFs, REITs) · Integrated Surveillance (ISD) · Enforcement · Legal Affairs · Investigations · **Information Technology (ITD)** · Economic and Policy Analysis (DEPA) · Office of Investor Assistance and Education (OIAE).

> **If asked which department you would want:** the Information Technology Department — and have reasons ready: MII system audits, cyber-resilience supervision under CSCRF, surveillance analytics, and SEBI's own digital infrastructure.

7. Accountability

- SEBI submits an **annual report** to the Central Government, laid before **both Houses of Parliament**.
- Accounts are audited by the **CAG**.

- The Central Government may issue **written policy directions (S.16)** and may **supersede the Board (S.17)**.
- SEBI is funded by **fees and charges** from market participants, not by a budgetary grant.

Exam pointers

1. 1988 = set up; 1992 = statutory.
2. Nine members; **at least three whole-time** among the "five other members".
3. S.11 functions · S.11B directions · S.11C investigation · S.12 registration · S.15-I adjudication · S.15T appeal · S.30 regulations.
4. SEBI regulates **securities markets** — not banks, insurance or pensions.
5. The preamble's word order — **protect, develop, regulate** — is itself a question.

Secondary Market — Exchanges, Indices, Trading and Settlement

Core preparation lesson

> One question here is nearly guaranteed, and the vocabulary in this note is what you will use for the rest of your career at SEBI.

1. The exchanges

| Exchange | Set up | Notes |

|---|---|---|

| **BSE** | 1875, as the Native Share and Stock Brokers' Association | Oldest exchange in Asia; benchmark **SENSEX** (30 stocks, base 1978-79 = 100) |

| **NSE** | 1992, began trading 1994 | Introduced screen-based nationwide trading; benchmark **NIFTY 50** (base 3 Nov 1995 = 1000) |

| **MSE** | Metropolitan Stock Exchange | Smaller third exchange |

| **NCDEX / MCX** | Commodity derivatives exchanges | Under SEBI since the FMC merger in 2015 |

Index methodology: both SENSEX and NIFTY use **free-float market capitalisation** weighting. Free float excludes promoter and locked-in holdings.

2. Trading mechanics

- **Order-driven market** with an **electronic limit order book**; orders match on **price-time priority**.
- **Order types:** market, limit, stop-loss, immediate-or-cancel (IOC), good-till-cancelled, disclosed quantity, after-market orders.
- **Trading hours:** equity 9:15 to 15:30, with a **pre-open session 9:00 to 9:08** (order entry 9:00–9:07, matching and price discovery 9:08–9:12).
- **Circuit breakers (market-wide):** index movement of **10%, 15% and 20%** triggers a market halt, with the halt duration depending on the level and time of day. Individual scrips have **price bands** (2%, 5%, 10%, 20%) and **dynamic price bands** for F&O scrips.
- **Rolling settlement:** currently **T+1**, with an optional **T+0** beta segment introduced in 2024.

3. Clearing and settlement

Clearing corporations act as the **central counterparty (CCP)**: through **novation** they become the buyer to every seller and the seller to every buyer, which is what removes counterparty risk.

Risk management toolkit: **VaR margin**, **extreme loss margin (ELM)**, **mark-to-market margin**, **peak margin reporting**, **upfront margin collection**, **Settlement Guarantee Fund (core SGF)**, **auction** in case of short delivery, and **default waterfall**.

Upstreaming of client funds and the **ASBA-like block mechanism for the secondary market** are recent investor-protection developments — know at least the principle: client money should not sit with brokers.

4. Surveillance measures (high-yield for the IT stream)

- **ASM** — Additional Surveillance Measure; **GSM** — Graded Surveillance Measure. Both impose higher margins or trade-for-trade settlement on stocks showing abnormal price behaviour.
- **Trade-for-trade segment** — compulsory delivery, no intraday netting.
- **Integrated market surveillance system** at SEBI plus exchange-level surveillance; alerts feed into investigation. *If you are asked in the interview how you would use technology at SEBI, this is the most natural answer: pattern detection over order and trade data.*

5. Market participants and accounts

- **Trading member, clearing member, professional clearing member, custodian.**
- **FPI** — Foreign Portfolio Investors, categorised and registered with SEBI through Designated Depository Participants (DDPs).
- **Retail direct access:** brokers, and for G-secs the **RBI Retail Direct** portal.
- **Client-level requirements:** PAN, KYC through a KRA, demat account, Unique Client Code, nomination, **running account settlement** of funds.

6. Corporate actions

Dividend, bonus, split, rights, buyback, merger/demerger. **Ex-date vs record date:** to be eligible, you must own the share **before the ex-date**; under T+1 settlement the ex-date and record date are effectively the same day. Price adjusts on the ex-date for bonus, split and dividend.

7. Other segments

- **Derivatives:** index and stock futures/options, currency derivatives, commodity derivatives, interest rate futures.
- **Debt segment:** corporate bonds, G-sec (NDS-OM for wholesale), **Request for Quote (RFQ)** platform, **Online Bond Platform Providers (OBPPs)** for retail.
- **SLB** — securities lending and borrowing through the clearing corporation.
- **Municipal bonds**, **InvIT/REIT units**, **ETFs** and **REITs** trade in the cash segment.

Exam pointers

1. BSE **1875** (Asia's oldest), NSE **1992/1994**. SENSEX base **1978-79 = 100**; NIFTY base **3 Nov 1995 = 1000**.
2. Both indices are **free-float market-cap weighted**.
3. Market-wide circuit breakers at **10 / 15 / 20 per cent**.
4. **Novation** is what makes a clearing corporation a CCP.

5. ****ASM and GSM**** are surveillance measures, not punishments — they raise the cost of speculation in a stock.

Static GK — The Shortlist Worth Memorising

Core preparation lesson

> Static GK has low marginal returns. Learn this list, then stop. Spend the time you save on securities market awareness, which is worth far more marks.

International financial institutions

| Institution | HQ | Note |

|---|---|---|

| IMF | Washington DC | 1944 Bretton Woods; SDR; Article IV consultations; World Economic Outlook |

| World Bank (IBRD) | Washington DC | IDA, IFC, MIGA, ICSID are the other four of the five WBG institutions |

| WTO | Geneva | 1995, successor to GATT |

| BIS | Basel | "Central bank of central banks"; **Basel norms** |

| **IOSCO** | Madrid | **International Organization of Securities Commissions — SEBI is a member; know this one** |

| FSB | Basel | Financial Stability Board, G20-linked |

| FATF | Paris | Anti-money-laundering standards; grey/black list |

| ADB | Manila | Asian Development Bank |

| AIIB | Beijing | NDB (BRICS bank) is at **Shanghai** |

| OECD | Paris | |

| UNCTAD | Geneva | World Investment Report |

Indian market institutions

| Institution | Set up | HQ |

|---|---|---|

| BSE | 1875 | Mumbai |

| NSE | 1992 | Mumbai |

| NSDL | 1996 | Mumbai |

| CDSL | 1999 | Mumbai |

| NPCI | 2008 | Mumbai |

| CCIL | 2001 | Mumbai |

| NABARD | 1982 | Mumbai |

| SIDBI | 1990 | Lucknow |

| EXIM Bank | 1982 | Mumbai |

| IRDAI | 1999 | Hyderabad |

| IFSCA | 2020 | Gandhinagar |

| IBBI | 2016 | New Delhi |

| NIC / MeitY bodies | — | New Delhi |

| CERT-In | 2004 | New Delhi |

Indices and reports (who publishes what)

- **Ease of Doing Business** — discontinued by the World Bank; replaced by **B-READY**.
- **Global Innovation Index** — WIPO. **Human Development Index** — UNDP.
- **World Economic Outlook** — IMF. **Global Financial Stability Report** — IMF.
- **Financial Stability Report** — **RBI**. **Trend and Progress of Banking in India** — RBI.
- **Economic Survey** — Ministry of Finance (CEA).
- **Global Competitiveness Report** — WEF. **Corruption Perceptions Index** — Transparency International.

Committees to recognise (securities market)

| Committee | Subject |

|---|---|

| Narasimham (1991, 1998) | Banking sector reforms |

| **Kotak Committee (2017)** | **Corporate governance** — shaped LODR amendments |

| Uday Kotak / SEBI committees on MII governance | Exchange governance |

| **Bimal Jalan Committee (2010)** | **MIl ownership and governance** |

| JR Varma | Derivatives risk containment |

| L C Gupta | Introduction of derivatives (1998) |

| **T K Viswanathan** | **Fair market conduct / PIT and PFUTP review** |

| Justice Sodhi Committee | Insider trading regulations review |

| Damodaran Committee | Investor protection and grievances |

| H R Khan | Municipal bonds / retail participation |

Exam pointers

1. **IOSCO** — Madrid. SEBI is a member and signed the **IOSCO MMoU**; expect a question.
2. **AIIB Beijing, NDB Shanghai** — routinely swapped in options.
3. **Financial Stability Report** = **RBI**; **Global Financial Stability Report** = **IMF**.
4. **Kotak Committee** = corporate governance; **Bimal Jalan** = MII ownership; **L C Gupta** = derivatives.
5. SIDBI is at **Lucknow** — the one institution that is not in Mumbai or Delhi.

Ethics and situational interview practice

Core preparation lesson

A useful response identifies the facts, stakeholders, applicable process, immediate action and how the decision will be recorded. Avoid promising actions beyond your authority.

Scenario

A colleague asks you to share restricted information to solve an urgent issue. Clarify the legitimate need, verify authorisation, use the approved channel and share only what is necessary. Escalate uncertainty through the appropriate process rather than quietly bypassing controls.

Failure story

Choose a real example. Explain your responsibility, corrective action and the check you introduced to prevent recurrence. Do not shift all responsibility to others or invent a perfect ending.

Mock interview routine

Practice one profile, one technical and one situational question aloud. Review clarity, evidence, judgement and honesty. If you do not know a fact, state that and explain how you would verify it. Save recurring weaknesses in the revision queue.

IT interview scenarios — original practice prompts

Core preparation lesson

These prompts are original exercises, not claims about past panel questions.

Scenario: duplicate transaction messages

Explain idempotency keys, a uniqueness constraint, transaction boundaries and safe retries. Discuss how you would reconcile a timeout when the sender does not know whether processing succeeded.

Scenario: suspicious login burst

Describe monitoring, evidence preservation, risk-based containment, escalation and recovery. Explain false positives and why you would avoid a blanket response without assessing impact.

Scenario: recovery readiness

Distinguish recovery time from acceptable data loss. Describe backups, restore drills, dependency checks, access control and communication responsibilities.

Daily practice

Pick one scenario. Spend two minutes structuring the answer, speak for two minutes and then challenge one assumption. End with a measurable check, such as successfully restoring a test dataset. Use official policy documents when discussing specific regulatory requirements.

Build your interview profile

Core preparation lesson

Prepare a one-page factual profile: education, two projects, responsibilities, a difficulty you handled, a mistake you corrected and what you learned. Be ready to explain each technical term you use.

Project explanation

Use problem, constraints, design, tradeoff, result and next improvement. For a database project explain why you chose the keys, how transactions fail safely, and how access is controlled. Describe your own contribution separately from your team's.

Practice

Record a 90-second introduction. Remove claims you cannot support. Ask yourself three follow-up questions per project: what fails under load; how do you test recovery; what would you change now? Write honest answers with concrete examples. Review your application details before the interview. These are practice prompts, not reported panel questions.

Prepare current SEBI examples for interview

Core preparation lesson

Use the official vacancies page for recruitment updates and SEBI publications for institutional information. Keep a dated evidence sheet with the document title, official link, publication date and a short explanation of why it matters.

Weekly evidence sheet

Choose one annual-report section, one circular and one investor-education page. For each write: problem addressed; affected participants; operational implication; open question. Separate a policy's stated objective from your interpretation of its effect.

Mock response

Explain a technical control in 60 seconds without jargon, then explain how you would measure whether it works. Do not memorise current officeholders or numerical thresholds from undated notes. Verify those facts again close to the interview.

[Official SEBI website](<https://www.sebi.gov.in/>)

Why SEBI and the IT stream

Core preparation lesson

Build an answer connecting public purpose, your technical experience and the work you want to learn. Understand the regulator's role through its official website and annual report. Avoid claiming knowledge of an internal system you have never used.

Answer structure

One sentence on investor protection and orderly markets; one example of relevant technical work; one sentence on how you want to contribute and develop. Mention reliability, data quality or secure systems only when you can discuss them concretely.

Follow-ups

Why regulation rather than a private technology role? How would you handle repetitive operational work? What is one knowledge gap you are actively closing? Prepare truthful answers instead of a memorised speech. Read a recent official report and explain one point in plain language.

Artificial Intelligence and Machine Learning

Supplementary reference / interview

Types of learning

| Type | Data | Goal | Examples |

|---|---|---|---|

| **Supervised** | Labelled | Predict a label/value | Regression, classification (fraud/not fraud) |

| **Unsupervised** | Unlabelled | Find structure | Clustering, dimensionality reduction, **anomaly detection** |

| **Semi-supervised** | Mostly unlabelled | Use a few labels | Common in fraud, where labels are scarce |

| **Reinforcement** | Reward signal | Learn a policy | Trading agents, control systems |

Core algorithms to recognise

Supervised: linear regression, logistic regression, decision trees, **random forest**, **gradient boosting** (XGBoost/LightGBM), SVM, k-NN, Naive Bayes, neural networks.

Unsupervised: **k-means**, hierarchical clustering, DBSCAN, **PCA**, isolation forest, autoencoders.

The workflow

Problem framing -> data collection -> EDA -> cleaning and feature engineering
-> train/validation/test split -> model selection -> training -> evaluation
-> deployment -> monitoring (drift) -> retraining

Feature engineering usually matters more than algorithm choice. **Data leakage** — letting information from the future or from the target into the features — is the most common cause of a model that works in testing and fails in production.

Evaluation

Confusion matrix → TP, FP, TN, FN.

- **Accuracy** = $(TP+TN)/total$ — **misleading on imbalanced data**. A fraud model that predicts "not fraud" always can be 99.9% accurate and useless.

- **Precision** = $TP/(TP+FP)$ — of what we flagged, how much was real. *This is what controls analyst workload in a surveillance system.*

- **Recall / Sensitivity** = $TP/(TP+FN)$ — of what was real, how much did we catch.

- **F1** = harmonic mean of precision and recall.

- **ROC-AUC**, **PR-AUC** (better for heavy imbalance).

- Regression: MAE, MSE, RMSE, R^2 .

Bias-variance trade-off: **underfitting** (high bias — too simple) vs **overfitting** (high variance — memorises noise). Controls: more data, **cross-validation**, **regularisation** (L1 lasso — sparse; L2 ridge), dropout, early stopping, pruning, ensembling.

Deep learning

Neural network = layers of weighted sums plus non-linear **activation functions** (ReLU, sigmoid, tanh, softmax), trained by **backpropagation** with gradient descent (SGD, Adam). **Vanishing/exploding gradients**, batch normalisation.

Architectures: **CNN** (images, spatial), **RNN/LSTM/GRU** (sequences, time series), **Transformer** (attention; the basis of modern **LLMs**), GAN (generation), diffusion models.

LLMs: pre-training, fine-tuning, **RAG** (retrieval-augmented generation), prompt engineering, **hallucination** as the core reliability problem, context windows, and the distinction between a model that *retrieves* a fact and one that *generates* a plausible-looking one.

AI in financial regulation — and its risks

Uses: surveillance anomaly detection, complaint triage and routing, document and filing analysis (NLP), KYC/identity matching, credit and risk scoring, chatbot investor assistance, and automating routine supervisory review.

Risks a regulator must weigh — know these, they make the difference in an interview:

- Explainability.** An enforcement action must be justified. Black-box outputs are weak evidence; SHAP/LIME help but do not fully solve it. Use ML for **triage**, human analysis for the case.
- Bias and fairness.** Models learn historical patterns, including historical unfairness.
- Model risk and drift.** Markets change; a model trained on last year's behaviour degrades. Requires monitoring and governance (validation, documentation, periodic review) — the same discipline banks apply under model risk management.
- Herding and correlated behaviour.** If many participants use similar models, they may act identically and amplify a move — a genuine systemic concern with AI in trading.
- Data privacy** under DPDP, and proportionality of surveillance.
- Accountability.** A decision must be attributable to a person, not "the model".
- Adversarial manipulation.** Deepfakes for fraud, and attempts to game known detection thresholds.

SEBI has been consulting on **responsible AI/ML usage by market participants**, including disclosure of AI/ML use by intermediaries. Know the direction of travel.

Exam pointers

- Accuracy is useless on imbalanced data** — quote precision, recall and PR-AUC instead.
- L1 = Lasso (feature selection), L2 = Ridge.**
- Overfitting = high variance; underfitting = high bias.**
- Transformers underpin LLMs; **attention** is the mechanism.
- For any AI-in-regulation answer, pair the use case with **explainability, model risk and accountability**.

Data Analytics and Visualisation

Supplementary reference / interview

Types of analytics

| Type | Question | Example |

|---|---|---|

| **Descriptive** | What happened? | Turnover by segment last month |

| **Diagnostic** | Why did it happen? | Which client group drove the spike |

| **Predictive** | What will happen? | Likelihood that an alert becomes a confirmed case |

| **Prescriptive** | What should we do? | Which alerts to assign to which analyst first |

Regulatory analytics rarely needs to be prescriptive; **descriptive and diagnostic done reliably** beat a speculative predictive model.

Statistics you must not fumble

- **Mean, median, mode**; median is robust to outliers — and financial data is full of outliers.
- **Variance, standard deviation, IQR**; **skewness** and **kurtosis** (fat tails are the norm in market returns).
- **Correlation vs causation**; **Simpson's paradox** — an aggregate trend can reverse within subgroups.
- **Hypothesis testing**: null and alternative, p-value, significance level, **Type I error (false positive)** vs **Type II error (false negative)**, power.
- **Confidence intervals**; **Central Limit Theorem**; sampling bias, survivorship bias.
- **z-score** for outlier detection: $(x - \mu)/\sigma$.

Visualisation

Chart choice:

| Purpose | Chart |

|---|---|

| Comparison across categories | Bar |

| Trend over time | Line |

| Distribution | Histogram, box plot |

| Relationship | Scatter |

| Composition | Stacked bar (**not** a pie chart with many slices) |

| Density over two dimensions | Heatmap |

| Networks | Node-link graph |

Principles: start bar-chart axes at zero, avoid dual axes where they imply false correlation, label directly rather than relying on legends, use colour for meaning rather than decoration, and design for colour-blind readers.

Chartjunk and **the lie factor** (Tufte) are worth naming.

Tools: Tableau, Power BI, Qlik; Python (matplotlib, seaborn, plotly), R (ggplot2), Grafana/Kibana for operational dashboards, D3 for custom work.

Dashboards for supervision

A supervisory dashboard should answer, at a glance: **what is unusual today**, **what is overdue**, and **where is risk concentrating**. Practical design rules:

- One screen, no scrolling, for the primary view.
- **Alert volume, ageing and closure rate** — a rising backlog is itself a supervisory signal.
- Drill-through from every aggregate to the underlying records, because supervisors must be able to see the evidence.
- **Data freshness indicators** — a dashboard that silently shows stale data is worse than no dashboard.

Data quality

Dimensions: **accuracy, completeness, consistency, timeliness, validity, uniqueness**. Techniques: validation at ingestion, reconciliation against a control total, deduplication and **identity resolution**, **lineage** tracking so any number can be traced to its source.

For a regulator, **lineage and reproducibility** are not optional — a figure used in an order must be reconstructible months later, from the same inputs, with the same result.

Exam pointers

1. **Type I = false positive, Type II = false negative.** Know which one a surveillance system fears more, and why the answer depends on cost.
2. **Median over mean** for skewed financial data.
3. **Simpson's paradox** — a good, quotable caution about aggregate statistics.
4. Pick charts by **purpose**; bar axes start at zero.
5. **Lineage and reproducibility** are regulatory requirements, not engineering niceties.

Big Data

Supplementary reference / interview

The Vs

Volume, Velocity, Variety (the original three), plus **Veracity** (trustworthiness) and **Value**. Some add **Variability**.

Market data is a textbook case of all five: tick-by-tick feeds are enormous and fast (volume, velocity), combined with filings, KYC records, news and social media (variety), of uneven quality (veracity), and useful only if it yields actionable surveillance signals (value).

The Hadoop ecosystem

| Component | Role |

---|---

| **HDFS** | Distributed file system; files split into blocks (default 128 MB), **replicated 3×**, **NameNode** holds metadata, **DataNodes** hold blocks |

| **MapReduce** | Batch processing: **Map** (transform into key-value pairs) → **Shuffle & Sort** → **Reduce** (aggregate) |

| **YARN** | Resource manager |

| **Hive** | SQL-like querying over HDFS |

| **Pig** | Dataflow scripting |

| **HBase** | Column-family NoSQL store on HDFS |

| **Sqoop / Flume / Kafka** | Ingestion (RDBMS / logs / streaming) |

| **Oozie** | Workflow scheduling |

| **ZooKeeper** | Coordination |

NameNode is the single point of failure in classic HDFS — hence the standby NameNode and high-availability configuration. Learn that; it is the standard question.

Spark and modern processing

Apache Spark replaced MapReduce for most workloads: **in-memory** processing, **RDD/DataFrame** abstractions, lazy evaluation with a **DAG scheduler**, and 10–100× speedups on iterative work. Modules: Spark SQL, Spark Streaming (micro-batch), MLlib, GraphX.

Batch vs stream:

| | Batch | Stream |

---|---

| Latency | Minutes to hours | Milliseconds to seconds |

| Tools | MapReduce, Spark, Hive | **Kafka**, Flink, Spark Streaming |

| Use | Reports, end-of-day reconciliation | **Real-time surveillance alerts**, fraud detection |

Kafka — distributed, partitioned, replicated **commit log**; producers, topics, partitions, consumer groups, offsets. It is the standard backbone for **event-driven architectures**, and conceptually the same pattern as a sequenced exchange message stream.

Lambda architecture (batch layer + speed layer + serving layer) vs **Kappa architecture** (stream only).
Data lake (raw, schema-on-read) vs **data warehouse** (curated, schema-on-write) vs **lakehouse** (both, with table formats like Delta/Iceberg).

Data governance

Cataloguing, **lineage** (where did this number come from — essential for a regulator), quality rules, master data management, retention and deletion policies, access control, and **privacy by design** under DPDP.

Application to markets

- Storing and querying **tick data** for surveillance replay and investigation.
- **Real-time alerting** on order/trade streams.
- Reconstructing the **order book at a point in time** to analyse an incident.
- Cross-referencing trading, KYC, holdings and fund-flow datasets for connectedness analysis.
- Capacity: an exchange can generate **billions of messages a day** — this is genuinely big data, not a buzzword.

Exam pointers

1. **3 Vs core, 5 Vs full.**
2. **HDFS: 128 MB blocks, 3× replication, NameNode is the metadata SPOF.**
3. MapReduce phases: **Map → Shuffle & Sort → Reduce**.
4. **Spark is in-memory**, hence much faster than MapReduce for iterative jobs.
5. **Kafka is a distributed commit log** — the same idea as a replayable exchange message stream.

Blockchain and Distributed Ledger Technology

Supplementary reference / interview

What it is

A **distributed, append-only ledger** replicated across nodes, where blocks are chained by **cryptographic hashes** and agreement is reached by a **consensus mechanism** rather than a central operator.

Block structure: previous block hash, timestamp, nonce, **Merkle root** of the transactions, plus the transaction list. Changing any transaction changes the Merkle root, which changes the block hash, which breaks every subsequent block — that is where immutability comes from.

Merkle tree — a binary tree of hashes allowing $O(\log n)$ proof that a transaction is included without downloading the whole block (**SPV proofs**).

Consensus mechanisms

| Mechanism | Basis | Trade-off |

|---|---|---|

| **PoW** | Computational work | Secure, **energy intensive**, slow |

| **PoS** | Staked value | Efficient; concerns about wealth concentration |

| **DPoS** | Delegated voting | Fast, more centralised |

| **PBFT / RAFT** | Voting among known nodes | Fast and final; needs **known participants** — hence the choice for permissioned ledgers |

| **PoA** | Trusted validators | Enterprise and consortium chains |

Byzantine Fault Tolerance: a system tolerating up to f faulty nodes out of $3f+1$ total. **Finality** — probabilistic in PoW, deterministic in BFT systems. For settlement in financial markets, **deterministic finality** matters more than throughput.

Public vs permissioned

| | Public (Bitcoin, Ethereum) | Permissioned (Hyperledger Fabric, Corda, Quorum) |

|---|---|---|

| Access | Anyone | Known, vetted participants |

| Consensus | PoW/PoS | BFT/RAFT |

| Throughput | Low | High |

| Privacy | Pseudonymous, public | Configurable, private channels |

| Fit for regulated markets | Poor | **Good** |

For securities markets, the realistic technology is permissioned DLT, not a public chain. Say that explicitly — it shows you can tell hype from architecture.

Smart contracts

Code deployed on-chain that executes deterministically when conditions are met. Enables **atomic DvP**: the securities leg and the funds leg either both complete or neither does, removing principal risk.

Risks: bugs are **immutable** once deployed (the standard cautionary tale is a large DAO exploit leading to a chain fork), **oracle risk** (on-chain code depends on off-chain data), gas/cost models, and the legal question of whether code alone constitutes a contract.

Applications in securities markets

- **Settlement and clearing** — atomic DvP, shorter cycles, reduced reconciliation. This is the strongest genuine use case.
- **Tokenised securities** — fractional ownership, programmable corporate actions.
- **Security and Covenant Monitoring** — SEBI has explored DLT for **monitoring security and covenants of corporate bonds**, shared between issuers, debenture trustees, credit rating agencies and depositories. A concrete, quotable Indian example.
- **KYC sharing** with consent and auditability.
- **Corporate actions** automation.
- **Proxy voting** and shareholder records.

What blockchain does *not* solve

Garbage in is still garbage in — immutability preserves errors as faithfully as truth. It does not remove the need for identity, legal enforceability, or governance. For a single trusted operator with a well-run database, a ledger is usually a worse engineering choice. The honest framing: DLT is valuable where **multiple mutually distrusting parties must share one authoritative record without a central operator** — which is a narrower set of problems than the hype suggests, but includes several real ones in post-trade infrastructure.

Crypto assets in India

Not legal tender; taxed (a flat rate on gains plus TDS on transfers since 2022); **VASPs** are reporting entities under PMLA, with CERT-In record-keeping obligations. **CBDC (e-Rupee)** is the central bank's own digital currency and is entirely distinct from private crypto assets — do not conflate them in an answer.

Exam pointers

1. **Merkle root** is what makes tampering detectable and SPV proofs possible.
2. **BFT tolerates f faults in $3f+1$ nodes**; finality is deterministic in BFT, probabilistic in PoW.
3. **Permissioned DLT** is the realistic choice for regulated markets.
4. **Atomic DvP** is the strongest financial use case.
5. Know SEBI's **DLT for bond security and covenant monitoring** as a concrete Indian example.

Cloud Computing — Models and Virtualisation

Supplementary reference / interview

NIST definition (SP 800-145) — five essential characteristics

1. **On-demand self-service**
2. **Broad network access**
3. **Resource pooling** (multi-tenancy)
4. **Rapid elasticity**
5. **Measured service** (pay per use)

Service models

| Model | You manage | Provider manages | Example |

|---|---|---|---|

| **IaaS** | OS, runtime, apps, data | Virtualisation, servers, storage, network | EC2, Azure VM |

| **PaaS** | Apps and data | + OS, runtime, middleware | App Engine, Elastic Beanstalk, RDS |

| **SaaS** | Configuration and your data | Everything else | Salesforce, Microsoft 365 |

| **FaaS / Serverless** | Function code | Everything, including scaling to zero | AWS Lambda |

Memorise the split by asking **"who patches the OS?"** — you in IaaS, the provider in PaaS and SaaS.

Deployment models

- **Public** — shared infrastructure, greatest elasticity, least control.
- **Private** — single organisation; on-premises or hosted.
- **Hybrid** — both, with portability between them.
- **Community** — shared by organisations with common concerns (a plausible model for a **regulated financial community cloud**).
- **Multi-cloud** — more than one provider; reduces concentration risk, raises complexity.

Virtualisation

Hypervisor types:

- **Type 1 (bare metal)** — runs directly on hardware (ESXi, Hyper-V, KVM, Xen). Better performance and isolation; used in data centres.
- **Type 2 (hosted)** — runs on a host OS (VirtualBox, VMware Workstation). For desktops and labs.

VM vs container:

| | Virtual machine | Container |

|---|---|---|

| Isolation | Full OS per VM, hardware-level | Shared kernel, namespace/cgroup isolation |

| Size | GBs | MBs |

| Boot | Minutes | Seconds |

| Density | Lower | Higher |

| Security boundary | **Stronger** | Weaker (kernel is shared) |

Containers are **not** a security boundary equivalent to a VM — which matters when regulators ask about tenant isolation.

Other forms: storage virtualisation, **network virtualisation (SDN, VXLAN)**, desktop virtualisation (**VDI**), application virtualisation.

Cloud economics and operations

- **CapEx → OpEx**, elasticity, pay-per-use, and the hidden costs: **egress charges**, over-provisioning, idle resources, and licensing.
- **Auto-scaling** (horizontal vs vertical), **load balancing**, **availability zones and regions**, **fault domains**.
- **Well-Architected pillars** (AWS): operational excellence, security, reliability, performance efficiency, cost optimisation, sustainability.
- **IaC** — Terraform, CloudFormation; **immutable infrastructure**, **GitOps**.

Storage types

Block (raw volumes — EBS), **File** (shared filesystem — EFS/NFS), **Object** (HTTP-addressed, immutable, metadata-rich — S3). Object storage with **versioning and object lock** is the standard ransomware-resistant backup target.

Exam pointers

1. **Five NIST characteristics** — learn them as a list; they get asked directly.
2. "Who patches the OS" distinguishes **IaaS** from **PaaS**.
3. **Type 1 hypervisor = bare metal.**
4. Containers share the **kernel** — a weaker isolation boundary than VMs.
5. **Object storage with object lock** is the backup answer for ransomware resilience.

SEBI's Framework for Adoption of Cloud Services

Supplementary reference / interview

> ☐ Verify current details on sebi.gov.in — this framework has been issued and clarified in phases.

Why SEBI issued it

Regulated entities were already adopting cloud, but with inconsistent contracts, unclear accountability and no common standard for exit or oversight. SEBI issued a **Framework for Adoption of Cloud Services by SEBI Regulated Entities** (2023) to set baseline expectations. It applies across REs — MIs, brokers, mutual funds, and other intermediaries.

The principles it establishes

1. **Accountability stays with the RE.** Using a cloud service provider (CSP) does not dilute any regulatory obligation. The board and management remain answerable.
2. **Risk assessment before adoption** — criticality of the workload, data classification, and the consequences of unavailability or compromise.
3. **Due diligence on the CSP** — security posture, certifications, financial stability, sub-contracting, track record, and incident history.
4. **Contractual clarity** — right to audit (by the RE, its auditors, and by SEBI), access to logs, incident notification timelines, service levels, liability, confidentiality, and data ownership.
5. **Regulatory access** — SEBI and its authorised persons must be able to access data and systems relating to the RE's regulated activity.
6. **Data localisation and jurisdiction** — data and logs relating to Indian operations kept within India, with clarity on where processing occurs.
7. **Security controls** — encryption with appropriate key management, identity and access management, network segregation, logging and monitoring, vulnerability management.
8. **Business continuity** — the CSP's resilience does not replace the RE's own BCP/DR obligations; the RE must plan for CSP outage.
9. **Concentration risk** — awareness at both entity and system level, including the possibility that many REs depend on the same provider or region.
10. **Exit strategy** — a documented, tested plan to migrate away, including data extraction in a usable format and a defined transition period. This is the requirement entities most often under-plan.
11. **Governance** — board-approved cloud policy, defined ownership, periodic review and reporting.

How it interacts with other requirements

An RE adopting cloud is simultaneously bound by:

- **CSCRF** — the cyber security and resilience framework, which applies to cloud-hosted systems as much as on-premises ones;
- **CERT-In directions** — six-hour incident reporting, **log retention within Indian jurisdiction**;

- **DPDP Act** — obligations as a Data Fiduciary over personal data, with the CSP typically a Data Processor;
- **System audit** requirements, now covering cloud configuration;
- Sector-specific requirements where the RE is also regulated by another authority.

How to discuss this well

A good answer moves past listing requirements to the actual tension:

> Cloud genuinely improves the security posture of **small and mid-size intermediaries**, who cannot staff a 24×7 SOC or patch reliably on their own. But at the **system level**, mass migration to two or three providers creates a new concentration risk that no individual entity can manage, and that only the regulator can see. The framework therefore has to do two things at once — enable adoption, and keep visibility of the aggregate dependency.

That is the kind of answer that distinguishes an IT officer candidate from a compliance-checklist recital.

Exam pointers

1. **Accountability is non-delegable** — the RE remains responsible for everything the CSP does.
2. **Right to audit and regulatory access** must be contractual.
3. **Exit strategy** is an explicit requirement, and the most neglected one.
4. **Concentration risk** is the systemic angle to raise.
5. Cloud adoption does not displace **CSCRF, CERT-In or DPDP** obligations.

Cloud Security and the Shared Responsibility Model

Supplementary reference / interview

Shared responsibility

The provider is responsible for **security OF the cloud**; the customer for **security IN the cloud**.

| Layer | IaaS | PaaS | SaaS |

|---|---|---|---|

| Physical, host, hypervisor | Provider | Provider | Provider |

| Network controls | Shared | Provider | Provider |

| OS patching | **Customer** | Provider | Provider |

| Application code | Customer | Customer | Provider |

| **Identity and access config** | **Customer** | **Customer** | **Customer** |

| **Data and its classification** | **Customer** | **Customer** | **Customer** |

The two rows that **never** move are **identity configuration** and **data**. Almost every publicised "cloud breach" is in fact a **customer misconfiguration** — a public storage bucket, an over-permissive IAM role, an exposed management port — not a provider failure. Say that precisely; it signals you understand the model.

Principal cloud risks

1. **Misconfiguration** — the dominant cause. Countered by **CSPM** tooling, IaC with policy-as-code, guardrails and secure baselines.
2. **Identity compromise** — over-permissive roles, long-lived keys, no MFA. Countered by least privilege, short-lived credentials, **just-in-time** access, key rotation, and **SCP/permission boundaries**.
3. **Data exposure** — encryption at rest (**customer-managed keys**, BYOK/HYOK) and in transit, tokenisation, DLP.
4. **Multi-tenancy risks** — noisy neighbours, side-channel attacks, hypervisor escape (rare but catastrophic).
5. **Supply chain** — compromised images, dependencies, and provider sub-processors.
6. **Concentration risk** — a handful of providers serve most of the financial sector; an outage or compromise is systemic. **This is the risk a regulator cares about most.**
7. **Vendor lock-in and exit** — the ability to actually leave, with data, within a defined time.
8. **Jurisdiction** — where data physically sits, and which government can compel access to it.
9. **Shared visibility gap** — you cannot deploy your own network taps in someone else's data centre; you depend on provider logs.

Controls

- **IAM**: roles over users, no root/keys in code, MFA everywhere, federated identity, **service control policies**.

- **Network:** VPC design, private subnets, security groups and NACLs, **private endpoints** instead of public exposure, egress filtering, **no direct internet access for data-layer resources**.
- **Encryption:** at rest with **KMS/HSM-backed keys**, in transit with TLS, key rotation, separation between who can use a key and who can manage it.
- **Logging:** provider audit logs (CloudTrail-style) shipped to an **immutable, separate account**, integrated into the SIEM.
- **Workload:** hardened images, vulnerability scanning of images and registries, runtime protection, **admission control** for Kubernetes.
- **Backup:** cross-region, **immutable/object-locked**, restore-tested.
- **Resilience:** multi-AZ by default, multi-region for critical systems, chaos/failure testing.

Compliance in the Indian context

- **MeitY empanelment** of cloud service providers for government workloads.
- **Data localisation** requirements vary by regulator — RBI's payment data localisation directive is the strictest well-known example; SEBI's cloud framework and CERT-In directions carry their own requirements on where logs and records are kept.
- **DPDP Act** obligations on the entity as a **Data Fiduciary**, regardless of who runs the infrastructure. Outsourcing the processing never outsources the accountability — a line worth using verbatim.
- **CERT-In empanelled auditors** for VAPT.
- **ISO 27017/27018**, **SOC 2 Type II** reports from providers as assurance artefacts.

Exam pointers

1. **Provider: security OF the cloud. Customer: security IN the cloud.**
2. **Identity configuration and data are always the customer's** — in every service model.
3. Most cloud breaches are **customer misconfiguration**.
4. **Concentration risk** is the systemic concern for a financial regulator.
5. "Outsourcing processing does not outsource accountability" — use it.

Application Layer Protocols

Core preparation lesson

DNS

Hierarchical, distributed name-to-address database. Runs on **UDP port 53** (TCP for zone transfers and large responses).

Hierarchy: root (13 logical root server clusters, anycast) → **TLD** (.com, .in) → authoritative → local resolver.

Resolution: **recursive** (the resolver does the work and returns the final answer) vs **iterative** (each server returns a referral).

Record types: **A** (IPv4), **AAAA** (IPv6), **CNAME** (alias), **MX** (mail), **NS** (name server), **PTR** (reverse), **SOA**, **TXT** (SPF, DKIM, domain verification), **SRV**.

Caching with **TTL**. **DNSSEC** signs records to prevent **DNS spoofing / cache poisoning**.
DoH/DoT encrypt DNS queries.

HTTP

Stateless request-response over TCP 80 (HTTPS 443).

Methods: **GET** (safe, idempotent), **POST** (not idempotent), **PUT** (idempotent, replaces), **PATCH**, **DELETE** (idempotent), HEAD, OPTIONS.

Status codes:

| Range | Meaning | Examples |

|---|---|---|

| 1xx | Informational | 101 Switching Protocols |

| 2xx | Success | **200 OK**, 201 Created, 204 No Content |

| 3xx | Redirection | **301 Moved Permanently**, 302 Found, 304 Not Modified |

| 4xx | **Client error** | 400 Bad Request, **401 Unauthorized**, **403 Forbidden**, **404 Not Found**, 405, 409, **429 Too Many Requests** |

| 5xx | **Server error** | **500 Internal Server Error**, 502 Bad Gateway, **503 Service Unavailable**, 504 Gateway Timeout |

401 vs 403: 401 means *not authenticated*; 403 means *authenticated but not allowed*. Asked constantly.

Versions: HTTP/1.1 (persistent connections, pipelining), **HTTP/2** (binary framing, multiplexing over one connection, header compression, server push), **HTTP/3** (over **QUIC/UDP**, eliminates head-of-line blocking).

State: cookies, sessions, tokens; **CORS** for cross-origin requests.

Email

- **SMTP (25, 587)** — sending; push protocol.
- **POP3 (110)** — downloads and usually deletes from the server; single device.
- **IMAP (143)** — keeps mail on the server, syncs folders across devices.
- **MIME** — allows non-ASCII content and attachments.
- Anti-spoofing: **SPF, DKIM, DMARC** — worth knowing, since email spoofing is the delivery mechanism for most financial fraud.

File transfer and remote access

- **FTP** — control on **21**, data on **20**; active vs passive mode; plaintext. **FTPS** (TLS) and **SFTP** (over SSH, port 22) are the secure options.
- **TFTP** — UDP 69, no authentication, used for device boot images.
- **SSH (22)** — encrypted remote shell, key-based authentication, tunnelling/port forwarding. **Telnet (23)** is its insecure predecessor and should never be used.

Other

- **DHCP** — **DORA**: Discover, Offer, Request, Acknowledge. Ports 67 (server) / 68 (client). Provides IP, mask, gateway, DNS, lease time.
- **SNMP** — 161/162; manager, agent, **MIB**, OID; v3 adds authentication and encryption. The backbone of network monitoring.
- **NTP** — port 123; time synchronisation. **Clock synchronisation** is a regulatory requirement in trading systems — exchanges and brokers must sync to a common reference so that order timestamps and audit trails are comparable.
- **WebSocket** — full-duplex over one TCP connection after an HTTP upgrade; how live price tickers work in a browser.

Exam pointers

1. **DNS = UDP 53**, TCP for zone transfer. **DORA** for DHCP.
2. **401** = not authenticated, **403** = forbidden.
3. **POP3** downloads, **IMAP** synchronises.
4. HTTP/3 runs over **QUIC**, which runs over **UDP**.
5. NTP and time-stamping matter in market systems — a good IT-stream talking point.

Data Link Layer — Framing, Error Control and Flow Control

Core preparation lesson

Responsibilities

Framing · physical (MAC) addressing · error detection and control · flow control · access control. Split into **LLC** (logical link control) and **MAC** sublayers.

Framing

- **Character/byte stuffing** — a flag byte marks boundaries; an escape byte (ESC) is inserted before any accidental flag or ESC in the payload.
- **Bit stuffing** — after **five consecutive 1s**, the sender inserts a **0**; the receiver removes it. The flag `01111110` therefore never appears inside data. *This is a guaranteed exam question — practise stuffing and de-stuffing a given bit string.*

Error detection

| Technique | How it works | Detects |

|---|---|---|

| **Parity** | One bit making the count of 1s even/odd | All odd numbers of bit errors; misses even |

| **2-D parity** | Parity per row and column | All 1, 2, 3-bit errors; can correct single-bit |

| **Checksum** | 1's complement sum of segments, complemented | Used by IP/TCP/UDP; weaker than CRC |

| **CRC** | Treat data as a polynomial; divide by a generator using modulo-2 (XOR) division; append the remainder | All single-bit errors, all burst errors shorter than the CRC length, and most others |

CRC procedure: append (degree of generator) zeros to the data, do modulo-2 division by the generator, append the remainder as the CRC. The receiver divides the whole received frame by the same generator — a **zero remainder means no detected error**.

Error correction

- **Hamming code** — for m data bits, choose r parity bits with $2^r \geq m + r + 1$. Parity bits sit at positions 1, 2, 4, 8, ... Each parity bit covers the positions whose binary representation has that bit set. The XOR of the failing parity checks gives the **position of the erroneous bit**.
- **Hamming distance** — minimum distance d detects $d - 1$ errors and corrects $\lfloor (d - 1)/2 \rfloor$ errors.
- **FEC vs ARQ** — forward error correction (add redundancy, no retransmission, used for satellite/wireless) vs automatic repeat request (retransmit).

Flow control / ARQ protocols

| Protocol | Sender window | Receiver window | Efficiency |

|---|---|---|---|

| ****Stop and Wait**** | 1 | 1 | $1/(1 + 2a)$ where $a = T_p/T_t$ |

| ****Go-Back-N**** | N | ****1**** | $N/(1 + 2a)$ capped at 1. Discards all frames after an error |

| ****Selective Repeat**** | N | ****N**** | $N/(1 + 2a)$. Buffers out-of-order frames; retransmits only the lost one |

****Sequence number bits:****

- Go-Back-N: window size $\leq 2^k - 1$

- Selective Repeat: window size $\leq 2^{(k-1)}$

****Efficiency formula:**** with $a = T_p/T_t$, throughput = (window size) / (1 + 2a), and utilisation cannot exceed 1.

****Piggybacking**** — carrying an acknowledgement inside a data frame going the other way.

Exam pointers

1. ****Bit stuffing: insert 0 after five 1s.****
2. CRC: remainder appended; ****zero remainder at the receiver = accepted****.
3. Hamming: $2^r \geq m + r + 1$; distance d corrects $\lfloor (d-1)/2 \rfloor$ errors.
4. ****GBN window $\leq 2^k - 1$; SR window $\leq 2^{(k-1)}$.**
5. Selective Repeat needs buffering ****and**** sorting at the receiver — which is its real cost.

Network Layer — IP Addressing and Subnetting

Core preparation lesson

> Subnetting is pure marks. Anyone can get these right with practice, and many candidates lose them by not practising.

IPv4 addressing

32 bits, written as four dotted decimal octets. Address = **network portion + host portion**, split by the **subnet mask**.

| Class | First octet range | Default mask | Networks / Hosts |

|---|---|---|---|

| A | 1–126 | /8 (255.0.0.0) | 2^7-2 networks, $2^{24}-2$ hosts |

| B | 128–191 | /16 (255.255.0.0) | 2^{14} networks, $2^{16}-2$ hosts |

| C | 192–223 | /24 (255.255.255.0) | 2^{21} networks, $2^8-2 = 254$ hosts |

| D | 224–239 | — | **Multicast** |

| E | 240–255 | — | Reserved / experimental |

127.x.x.x is loopback. **169.254.x.x** is APIPA/link-local.

Private ranges (RFC 1918):

- **10.0.0.0/8**

- **172.16.0.0 – 172.31.255.255 (/12)**

- **192.168.0.0/16**

Subnetting method

For a /n network:

- **Host bits** = $32 - n$; **usable hosts** = $2^{(32-n)} - 2$ (subtract network and broadcast addresses).

- **Block size** = $256 - (\text{the mask octet value})$ in the "interesting" octet.

Worked example: 192.168.10.0/26

- Mask = 255.255.255.192, block size = $256 - 192 = 64$.

- Subnets: .0, .64, .128, .192 → **4 subnets**, each with **62 usable hosts**.

- For the .64 subnet: network **192.168.10.64**, first host **.65**, last host **.126**, broadcast **.127**.

Handy table:

| CIDR | Mask | Block | Usable hosts |

|---|---|---|---|

| /24 | 255.255.255.0 | 256 | 254 |

/25	.128	128	126
/26	.192	64	62
/27	.224	32	30
/28	.240	16	14
/29	.248	8	6
/30	.252	4	****2**** (point-to-point links)

****VLSM**** — using different mask lengths inside one network, allocating largest subnets first.

****Supernetting/CIDR**** — aggregating routes (e.g. four /24s into one /22) to shrink routing tables.

IPv4 header (20 bytes minimum)

Version · IHL · ****DSCP/ECN**** · Total length · ****Identification, Flags, Fragment offset**** · ****TTL**** · Protocol ·
Header checksum · Source IP · Destination IP · Options.

****Fragmentation:**** fragment offset is measured in ****units of 8 bytes****; the ****MF**** (more fragments) flag is 1 on all but the last fragment; ****DF**** means do not fragment. TTL is decremented by each router and the packet is discarded at 0 (which is how ****traceroute**** works). Note: ****IPv6 routers do not fragment**** — the source must do path MTU discovery.

NAT

Maps private addresses to public ones. ****Static NAT****, ****dynamic NAT****, and ****PAT/NAT overload**** (many private hosts behind one public IP, distinguished by port). Solves address exhaustion; breaks end-to-end addressing and complicates peer-to-peer.

ICMP

Error reporting and diagnostics: ****echo request/reply (ping)****, destination unreachable, time exceeded (used by ****traceroute****), redirect, source quench. ICMP has no ports and is not reliable.

IPv6

- ****128 bits****, written as eight groups of four hex digits; `::`` compresses one run of zero groups.
- ****No broadcast**** — replaced by ****multicast and anycast****. No header checksum. Fixed ****40-byte header**** with ****extension headers****.
- Address types: global unicast (2000::/3), link-local (****fe80::/10****), unique local, multicast (ff00::/8), loopback (::1).
- Transition: ****dual stack****, ****tunnelling**** (6to4, Teredo), ****NAT64/DNS64****.
- ****SLAAC**** — stateless address autoconfiguration with ****NDP**** (which replaces ARP).

Exam pointers

1. **Usable hosts = $2^{(\text{host bits})} - 2$** ; /30 gives exactly 2.
2. Memorise the private ranges — **172.16–172.31**, not 172.x generally.
3. **Fragment offset is in 8-byte units.**
4. IPv6 has **no broadcast and no router fragmentation**.
5. Practise five subnetting questions a day for a week; then they are free marks forever.

MAC Sublayer, Ethernet and LAN Devices

Core preparation lesson

Multiple access protocols

Random access: ALOHA, Slotted ALOHA, CSMA, CSMA/CD, CSMA/CA
Controlled: Reservation, Polling, Token passing
Channelisation: FDMA, TDMA, CDMA

Efficiency: Pure ALOHA maximum throughput **18.4%** ($S = G \cdot e^{(-2G)}$); Slotted ALOHA **36.8%** ($S = G \cdot e^{(-G)}$). These two numbers are asked directly.

CSMA variants

- **1-persistent** — transmit as soon as the channel is idle (high collision chance).
- **Non-persistent** — wait a random time, then sense again.
- **p-persistent** — for slotted channels; transmit with probability p .

CSMA/CD (Ethernet): sense, transmit, and keep listening; on collision send a **jam signal** and back off using **binary exponential backoff** (wait a random number of slot times in $[0, 2^n - 1]$, n capped at 10, give up after 16 attempts).

Minimum frame size for collision detection: transmission time must be at least **$2 \times$ propagation delay**, which is why Ethernet has a **64-byte minimum frame** and a maximum segment length.

CSMA/CA (Wi-Fi): collisions cannot be detected on a half-duplex radio, so they are **avoided** — IFS, contention window, **RTS/CTS** handshake, and ACKs. Solves the **hidden terminal** problem (**exposed terminal** is the mirror problem).

Ethernet

- **MAC address** — 48 bits, written as 6 hex pairs. First 24 bits = **OUI** (manufacturer). A broadcast address is **FF:FF:FF:FF:FF:FF**. **Unicast/multicast** is indicated by the least significant bit of the first byte.
- **Frame format:** Preamble (7B) + SFD (1B) + Destination (6B) + Source (6B) + Type/Length (2B) + Data (46–1500B) + FCS (4B). Minimum frame **64 bytes**, maximum **1518 bytes**; **MTU = 1500 bytes**.
- Standards: 10Base5, 10Base2, **10BaseT**, 100BaseTX (Fast Ethernet), 1000BaseT (Gigabit), 10GBase.

Switches vs hubs vs bridges

Hub Switch
--- --- ---
Layer 1 2
Collision domains One for all ports One per port
Broadcast domains One One (unless VLANs)
Forwarding Flood everything MAC address table , forward selectively

| Duplex | Half | Full |

Switch operation: learn source MAC → port mapping; forward if known, **flood** if unknown; **filter** if the destination is on the same port. **Switching methods:** store-and-forward, cut-through, fragment-free.

STP — Spanning Tree Protocol (802.1D) prevents loops (and therefore broadcast storms) by blocking redundant links; **RSTP** converges faster.

VLAN

A **VLAN** creates separate **broadcast domains** on one physical switch. Inter-VLAN traffic must go through a **router or L3 switch**. **802.1Q** adds a 4-byte tag to the Ethernet frame; a **trunk** port carries multiple VLANs, an **access** port carries one.

ARP and friends

- **ARP** — IP → MAC, within a LAN; broadcast request, unicast reply; cached in the ARP table. **Gratuitous ARP**, **proxy ARP**, and **ARP spoofing/poisoning** as the attack.

- **RARP / BOOTP / DHCP** — MAC → IP (DHCP has replaced the older two).

Exam pointers

1. **Pure ALOHA 18.4%, Slotted ALOHA 36.8%.**
2. Ethernet minimum frame **64 bytes**, MTU **1500 bytes**.
3. A switch gives **one collision domain per port**, but **one broadcast domain** unless VLANs are used.
4. Binary exponential backoff: random wait in **$[0, 2^n - 1]$** slots, up to 16 attempts.
5. **ARP maps IP to MAC** — and ARP spoofing is the classic LAN attack because ARP has no authentication.

OSI and TCP/IP Models

Core preparation lesson

OSI — seven layers (top to bottom)

| # | Layer | Job | PDU | Devices / protocols |

|---|---|---|---|

| 7 | **Application** | Services to the user | Data | HTTP, FTP, SMTP, DNS, DHCP, SNMP |

| 6 | **Presentation** | Translation, **encryption**, **compression** | Data | SSL/TLS (arguably), JPEG, ASCII |

| 5 | **Session** | Establish, manage, terminate sessions; **synchronisation**, dialog control, checkpointing | Data | NetBIOS, RPC, PPTP |

| 4 | **Transport** | **End-to-end** delivery, segmentation, flow and error control | **Segment** (TCP) / Datagram (UDP) | TCP, UDP, SCTP |

| 3 | **Network** | **Logical addressing and routing** across networks | **Packet** | IP, ICMP, IGMP, routers, L3 switches |

| 2 | **Data link** | Node-to-node delivery, **framing**, MAC addressing, error detection | **Frame** | Ethernet, PPP, switches, bridges, NIC |

| 1 | **Physical** | Bits on the medium | **Bit** | Cables, hubs, repeaters, modems |

Mnemonic (top-down): **A**ll **P**eople **S**ee **T**o **N**eed **D**ata **P**rocessing.

TCP/IP model

| TCP/IP layer | Maps to OSI |

|---|

| **Application** | Application + Presentation + Session |

| **Transport** | Transport |

| **Internet** | Network |

| **Network Access / Link** | Data link + Physical |

The TCP/IP model is what the internet actually runs on; OSI is the teaching and troubleshooting reference. TCP/IP was **built first and modelled after**; OSI was modelled first and largely not implemented.

Encapsulation

```
Application data
+ TCP header      -> Segment
+ IP header       -> Packet / Datagram
+ Frame header and trailer -> Frame
-> bits on the wire
```

Each layer adds its own header (and the data link layer adds a trailer with the FCS). The receiving side strips them in reverse — **de-encapsulation**.

Which device works at which layer

| Device | Layer |

|---|---|

| Hub, repeater, cable | 1 |

| Switch, bridge, NIC, access point | 2 |

| Router, L3 switch | 3 |

| Firewall | 3/4 (stateful), 7 for next-gen |

| Gateway, proxy, load balancer | up to 7 |

Key distinctions

- **Connection-oriented vs connectionless** — TCP vs UDP; virtual circuit vs datagram at the network layer.
- **Flow control** (protect the receiver from being overrun) vs **congestion control** (protect the network) vs **error control**.
- **Node-to-node** (data link) vs **host-to-host** (network) vs **process-to-process / end-to-end** (transport).
- **Reliability** appears at both layer 2 (hop by hop) and layer 4 (end to end) — the **end-to-end argument** says the real guarantee must be at layer 4.

Exam pointers

1. PDU names: **bit, frame, packet, segment**. Asked every single year.
2. **Encryption and compression** → Presentation layer; dialog control → Session layer.
3. **Transport is process-to-process**; Network is host-to-host.
4. A switch is layer 2, a router layer 3, a hub layer 1.
5. TCP/IP has **four** layers (five if you split the link layer); OSI has seven.

Wireless, Mobile, SDN and Troubleshooting

Core preparation lesson

Wireless standards (802.11)

| Standard | Common name | Band | Max rate (theoretical) |

|---|---|---|---|

| 802.11a | — | 5 GHz | 54 Mbps |

| 802.11b | — | 2.4 GHz | 11 Mbps |

| 802.11g | — | 2.4 GHz | 54 Mbps |

| 802.11n | Wi-Fi 4 | Both | 600 Mbps (MIMO) |

| 802.11ac | Wi-Fi 5 | 5 GHz | Gbps range (MU-MIMO) |

| 802.11ax | **Wi-Fi 6 / 6E** | 2.4/5/6 GHz | OFDMA, higher density |

| 802.11be | Wi-Fi 7 | Multi-band | Multi-link operation |

****2.4 GHz**** — longer range, more interference, 3 non-overlapping channels (1, 6, 11). ****5 GHz**** — shorter range, more channels, higher throughput.

Other 802 standards to recognise: ****802.3 Ethernet****, ****802.1Q VLAN****, ****802.1X port-based access control****, ****802.15.1 Bluetooth****, ****802.15.4 Zigbee****, ****802.16 WiMAX****.

Mobile generations

| Gen | Technology | Characteristic |

|---|---|---|

| 1G | Analog | Voice only |

| 2G | GSM/CDMA, ****digital**** | SMS, GPRS/EDGE (2.5G) |

| 3G | UMTS/HSPA | Mobile data |

| 4G | ****LTE****, all-IP | Broadband, VoLTE |

| ****5G**** | NR | ****eMBB, URLLC, mMTC****; network slicing, edge computing, sub-6 GHz and mmWave |

****Handoff/handover****, cell structure, frequency reuse, MIMO, beamforming.

SDN and NFV

****Software Defined Networking**** separates the ****control plane**** (decides where traffic goes) from the ****data plane**** (forwards it), with a ****centralised controller**** programming switches through a southbound API (****OpenFlow****) and exposing a northbound API to applications.

Benefits: central policy, automation, vendor independence, rapid reconfiguration. Risks: ****the controller is a single point of failure and a high-value target****.

NFV — running network functions (firewall, load balancer, router) as software on commodity hardware rather than dedicated appliances. **VNF**, service chaining.

Related: **intent-based networking**, **SD-WAN**, **network automation** (Ansible, NETCONF/YANG).

Cloud and data centre networking

Leaf-spine topology, **east-west vs north-south traffic**, **overlay networks (VXLAN)**, **container networking (CNI)**, **service mesh (Envoy/Istio)** for service-to-service traffic, **load balancers** (L4 vs L7, round robin, least connections, IP hash), **CDN** for edge caching.

Troubleshooting toolkit

| Tool | Use |

|---|---|

| ``ping`` | Reachability and RTT (ICMP echo) |

| ``traceroute`` / ``tracert`` | Path and per-hop latency (TTL expiry) |

| ``nslookup`` / ``dig`` | DNS resolution |

| ``netstat -tulnp`` / ``ss`` | Listening ports and connections |

| ``ipconfig`` / ``ip a`` | Interface configuration |

| ``arp -a`` | ARP cache |

| ``tcpdump`` / **Wireshark** | Packet capture and analysis |

| ``iperf`` | Throughput measurement |

| ``mtr`` | Continuous traceroute + ping |

A structured method: check physical → IP configuration → gateway → DNS → application. Or work the OSI layers bottom-up; either way, be systematic rather than random.

Latency in trading networks (stream-relevant)

Every microsecond matters in low-latency trading: **co-location** puts the member's server in the exchange data centre; **kernel bypass** (DPDK, solarflare), **FPGA** based order handling, **multicast market data**, **precision time protocol (PTP)** for sub-microsecond clock sync. SEBI's concern is **fair and equal access** — which is why co-location, tick-by-tick data dissemination and latency measurement are regulated subjects rather than purely technical ones.

Exam pointers

- 2.4 GHz non-overlapping channels: 1, 6, 11.**
- 802.1X = port-based network access control** (not a wireless standard as such).
- 5G's three pillars: eMBB, URLLC, mMTC.**

4. SDN separates **control plane from data plane**; OpenFlow is the southbound protocol.
5. Traceroute works by **incrementing TTL** and reading ICMP time-exceeded messages.

Physical Layer, Media, Topologies and Switching

Core preparation lesson

Transmission media

| Medium | Notes |

|---|---|

| **Twisted pair (UTP/STP)** | Cat 5e (1 Gbps), Cat 6/6a (10 Gbps), 100 m limit; cheap, susceptible to EMI |

| **Coaxial** | Older LANs, cable TV |

| **Fibre optic** | **Single-mode** (long haul, laser, small core) vs **multi-mode** (short distance, LED). Immune to EMI, highest bandwidth, hardest to tap |

| **Wireless** | Radio, microwave, infrared, satellite (GEO ~35,786 km with ~250 ms one-way delay, MEO, LEO) |

Topologies

| Topology | Cable need | Failure impact | Notes |

|---|---|---|---|

| **Bus** | Least | Backbone failure kills all | Obsolete |

| **Star** | Medium | Hub/switch is a single point of failure | The standard today |

| **Ring** | Medium | One break can kill the ring (dual ring helps) | Token passing |

| **Mesh** | $n(n-1)/2$ links for full mesh | Highest fault tolerance | Backbone/WAN |

| **Tree / hybrid** | — | — | Hierarchical enterprise design |

Full mesh requires $n(n-1)/2$ links and $n-1$ ports per node — a standard one-mark calculation.

Transmission concepts

- **Bandwidth** (Hz or bps), **throughput**, **latency**, **jitter**, **attenuation**, **noise**, **distortion**.

- **Nyquist (noiseless):** $C = 2B \log_2(L)$ bits/s for L signal levels.

- **Shannon (noisy):** $C = B \log_2(1 + S/N)$ bits/s. SNR in dB = $10 \log_{10}(S/N)$.

- **Total delay = transmission delay + propagation delay + queuing delay + processing delay.**

- Transmission delay = **packet size ÷ bandwidth**

- Propagation delay = **distance ÷ propagation speed**

- **Bandwidth-delay product** = bandwidth × RTT = the number of bits "in flight" — this is why long fat pipes need large TCP windows.

Multiplexing

FDM (frequency), **TDM** (time — synchronous vs statistical), **WDM/DWDM** (wavelength, for fibre), **CDM/CDMA** (code). **Statistical TDM** allocates slots on demand and is more efficient than synchronous

TDM.

Transmission modes

Simplex (one way), **half-duplex** (both ways, one at a time — walkie-talkie), **full-duplex** (both simultaneously). **Serial vs parallel**; **synchronous vs asynchronous**.

Switching

| Type | How |

|---|---|

| **Circuit switching** | Dedicated path reserved end to end (telephone). Setup delay, guaranteed bandwidth, wasteful when idle |

| **Packet switching — datagram** | Each packet routed independently; may arrive out of order (the Internet) |

| **Packet switching — virtual circuit** | Path set up once, all packets follow it (MPLS, Frame Relay, ATM) |

| **Message switching** | Store and forward whole messages; obsolete |

Exam pointers

1. **Full mesh = $n(n-1)/2$ links.**
2. **Single-mode fibre for long distance**, multi-mode for short.
3. Transmission delay depends on **bandwidth**; propagation delay on **distance**.
4. Shannon gives the theoretical maximum with noise; Nyquist assumes no noise.
5. **Statistical TDM** beats synchronous TDM because idle slots are reused.

Routing Algorithms and Protocols

Core preparation lesson

Classification

Basis	Types
Information source	**Static** (manual) vs **Dynamic** (learned)
Algorithm	**Distance vector**, **Link state**, **Path vector**
Scope	**IGP** (within an AS) vs **EGP** (between AS)
Behaviour	Classful vs **classless** (supports VLSM)

Distance vector (Bellman-Ford)

Each router shares its **whole routing table** with its neighbours, periodically. Metric: **hop count**.

- **RIP** — max hop count **15** (16 = unreachable), updates every **30 s**, uses **UDP port 520**. RIPv2 is classless with authentication.
- Problem: **count-to-infinity** and slow convergence. Fixes: **split horizon** (do not advertise a route back on the interface you learned it from), **split horizon with poison reverse**, **hold-down timers**, **triggered updates**.

Link state (Dijkstra)

Each router floods **link-state advertisements** about its own links to **everyone**, builds an identical topology map, then runs **Dijkstra's shortest path first** locally.

- **OSPF** — IGP, classless, metric is **cost = reference bandwidth / interface bandwidth**, uses **areas** (area 0 is the backbone) to scale, runs directly on **IP protocol 89**, forms adjacencies with DR/BDR on broadcast segments. Faster convergence, more CPU and memory.
- **IS-IS** — similar, common in service-provider networks.

	Distance vector	Link state
Knowledge	Neighbours only	Entire topology
Shared with	Neighbours	All routers (flooding)
Update	Periodic, full table	Triggered, only changes
Convergence	Slow	Fast
Resource use	Low	High
Loops	Count-to-infinity	Rare

Path vector

BGP — the routing protocol **between autonomous systems**, and therefore the routing protocol of the Internet.

- Runs over **TCP port 179**; eBGP between AS, iBGP within.
- Advertises the **AS_PATH**, which both prevents loops and enables **policy-based routing** — BGP chooses on policy, not shortest path.
- Attributes in decision order (simplified): **weight** → local preference → locally originated → AS_PATH length → origin → MED → eBGP over iBGP → lowest IGP metric.
- **Route hijacking / route leaks** happen because BGP trusts announcements. Mitigations: **RPKI**, route filtering, **BGPsec**. *Worth knowing: a BGP hijack affecting an exchange or a broker is a realistic market-infrastructure risk.*

Dijkstra vs Bellman-Ford

Dijkstra Bellman-Ford
--- --- ---
Negative weights Not allowed Allowed (detects negative cycles)
Complexity $O(E \log V)$ with a heap $O(V \cdot E)$
Approach Greedy Dynamic programming

Other routing concepts

- **Routing table entries**: destination network, mask, next hop, interface, metric, administrative distance.
- **Longest prefix match** decides which route wins when several match.
- **Administrative distance** — preference between protocols (connected 0, static 1, eBGP 20, OSPF 110, RIP 120).
- **Default route** 0.0.0.0/0. **Floating static route** as a backup.
- **Unicast, broadcast, multicast (IGMP, PIM), anycast** (used by DNS root servers and CDNs).
- **MPLS** — label switching, faster forwarding and traffic engineering; often used for **leased-line quality links** between brokers, exchanges and clearing corporations.

Exam pointers

1. **RIP hop limit 15**; OSPF metric is **cost based on bandwidth**; BGP uses **AS_PATH**.
2. **OSPF area 0 is the backbone.**
3. BGP runs on **TCP 179** — the only routing protocol using TCP.
4. **Longest prefix match** always wins, regardless of metric.
5. Bellman-Ford handles negative weights; **Dijkstra does not**.

Network Security Devices and Controls

Core preparation lesson

Firewalls

| Generation | How it decides |

|---|---|

| **Packet filter (stateless)** | ACLs on IP, port, protocol. Fast, no context |

| **Stateful inspection** | Tracks connection state; allows return traffic for established sessions |

| **Application/proxy firewall** | Terminates and inspects at layer 7 |

| **NGFW** | Adds application awareness, user identity, IPS, TLS inspection, threat intelligence |

| **WAF** | Specifically protects web applications — SQLi, XSS, OWASP rules |

Default-deny is the correct posture: permit what is required, deny everything else.

DMZ — a screened subnet holding internet-facing services (web, mail, DNS) between two firewalls, so that a compromise there does not reach the internal network. In a broker or exchange architecture, the trading core sits several zones deep, never in the DMZ.

IDS and IPS

| | IDS | IPS |

|---|---|---|

| Position | Out of band (span port) | **In line** |

| Action | Alerts | **Blocks** |

| Risk | Missed attacks | False positives can drop legitimate traffic |

Detection methods: **signature-based** (known patterns, cannot catch zero-days) vs

anomaly/behaviour-based (baseline deviation, more false positives) vs **heuristic**. Types: **NIDS** (network) and **HIDS** (host).

Know the four outcomes: **true positive**, **true negative**, **false positive (false alarm)**, **false negative (missed attack)**. In a regulated environment, false negatives are the dangerous ones.

VPN

Creates an encrypted tunnel over a public network.

- **IPsec** — layer 3. **AH** (authentication and integrity only) vs **ESP** (**encryption** plus authentication).

Transport mode (payload only, host to host) vs **tunnel mode** (whole packet encapsulated, site to site).

IKE for key exchange.

- **SSL/TLS VPN** — layer 7, clientless browser access.

- **Site-to-site** vs **remote access** VPN. **Split tunnelling** (and why security teams disable it).

- Modern successors: **SASE**, **ZTNA** — identity-based access replacing network-perimeter trust.

Other controls

- **Proxy** (forward: controls outbound user access; reverse: fronts servers, load balancing and TLS termination).
- **NAC** — network access control, posture checking before admitting a device.
- **Network segmentation and micro-segmentation**; **air-gapped** networks for the most critical systems.
- **DDoS protection** — rate limiting, anycast scrubbing centres, upstream filtering. **Volumetric vs protocol vs application-layer** attacks.
- **Honeypot / honeynet** — decoy systems to study attackers.
- **SIEM** — aggregates logs from everything, correlates, alerts; the heart of a **SOC**. **SOAR** adds automated response.
- **Network taps**, span ports, NetFlow, packet capture (Wireshark, tcpdump) for visibility.

Wireless security

WEP (broken) → WPA (TKIP) → **WPA2 (AES-CCMP)** → **WPA3 (SAE, forward secrecy, protection against offline dictionary attacks)**. Enterprise mode uses **802.1X with RADIUS** and EAP; personal mode uses a pre-shared key. **Evil twin**, **deauthentication** and **rogue AP** are the standard attacks.

Exam pointers

1. **Stateless filters cannot track sessions**; stateful firewalls can.
2. **IDS detects, IPS prevents**; IPS is **in line**.
3. IPsec: **AH = integrity only, ESP = encryption + integrity**; tunnel mode encapsulates the whole packet.
4. **WPA2 = AES-CCMP; WPA3 = SAE.**
5. A **DMZ** exists so that a compromised public service does not equal a compromised internal network.

Transport Layer — TCP and UDP

Core preparation lesson

TCP vs UDP

|| TCP | UDP |

---|---|---

| Connection | Connection-oriented (**3-way handshake**) | Connectionless |

| Reliability | Acknowledged, retransmitted, ordered | **Best effort**, no ordering |

| Header size | **20 bytes** minimum | **8 bytes** |

| Flow control | Yes (sliding window) | No |

| Congestion control | Yes | No |

| Speed / overhead | Slower, heavier | Faster, lighter |

| Uses | HTTP, FTP, SMTP, SSH, **trading order entry** | DNS, DHCP, TFTP, SNMP, VoIP, video streaming, **market data multicast** |

> In markets, **order entry is TCP** (you cannot lose an order) while **market data feeds are frequently UDP multicast** (stale prices are worse than missing ones, and one sender must reach thousands of subscribers). This contrast is an excellent interview answer.

TCP header fields to know

Source port · Destination port · **Sequence number** · **Acknowledgement number** · Data offset · Flags (**URG, ACK, PSH, RST, SYN, FIN**) · **Window size** · Checksum · Urgent pointer · Options (MSS, window scaling, SACK, timestamps).

Three-way handshake

```
Client -> SYN (seq = x)                -> Server
Client <- SYN-ACK (seq = y, ack = x+1)  <- Server
Client -> ACK (seq = x+1, ack = y+1)    -> Server
```

Connection termination is four-way (FIN, ACK, FIN, ACK) because each direction closes independently — **half-close** is legal. The closing side then waits in **TIME_WAIT** for $2 \times \text{MSL}$.

SYN flood attacks exploit the half-open state; **SYN cookies** are the defence.

Flow control

Sliding window, advertised by the receiver in the **window size** field. Problems and fixes:

- **Silly window syndrome** — tiny segments; fixed by **Nagle's algorithm** (sender side) and **Clark's solution / delayed ACK** (receiver side).

- **Zero window probe** keeps the connection from deadlocking when the advertised window is 0.

Congestion control

Congestion window (cwnd); the sender is limited by **$\min(\text{cwnd}, \text{receiver window})$** .

1. **Slow start** — cwnd starts at 1 MSS and **doubles every RTT** (exponential) until it reaches **ssthresh**.
2. **Congestion avoidance** — cwnd increases by **1 MSS per RTT** (linear, additive increase).
3. **Congestion detection:**
 - **Timeout** (severe): $\text{ssthresh} = \text{cwnd}/2$, **cwnd = 1**, back to slow start. (**TCP Tahoe**)
 - **Three duplicate ACKs** (mild): **fast retransmit** the missing segment, then **fast recovery** — $\text{ssthresh} = \text{cwnd}/2$ and $\text{cwnd} = \text{ssthresh}$, continue in congestion avoidance. (**TCP Reno**)

This is **AIMD** — additive increase, multiplicative decrease — and it is what keeps the Internet stable.

Variants: NewReno, **CUBIC** (Linux default), **BBR** (models bandwidth and RTT rather than reacting to loss).

Retransmission timeout uses a smoothed RTT estimate: $\text{EstimatedRTT} = (1-\alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$ (α typically 0.125), with a deviation term; **Karn's algorithm** says do not use retransmitted segments for RTT samples.

Ports

Well-known **0–1023**, registered **1024–49151**, dynamic/ephemeral **49152–65535**.

Port	Service
---	---
20/21	FTP data / control
22	SSH, SCP, SFTP
23	Telnet
25	SMTP
53	DNS (UDP, TCP for zone transfer and large responses)
67/68	DHCP server / client
69	TFTP
80	HTTP
110	POP3
143	IMAP
161/162	SNMP / SNMP trap
179	BGP
389 / 636	LDAP / LDAPS
443	HTTPS
445	SMB

| 3306 | MySQL |

| 3389 | RDP |

| 5432 | PostgreSQL |

A **socket** = IP address + port. A TCP connection is identified by the **4-tuple** (source IP, source port, destination IP, destination port).

Exam pointers

1. TCP header **20 bytes**, UDP **8 bytes**.
2. Handshake is **3-way**, termination is **4-way**.
3. **Three duplicate ACKs** → fast retransmit; timeout → back to slow start.
4. Slow start is **exponential**, congestion avoidance is **linear**.
5. Memorise the port table — it is free marks, and it comes up in security questions too.

Boolean Algebra and Logic Minimisation

Supplementary reference / interview

Laws

| Law | Form |

|---|---|

| Identity | $A + 0 = A$; $A \cdot 1 = A$ |

| Null | $A + 1 = 1$; $A \cdot 0 = 0$ |

| Idempotent | $A + A = A$; $A \cdot A = A$ |

| Complement | $A + A' = 1$; $A \cdot A' = 0$ |

| Involution | $(A')' = A$ |

| Commutative | $A + B = B + A$ |

| Associative | $(A+B)+C = A+(B+C)$ |

| Distributive | $A(B+C) = AB + AC$; $A + BC = (A+B)(A+C)$ |

| **Absorption** | $A + AB = A$; $A(A+B) = A$ |

| Consensus | $AB + A'C + BC = AB + A'C$ |

| **De Morgan** | $(A+B)' = A'B'$; $(AB)' = A' + B'$ |

De Morgan's laws are the most-used identities in the paper — know both directions instantly.

Gates

| Gate | Output |

|---|---|

| AND | 1 only if all inputs are 1 |

| OR | 1 if any input is 1 |

| NOT | Inverts |

| **NAND** | NOT(AND) — **universal gate** |

| **NOR** | NOT(OR) — **universal gate** |

| **XOR** | 1 if inputs **differ** (odd number of 1s) |

| XNOR | 1 if inputs are **equal** |

NAND and NOR are functionally complete — any circuit can be built from either alone. NOT from NAND: tie both inputs together.

XOR properties: $A \oplus 0 = A$; $A \oplus A = 0$; $A \oplus 1 = A'$; commutative and associative. Used in parity, checksums, swapping without a temporary, and toggle logic.

Canonical forms

- **SOP (sum of products)** — sum of **minterms**; a minterm is 1 for exactly one input combination. Written $\Sigma m(\dots)$.
- **POS (product of sums)** — product of **maxterms**. Written $\Pi M(\dots)$.
- For n variables there are 2^n minterms; minterm i and maxterm i are complements.

Karnaugh maps

Minimisation by grouping adjacent 1s.

Rules:

1. Groups must be **powers of two** in size (1, 2, 4, 8, 16).
2. Groups must be **rectangular**, and may **wrap around** edges and corners.
3. Make groups **as large as possible** and use **as few as possible**.
4. Overlapping is allowed.
5. **Don't-care (X)** terms may be included when they help make a bigger group, and ignored otherwise.

A group of 2^k cells eliminates k variables. Cell ordering follows **Gray code** (00, 01, 11, 10) so that adjacent cells differ in one variable.

Prime implicant — a group that cannot be made larger. **Essential prime implicant** — one that covers a minterm covered by no other prime implicant; these must all appear in the minimal expression.

For more than 4–5 variables, use the **Quine-McCluskey** tabular method — systematic and machine-implementable, unlike visual K-maps.

Worked pattern

> $F(A,B,C,D) = \Sigma m(0,1,2,5,8,9,10)$ — draw the 4×4 map, group the four corners $(0,2,8,10) \rightarrow B'D'$, group $(0,1,8,9) \rightarrow B'C'$, group $(1,5) \rightarrow A'C'D$. Result: $F = B'D' + B'C' + A'C'D$.

Always verify a minimised expression by spot-checking two or three input combinations against the original.

Exam pointers

1. **De Morgan both directions** — the single most-used law.
2. **NAND and NOR are universal**; XOR is not.
3. K-map groups are **powers of two**, may wrap, and should be as **few and as large** as possible.
4. **Essential prime implicants must be in the answer**.
5. K-map cell order is **Gray code**, not binary counting order.

Combinational Circuits

Supplementary reference / interview

Output depends **only on the current inputs** — no memory, no clock.

Adders

- **Half adder:** $\text{Sum} = A \oplus B$; $\text{Carry} = A \cdot B$. Two inputs, no carry-in.
- **Full adder:** $\text{Sum} = A \oplus B \oplus C_{in}$; $\text{Carry} = AB + C_{in}(A \oplus B)$. Built from **two half adders and an OR gate**.
- **Ripple carry adder:** n full adders chained; simple but **slow** — delay grows linearly with n , because each stage waits for the previous carry.
- **Carry look-ahead adder:** computes carries in parallel using **generate** $G = A \cdot B$ and **propagate** $P = A \oplus B$, with $C_{i+1} = G(i) + P(i) \cdot C(i)$. Delay becomes roughly **$O(\log n)$** at the cost of more gates — the classic speed-versus-area trade-off.
- **Subtractor** — use a full adder with B inverted and $C_{in} = 1$ (2's complement).

Multiplexer (MUX)

2^n data inputs, n select lines, 1 output. "Data selector."

- A **2^n -to-1 MUX** can implement **any Boolean function of n variables** directly (connect minterms to inputs), and any function of **$n+1$** variables using one extra inverter.
- Building bigger from smaller: a 4:1 MUX needs **three 2:1 MUXes**; an 8:1 needs **seven 2:1**. In general a 2^n :1 MUX needs **$2^n - 1$** 2:1 MUXes.

Demultiplexer (DEMUX) — one input, n select lines, 2^n outputs. The inverse of a MUX.

Decoder and encoder

- **Decoder:** n inputs $\rightarrow 2^n$ outputs, exactly one active. An $n:2^n$ decoder plus OR gates implements any SOP function (each output is a minterm). Used for memory address decoding.
- **Encoder:** 2^n inputs $\rightarrow n$ outputs. A plain encoder fails if two inputs are active; a **priority encoder** resolves this by encoding the highest-priority active input — which is how interrupt controllers work.

Comparator

Compares two n -bit numbers producing $A > B$, $A = B$, $A < B$. Equality uses **XNOR** on each bit position, ANDed together.

Code converters and parity

Binary $\leftrightarrow$ Gray, binary $\leftrightarrow$ BCD, seven-segment decoder. **Parity generator/checker** built from XOR trees: the XOR of all bits gives odd/even parity.

Timing concerns

Propagation delay limits the maximum clock frequency. **Glitches/hazards** occur when different paths through a circuit have different delays:

- **Static-1 hazard** — output should stay 1 but momentarily dips to 0.
- **Static-0 hazard** — the reverse.
- **Dynamic hazard** — output changes more than once during a single transition.

Removed by adding **redundant (consensus) terms** to the expression.

Exam pointers

1. Full adder = **two half adders + OR**.
2. **Carry look-ahead** is faster than ripple carry — generate and propagate.
3. A **$2^n:1$ MUX** implements any n -variable function, and needs **$2^n - 1$ $2:1$ MUXes** to build.
4. **Decoder outputs are minterms.**
5. Hazards are removed with **redundant consensus terms**.

CPU Organisation and Instruction Execution

Supplementary reference / interview

Components

ALU (arithmetic and logic), **control unit**, **registers** (PC, IR, MAR, MDR, accumulator, general purpose, stack pointer, status/flag register), and the **buses** connecting them: **address bus** (unidirectional, width determines addressable memory), **data bus** (bidirectional, width = word size transferred), **control bus**.

> If the address bus is n bits wide, the CPU can address 2^n locations. With a 32-bit address bus and byte addressing: 4 GB. A standard question.

Instruction cycle

```
FETCH      : MAR <- PC ; MDR <- Memory[MAR] ; IR <- MDR ; PC <- PC + 1
DECODE     : control unit interprets the opcode
EXECUTE    : ALU operation / memory access / branch
(INTERRUPT check at the end of each cycle)
```

Machine cycle vs instruction cycle vs clock cycle: one instruction takes several machine cycles; each machine cycle takes several clock cycles (in a non-pipelined design).

Control unit design

- **Hardwired** — fixed logic; **fast**, hard to modify; suits RISC.
- **Microprogrammed** — a **control memory** holds microinstructions; **flexible and easier to modify**, but slower. Horizontal (wide, parallel, less decoding) vs vertical (narrow, encoded, needs decoding) microinstructions.

Instruction formats

3-address, 2-address, 1-address (accumulator), 0-address (stack-based, using postfix). Fewer addresses means shorter instructions but more instructions per task.

Addressing modes

| Mode | Effective address | Example use |

|---|---|---|

| **Immediate** | Operand is in the instruction | Constants |

| **Register** | In a register | Fastest |

| **Direct/absolute** | Given in the instruction | Globals |

| **Indirect** | Address of the address | Pointers |

| **Register indirect** | Register holds the address | Pointer dereference |

| **Indexed** | Base + index register | **Arrays** |

Base + displacement	Base register + offset	Struct fields, stack frames
Relative (PC-relative)	PC + offset	**Branches**, position-independent code
Auto-increment/decrement	Address register updated after use	Stacks, sequential access

RISC vs CISC

	RISC	CISC
Instruction set	Small, simple, fixed length	Large, complex, variable length
Cycles per instruction	Mostly 1 (pipelined)	Many
Addressing modes	Few	Many
Registers	Many	Fewer
Memory access	Load/store only	Most instructions can access memory
Complexity in	Compiler/software	Hardware/microcode
Examples	ARM, RISC-V, MIPS	x86

Modern x86 processors decode CISC instructions into RISC-like micro-operations internally — so the distinction is now architectural rather than absolute.

Performance

CPU time = Instruction count × CPI × Clock cycle time

MIPS = clock rate / (CPI × 10⁶)

Amdahl's law: speedup = $1 / ((1 - f) + f/s)$, where f is the fraction improved and s the speedup of that fraction.
 The lesson: the **unimproved portion sets the ceiling** — optimising 10% of runtime infinitely still gives at most 1.11× overall.

Exam pointers

- n-bit address bus → 2ⁿ addressable locations.**
- Hardwired = fast/inflexible; microprogrammed = flexible/slower.**
- Indexed addressing is for arrays; PC-relative for branches.**
- RISC = load/store architecture**, complexity moved to the compiler.
- CPU time = IC × CPI × clock time**; Amdahl's law caps the benefit of partial optimisation.

I/O Organisation and Buses

Supplementary reference / interview

I/O techniques

| Technique | CPU involvement | When used |

|---|---|---|

| **Programmed I/O (polling)** | CPU busy-waits on a status flag | Simple, low-rate devices |

| **Interrupt-driven** | Device signals when ready; CPU services it | Moderate rate, unpredictable timing |

| **DMA** | Controller moves data directly to/from memory; CPU interrupted **once at completion** |

High-volume transfers (disk, network) |

DMA transfer modes: **burst** (holds the bus until done — fastest, starves the CPU), **cycle stealing** (one word at a time, using cycles the CPU does not need), **transparent** (only when the CPU is idle).

Interrupts

Classification: hardware vs software; **maskable** vs **non-maskable (NMI)**; vectored (device supplies the vector) vs **non-vectored** (fixed address); internal (traps/exceptions) vs external.

Handling: finish the current instruction → save PC and status → disable lower-priority interrupts → fetch the vector → execute the **ISR** → restore context → return.

Priority resolution: **daisy chaining** (hardware priority by physical position), **polling**, or a **programmable interrupt controller** with a **priority encoder**.

Interrupt latency — time from the request to the first ISR instruction; critical in real-time systems.

Memory-mapped vs isolated I/O

| | Memory-mapped I/O | Isolated (port-mapped) I/O |

|---|---|---|

| Address space | Shares the memory address space | Separate I/O address space |

| Instructions | Ordinary load/store | Special IN/OUT instructions |

| Address space cost | Consumes memory addresses | None |

| Typical of | ARM, RISC-V, most modern designs | x86 (which supports both) |

Buses

- **Types:** address, data, control. **Synchronous** (clocked, simple, limited by the slowest device) vs **asynchronous** (handshaking, flexible).

- **Bus arbitration:** centralised (daisy chain, polling, independent request/grant) vs distributed. Concepts: bus master, bus grant, bus request.

- **Standards to recognise:** PCI → **PCIe** (serial, point-to-point lanes), **USB** (host-controlled, tiered star), SATA/NVMe, Thunderbolt, SCSI/SAS, and in embedded systems **I²C**, **SPI**, **UART**, **CAN**.

Bus bandwidth = bus width (bytes) × bus clock frequency. Modern high-speed interconnects are **serial with multiple lanes** rather than wide parallel buses, because parallel skew limits clock rates.

Storage interfaces and RAID

Covered in the OS note — but remember the **RAID levels** and that RAID is **availability, not backup**: it protects against disk failure, not against deletion, corruption or ransomware.

Exam pointers

1. **DMA interrupts the CPU once**, at completion.
2. **Cycle stealing** takes single bus cycles; burst mode holds the bus.
3. **NMI cannot be masked** — used for critical faults like power failure.
4. **Memory-mapped I/O uses ordinary load/store instructions.**
5. **RAID is not backup** — a one-line answer worth having ready.

Memory Hierarchy and Cache

Supplementary reference / interview

The hierarchy

Registers (fastest, smallest, most expensive per bit)
L1 / L2 / L3 cache (SRAM)
Main memory (DRAM)
SSD / Disk
Tape / archival (slowest, largest, cheapest)

As you go down: **capacity and cost-efficiency increase; speed decreases**. The hierarchy works because of **locality of reference** — **temporal** (recently used data will be used again) and **spatial** (nearby addresses will be used soon).

SRAM vs DRAM: SRAM uses flip-flops (6 transistors), is fast, expensive, and needs no refresh — used for cache. DRAM uses a capacitor per bit, is dense and cheap, and **must be refreshed** periodically — used for main memory.

ROM family: ROM, PROM (one-time), EPROM (UV erase), EEPROM (electrical erase), **Flash** (block erase; NAND vs NOR).

Cache mapping

| Mapping | Placement | Comparators | Flexibility |

|---|---|---|---|

| **Direct mapped** | Block goes to exactly one line: (block number) mod (number of lines) | 1 | Least; conflict misses |

| **Fully associative** | Anywhere | One per line (expensive) | Most |

| **k-way set associative** | Anywhere within one set | k | Practical compromise |

Address breakdown:

- Direct mapped: **Tag | Line (index) | Block offset**

- Set associative: **Tag | Set index | Block offset**

- Fully associative: **Tag | Block offset**

> **Worked example.** 32-bit address, cache 64 KB, block size 16 bytes, 4-way set associative.

> Block offset = $\log_2 16 = 4$ bits. Number of blocks = $64 \text{ KB} / 16 \text{ B} = 4096$. Sets = $4096 / 4 = 1024 \rightarrow$ set index = $\log_2 1024 = 10$ bits. Tag = $32 - 10 - 4 = 18$ bits.

Replacement policies: LRU, FIFO, random, LFU. Direct-mapped caches need **no replacement policy** — there is only one candidate line.

Write policies:

- **Write-through** — write to cache and memory simultaneously; simple, consistent, more traffic. Often paired with a **write buffer**.

- **Write-back** — write only to cache, mark the line **dirty**, write to memory on eviction; less traffic, more complex, risk on power failure.
- On a write miss: **write-allocate** (fetch then write) vs **no-write-allocate** (write straight to memory).

Performance

Average Memory Access Time (AMAT) = Hit time + Miss rate × Miss penalty

For multiple levels: $AMAT = H_1 + M_1 \times (H_2 + M_2 \times (H_3 + M_3 \times \text{memory time}))$.

The three Cs of misses:

- **Compulsory** (cold start) — first access; reduced by prefetching or larger blocks.
- **Capacity** — cache too small; reduced by a bigger cache.
- **Conflict** — too many blocks mapping to the same set; reduced by **higher associativity**. Fully associative caches have no conflict misses.

Cache coherence in multiprocessors: the **MESI protocol** (Modified, Exclusive, Shared, Invalid), snooping vs directory-based schemes. **False sharing** — two cores updating different variables in the same cache line, causing needless invalidation traffic.

Exam pointers

1. Practise the **address-split calculation** until it is automatic; it appears every year.
2. **Direct mapped** needs no replacement policy.
3. **AMAT = hit time + miss rate × miss penalty.**
4. **Three Cs: compulsory, capacity, conflict** — and which cache change fixes each.
5. **Write-back** marks lines **dirty**; **write-through** keeps memory always current.

Number Systems and Computer Arithmetic

Supplementary reference / interview

Bases and conversion

| Base | Digits |

|---|---|

| Binary (2) | 0–1 |

| Octal (8) | 0–7 |

| Decimal (10) | 0–9 |

| Hexadecimal (16) | 0–9, A–F |

- **Binary → octal:** group bits in **3s** from the right. **Binary → hex:** group in **4s**.

- **Decimal → binary:** repeated division by 2, reading remainders **bottom-up**. For fractions, repeatedly multiply by 2 and read the integer parts **top-down**.

Signed representations (n bits)

| Representation | Range | Zero |

|---|---|---|

| **Sign-magnitude** | $-(2^{n-1}-1)$ to $+(2^{n-1}-1)$ | **Two zeros** |

| **1's complement** | $-(2^{n-1}-1)$ to $+(2^{n-1}-1)$ | **Two zeros** |

| **2's complement** | -2^{n-1} to $+2^{n-1}-1$ | **One zero** |

2's complement = invert all bits and add 1. It wins because addition and subtraction use the same circuit and there is a single zero. For 8 bits: **-128 to +127**.

Overflow rule (2's complement addition): overflow occurs when adding two numbers of the **same sign** produces a result of the **opposite sign**. Equivalently, carry-in to the sign bit $\neq$ carry-out.

Subtraction: $A - B = A + (2\text{'s complement of } B)$.

Floating point — IEEE 754

| Format | Total | Sign | Exponent | Mantissa | Bias |

|---|---|---|---|---|---|

| **Single** | 32 | 1 | **8** | **23** | **127** |

| **Double** | 64 | 1 | **11** | **52** | **1023** |

Value = $(-1)^{\text{sign}} \times 1.\text{mantissa} \times 2^{(\text{exponent} - \text{bias})}$, with an **implicit leading 1** for normalised numbers.

Special values: exponent all 0s with mantissa 0 → **±zero**; exponent all 0s with non-zero mantissa → **denormalised**; exponent all 1s with mantissa 0 → **±infinity**; exponent all 1s with non-zero mantissa →

NaN.

Why $0.1 + 0.2 \neq 0.3$: 0.1 and 0.2 have no exact binary representation. Never compare floats with `==`; compare within an epsilon. In financial systems, use **integer paise/cents** or a decimal type — a genuinely relevant point for a market IT role.

Codes

- **BCD** — each decimal digit in 4 bits; wastes range but simplifies decimal display.
- **Gray code** — successive values differ in exactly **one bit**; used in position encoders and K-maps to avoid transition errors.
- **ASCII** — 7 bits, 128 characters ('A' = 65, 'a' = 97, '0' = 48). **Unicode/UTF-8** — variable width, 1–4 bytes, backwards compatible with ASCII.
- **Parity, Hamming code** — see the data link note.

Multiplication and division algorithms

Booth's algorithm for signed multiplication (handles runs of 1s efficiently), **restoring** and **non-restoring** division. Know that they exist and what problem each solves.

Exam pointers

1. **2's complement** has one zero and an asymmetric range (−128 to +127 for 8 bits).
2. **Overflow**: same signs in, opposite sign out.
3. IEEE 754 single: **1 + 8 + 23**, bias 127. Double: **1 + 11 + 52**, bias 1023.
4. Binary→hex groups of **4**, binary→octal groups of **3**.
5. **Gray code** changes one bit at a time — that is the whole point of it.

Pipelining and Parallel Processing

Supplementary reference / interview

The idea

Overlap instruction execution like an assembly line. Classic 5-stage RISC pipeline:

IF (fetch) → ID (decode/register read) → EX (execute/ALU) → MEM (memory) → WB (write back)

Performance: for n instructions in a k -stage pipeline with cycle time t :

- Non-pipelined time = $n \times k \times t$
- Pipelined time = $(k + n - 1) \times t$
- **Speedup** = $nk / (k + n - 1)$ → approaches k for large n .
- **Throughput** ideally becomes one instruction per cycle.

The clock period is set by the **slowest stage** (plus register overhead) — which is why stages are balanced.

Hazards

1. Structural hazard

Two instructions need the same hardware resource in the same cycle. Fixed by duplicating resources — the classic case is **separate instruction and data caches (Harvard-style)** so IF and MEM do not collide.

2. Data hazard

An instruction needs a result that is not ready.

- **RAW (read after write)** — the true dependency, and the common one.
- **WAR (write after read)** and **WAW (write after write)** — name dependencies, appearing with out-of-order execution; fixed by **register renaming**.

Solutions: **forwarding/bypassing** (route the ALU result directly to the next instruction), **stalling** (pipeline bubbles), compiler **instruction scheduling**. A load followed immediately by a use of its result still needs **one stall cycle** even with forwarding — the **load-use hazard**.

3. Control hazard

A branch changes the PC, and instructions already fetched may be wrong.

Solutions: stall, **branch prediction** (static: always-taken/not-taken; dynamic: 1-bit and 2-bit saturating counters, branch history tables, branch target buffers, correlating predictors), **delayed branch** (fill the slot with a useful instruction), and **speculative execution** (which, incidentally, is the root of the Spectre/Meltdown class of vulnerabilities — a good cross-topic point).

Beyond simple pipelining

- **Superscalar** — multiple instructions issued per cycle using duplicated functional units.

- **Superpipelined** — more, shorter stages and a faster clock.
- **VLIW** — the compiler packs independent operations into one long instruction.
- **Out-of-order execution** with **Tomasulo's algorithm** (reservation stations, register renaming, common data bus) and a **reorder buffer** for in-order commit.
- **SIMD** (vector instructions: SSE, AVX, NEON) and **MIMD**.
- **Multicore** and **SMT/hyper-threading** — sharing one core's resources between threads.

Flynn's taxonomy

| | Single instruction | Multiple instruction |

|---|---|---|

| **Single data** | SISD (classic uniprocessor) | MISD (rare, fault-tolerant systems) |

| **Multiple data** | **SIMD** (vector, GPU) | **MIMD** (multicore, clusters) |

Amdahl's law caps parallel speedup by the serial fraction; **Gustafson's law** counters that larger problems have proportionally more parallel work, so scaling is more useful than Amdahl suggests.

Exam pointers

1. **Speedup = $nk / (k + n - 1)$** , approaching **k**.
2. **RAW** is the true dependency; **WAR/WAW** are fixed by renaming.
3. **Forwarding** does not eliminate the load-use stall.
4. **Flynn: SISD, SIMD, MISD, MIMD.**
5. Separate I-cache and D-cache exist to remove a **structural hazard**.

Sequential Circuits

Supplementary reference / interview

Output depends on **current inputs and stored state**. Requires memory and (usually) a clock.

Latches vs flip-flops

- **Latch** — **level triggered**; transparent while the enable is active.
- **Flip-flop** — **edge triggered**; changes state only on a clock edge. This is what makes synchronous design possible.

Flip-flop types

| Type | Characteristic equation | Behaviour |

|---|---|---|

| **SR** | $Q(\text{next}) = S + R'Q$ | **S = R = 1 is forbidden** |

| **D** | $Q(\text{next}) = D$ | Data/delay; the workhorse of real design |

| **JK** | $Q(\text{next}) = JQ' + K'Q$ | Like SR but **J = K = 1 toggles** |

| **T** | $Q(\text{next}) = T \oplus Q$ | Toggle when **T = 1** |

Excitation tables (what inputs are needed for a given transition) are the tool for designing counters — learn to derive them rather than memorise them.

Timing: **setup time** (data stable before the edge), **hold time** (stable after the edge), **propagation delay**. A violation causes **metastability**, resolved in practice by **synchroniser chains** (two flip-flops in series) when crossing clock domains.

Registers

- **Parallel-in parallel-out (PIPO)**, **SISO**, **SIPO**, **PISO**.
- **Shift register** — shifts data one position per clock; used for serial-parallel conversion, delay lines and **LFSRs** (pseudorandom sequence generation, CRC).
- **Universal shift register** — load, shift left, shift right, hold.

Counters

| Type | Nature |

|---|---|

| **Asynchronous (ripple)** | Each flip-flop clocked by the previous output. Simple; **cumulative delay**, glitches |

| **Synchronous** | All flip-flops share one clock. Faster, cleaner; more combinational logic |

| **Mod-N** | Counts 0 to N-1; needs $\lceil \log_2 N \rceil$ flip-flops |

| Up/down, ring, **Johnson (twisted ring)** | Ring counter: n states with n flip-flops; Johnson: **$2n$ states** with n flip-flops |

n flip-flops give 2^n states — the most common one-mark question here.

State machines

- **Moore machine** — output depends **only on the state**; output changes are synchronous with state transitions, hence glitch-free.

- **Mealy machine** — output depends on **state and input**; reacts one cycle earlier and usually needs **fewer states**, but can glitch.

Design procedure: state diagram $\rightarrow$ state table $\rightarrow$ state assignment $\rightarrow$ **state minimisation** (merge equivalent states) $\rightarrow$ choose flip-flops $\rightarrow$ excitation table $\rightarrow$ K-map for each flip-flop input and output $\rightarrow$ circuit.

Exam pointers

1. **Latch = level triggered, flip-flop = edge triggered.**
2. **SR: $S=R=1$ forbidden; JK: $J=K=1$ toggles.**
3. **Mod- N counter needs $\lceil \log_2 N \rceil$ flip-flops**; n flip-flops give 2^n states.
4. **Johnson counter gives $2n$ states with n flip-flops**; ring counter gives n .
5. **Moore = output from state only; Mealy = state + input**, fewer states.

Python analytics: slicing, regex, files and dataframes

Core preparation lesson

Lists preserve order; sets remove duplicate values; dictionaries map keys to values. A slice `a[start:stop:step]` excludes stop. A shallow copy creates a new outer collection while retaining references to nested objects.

Worked examples

```
a = [10, 20, 30, 40, 50]
print(a[1:4:2]) # [20, 40]
print(a[::-1]) # [50, 40, 30, 20, 10]
counts = {}
for x in ['A', 'B', 'A']:
    counts[x] = counts.get(x, 0) + 1
```

In pandas, `loc` selects labels and `iloc` selects integer positions. Filter with a boolean mask; combine masks with parenthesised `&` and `|`. `groupby` splits rows into groups before aggregation. `pivot` reshapes unique index-column pairs; `pivot_table` can aggregate duplicates. `merge` performs relational joins; verify key uniqueness to avoid multiplying rows unexpectedly.

Files and cleaning

Use a context manager for files. Choose encodings explicitly. Distinguish missing values from zeros, validate types and document how duplicates are handled. Regex anchors `^` and `$` constrain boundaries; `search` finds a match anywhere, while `fullmatch` covers the entire input. Practise predicting shapes and row counts before running code.

[Python tutorial](<https://docs.python.org/3/tutorial/>) · [pandas user guide](https://pandas.pydata.org/docs/user_guide/index.html)

R essentials for data analysis

Core preparation lesson

R vectors are normally one-indexed. `c(2,4,6)[2]` gives 4. A negative index excludes a position. Vector operations are elementwise; shorter vectors may be recycled, so check lengths. NA represents missingness and is tested with `is.na`, not equality.

Worked example

```
x <- c(2, NA, 6)
mean(x)           # NA
mean(x, na.rm = TRUE) # 4
d <- data.frame(team=c("A", "A", "B"), value=c(2,4,9))
aggregate(value ~ team, data=d, FUN=sum)
```

Use `read.csv` and `write.csv` for tabular files; inspect `str`, `head` and `summary` after import. A `data.frame` can hold columns of different types. Matrix elements share a type. Lists can contain heterogeneous objects. A factor encodes categories and levels, not arbitrary numeric magnitudes.

Drill

Predict `x[c(TRUE,FALSE,TRUE)]`, compare list indexing with `[` and `[[`, and reshape a small table from wide to long. Practise missing-value handling, functions and regular expressions. Compare with Python explicitly to avoid confusing zero-based and one-based positions.

[Official R manuals](<https://cran.r-project.org/manuals.html>)

Concurrency Control

Core preparation lesson

Lock-based protocols

Shared (S) locks for reading, **Exclusive (X)** locks for writing.

| | S requested | X requested |

|---|---|---|

| S held | Compatible | Conflict |

| X held | Conflict | Conflict |

Two-Phase Locking (2PL)

Every transaction has a **growing phase** (only acquires locks) and a **shrinking phase** (only releases). Once it releases any lock, it may not acquire another.

- **2PL** guarantees conflict serialisability but **not** freedom from deadlock, and not recoverability.
- **Strict 2PL** — all **exclusive** locks held until commit/abort. Avoids cascading rollback. This is what real systems use.
- **Rigorous 2PL** — **all** locks held until commit. Simplest to reason about.
- **Conservative 2PL** — acquire all locks up front; **deadlock free**, but poor concurrency and requires knowing the lock set in advance.

Lock granularity: database → table → page → row. Finer granularity gives more concurrency and more locking overhead. **Intention locks (IS, IX, SIX)** in **multiple-granularity locking** let the system lock a table while row locks exist beneath it.

Deadlock in databases

- **Prevention by timestamps** (older transaction = higher priority):
- **Wait-Die** — an **older** transaction **waits** for a younger one; a **younger** one **dies** (rolls back and restarts). Non-preemptive.
- **Wound-Wait** — an **older** transaction **wounds** (preempts) the younger; a younger one **waits**. Preemptive.

Both avoid cycles because waiting always goes in one direction along timestamps.

- **Detection** — **wait-for graph**; a cycle means deadlock; choose a **victim** by cost, and beware **starvation**.
- **Timeouts** — crude but common in practice.

Timestamp ordering protocol

Each transaction gets a timestamp $TS(T)$ at start. Each item X has **$R\text{-timestamp}(X)$** and **$W\text{-timestamp}(X)$** .

- **Read(X)** by T: if $TS(T) < W-TS(X)$, the value is already too new → **reject and roll back T**; else read and set $R-TS(X) = \max(R-TS(X), TS(T))$.
- **Write(X)** by T: if $TS(T) < R-TS(X)$ or $TS(T) < W-TS(X)$ → **reject and roll back**; else write and set $W-TS(X) = TS(T)$.

Thomas's Write Rule relaxes the second case: an obsolete write ($TS(T) < W-TS(X)$) can simply be **ignored** rather than causing a rollback, which allows some view-serialisable schedules.

Timestamp ordering is **deadlock free** (no waiting) but can cause **starvation** through repeated restarts.

Optimistic concurrency control (validation-based)

Three phases: **Read** (work on a private copy), **Validation** (check for conflicts with concurrent transactions), **Write** (apply if validation succeeds). Good when conflicts are rare — typical of read-heavy workloads.

MVCC — Multiversion Concurrency Control

Each write creates a **new version** with a timestamp; readers are given the version consistent with their snapshot. **Readers never block writers and writers never block readers**, which is why PostgreSQL, Oracle and InnoDB all use it. Cost: version storage and **vacuum/garbage collection**. **Snapshot isolation** is the common level built on MVCC, and it permits the **write skew** anomaly, which is why "serializable snapshot isolation" exists.

Exam pointers

1. **2PL** □ conflict serialisable, but deadlock is still possible.
2. **Strict 2PL** holds X locks to commit → avoids cascading aborts.
3. **Wait-Die**: old waits, young dies. **Wound-Wait**: old wounds, young waits. Remember "older is privileged in both".
4. Timestamp ordering is deadlock-free but starvation-prone.
5. MVCC: **readers do not block writers**; snapshot isolation allows **write skew**.

Indexing, B+ Trees and File Organisation

Core preparation lesson

File organisation

| Organisation | Insert | Search | Notes |

|---|---|---|---|

| **Heap (unordered)** | Fast (append) | Linear scan | Good for bulk loads |

| **Sequential (ordered)** | Expensive | Binary search possible | Needs overflow handling |

| **Hashed** | $O(1)$ average | $O(1)$ for equality; **useless for ranges** | Static vs dynamic hashing |

| **Clustered** | — | Fast on the clustering key | Rows physically ordered |

Index classification

- **Primary index** — on the ordering key field of an ordered file; **sparse** (one entry per block).

- **Clustering index** — on a non-key ordering field.

- **Secondary index** — on a non-ordering field; must be **dense**.

Dense index: one entry per record. **Sparse index:** one entry per block — smaller, but requires the file to be ordered on that field.

Multi-level index: treat the index itself as a file and index it — which converges on the B+ tree.

B+ tree — the structure every DBMS uses

- All **data pointers** are in the leaf nodes; internal nodes hold only keys for routing.

- **Leaves are linked** in a list, which makes **range queries** efficient — the main reason B+ trees beat B trees for databases.

- **Balanced**: all leaves at the same level, so every search costs the same $O(\log n)$.

- A node of order n holds at most $n-1$ keys and n pointers; internal nodes must be at least half full (except the root).

| | B tree | B+ tree |

|---|---|---|

| Data in internal nodes | Yes | **No** |

| Sequential access | Harder | **Easy** — linked leaves |

| Height | Can be taller (keys spread out) | Shorter (more keys per internal node) |

| Search cost | Can terminate early | Always reaches a leaf |

Insertion: insert into the leaf; if it overflows, **split** and push the middle key up; splits can cascade to the root, which is how the tree grows in height (upwards).

Deletion: remove; if underflow, **borrow from a sibling** or **merge**, possibly shrinking the height.

Hashing

- **Static hashing** — fixed number of buckets; **overflow chains** degrade performance as the file grows.
- **Extendible hashing** — a directory of 2^d entries with a **global depth** and per-bucket **local depth**; on overflow, split just that bucket and double the directory only when local depth = global depth.
- **Linear hashing** — splits buckets in a fixed order, avoiding a directory.

Hash index: excellent for equality (`=`), **useless** for `<`, `>`, `BETWEEN`, `ORDER BY` and prefix `LIKE`. This is the standard comparison question against B+ trees.

Query processing and optimisation

Parse → translate to relational algebra → **optimise** → execute.

- **Cost-based optimisation** using statistics (cardinality, selectivity, histograms).
- **Heuristic rules:** push **selection** down, push **projection** down, do the most restrictive selection first, avoid Cartesian products.
- **Join algorithms:** nested loop, **block nested loop**, **index nested loop**, **sort-merge join**, **hash join**. Cost is measured in **disk block transfers and seeks**.
- Reading an **EXPLAIN plan** and spotting a full table scan where an index should be used is a genuinely useful practical skill.

Exam pointers

1. In a **B+ tree**, all records are in the leaves, and the leaves are **linked**.
2. Secondary index must be **dense**; primary index can be **sparse**.
3. **Hash indexes cannot do range queries.**
4. B+ tree height grows by **root split**, upwards, not downwards.
5. The first heuristic of query optimisation is always: **push selections down**.

DBMS Architecture and Data Modelling

Core preparation lesson

Why a DBMS over file systems

File systems suffer from **data redundancy and inconsistency, difficulty of access, data isolation, integrity problems, atomicity problems, concurrent-access anomalies and security problems**. A DBMS solves all seven.

Three-schema architecture (ANSI/SPARC)

External level -> many user views
| logical data independence
Conceptual level -> the whole logical schema (tables, relationships, constraints)
| physical data independence
Internal level -> storage structures, files, indexes

- **Logical data independence** — change the conceptual schema without changing external views. Harder to achieve.

- **Physical data independence** — change storage/indexes without touching the logical schema. Easier, and the one usually achieved.

Components

Query processor (DDL interpreter, DML compiler, query optimiser, execution engine) and **storage manager** (authorisation and integrity manager, transaction manager, file manager, buffer manager), plus the **data dictionary / system catalog**.

Database users: naive users, application programmers, sophisticated users, **DBA** (schema definition, access authorisation, performance tuning, backup and recovery).

ER model

| Concept | Notation | Notes |

|---|---|---|

| Entity set | Rectangle | A thing; entity **instance** is one row |

| Attribute | Ellipse | Simple, composite, multivalued (double ellipse), derived (dashed) |

| Key attribute | Underlined | |

| Relationship | Diamond | Degree: unary, binary, ternary |

| Weak entity | Double rectangle | No key of its own; depends on an **identifying/owner entity** via a **double diamond** identifying relationship; has a **partial key** (dashed underline) |

Cardinality: 1:1, 1:N, M:N. **Participation:** total (double line, every entity must participate) vs partial.

EER extensions

- **Specialisation / generalisation** — an ISA hierarchy.

- **Constraints:** **disjoint vs overlapping**, **total vs partial**.
- **Aggregation** — treating a relationship as an entity so it can participate in another relationship.

ER to relational mapping — the rules that get tested

1. **Strong entity** → its own table with the key as primary key.
2. **Weak entity** → table with (owner's PK + partial key) as the composite primary key; the owner's PK is also a foreign key with ON DELETE CASCADE.
3. **1:N relationship** → put the **foreign key on the N side**. No extra table needed.
4. **M:N relationship** → **a separate table** with both keys as a composite primary key.
5. **1:1 relationship** → foreign key on either side, preferably the one with **total participation**.
6. **Multivalued attribute** → its own table.
7. **Ternary relationship** → its own table with all three keys.

Minimum number of tables questions follow directly from these rules: 1:1 with total participation on one side can collapse into a single table; M:N always needs three.

Exam pointers

1. **Physical data independence** is easier to achieve than logical.
2. For **1:N**, the foreign key goes on the **many** side.
3. **M:N** always needs a separate relation.
4. A weak entity's PK = owner PK + partial key.
5. Total participation is drawn as a **double line**.

Normalisation and Functional Dependencies

Core preparation lesson

> The most reliably examined DBMS topic. Learn to find candidate keys mechanically and the rest follows.

Functional dependency

$X \rightarrow Y$ means: for any two tuples with the same X value, the Y values must also be equal. X is the determinant.

Armstrong's axioms (sound and complete):

- **Reflexivity:** if $Y \subseteq X$ then $X \rightarrow Y$ (trivial dependency)
- **Augmentation:** if $X \rightarrow Y$ then $XZ \rightarrow YZ$
- **Transitivity:** if $X \rightarrow Y$ and $Y \rightarrow Z$ then $X \rightarrow Z$

Derived: **union** ($X \rightarrow Y, X \rightarrow Z \Rightarrow X \rightarrow YZ$), **decomposition** ($X \rightarrow YZ \Rightarrow X \rightarrow Y$ and $X \rightarrow Z$), **pseudo-transitivity**.

Attribute closure — the workhorse

To find **X^+** : start with X; repeatedly add the right-hand side of any FD whose left-hand side is already inside; stop when nothing changes.

X is a candidate key if and only if $X^+ =$ all attributes and no proper subset of X has that property.

Finding candidate keys fast:

1. Attributes that never appear on the **right** of any FD must be in **every** candidate key. Start there.
2. Attributes that appear on neither side are also in every key.
3. Take the closure of that essential set; if it is everything, it is the unique candidate key.
4. Otherwise, add the remaining attributes one at a time and test.

Normal forms

| Form | Condition |

|---|---|

| **1NF** | All attributes atomic; no repeating groups |

| **2NF** | 1NF + **no partial dependency** — no non-prime attribute depends on part of a candidate key |

| **3NF** | 2NF + **no transitive dependency** — for every FD $X \rightarrow Y$, either **X is a super key** or **Y is a prime attribute** |

| **BCNF** | For every non-trivial FD $X \rightarrow Y$, **X must be a super key** |

| **4NF** | BCNF + no non-trivial **multivalued dependency** unless the determinant is a super key |

| **5NF (PJNF)** | No **join dependency** that is not implied by candidate keys |

Prime attribute = part of some candidate key. A **partial dependency** can only exist when the candidate key is composite — so a relation with all single-attribute candidate keys and in 1NF is automatically in 2NF.

The two trade-offs

| Property | 3NF | BCNF |

|---|---|---|

| Lossless join decomposition | Always achievable | Always achievable |

| **Dependency preservation** | **Always achievable** | **Not always** |

This is the classic answer: **BCNF is stricter but may lose dependency preservation; 3NF always preserves dependencies.**

Decomposition

- **Lossless join:** a decomposition of R into R1 and R2 is lossless if $R1 \cap R2 \rightarrow R1$ or $R1 \cap R2 \rightarrow R2$ — that is, the common attributes form a key of at least one fragment.
- **Dependency preserving:** the union of the FDs projected onto each fragment implies the original FD set.

Worked example

R(A, B, C, D) with FDs: $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$.

- $A \rightarrow B$: $\{A, B, C, D\} \rightarrow$ **A is the only candidate key**. Prime attribute: A.
- $A \rightarrow B$: A is a key, fine.
- **$B \rightarrow C$** : B is not a super key, and C is not prime $\rightarrow$ violates **3NF** (transitive dependency $A \rightarrow B \rightarrow C$).
- So R is in **2NF but not 3NF**. Decompose into R1(A, B), R2(B, C), R3(C, D) — all in BCNF, lossless and dependency-preserving.

Denormalisation

Deliberately reintroducing redundancy for read performance — standard in **data warehouses** (star schema) and reporting systems. Normalise for OLTP, denormalise for OLAP.

Exam pointers

1. **3NF: X is a super key OR Y is prime. BCNF: X must be a super key.**
2. Partial dependency requires a **composite** candidate key.
3. **BCNF may not preserve dependencies; 3NF always does.**
4. Lossless join test: **common attributes must be a key of one fragment**.
5. Always compute candidate keys **first** — every normal-form question collapses once you have them.

NoSQL, Data Warehousing and Distributed Databases

Core preparation lesson

Why NoSQL

Relational systems struggle with **horizontal scale**, **schema flexibility** and **very high write throughput**. NoSQL trades some of ACID for those.

| Family | Model | Examples | Fits |

|---|---|---|---|

| **Key-value** | Hash map | Redis, DynamoDB | Caching, sessions, counters |

| **Document** | JSON/BSON documents | MongoDB, CouchDB | Semi-structured content, catalogues |

| **Column-family** | Sparse, wide rows | Cassandra, HBase | Time series, huge write volume |

| **Graph** | Nodes and edges | Neo4j, JanusGraph | Relationships, **fraud rings**, ownership networks |

> A graph database detecting **circular trading** or linked beneficial owners is a good, concrete example to give in a SEBI interview.

CAP theorem (Brewer)

In the presence of a network **Partition**, a distributed system must choose between **Consistency** and **Availability**. You cannot have all three simultaneously.

- **CP** — MongoDB (default), HBase: refuse writes rather than diverge.
- **AP** — Cassandra, DynamoDB: accept writes, reconcile later.
- **CA** — only meaningful for a single-node or non-partitioned system.

PACELC extends it: ***if Partition then A or C, Else Latency or Consistency*** — the everyday trade-off even when nothing is broken.

BASE (Basically Available, Soft state, Eventual consistency) is the NoSQL counterpart to ACID. **Eventual consistency** means replicas converge if writes stop.

Distributed database concepts

- **Fragmentation** — horizontal (rows), vertical (columns), mixed. **Replication** — full or partial; improves availability and read performance at the cost of update complexity.
- **Transparency** — location, fragmentation, replication transparency.
- **Two-Phase Commit (2PC)** — prepare then commit, with a coordinator. **Blocking** if the coordinator fails. **3PC** adds a phase to reduce blocking.
- **Consensus:** **Paxos**, **Raft** — how replicas agree on a log. **Quorum** rule: $R + W > N$ guarantees a read sees the latest write.
- **Sharding** — partition by key (range, hash, directory). A bad shard key creates **hot spots**.

Data warehousing

| OLTP | OLAP |

|---|---|---|

| Purpose | Run the business | Analyse the business |

| Queries | Short, many, write-heavy | Long, few, read-heavy |

| Design | **Normalised (3NF)** | **Denormalised (star/snowflake)** |

| Data | Current, detailed | Historical, aggregated |

Schemas: **star** (one fact table, denormalised dimensions) · **snowflake** (normalised dimensions) · **fact constellation / galaxy** (multiple fact tables sharing dimensions).

ETL vs ELT, **data mart** (a subject-specific subset), **data lake** (raw, schema-on-read), **lakehouse**, **slowly changing dimensions** (Type 1 overwrite, Type 2 new row with validity dates, Type 3 new column).

OLAP operations: **roll-up** (aggregate up a hierarchy), **drill-down**, **slice** (one dimension fixed), **dice** (sub-cube), **pivot**.

Architectures: ROLAP (relational), MOLAP (multidimensional cube), HOLAP (hybrid).

Data mining, briefly

Classification, clustering, **association rule mining** (support, confidence, lift; the **Apriori** algorithm), regression, anomaly detection. **Anomaly detection on trade data is exactly what a market surveillance system does** — connect the two in any descriptive answer.

Exam pointers

1. CAP: under a partition you choose **C** or **A**; "CA" systems are not distributed.
2. Quorum consistency requires $R + W > N$.
3. **Star schema = denormalised dimensions; snowflake = normalised.**
4. OLTP normalised, OLAP denormalised.
5. **2PC blocks if the coordinator fails** — this is the standard criticism, and why Raft/Paxos matter.

Relational Model, Keys and Relational Algebra

Core preparation lesson

Terminology

Relation (table), **tuple** (row), **attribute** (column), **domain** (allowed values), **degree** (number of attributes), **cardinality** (number of tuples). A relation is a **set** of tuples — so duplicates do not exist in the pure model, though SQL allows them.

Keys

| Key | Definition |

---|---

| **Super key** | Any set of attributes that uniquely identifies a tuple |

| **Candidate key** | A **minimal** super key (no proper subset is a super key) |

| **Primary key** | The chosen candidate key; **NOT NULL** and **unique** |

| **Alternate key** | Candidate keys not chosen as primary |

| **Composite key** | A key of more than one attribute |

| **Foreign key** | Attribute(s) referencing the primary key of another (or the same) relation |

| **Surrogate key** | System-generated identifier with no business meaning |

Integrity constraints

- **Domain** — values come from the declared domain.

- **Entity integrity** — the primary key cannot be NULL.

- **Referential integrity** — a foreign key value must either match an existing primary key or be NULL. Violations handled by **CASCADE**, **SET NULL**, **SET DEFAULT**, **RESTRICT/NO ACTION**.

- **Key constraint**, **check constraint**, **not null**, **unique**.

Relational algebra — the six basic operators

| Operator | Symbol | Meaning |

---|---|---

| Selection | σ | Choose **rows** matching a predicate |

| Projection | π | Choose **columns** (and eliminate duplicates) |

| Union | $\sqcup$ | Requires **union compatibility** (same degree, same domains) |

| Set difference | $-$ | Union compatible |

| Cartesian product | $\times$ | Every row with every row |

| Rename | ρ |

Derived operators: intersection ($A \cap B = A - (A - B)$), **join** (θ -join, equijoin, **natural join** $\bowtie$), **division** ($\div$) — used for "for all" queries, **outer joins** (left, right, full).

> **Division** is the answer to questions like "find suppliers who supply **all** parts". Recognise the pattern.

Relational calculus

Tuple relational calculus (TRC) — $\{ t \mid P(t) \}$; **Domain relational calculus (DRC)** — $\{ \langle x, y \rangle \mid P(x, y) \}$. Both are **non-procedural** (say *what*, not *how*) whereas algebra is **procedural**. All three are **equivalent in expressive power** (relationally complete). A calculus expression may be **unsafe** if it produces an infinite result.

Joins

| Join | Result |

|---|---|

| Inner / equi | Matching rows only |

| Natural | Equi-join on common attributes, with the duplicate column removed |

| Left outer | All left rows + matching right (NULLs otherwise) |

| Right outer | Mirror image |

| Full outer | All rows from both |

| Self join | A table joined to itself (employee–manager) |

| Semi / anti join | Rows of one relation that have / do not have a match |

Cardinality bounds are a favourite: for R (m tuples) and S (n tuples),

- **$R \times S$** → exactly $m \times n$ rows.

- **Natural join** → between **0** and **$m \times n$** rows, depending on the common attribute.

- **Left outer join** → at least m rows.

Exam pointers

1. Candidate key = **minimal** super key. Every candidate key is a super key; the converse is false.
2. **Projection eliminates duplicates** in relational algebra (SQL's SELECT does not, unless DISTINCT).
3. Union and set difference need **union compatibility**.
4. **Division answers "for all"** queries.
5. Algebra is procedural; calculus is declarative; both are equally expressive.

Advanced SQL — Views, Indexes, Triggers, Window Functions

Core preparation lesson

Views

A **virtual table** defined by a query. Benefits: security (expose only some columns), simplicity, logical data independence.

- **Updatable view** conditions: based on a single table, includes the primary key, no aggregates, no DISTINCT, no GROUP BY/HAVING, no set operators.
- **Materialised view** — the result is **physically stored** and refreshed (on commit, on demand, or scheduled). Used for expensive aggregations in reporting/warehouse workloads.
- `WITH CHECK OPTION` prevents updates through the view that would make the row invisible to the view.

Indexes

An index trades **write cost and storage** for **read speed**.

Type	Meaning
------	---------

---	---
-----	-----

Clustered	Determines the physical order of rows; one per table
------------------	--

Non-clustered / secondary	Separate structure with pointers; many allowed
----------------------------------	--

Dense	One index entry per record
--------------	-----------------------------------

Sparse	One entry per block/page — only possible on a clustered/ordered field
---------------	--

Composite	Multiple columns; useful only left-to-right (leftmost prefix rule)
------------------	---

Covering	Contains every column the query needs, so the table is never touched
-----------------	--

Unique, partial, function-based, bitmap (low-cardinality columns), full-text	Specialised
--	-------------

When an index is not used: a function applied to the column (`WHERE YEAR(dt) = 2024`), a leading wildcard (`LIKE '%x'`), implicit type conversion, or when the optimiser estimates that a large fraction of rows will match.

Stored procedures and functions

Procedure	Function
-----------	----------

---	---
-----	-----

Returns	Zero or more values via OUT parameters	Exactly one value
---------	--	-------------------

Called from SQL	No (CALL / EXEC)	Yes, inside a SELECT
-----------------	------------------	----------------------

DML allowed	Yes	Usually restricted
-------------	-----	--------------------

Benefits: reduced network traffic, precompiled plans, centralised business logic, permission granularity.

Triggers

Code that fires automatically on an event.

- Timing: **BEFORE, AFTER, INSTEAD OF** (for views).
- Event: INSERT, UPDATE, DELETE.
- Granularity: **row-level** (FOR EACH ROW) vs **statement-level**.
- Pseudo-tables: `'NEW'/'OLD'` (MySQL, Oracle `:NEW`, `:OLD`), `'inserted'/'deleted'` (SQL Server).

Typical uses: **audit trails**, enforcing complex constraints, maintaining derived values. Risks: hidden control flow, cascading triggers, performance.

> An **audit trigger writing to an immutable audit table** is exactly what a regulator expects in a financial system — worth mentioning in the descriptive paper.

Window (analytic) functions

Compute across a set of rows **without collapsing them** — the key difference from GROUP BY.

```
SELECT trader_id, trade_date, pnl,
       SUM(pnl)      OVER (PARTITION BY trader_id ORDER BY trade_date) AS running_pnl,
       ROW_NUMBER()  OVER (PARTITION BY trader_id ORDER BY pnl DESC)   AS rn,
       RANK()         OVER (ORDER BY pnl DESC)                         AS rnk,
       DENSE_RANK()   OVER (ORDER BY pnl DESC)                         AS drnk,
       LAG(pnl, 1)     OVER (PARTITION BY trader_id ORDER BY trade_date) AS prev_pnl,
       AVG(pnl)        OVER (PARTITION BY trader_id
                             ORDER BY trade_date
                             ROWS BETWEEN 4 PRECEDING AND CURRENT ROW) AS moving_avg_5
FROM   daily_pnl;
```

ROW_NUMBER vs RANK vs DENSE_RANK on values 100, 90, 90, 80:

- ROW_NUMBER → 1, 2, 3, 4
- RANK → 1, 2, 2, **4**
- DENSE_RANK → 1, 2, 2, **3**

CTEs (`'WITH x AS (...)'`) improve readability; **recursive CTEs** traverse hierarchies (org charts, bill of materials).

Exam pointers

1. **One clustered index per table**; sparse indexes need ordered data.
2. **Leftmost prefix rule** for composite indexes.
3. RANK leaves gaps; DENSE_RANK does not; ROW_NUMBER is always distinct.
4. INSTEAD OF triggers exist for **views**.
5. Window functions **do not reduce the number of rows**; GROUP BY does.

SQL — The Core You Must Be Able to Write

Core preparation lesson

> The descriptive paper can ask you to **write** a query. Practise on paper, not just by reading.

Sub-languages

- **DDL** — CREATE, ALTER, DROP, TRUNCATE, RENAME (auto-commit).

- **DML** — SELECT, INSERT, UPDATE, DELETE.

- **DCL** — GRANT, REVOKE.

- **TCL** — COMMIT, ROLLBACK, SAVEPOINT.

DELETE vs TRUNCATE vs DROP — the single most-asked SQL question:

| | DELETE | TRUNCATE | DROP |

|---|---|---|---|

| Type | DML | DDL | DDL |

| WHERE clause | Yes | No | No |

| Rollback | Yes | No (auto-commit) | No |

| Triggers fired | Yes | No | No |

| Structure kept | Yes | Yes | **No** |

| Speed | Slow | Fast | Fast |

Query execution order (not the written order)

FROM -> ON -> JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> LIMIT

This explains two classic rules: you **cannot use a column alias in WHERE** (SELECT runs later), and **WHERE filters rows while HAVING filters groups**.

Essential syntax

```
SELECT d.dept_name, COUNT(*) AS staff, AVG(e.salary) AS avg_sal
FROM   employee e
JOIN   department d ON e.dept_id = d.dept_id
WHERE  e.status = 'ACTIVE'
GROUP BY d.dept_name
HAVING COUNT(*) > 5
ORDER BY avg_sal DESC
LIMIT 10;
```

Aggregates: COUNT, SUM, AVG, MIN, MAX. **COUNT(*)** counts rows including NULLs; **COUNT(col)** ignores NULLs — a guaranteed exam point.

NULL handling: any arithmetic with NULL is NULL; comparisons use **IS NULL / IS NOT NULL**, never `= NULL`. `COALESCE(a, b, c)` returns the first non-null; `NULLIF(a, b)` returns NULL if `a = b`.

Subqueries

```
-- scalar
SELECT name FROM employee WHERE salary > (SELECT AVG(salary) FROM employee);

-- IN
SELECT name FROM employee WHERE dept_id IN (SELECT dept_id FROM department WHERE location =
'Mumbai');

-- correlated: runs once per outer row
SELECT e.name FROM employee e
WHERE e.salary > (SELECT AVG(x.salary) FROM employee x WHERE x.dept_id = e.dept_id);

-- EXISTS
SELECT d.dept_name FROM department d
WHERE EXISTS (SELECT 1 FROM employee e WHERE e.dept_id = d.dept_id);
```

****NOT IN vs NOT EXISTS:**** if the subquery returns any NULL, ****NOT IN** returns no rows at all**. **NOT EXISTS** is the safe choice.

The classic problems

****Nth highest salary:****

```
-- using a window function (preferred)
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employee
) t WHERE rnk = 3;

-- using a correlated subquery
SELECT DISTINCT e.salary FROM employee e
WHERE 3 = (SELECT COUNT(DISTINCT s.salary) FROM employee s WHERE s.salary >= e.salary);
```

****Find duplicates:****

```
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;
```

****Employees earning more than their manager:****

```
SELECT e.name FROM employee e JOIN employee m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Set operations

UNION (removes duplicates, sorts) vs ****UNION ALL**** (keeps duplicates, faster), **INTERSECT**, **EXCEPT/MINUS**. All require the same number of columns with compatible types.

Constraints and joins in DDL

```
CREATE TABLE trade (
    trade_id    BIGINT PRIMARY KEY,
    client_id   INT NOT NULL REFERENCES client(client_id) ON DELETE RESTRICT,
    symbol      VARCHAR(20) NOT NULL,
    qty         INT CHECK (qty > 0),
    price       DECIMAL(12,2) NOT NULL,
    traded_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_trade UNIQUE (client_id, symbol, traded_at)
);
```

Exam pointers

1. ****WHERE** filters rows, **HAVING** filters groups**; aliases are unavailable in **WHERE**.

2. **COUNT(*)** includes NULLs, **COUNT(col)** does not.
3. **NOT IN** with NULLs returns nothing — use **NOT EXISTS**.
4. **TRUNCATE** is DDL and cannot be rolled back; **DELETE** is DML and can.
5. **UNION** removes duplicates; **UNION ALL** is faster because it does not.

Transactions, ACID and Recovery

Core preparation lesson

ACID

| Property | Meaning | Ensured by |

|---|---|---|

| **Atomicity** | All or nothing | Transaction manager / recovery (undo) |

| **Consistency** | Takes the DB from one valid state to another | Application + integrity constraints |

| **Isolation** | Concurrent execution appears serial | Concurrency control |

| **Durability** | Committed changes survive crashes | Recovery manager (logs, redo) |

Transaction states

```
active -> partially committed -> committed
      |           |
      v           v
failed -> aborted (rollback)
```

Concurrency problems

| Problem | Pattern |

|---|---|

| **Dirty read** (WR conflict) | T2 reads data written by uncommitted T1; T1 then aborts |

| **Lost update** (WW) | Two writes; one overwrites the other |

| **Unrepeatable read** (RW) | T1 reads the same row twice and gets different values |

| **Phantom read** | T1 re-runs a range query and sees **new rows** inserted by T2 |

Isolation levels (SQL standard)

| Level | Dirty read | Unrepeatable read | Phantom |

|---|---|---|---|

| Read Uncommitted | Possible | Possible | Possible |

| **Read Committed** | No | Possible | Possible |

| **Repeatable Read** | No | No | Possible |

| **Serializable** | No | No | No |

Higher isolation = lower concurrency. Most databases default to Read Committed; MySQL/InnoDB defaults to Repeatable Read.

Schedules and serialisability

A **schedule** is an interleaving of operations. It is **serialisable** if it is equivalent to some serial schedule.

Conflict serialisability — two operations conflict if they are from **different transactions**, on the **same data item**, and **at least one is a write**. Build a **precedence graph**: a node per transaction, an edge $T_i \rightarrow T_j$ for each conflicting pair where T_i 's operation comes first. The schedule is **conflict serialisable** if and only if the graph is **acyclic**; topological order gives the equivalent serial schedule.

View serialisability is weaker (every conflict-serialisable schedule is view-serialisable, not conversely) and is NP-hard to test.

Recoverability:

- **Recoverable schedule** — T_j commits only after every T_i it read from has committed.
- **Cascadeless (ACA)** — a transaction reads only committed data; avoids cascading rollback.
- **Strict** — no read or write of an uncommitted written value. Strict $\square$ cascadeless $\square$ recoverable.

Recovery

Log-based recovery with **write-ahead logging (WAL)**: the log record must reach stable storage **before** the data page. Log record: `<Tn, X, old_value, new_value>`.

- **Deferred update (NO-UNDO/REDO)** — writes applied only after commit; recovery needs only redo.
- **Immediate update (UNDO/REDO)** — writes may reach disk before commit; recovery needs both.

Checkpointing limits how far back recovery must scan. **ARIES** is the standard algorithm: three phases — **Analysis, Redo, Undo** — using the LSN, dirty page table and transaction table, with **compensation log records (CLRs)** so that undo is itself recoverable.

Shadow paging is an alternative with no log, but poor concurrency and fragmentation.

Exam pointers

1. **Only Serializable prevents phantoms**; Repeatable Read does not (in the standard).
2. Conflict serialisable $\square$ **acyclic precedence graph**.
3. **WAL: log first, data later**. This is what makes durability possible.
4. Cascading rollback is avoided by **cascadeless** schedules.
5. ARIES phases in order: **Analysis, Redo, Undo**.

How to Write the IT Descriptive Paper

Supplementary reference / interview

> Phase II Paper 2 has a descriptive component, and most candidates lose marks here not from ignorance but from formless answers. This is a method note — use it every time you practise.

The structure that always works

For a 10–15 mark question, in roughly 250–300 words:

1. DEFINITION (1-2 lines) What the thing is, precisely.
2. CONTEXT (1-2 lines) Where it sits / why it exists.
3. BODY (main marks) Structured points - a table, a labelled list, or a diagram.
4. EXAMPLE (2-3 lines) A concrete instance, ideally from financial markets.
5. LIMITATION / REGULATORY ANGLE (2-3 lines)
6. CONCLUSION (1 line) One sentence that answers the question asked.

The examiner is scanning for **structure and keywords**, not prose. Headings, numbered points and a diagram earn more than a beautifully written paragraph.

Five rules

1. **Answer the verb.** "Explain" needs mechanism; "compare" needs a table; "discuss" needs both sides; "with reference to SEBI" means an example from the securities market is mandatory, not optional.
2. **Draw the diagram.** A three-way TCP handshake, an OSI stack, a DMZ layout, a Merkle tree — one diagram is worth a paragraph and takes 40 seconds.
3. **Use the correct technical vocabulary.** "Novation", "idempotent", "AIMD", "RPO/RTO", "defence in depth". Precise terms are what separate an IT specialist's answer from a generalist's.
4. **Always land the market angle.** Almost any IT topic can be tied back to markets: databases → audit trails and the Structured Digital Database; networks → co-location and latency fairness; security → CSCRf; AI → surveillance and model risk. SEBI is hiring an IT officer *for a regulator*.
5. **Time-box ruthlessly.** Allocate minutes = marks. Three-quarters of a good answer scores far more than one complete answer and one blank.

Worked skeleton

> **Q. Explain the role of a Security Operations Centre in a market infrastructure institution.**

Def: A SOC is a centralised function that monitors, detects, analyses and responds to cyber security incidents on a 24x7 basis.

Context: Under SEBI's cyber resilience requirements, MIIs and larger regulated entities are expected to have SOC coverage.

Body: People / Process / Technology

- People: L1 monitoring, L2 analysis, L3 threat hunting, incident manager
- Process: use cases, playbooks, escalation matrix, incident reporting timelines
- Technology: SIEM, EDR, NDR, threat intelligence feeds, SOAR

Metrics: MTTD, MTTR, false-positive rate

Example: Detecting anomalous after-hours access to an order management system, correlated across VPN logs, EDR alerts and database audit logs.

Limits: Alert fatigue, cost, skills shortage; a SOC detects but does not prevent - it must sit on top of hardening and segmentation.

Reg angle: Incident reporting obligations to SEBI/CERT-In; system audit; CSCRf expectations on continuous monitoring.

Conclusion: A SOC converts scattered telemetry into timely, accountable response - which is what "cyber resilience", as opposed to "cyber security", means.

Practice protocol

One question a day from the roadmap's Phase B onwards:

1. Set a timer for the marks-proportional time.
2. Write by hand or type — whatever the exam uses.
3. Then score yourself out of 10: structure (3), technical accuracy (3), example (2), market angle (1), conclusion (1).
4. Log the score in your own note on this page and watch it move.

Exam pointers

1. Structure first, content second — the marks live in the skeleton.
2. ****Every IT answer gets a market angle.**** Every one.
3. A diagram in 40 seconds beats a paragraph in three minutes.
4. Never leave a question blank; a structured half-answer still scores.
5. Practise under a clock from day one of Phase B — descriptive writing is a motor skill.

Complexity Analysis

Core preparation lesson

Asymptotic notation

| Notation | Meaning | Bound |

|---|---|---|

| $O(g)$ | f grows no faster than g | Upper |

| $\Omega(g)$ | f grows at least as fast as g | Lower |

| $\Theta(g)$ | Both | Tight |

| $o(g)$ / $\omega(g)$ | Strictly slower / faster | Strict |

Growth order: $1 < \log \log n < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n! < n^n$

Always analyse **worst, average and best** case separately. Quicksort is $O(n^2)$ worst but $\Theta(n \log n)$ average — and the average is what matters in practice.

Space complexity

Total space = input space + **auxiliary space**. When a question asks for "space complexity of merge sort", the answer usually means **auxiliary**: $O(n)$ for merge sort, $O(\log n)$ for quicksort (recursion stack), $O(1)$ for heap sort.

Recurrence relations

Master theorem for $T(n) = aT(n/b) + f(n)$, with $a \geq 1$, $b > 1$:

Let $c = \log_b(a)$. Compare $f(n)$ with n^c :

1. If $f(n) = O(n^{c-\epsilon}) \rightarrow T(n) = \Theta(n^c)$ (work dominated by leaves)
2. If $f(n) = \Theta(n^c) \rightarrow T(n) = \Theta(n^c \cdot \log n)$ (work evenly spread)
3. If $f(n) = \Omega(n^{c+\epsilon})$ **and** regularity holds $\rightarrow T(n) = \Theta(f(n))$ (root dominates)

Standard results to memorise:

| Recurrence | Solution | Example |

|---|---|---|

| $T(n) = T(n/2) + O(1)$ | $\Theta(\log n)$ | Binary search |

| $T(n) = T(n/2) + O(n)$ | $\Theta(n)$ | Quickselect (average) |

| $T(n) = 2T(n/2) + O(1)$ | $\Theta(n)$ | Tree traversal |

| $T(n) = 2T(n/2) + O(n)$ | $\Theta(n \log n)$ | Merge sort |

| $T(n) = 2T(n/2) + O(n^2)$ | $\Theta(n^2)$ | |

| $T(n) = T(n-1) + O(1)$ | $\Theta(n)$ | Linear recursion |

| $T(n) = T(n-1) + O(n)$ | $\Theta(n^2)$ | Selection/insertion sort worst |

| $T(n) = 2T(n-1) + O(1)$ | $\Theta(2^n)$ | Towers of Hanoi, naive subsets |

Other methods: **substitution** (guess and prove by induction) and the **recursion tree**.

Amortised analysis

The average cost per operation over a worst-case sequence. **Dynamic array doubling**: a single push can cost $O(n)$ when it resizes, but the **amortised cost is $O(1)$** because resizes are geometrically rare. Methods: aggregate, accounting (banker's), potential.

Complexity classes

- **P** — solvable in polynomial time.
- **NP** — verifiable in polynomial time.
- **NP-complete** — in NP and every NP problem reduces to it (SAT, 3-SAT, clique, vertex cover, Hamiltonian cycle, subset sum, TSP decision version, graph colouring).
- **NP-hard** — at least as hard as NP-complete, need not be in NP (halting problem, TSP optimisation).
- **P vs NP** is open. If any NP-complete problem is in P, then $P = NP$.

Exam pointers

1. Learn the **recurrence table** above by heart — it answers most complexity questions instantly.
2. Master theorem case 2 gives the **$n \log n$** family.
3. Amortised $O(1)$ for dynamic array append; do not confuse amortised with average.
4. **NP-complete** $\not\subseteq$ NP; NP-hard need not be.
5. When asked "space complexity", clarify whether auxiliary space is meant — and say so in a descriptive answer.

Graphs

Core preparation lesson

Representation

| | Adjacency matrix | Adjacency list |

|---|---|---|

| Space | $O(V^2)$ | $O(V + E)$ |

| Edge lookup | $O(1)$ | $O(\text{degree})$ |

| Iterate neighbours | $O(V)$ | $O(\text{degree})$ |

| Best for | Dense graphs | **Sparse graphs** (most real graphs) |

Terms: directed/undirected, weighted, **degree** (in-degree/out-degree), path, cycle, connected, **strongly connected** (directed), DAG, tree (connected acyclic, $V - 1$ edges), complete graph ($V(V-1)/2$ edges), bipartite (2-colourable $\square$ no odd cycle).

Traversals

| | BFS | DFS |

|---|---|---|

| Data structure | **Queue** | **Stack / recursion** |

| Complexity | $O(V + E)$ | $O(V + E)$ |

| Finds | **Shortest path in an unweighted graph** | Cycles, topological order, connectivity, SCC |

| Space | $O(V)$ for the queue | $O(V)$ recursion depth |

DFS edge types (directed): tree, **back** (indicates a cycle), forward, cross.

Core algorithms

| Algorithm | Problem | Complexity | Notes |

|---|---|---|---|

| **Dijkstra** | Single-source shortest path | $O((V+E) \log V)$ with a heap | **Greedy**; **no negative weights** |

| **Bellman-Ford** | Single-source shortest path | $O(V \cdot E)$ | **Handles negative weights**; detects negative cycles; DP |

| **Floyd-Warshall** | All-pairs shortest paths | $O(V^3)$ | DP; simple triple loop; handles negative edges, not negative cycles |

| **Prim** | Minimum spanning tree | $O(E \log V)$ | Greedy, grows one tree; good for dense graphs |

| **Kruskal** | Minimum spanning tree | $O(E \log E)$ | Greedy on sorted edges + **union-find**; good for sparse graphs |

| **Topological sort** | Ordering of a DAG | $O(V + E)$ | **Kahn's algorithm** (in-degrees + queue) or DFS finish times reversed |

| **Tarjan / Kosaraju** | Strongly connected components | $O(V + E)$ | Kosaraju = two DFS passes with a transposed graph |

| **Union-Find (DSU)** | Connectivity, cycle detection | $\sim O(\alpha(n))$ | With **path compression + union by rank** |

Floyd-Warshall core:

```
for k in V:
    for i in V:
        for j in V:
            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

The **k loop must be outermost** — a favourite trick question.

MST facts: an MST has $V - 1$ edges; it is unique if all edge weights are distinct; the **cut property** (the lightest edge crossing any cut is in some MST) is what makes both greedy algorithms correct.

Cycle detection

- **Undirected:** DFS finding an already-visited vertex that is not the parent, or union-find.

- **Directed:** DFS with a recursion stack (a **back edge** means a cycle), or Kahn's algorithm — if the topological sort output has fewer than V vertices, there is a cycle.

Applications worth naming in an answer

Routing (OSPF uses Dijkstra), dependency resolution (topological sort), social and ownership networks, **detecting circular trading and related-party rings in market surveillance**, deadlock detection (wait-for graph), PageRank, network flow (max-flow min-cut, Ford-Fulkerson / Edmonds-Karp).

Exam pointers

1. **Dijkstra fails with negative edges; Bellman-Ford does not.**
2. **BFS gives the shortest path only in unweighted graphs.**
3. Floyd-Warshall is $O(V^3)$ with **k outermost**.
4. MST has exactly $V - 1$ edges; Kruskal needs union-find.
5. A **back edge in DFS means a cycle** — both for directed and (with the parent check) undirected graphs.

Hashing

Core preparation lesson

Idea

Map a key to an index with a **hash function**, giving average **$O(1)$** insert, search and delete. Worst case degrades to $O(n)$ when everything collides.

Hash functions

- **Division method:** $h(k) = k \bmod m$. Choose **m prime**, and not close to a power of 2.
- **Multiplication method:** $h(k) = \lfloor m(kA \bmod 1) \rfloor$, $A \approx 0.618$ (golden ratio).
- **Universal hashing** — pick randomly from a family to defeat adversarial inputs. Relevant to **hash-collision DoS attacks**.
- **Cryptographic hashes** (SHA-256) are a different thing entirely — see the security notes.

A good hash function is **deterministic, uniform, fast, and avalanche-sensitive**.

Collision resolution

1. Separate chaining

Each bucket holds a linked list (or tree, in Java 8+ HashMap once a bucket gets long).

- **Load factor $\alpha = n/m$** can exceed 1.
- Average search: **unsuccessful $\approx \alpha$** , **successful $\approx 1 + \alpha/2$** .
- Simple deletion; extra pointer memory.

2. Open addressing

All elements live in the table itself; on collision, **probe** for another slot.

| Probing | Formula | Problem |

| --- | --- | --- |

| **Linear** | $(h(k) + i) \bmod m$ | **Primary clustering** |

| **Quadratic** | $(h(k) + c_1 i + c_2 i^2) \bmod m$ | **Secondary clustering**; may not find an empty slot unless m is prime and $\alpha < 0.5$ |

| **Double hashing** | $(h_1(k) + i \cdot h_2(k)) \bmod m$ | Best distribution; h_2 must never return 0 and should be coprime with m |

- Load factor **$\alpha < 1$** always; performance degrades sharply above ~ 0.7 , which is when **rehashing** (usually doubling m) happens.
- Average probes for unsuccessful search $\approx 1/(1 - \alpha)$; for successful $\approx (1/\alpha) \cdot \ln(1/(1 - \alpha))$.

- **Deletion needs a tombstone** marker; otherwise probe chains break. This is the most-asked open-addressing subtlety.

Applications

Dictionaries and symbol tables, database indexing, caches, **de-duplication**, set membership, **Bloom filters** (probabilistic, no false negatives but possible false positives, used for "have I seen this before" at huge scale), **consistent hashing** (distributes keys across nodes so that adding or removing a node moves only $\sim 1/n$ of the keys — the basis of distributed caches and sharded databases).

Exam pointers

1. **Separate chaining allows $\alpha > 1$; open addressing does not.**
2. Linear probing $\rightarrow$ **primary clustering**; quadratic $\rightarrow$ secondary; double hashing is best.
3. **Deletion in open addressing requires tombstones.**
4. Probes for unsuccessful search under open addressing $\approx 1/(1 - \alpha)$.
5. A **Bloom filter** can give false positives but never false negatives.

Arrays, Linked Lists, Stacks and Queues

Core preparation lesson

Arrays

Contiguous memory, **$O(1)$ random access** by index, fixed size (unless dynamic).

Address calculation — a routine exam question.

- 1-D: $\text{addr}(A[i]) = \text{base} + (i - \text{lower}) \times \text{size}$
- 2-D **row-major**: $\text{base} + ((i - \text{lr}) \times \text{total_cols} + (j - \text{lc})) \times \text{size}$
- 2-D **column-major**: $\text{base} + ((j - \text{lc}) \times \text{total_rows} + (i - \text{lr})) \times \text{size}$

| Operation | Array | Linked list |

|---|---|---|

| Access by index | **$O(1)$** | $O(n)$ |

| Search | $O(n)$ | $O(n)$ |

| Insert/delete at front | $O(n)$ | **$O(1)$** |

| Insert/delete at end | $O(1)$ amortised | $O(n)$, or $O(1)$ with a tail pointer |

| Memory | Contiguous, cache-friendly | Scattered, pointer overhead |

| Size | Fixed / amortised growth | Dynamic |

Linked lists

- **Singly** — one next pointer; cannot traverse backwards.
- **Doubly** — prev and next; deletion given only a node pointer is $O(1)$.
- **Circular** — last points to the first; used in round-robin scheduling and buffers.

Standard problems and techniques:

- **Reverse a list** — iterative with three pointers (prev, curr, next).
- **Cycle detection** — **Floyd's tortoise and hare**: slow moves 1, fast moves 2; they meet inside the cycle. To find the cycle start, reset one pointer to the head and move both one step at a time.
- **Middle element** — slow/fast pointer in one pass.
- **Nth from end** — two pointers n apart.
- **Merge two sorted lists**, **detect intersection**, **remove duplicates**.

Stack — LIFO

Operations: push, pop, peek, isEmpty — all **$O(1)$** . Overflow when full, **underflow** when popping an empty stack.

Applications:

- **Function call stack** and recursion.
- **Expression conversion and evaluation** — infix $\rightarrow$ postfix (**Shunting-yard**), postfix evaluation.
- **Balanced parentheses checking**.
- **Undo/redo**, browser back, **DFS**, backtracking.
- **Next greater element** and histogram problems, using a monotonic stack.

Postfix evaluation: scan left to right; push operands; on an operator pop **two** operands (second popped is the left operand), apply, push the result.

Infix to postfix rules: operands go straight to output; operators are pushed, but before pushing, pop all operators of **greater or equal precedence** (for left-associative operators); '(' is pushed; ')' pops until '('.

Queue — FIFO

Operations: enqueue (rear), dequeue (front) — $O(1)$.

- **Circular queue** — avoids the false-full condition of a linear array queue. With array size n , one slot is usually left empty so that 'full' and 'empty' are distinguishable: **full** when $(\text{rear} + 1) \% n == \text{front}$.
- **Deque** — insert/delete at both ends.
- **Priority queue** — served by priority, not arrival; implemented with a **heap** ($O(\log n)$ insert and extract).

Applications: CPU and disk scheduling, BFS, buffers, printer spooling, **order queues in a matching engine**.

Classic conversions: implement a queue using two stacks (amortised $O(1)$ dequeue), and a stack using two queues.

Exam pointers

1. Row-major vs column-major address calculation — read the question carefully.
2. Floyd's cycle detection: **slow 1, fast 2**, then reset one to the head.
3. Circular queue full condition: $(\text{rear} + 1) \% \text{size} == \text{front}$.
4. In postfix evaluation the **second popped operand is the left one** — this is where most mistakes happen.
5. A priority queue is not a queue: **priority, not arrival order**.

Algorithm Design Paradigms

Core preparation lesson

1. Divide and conquer

Divide → conquer (recurse) → combine. Examples: merge sort, quicksort, binary search, Strassen's matrix multiplication, closest pair of points, **Karatsuba** multiplication, FFT.

Analysis is by recurrence, usually solved with the Master theorem.

2. Greedy

Make the **locally optimal choice** at each step and never reconsider. Correct only when the problem has **greedy-choice property** and **optimal substructure**.

Works: Dijkstra, Prim, Kruskal, Huffman coding, **activity selection** (sort by finish time), fractional knapsack (sort by value/weight), job sequencing with deadlines, coin change **with canonical coin systems**.

Fails: **0/1 knapsack**, coin change with arbitrary denominations (e.g. coins {1, 3, 4} for 6: greedy gives $4+1+1 = 3$ coins, optimal is $3+3 = 2$ coins), longest path.

3. Dynamic programming

For problems with **overlapping subproblems** and **optimal substructure**. Two styles: **memoisation** (top-down recursion + cache) and **tabulation** (bottom-up table).

| Problem | Recurrence | Complexity |

|---|---|---|

| **Fibonacci** | $f(n) = f(n-1) + f(n-2)$ | $O(n)$ |

| **0/1 Knapsack** | $dp[i][w] = \max(dp[i-1][w], v[i] + dp[i-1][w-w[i]])$ | $O(nW)$ — **pseudo-polynomial** |

| **LCS** | if equal: $1 + dp[i-1][j-1]$; else $\max(dp[i-1][j], dp[i][j-1])$ | $O(mn)$ |

| **Edit distance** | $\min(\text{insert, delete, replace}) + 1$ | $O(mn)$ |

| **LIS** | $O(n^2)$ DP, or $O(n \log n)$ with patience/binary search | |

| **Matrix chain multiplication** | $dp[i][j] = \min \text{ over } k \text{ of } dp[i][k] + dp[k+1][j] + \text{cost}$ | $O(n^3)$ |

| **Coin change** | min coins / number of ways | $O(n \cdot \text{amount})$ |

| **Subset sum / partition** | boolean DP over sums | $O(n \cdot \text{sum})$ |

| **Floyd-Warshall** | all-pairs shortest paths | $O(V^3)$ |

| **Bellman-Ford** | relax all edges $V-1$ times | $O(VE)$ |

DP vs divide and conquer: DP's subproblems **overlap** and are therefore stored; D&C subproblems are disjoint and recomputed freely.

DP vs greedy: DP explores all choices and keeps the best; greedy commits immediately. Every greedy-solvable problem can be solved by DP, more slowly.

4. Backtracking

Systematic search that **prunes** branches that cannot lead to a solution: N-Queens, Sudoku, graph colouring, Hamiltonian path, subset generation, rat in a maze, word search.

5. Branch and bound

Backtracking plus a **bound** on the best achievable solution, used to prune aggressively in optimisation problems: 0/1 knapsack, TSP, job assignment.

6. Others worth naming

- **Randomised algorithms** — randomised quicksort, Monte Carlo (fast, possibly wrong) vs Las Vegas (always right, time varies).
- **Approximation algorithms** — when exact is NP-hard: vertex cover 2-approximation, TSP with triangle inequality.
- **Two pointers / sliding window** — $O(n)$ solutions for subarray and string problems.
- **Bit manipulation** — subsets via bitmasks, XOR tricks ($a \oplus a = 0$ gives the single non-repeating element), $n \& (n-1)$ clears the lowest set bit.

Exam pointers

1. **DP needs overlapping subproblems + optimal substructure.** Say both in a descriptive answer.
2. **0/1 knapsack is DP; fractional knapsack is greedy.** Know why.
3. Knapsack's $O(nW)$ is **pseudo-polynomial**, not polynomial in the input size.
4. **LIS in $O(n \log n)$** using binary search — worth remembering as the non-obvious one.
5. Greedy needs a **proof**; in a descriptive answer, give the exchange argument or the counterexample.

Sorting and Searching

Core preparation lesson

Comparison sorts

Algorithm	Best	Average	Worst	Space	Stable	In-place
	---	---	---	---	---	---
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes
Shell	$O(n \log n)$	depends on gaps	$O(n^2)$	$O(1)$	No	Yes

* Bubble sort is $O(n)$ in the best case only with the "swapped" flag optimisation.

Lower bound: any comparison-based sort needs $\Omega(n \log n)$ comparisons in the worst case (decision-tree argument: $n!$ leaves, height $\geq \log_2(n!) = \Theta(n \log n)$).

Choosing one

- **Nearly sorted data** $\rightarrow$ insertion sort (genuinely $O(n)$).
- **Guaranteed $O(n \log n)$ and stability required** $\rightarrow$ merge sort.
- **Fastest in practice, memory tight** $\rightarrow$ quicksort (with a randomised or median-of-three pivot).
- **Guaranteed $O(n \log n)$ and $O(1)$ space** $\rightarrow$ heap sort.
- **Real libraries** use hybrids: **Timsort** (merge + insertion, stable — Python, Java objects) and **Introsort** (quick $\rightarrow$ heap on deep recursion — C++ STL).

Quicksort worst case is an already-sorted array with a first/last-element pivot — fixed by randomising the pivot or using median-of-three.

Non-comparison sorts

Algorithm	Complexity	Condition
	---	---
Counting sort	$O(n + k)$	Integer keys in a small range k . Stable
Radix sort	$O(d(n + k))$	Fixed-width keys; uses a stable sort (usually counting) per digit, least significant digit first
Bucket sort	$O(n)$ average	Input uniformly distributed

These beat the $\Omega(n \log n)$ bound because they **do not compare elements**.

Stability — why it matters

A stable sort preserves the relative order of equal keys. It matters when sorting on multiple fields in sequence — sort by price, then stably by time, and you get **price-time priority**, which is exactly how an exchange order book is ordered.

Searching

- **Linear search** — $O(n)$, unsorted data.
- **Binary search** — $O(\log n)$, requires a **sorted array with random access**. Recurrence $T(n) = T(n/2) + O(1)$.
- Overflow-safe midpoint: `mid = low + (high - low) / 2`.
- Variants: first/last occurrence, lower/upper bound, search in a rotated sorted array, binary search on the answer.
- **Ternary search** — for unimodal functions; not better than binary for sorted arrays.
- **Interpolation search** — $O(\log \log n)$ for uniformly distributed data, $O(n)$ worst.
- **Exponential search** — for unbounded/infinite lists.

Exam pointers

1. **Stable**: bubble, insertion, merge, counting. Not stable: selection, quick, heap.
2. **Merge sort** needs $O(n)$ auxiliary space; **heap sort** needs $O(1)$.
3. **Quicksort** worst case $O(n^2)$ — on sorted input with a bad pivot.
4. Counting/radix/bucket beat $n \log n$ because they **do not compare**.
5. Binary search requires **sorted + random access** — it cannot be used on a linked list efficiently.

Trees, BSTs, Heaps and Balanced Trees

Core preparation lesson

Terminology

Root, parent, child, leaf, sibling, **degree**, **height** (longest path root→leaf, edges), **depth/level**, subtree, forest.

Binary tree facts:

- Maximum nodes at level $l = 2^l$ (root at level 0).
- Maximum nodes in a tree of height $h = 2^{h+1} - 1$.
- Minimum height for n nodes = $\lceil \log_2 n \rceil$.
- In a tree with n_l leaves and n_i nodes of degree 2: $n_l = n_i + 1$.
- A **full/strict** binary tree has 0 or 2 children per node; a **complete** binary tree is filled left to right; a **perfect** tree has all leaves at the same level.

Traversals

| Traversal | Order | Use |

|---|---|---|

| **Preorder** | Root, Left, Right | Copy a tree, prefix expression |

| **Inorder** | Left, Root, Right | **Sorted output of a BST** |

| **Postorder** | Left, Right, Root | Delete a tree, postfix expression |

| **Level order** | BFS using a queue | Shortest path in an unweighted tree |

Reconstruction: inorder + preorder, or inorder + postorder, uniquely determines a binary tree. **Preorder + postorder does not** (unless the tree is full).

Binary Search Tree

Invariant: left subtree < node < right subtree, for all nodes.

| Operation | Average | Worst (skewed) |

|---|---|---|

| Search / insert / delete | $O(\log n)$ | $O(n)$ |

Deletion cases: leaf → remove; one child → splice; **two children** → replace with inorder successor (smallest in the right subtree) or predecessor, then delete that node.

Number of distinct BSTs with n keys = the **n th Catalan number** = $C(2n, n)/(n+1)$. For $n = 3$ it is 5; for $n = 4$ it is 14.

Balanced trees

- **AVL tree** — for every node, **balance factor** = $\text{height}(\text{left}) - \text{height}(\text{right}) \in \{-1, 0, 1\}$. Rebalanced by **LL**, **RR**, **LR**, **RL** rotations. Strictly balanced → faster lookups, more rotations on write.
- **Red-Black tree** — looser balance ($\text{longest path} \leq 2 \times \text{shortest}$), fewer rotations, used in `std::map` and Linux's scheduler. Properties: root black, no two consecutive reds, every root-to-NULL path has the same number of black nodes.
- **B tree / B+ tree** — high branching factor for **disk-based** storage; see the DBMS indexing note.
- **Trie** — prefix tree for strings; search is $O(\text{length of key})$, independent of the number of keys. Used in autocomplete, IP routing tables, dictionary lookups.
- **Segment tree / Fenwick (BIT)** — range queries with point updates in $O(\log n)$.

Heaps

A **complete binary tree** stored in an array, satisfying the heap property: $\text{parent} \geq \text{children}$ (**max heap**) or $\text{parent} \leq \text{children}$ (**min heap**).

- Array indices (0-based): parent of $i = (i-1)/2$; children = $2i+1$ and $2i+2$.
- Insert: append at the end, **sift up** → $O(\log n)$.
- Extract root: swap with the last element, remove, **sift down** → $O(\log n)$.
- **Build heap from an unsorted array**: $O(n)$, not $O(n \log n)$ — a classic exam trap.
- **Heap sort**: build heap $O(n)$, then extract n times → $O(n \log n)$, in-place, **not stable**.

Applications: priority queues, **top-K** problems, Dijkstra and Prim, median maintenance with two heaps, **order books by price-time priority**.

Exam pointers

1. **Inorder traversal of a BST gives sorted order.**
2. Preorder + postorder does **not** uniquely determine a tree.
3. **Build-heap is $O(n)$** ; heapsort is $O(n \log n)$.
4. BST worst case is $O(n)$ — which is exactly why AVL and Red-Black trees exist.
5. Number of BSTs with n distinct keys = **Catalan(n)**.

IoT, Edge and Quantum Computing

Supplementary reference / interview

Internet of Things

Networked physical devices with sensors and actuators. Architecture: **perception layer** (sensors) → **network layer** (connectivity) → **middleware/platform** → **application layer**.

Protocols: **MQTT** (lightweight publish-subscribe, the default for constrained devices), CoAP, Zigbee, **BLE**, LoRaWAN, NB-IoT, 6LoWPAN.

Security problems: default and hard-coded credentials, no update mechanism, weak or absent encryption, physical access, and enormous scale — which is why IoT devices are the standard building block for **DDoS botnets** (the well-known Mirai-class botnets recruited devices with default passwords). Defences: device identity and certificates, secure boot, signed OTA updates, network segmentation, and lifecycle management.

Relevance to financial infrastructure: mostly indirect — building management systems, CCTV, access control and UPS systems sit on the same corporate networks as critical systems, and are a common initial access vector. That is a good, non-obvious point to make.

Edge and fog computing

Processing near the data source instead of in a distant data centre.

Why: latency, bandwidth cost, resilience when connectivity fails, and data-residency constraints. **Fog** is the intermediate layer between edge devices and cloud.

Cloud vs edge trade-off: cloud gives elastic compute and centralised management; edge gives **deterministic low latency** and local autonomy. Co-location in trading is, conceptually, edge computing for markets — compute placed as close to the event source as possible.

Quantum computing

Qubits exploit **superposition** and **entanglement**; measurement collapses the state. Quantum computers are not "faster computers" — they are faster only for specific problem structures.

| Algorithm | Effect |

|---|

| **Shor's** | Factors integers and solves discrete logs in polynomial time → **breaks RSA, DH, ECC** |

| **Grover's** | Quadratic speedup for unstructured search → **halves effective symmetric key strength** (AES-256 stays adequate; AES-128 becomes marginal) |

Current state: noisy intermediate-scale devices; error correction remains the bottleneck; no cryptographically relevant quantum computer exists today.

Why financial institutions care now — "harvest now, decrypt later": an adversary can record encrypted traffic today and decrypt it when capability arrives. Long-lived confidential data (client records, contracts) is therefore already at risk.

Response:

- **Post-quantum cryptography** — NIST-standardised algorithms: **ML-KEM (Kyber)** for key establishment, **ML-DSA (Dilithium)** and **SLH-DSA (SPHINCS+)** for signatures.
- **Crypto-agility** — design systems so that algorithms can be replaced without re-architecting. This is the practical recommendation to give.
- **QKD** — quantum key distribution; physics-based key exchange, but requires special hardware and has limited range; not a general replacement for PQC.
- **Inventory your cryptography first** — you cannot migrate what you have not catalogued.

Other technologies to recognise

AR/VR/metaverse, **digital twins**, **robotic process automation (RPA)** — heavily used for back-office reconciliation in financial firms, **5G network slicing**, **green computing** and data-centre energy efficiency.

Exam pointers

1. **MQTT is the standard IoT protocol** — lightweight publish-subscribe.
2. IoT botnets exploit **default credentials**; segmentation is the practical control.
3. **Shor breaks asymmetric crypto; Grover halves symmetric strength.**
4. **"Harvest now, decrypt later"** is why PQC migration starts before quantum computers arrive.
5. The deliverable answer is **crypto-agility plus a cryptographic inventory**, not "buy QKD".

Algorithmic and High-Frequency Trading

Supplementary reference / interview

Definitions

Algorithmic trading — any order generated using automated execution logic. **HFT** is a subset characterised by very high order rates, very short holding periods and latency sensitivity. **DMA (Direct Market Access)** lets a client's order reach the exchange through the broker's infrastructure without manual intervention.

Common strategy families

| Strategy | Idea |

|---|---|

| **Execution algos** (TWAP, VWAP, POV, implementation shortfall) | Minimise market impact of a large order |

| **Market making** | Quote both sides, earn the spread, manage inventory |

| **Statistical arbitrage / pairs** | Mean reversion between related instruments |

| **Index arbitrage / cash-futures** | Exploit basis differences |

| **Momentum / trend following** | Directional |

| **Latency arbitrage** | Exploit stale prices across venues — the controversial one |

Regulatory framework in India

SEBI's algorithmic trading framework has developed in stages. The consistent themes:

- **Approval and testing:** algorithms must be approved by the exchange; changes require re-approval; **mock/simulated environment testing** before deployment.
- **Unique identifiers** for algorithms so that every order can be traced to the algorithm that generated it.
- **Risk controls:** price checks, quantity limits, **order-per-second limits**, cumulative exposure checks, and an **automatic kill switch**.
- **Order-to-trade ratio penalties** to discourage excessive messaging.
- **Audit trail** of every algorithmic order, retained and reproducible.
- **System audit** of algorithmic trading systems.
- **Retail algo framework:** SEBI moved to bring **retail algorithmic trading and third-party algo providers** into a formal structure — broker responsibility for algos offered to clients, registration/tagging of algos, and restrictions on advertising assured returns from algos. This has been through consultation and phased implementation; **verify the current position** before quoting specifics.

Why a regulator worries

1. **Systemic risk** — a runaway algorithm can move a market in seconds. International precedents (flash crashes, a firm losing hundreds of millions in under an hour through a faulty deployment) are the standard

cautionary examples.

2. **Fairness** — unequal access to speed and data.
3. **Manipulation risk** — **spoofing** (placing orders without intent to execute), **layering**, **quote stuffing**, **momentum ignition**. These are manipulative practices under **PFUTP**, regardless of whether a human or an algorithm did them.
4. **Investor protection** — retail investors being sold "guaranteed return" algo strategies.
5. **Accountability** — if no one can say which algorithm produced an order, no one can be held responsible. Hence unique tagging.

The technology controls that actually matter

- **Pre-trade risk checks in the order path**, not as an afterthought — fat-finger limits, price collars, max order value, max position.
- **Kill switch** at multiple levels: strategy, session, member, exchange.
- **Deterministic, replayable logs** — the ability to reconstruct exactly what the algorithm saw and did.
- **Change management and version control** on trading logic, with segregation between developer and deployer.
- **Simulation environment** that faithfully mirrors production.
- **Monitoring** of message rates, rejection rates, and abnormal behaviour in real time.

> This is a question a panel could reasonably ask an IT candidate: "How would you supervise algorithmic trading?" The answer is not "ban it" — it is **tagging, pre-trade controls, kill switches, audit-trail reproducibility, and surveillance for manipulative patterns**.

Exam pointers

1. Algo orders need **exchange approval, unique tagging and an audit trail**.
2. **Spoofing and layering are PFUTP violations** — the algorithm is not a defence.
3. **Kill switch and pre-trade risk checks** are the two controls to name first.
4. **OTR penalties** discourage message flooding.
5. Retail algo regulation is recent and evolving — describe the direction, verify the details.

Business Continuity for Market Infrastructure

Supplementary reference / interview

Why the standard is higher here

For an ordinary company, an outage is a commercial loss. For an exchange or clearing corporation, an outage during market hours means **price discovery stops**, positions cannot be closed, margins cannot be met, and the effects propagate across every participant. Continuity is therefore a **regulatory obligation**, not a business decision.

The obligations (know the shape, verify the numbers)

- **Disaster Recovery site** in a different seismic zone from the primary data centre, with sufficient separation.
- Defined **RTO and RPO** for critical systems — for market systems the expectation is **intraday recovery**, measured in minutes, with near-zero data loss.
- **Periodic live DR drills**, including running live trading from the DR site, not merely a failover test.
- **BCP policy** approved by the governing board, with a designated owner.
- **Technical glitch framework**: reporting of glitches to SEBI within defined timelines, **root cause analysis** submission, and financial disincentives for MIs and large brokers where recovery timelines are missed. *Verify current parameters.*
- **Capacity planning** — systems sized for peak load with headroom, with periodic capacity assessment and stress testing against event-day volumes.
- **System audit** covering continuity arrangements.

How to design for it (a good descriptive answer)

Primary DC <--- synchronous replication ---> Near-site (same region, zero RPO)

 \
 \--- asynchronous replication ---> DR site (different seismic zone)

Controls: active-active where feasible; automated health checks and failover; sequenced, replayable message logs so state can be rebuilt deterministically; tested runbooks; out-of-band communication; participant notification protocol.

Key design ideas to name:

- **RPO near zero** requires **synchronous replication**, which constrains distance (latency); hence the near-site plus far-site pattern.
- **Deterministic replay** from a sequenced message log is how a matching engine rebuilds its book after failure — far more reliable than restoring a database snapshot.
- **Failover must be tested in production conditions**, because untested failover usually fails.
- **Participant readiness** matters: if the exchange fails over but members cannot connect to the DR environment, the drill is meaningless — hence member participation in drills.
- **Communication plan**: participants and the public must be told promptly. A silent outage is worse than a disclosed one.

Cyber-resilience overlap

Modern BCP must assume **ransomware**, not just fire and flood. That changes the design:

- Backups must be **immutable/offline**, or the attacker encrypts them too.
- DR that replicates continuously will faithfully replicate the corruption — so you need **point-in-time recovery** and known-good restore points.
- **Recovery must be practised against a scenario where the primary environment is untrusted**, requiring rebuild rather than failover.

This distinction — **disaster recovery assumes the infrastructure is intact; cyber recovery assumes it is compromised** — is a genuinely sophisticated point and very quotable.

Exam pointers

1. **DR site in a different seismic zone**; live drills, not paper tests.
2. **RTO = time to restore; RPO = tolerable data loss.** Market systems need intraday RTO and near-zero RPO.
3. Synchronous replication for near-zero RPO, asynchronous for distance.
4. **Deterministic replay of a sequenced log** is how trading state is rebuilt.
5. **Cyber recovery ≠ disaster recovery** — immutable backups and rebuild, not failover.

Investor-Facing Technology and Market Access

Supplementary reference / interview

Channels through which investors reach the market

- **Internet-Based Trading (IBT)** and mobile apps, with **two-factor authentication mandated** for client logins.
- **Securities Trading using Wireless Technology (STWT)**.
- **Call and trade**, with recorded lines.
- **Execution-only platforms** for mutual funds; **RIA/distributor** platforms.
- **Online Bond Platform Providers (OBPPs)** — SEBI brought retail bond platforms into a registration framework, addressing unregulated platforms selling debt to retail investors.

Technology-enabled investor protection

| Mechanism | What it does |

|---|---|

| **Two-factor authentication** | Mandatory for trading account access |

| **DDPI replacing PoA** | Demat Debit and Pledge Instruction — narrowly scoped authority instead of a broad power of attorney, after misuse of client securities |

| **Direct payout of securities** | Securities credited straight to the client's demat account |

| **Upstreaming of client funds** | Client money moves to the clearing corporation |

| **ASBA-like block for secondary market** | Funds stay in the client's own bank account |

| **SMS/email alerts** from exchanges and depositories on every trade and demat debit | Independent confirmation outside the broker's channel |

| **Consolidated Account Statement (CAS)** | Single view across depositories and mutual funds |

| **Nomination** mandatory or explicit opt-out | Reduces unclaimed assets |

| **Investor charters and disclosure of complaint data** | Transparency on service quality |

> Notice the pattern: several of these controls exist so that an investor receives **independent confirmation** that does not pass through the intermediary being monitored. That design principle — out-of-band verification — is worth naming.

Account Aggregator and consented data sharing

The **Account Aggregator (AA)** framework lets a user share financial data across institutions with **explicit, revocable, purpose-limited consent**, through an RBI-regulated intermediary that carries data but never stores it. Securities market participants have been joining as **Financial Information Providers/Users**. It is the clearest Indian example of **consent-based data portability**, and it aligns with the DPDP Act's consent architecture.

Digital fraud targeting investors

The realistic threat list: fake trading apps and cloned websites, **social media tip groups** and unregistered advisers promising assured returns, **deepfake videos** of known market figures, **dabba trading** platforms, fraudulent "IPO allotment" and "refund" messages, remote-access-tool scams, and mule accounts used to launder proceeds.

Countermeasures: registration verification tools, SEBI investor alerts and caution lists, mandatory risk disclosures, takedown mechanisms, platform accountability, and investor education. A structural point worth making: **detection at the platform layer scales better than case-by-case enforcement** — which is why regulators press intermediaries and social platforms rather than chasing individual fraudsters alone.

Accessibility and inclusion

Regional language interfaces, simplified onboarding, low-ticket products (SIPs from ₹100, fractional access debates), and the risk that easy access without understanding pushes retail investors into **derivatives speculation** — which is exactly the harm SEBI's recent disclosures about F&O losses were aimed at.

Exam pointers

1. **2FA is mandatory** for internet-based trading logins.
2. **DDPI** replaced the broad PoA model after misuse of client securities.
3. Exchange/depository **SMS and email alerts are deliberately out-of-band**.
4. **OBPPs** brought retail bond platforms under registration.
5. **Account Aggregator = consented, revocable, purpose-limited data sharing** — a strong India Stack example.

KYC Infrastructure in the Securities Market

Supplementary reference / interview

Why KYC matters technically

Every regulatory outcome — surveillance, connectedness analysis in insider trading, grievance redress, settlement of claims — depends on knowing **who** is behind an account. KYC is therefore an identity-infrastructure problem, not a paperwork problem.

The players

| Entity | Role |

|---|---|

| **KRA — KYC Registration Agency** | SEBI-registered; holds KYC records centrally so that a client verified once can be onboarded by any intermediary |

| **CKYCR (CERSAI)** | **Central KYC Records Registry** across the whole financial sector — banking, insurance, securities |

| **Intermediary** | Does the verification and uploads the record |

| **UIDAI** | Aadhaar authentication and e-KYC |

| **Depositories/DPs** | Demat account opening, linked to the same KYC |

Principle: verify once, reuse across intermediaries. This is the securities-market instance of the India Stack pattern.

Modes of verification

- **Aadhaar-based e-KYC** — OTP or biometric, with the constraints set by the Supreme Court's Aadhaar judgment and subsequent rules; only entities permitted to perform Aadhaar authentication may do so, and Aadhaar numbers must be stored in a compliant manner (**Aadhaar Vault**, virtual IDs, masking).
- **Video-based Customer Identification Process (V-CIP / IPV)** — live video verification with liveness checks, geotagging and recording.
- **Digital KYC** with OVD upload and verification.
- **In-person verification (IPV)** — the traditional route.
- **DigiLocker**-sourced documents.

PAN remains the unique identifier for securities market transactions, with **PAN-Aadhaar linkage** requirements. **UCC (Unique Client Code)** ties trades to a client at the exchange level. **Nomination** is mandatory or requires an explicit opt-out.

Risk-based approach under PMLA

Every intermediary is a **reporting entity** under the **PMLA, 2002**:

- **Customer Due Diligence (CDD)** at onboarding; **Enhanced Due Diligence** for higher-risk clients; **simplified** for low risk.
- **Beneficial ownership** identification for non-individual clients — the threshold-based test for who ultimately controls an entity.
- **PEP** (politically exposed person) screening, **sanctions screening** (UN lists), adverse media screening.
- **Ongoing monitoring** of transactions against the client's profile.
- **STR / CTR reporting to FIU-IND**; **record retention for five years**.
- **Designated Director and Principal Officer** appointments.

Technology issues worth discussing

- **Identity resolution and deduplication** — the same person with variations in name and address across records.
- **Fuzzy matching** for sanctions and PEP screening, and the **false positive** burden it creates.
- **Data minimisation and storage** obligations under **DPDP** versus retention obligations under **PMLA** — a genuine tension worth naming.
- **Deepfakes and injection attacks against V-CIP** — liveness detection is now an adversarial problem.
- **Security of KYC repositories** — a KRA or CKYCR breach would be a national-scale PII incident, which is exactly why they fall under the cyber framework.
- **Interoperability** — the reason a central registry beats every intermediary holding its own copy.

Exam pointers

1. **KRA** is securities-market specific; **CKYCR (CERSAI)** is sector-wide.
2. **PAN** is the unique identifier; **UCC** ties trades to the client at the exchange.
3. **PMLA**: **STR/CTR to FIU-IND**, records for **five years**, **Designated Director + Principal Officer**.
4. **Beneficial ownership identification** is the heart of AML for non-individual clients.
5. Name the **DPDP vs PMLA retention tension** — it shows you think like a regulator, not a checklist.

Market Infrastructure Institutions and Their Technology

Supplementary reference / interview

> This note is the bridge between your degree and the job. Most IT candidates can explain a load balancer; very few can explain what a clearing corporation does and why its uptime is a public-interest question.

What an MII is

Market Infrastructure Institutions = **stock exchanges + clearing corporations + depositories**. They are systemically important: a failure in any of them is not a company problem, it is a **market** problem. Hence SEBI regulates their **ownership, governance, technology and resilience** more tightly than any other entity — through the **SECC Regulations, 2018**, the Depositories and Participants Regulations, and the cyber framework.

What each one actually runs

| MII | Core system | The technology problem it solves |

|---|---|---|

| **Stock exchange** | **Order matching engine**, market data dissemination, surveillance, risk management | Deterministic, low-latency matching at very high message rates, with fair access |

| **Clearing corporation** | Clearing and settlement engine, **risk and margin computation**, collateral management | Computing exposure and margins in near real time; acting as CCP through **novation** |

| **Depository** | Beneficial owner database, **book-entry transfer**, corporate actions | An extremely high-integrity ledger with total auditability |

Architecture of an exchange (how to describe it)

```
Members / brokers
| (leased lines, co-location, internet)
Trading Access Point / gateway -> authentication, risk checks, order validation
|
ORDER MATCHING ENGINE -> price-time priority order book, in-memory, deterministic
|
| \
|  -> Market data feed (broadcast/multicast: tick-by-tick, snapshots, depth)
|
Trade file -> Clearing Corporation -> obligations, margins
|
|      Depository (securities leg) + Clearing banks (funds leg)
|
Surveillance systems, audit trail, regulatory reporting
```

Design characteristics you should be able to name:

- **Determinism over raw speed** — matching must be reproducible and auditable; two identical inputs must produce the same book.
- **In-memory order book**, with persistence for recovery.
- **Price-time priority** — the reason a **stable** sort matters (see the sorting note).
- **Sequenced, replayable message streams** — the standard way to rebuild state after failure.
- **Multicast for market data** — one publisher, thousands of subscribers, so UDP multicast rather than per-client TCP.

- **Capacity headroom** — sized for peak (event-day) volumes, not average.
- **Hot DR** in a different seismic zone, with drills.

Governance requirements worth knowing

- MIs must have **public interest directors**; the **Bimal Jalan Committee (2010)** shaped ownership and governance norms.
- Key managerial personnel, and specifically the roles responsible for **regulatory, compliance and technology** functions, have defined obligations.
- MIs run **investor protection funds**, **core SGF**, and must maintain a minimum net worth.
- **System audit** by independent auditors, on SEBI's terms of reference, with findings reported to SEBI and the governing board.
- **Business continuity**: defined **RTO/RPO**, intraday recovery expectations, periodic **live DR drills**.
- **Technical glitch framework** — SEBI has prescribed reporting, root cause analysis and financial disincentives for major technical glitches at MIs and large brokers, with defined recovery timelines. *Know this exists; verify the current parameters.*

Why this matters for your interview

If asked "why SEBI for an IT person", the strongest available answer is here: the technology of MIs is **critical national infrastructure operating under a public-interest mandate**, and the regulator's job is to supervise that infrastructure — which needs people who genuinely understand distributed systems, latency, capacity and resilience, not just policy.

Exam pointers

1. **MII = exchange + clearing corporation + depository.** Governed largely by the **SECC Regulations, 2018**.
2. **Novation** is what makes a clearing corporation a CCP.
3. Matching engines are **deterministic and in-memory**, with **price-time priority**.
4. Market data is **multicast**; order entry is **TCP**.
5. **Bimal Jalan Committee** = MI ownership and governance.

Payment Rails Behind the Securities Market

Supplementary reference / interview

Why an IT officer should know this

Every trade has a **“funds leg”** and a **“securities leg”**. The securities leg runs through depositories; the funds leg runs through the banking system. Most settlement failures and most investor complaints about money are, underneath, payment-system problems.

The rails

| Rail | Operator | Characteristic |

|---|---|---|

| **RTGS** | RBI | **Real-time gross settlement**, high value (₹2 lakh minimum), **24×7** since December 2020, irrevocable |

| **NEFT** | RBI | Deferred net settlement in half-hourly batches, **24×7** since December 2019, no minimum |

| **IMPS** | NPCI | Instant, 24×7, retail, mobile-first |

| **UPI** | NPCI | Instant, interoperable, VPA-addressed, 24×7; underpins IPO applications and the block mechanism |

| **NACH** | NPCI | Bulk mandates — SIP debits, dividends, refunds |

| **AePS / BBPS / FASTag / RuPay** | NPCI | Aadhaar-enabled, bill payments, tolls, cards |

Gross vs net settlement is a key distinction: RTGS settles each transaction individually (no netting, no settlement risk, high liquidity need); NEFT nets within a batch (efficient, small residual risk).

Where they touch the securities market

- **IPO applications:** **ASBA** blocks funds in the investor's own bank account; the **UPI mandate** is the retail block mechanism, routed through a **Sponsor Bank** to the **SCSB**, with the **Registrar** reconciling.
- **Secondary market block mechanism (ASBA-like):** funds stay in the client's bank account, blocked in favour of the clearing corporation, debited only on settlement — reducing the amount of client money sitting with brokers.
- **Upstreaming of client funds:** brokers must move client funds to clearing corporations rather than retaining them, in specified instruments. This is a direct response to broker-default episodes.
- **Mutual fund SIPs** run on **NACH mandates** and increasingly **UPI Autopay**.
- **Dividends, redemptions and refunds** are paid electronically to the registered bank account.
- **Payouts:** direct payout of securities to client demat accounts, and funds to the client's registered account.

CBDC (e-Rupee)

RBI's **central bank digital currency**, piloted in **wholesale** (settlement of secondary-market G-sec transactions) and **retail** (token-based wallets) forms since 2022. Relevance to securities markets: **atomic**

delivery-versus-payment (DvP)** — settling the securities leg and the funds leg simultaneously and irrevocably, which removes principal risk. Being able to explain **DvP and why atomic settlement matters** is a strong, specific answer.

Risk vocabulary

Settlement risk, **Herstatt risk** (time-zone settlement risk in FX), **principal risk**, **liquidity risk**, **DvP models (DvP-1 gross/gross, DvP-2 gross securities/net funds, DvP-3 net/net)** , **finality of settlement** , **payment versus payment**.

Exam pointers

1. **RTGS = gross, high value, real-time; NEFT = net, batched.** Both 24×7 now.
2. **ASBA blocks funds in the investor's own account** ; UPI is the retail block mechanism.
3. **Upstreaming** keeps client money out of broker accounts.
4. **DvP = simultaneous exchange of securities and funds** ; CBDC enables atomic settlement.
5. NPCI runs UPI/IMPS/NACH; **RBI runs RTGS and NEFT**.

RegTech, SupTech and SEBI's Digital Initiatives

Supplementary reference / interview

Definitions

- **FinTech** — technology applied to financial services.
- **RegTech** — technology used by **regulated entities** to meet compliance obligations more cheaply and reliably: automated reporting, KYC/AML screening, transaction monitoring, trade surveillance, policy-as-code.
- **SupTech** — technology used by the **supervisor** itself: data collection, analytics, risk-based supervision, machine-readable regulation.

The distinction is worth stating precisely, because it is exactly the axis on which a SEBI IT officer works: you are on the **SupTech** side, supervising entities that rely on **RegTech**.

Why regulators are moving this way

Supervisory capacity does not scale with market growth. Manual inspections cover a fraction of entities; periodic reporting is stale by the time it arrives. The response is **continuous, data-driven, risk-based supervision** — collect structured data directly, run analytics, and target inspections where the risk signals are.

SEBI's digital platforms to know by name

| Platform | Purpose |

|---|---|

| **SCORES / SCORES 2.0** | Investor grievance redress, with auto-routing and escalation |

| **SMART ODR portal** | Online dispute resolution — conciliation then arbitration |

| **SEBI Intermediary (SI) Portal** | Registration and compliance filings by intermediaries |

| **Integrated surveillance systems** | Cross-market alert generation |

| **Saaṛthi app / investor website** | Investor education |

| **e-filing and XBRL-based disclosure** by listed entities | Structured, machine-readable filings |

| **KYC infrastructure (KRA / CKYC)** | Shared identity verification |

XBRL deserves a mention of its own: structured, tagged financial reporting makes disclosure **machine-readable**, which is the precondition for automated analysis of thousands of filings.

The technology agenda for a regulator

1. **Data infrastructure** — a supervisory data warehouse or lakehouse, standardised schemas, identity resolution across datasets.
2. **Automated validation** at the point of submission rather than manual scrutiny afterwards.
3. **Risk scoring of entities** to prioritise inspections.
4. **Analytics and ML** for surveillance and complaint triage.

5. **APIs** for machine-to-machine reporting instead of file uploads.
6. **Machine-readable regulation** — the ambitious end-state, where a rule can be consumed and applied by systems directly.
7. **The regulator's own cyber resilience** — a regulator holding market-wide data is itself a target.

Related public digital infrastructure

Account Aggregator framework (consented financial data sharing, RBI-regulated, with securities market participation), **DigiLocker** for documents, **UPI** as a payment rail inside the securities market, **ONDC**-style open network thinking, and the **India Stack** design pattern of consent-based, interoperable, API-first public infrastructure. These come up constantly in interviews because they are the government's flagship technology narrative.

Risks to acknowledge

Model risk and bias, over-reliance on vendors, concentration risk when many entities use the same RegTech provider, data quality garbage-in-garbage-out, the privacy implications of granular supervisory data under **DPDP**, and the danger of **automation without accountability** — a supervisory decision must always be attributable to a person.

Exam pointers

1. **RegTech** = compliance side; **SupTech** = supervisor side. State the difference crisply.
2. **XBRL** makes filings machine-readable — the precondition for supervisory analytics.
3. **SCORES**, **SMART ODR**, **SI Portal** — know the platform names.
4. Risk-based supervision means **targeting scarce inspection capacity using data**.
5. Always pair the opportunity with a **risk** (privacy, model risk, vendor concentration) — panels reward balance.

Market Surveillance and Data Analytics

Supplementary reference / interview

> If you want one answer ready for "how would you use your IT skills at SEBI", make it this one.

The surveillance stack

Exchanges (first-level surveillance)

- real-time alerts, price/volume monitoring, ASM/GSM, trade-for-trade

Clearing corporations and depositories

- settlement, position and holding data

SEBI (second-level surveillance)

- integrated surveillance across exchanges and segments
- data warehouse, pattern analytics, alert triage, referral to investigation

Multi-exchange and cross-segment surveillance matters because a manipulation can be split across venues and between cash and derivatives to stay under each venue's radar.

Data sources

Order and trade data (with timestamps), **order book snapshots**, client/UCC data, **demat holdings and transfers**, bank flows where available, KYC and beneficial ownership, corporate announcements and filings, **social media and messaging channels**, news feeds.

Patterns that alerts are built for

| Pattern | Signature |

|---|---|

| **Circular / wash trading** | Trades among a closed set of connected accounts with no real change in beneficial ownership — a **graph problem** |

| **Pump and dump** | Coordinated promotion, price and volume spike, promoter/operator exit |

| **Front running** | A broker or connected account trading ahead of a large client order — requires correlating order arrival with account activity |

| **Insider trading** | Trading in a window before a price-sensitive announcement by connected persons — correlate UPSI dates with trading records |

| **Spoofing / layering** | Large orders placed and cancelled near touch to move the price; high **order-to-cancel** ratio |

| **Synchronised trades** | Matching buy and sell orders entered within milliseconds at identical prices and quantities |

| **Price manipulation at close** | Concentrated activity in the closing session to set a reference price |

Analytical techniques

- **Rule-based alerts** — thresholds and patterns. Transparent and explainable, but rigid and easy to game.
- **Statistical anomaly detection** — z-scores, seasonality-adjusted baselines, peer-group comparisons.

- **Machine learning** — unsupervised clustering and outlier detection (isolation forests, autoencoders) for unknown patterns; supervised models trained on confirmed cases, which are scarce — a **class-imbalance** problem.
- **Graph/network analytics** — the single best fit for market abuse: accounts as nodes, trades and fund transfers as edges, then look for **communities, cycles and unusual centrality**. Common beneficial ownership, shared addresses, shared IP or device fingerprints, and fund-flow chains all become edges.
- **NLP** — parsing filings, announcements and social media for manipulation signals and for **rumour verification** requirements.
- **Link analysis with KYC/CKYC data** to establish connectedness — which is the legal crux of most insider trading and PFUTP cases.

The hard parts (say these in an interview — they show judgement)

1. **False positives.** Alert volume is the binding constraint; an analyst's time is the scarce resource. Precision matters more than recall at the alert stage.
2. **Explainability.** An enforcement order must survive appeal at SAT. A model that cannot explain *why* it flagged something is evidence-weak — so ML is best used for **triage and lead generation**, with rule-based and human analysis for the case itself.
3. **Data quality and identity resolution.** Everything depends on knowing that two accounts are the same person or connected persons.
4. **Adversarial adaptation.** Manipulators change behaviour once they learn the thresholds; static rules decay.
5. **Privacy and proportionality.** Surveillance of financial behaviour engages the DPDP Act and constitutional privacy considerations; powers must be exercised within the statute.

Related SEBI initiatives

Integrated surveillance systems, **data warehousing and analytics projects**, the use of **AI/ML for alert generation**, social-media monitoring for unregistered advisory and manipulation, and the **rumour verification** requirement under LODR for large listed entities.

Exam pointers

1. **Two-level surveillance:** exchange first, SEBI second.
2. **Circular trading is a graph problem**; say so, and explain why.
3. ML is best for **triage**; enforcement needs **explainable** evidence.
4. **False positive rate** is the practical constraint on any surveillance system.
5. Manipulative practices are actionable under **PFUTP**, and insider trading under **PIT + S.15G**.

Trading Systems, Latency and Co-location

Supplementary reference / interview

The order lifecycle

Client -> broker's OMS/RMS -> risk checks (margin, limits, price band)
-> exchange gateway -> matching engine -> order book
-> execution -> trade confirmation -> clearing -> settlement (T+1)

OMS (order management) and **RMS** (risk management) sit at the broker. SEBI requires **broker-level risk checks before an order reaches the exchange** — margin availability, price bands, quantity limits, and order-per-second throttles.

Matching engine mechanics

- **Continuous double auction** with **price-time priority**: better price wins; at the same price, the earlier order wins.
- **Order types**: market, limit, stop-loss, IOC, disclosed quantity, iceberg, GTC/GTD, auction orders.
- **Pre-open session** uses **call auction** price discovery: orders collected, then an equilibrium price computed that maximises executable volume.
- **Price bands and circuit filters** are enforced at the engine level.
- **Order-to-trade ratio (OTR)** penalties discourage excessive order flooding.

Latency

End-to-end latency = network + gateway + risk checks + matching + response. Measured in microseconds at the exchange level.

Techniques used by low-latency participants: **kernel bypass** (DPDK, Solarflare/onload), **FPGA**-based order handling, busy-polling, NUMA pinning, pre-allocated memory, **binary protocols** rather than text, and co-location.

Jitter (variance in latency) often matters more than mean latency, because strategies must be predictable.

Co-location

Exchanges offer rack space in their own data centre so that a member's servers sit metres from the matching engine. SEBI's concerns are **fairness and equal access**, not speed itself:

- **Equal-length cabling** within the co-location facility so no rack is physically advantaged.
- **Randomisation** of order entry in some contexts, and **tick-by-tick data** access arrangements.
- A **level-playing-field** requirement between co-located and non-co-located participants, including **managed co-location services through third parties** so that smaller members can access it.
- Transparent, non-discriminatory **pricing and allocation** of co-location facilities.
- The well-known co-location matter of the past decade — where preferential access to a tick-by-tick data dissemination architecture became an enforcement issue — is the reason this is regulated so specifically.

Understand the *principle* (equal access to the same information at the same time); avoid asserting case specifics you have not verified.

Market data

- **Tick-by-tick (TBT)** feeds vs **snapshot** feeds; **market depth** (best 5/20 levels).
- Dissemination architecture matters: a **broadcast/multicast** model treats all subscribers equally, whereas a sequential **TCP push** to subscribers in order creates an advantage for whoever is served first. That distinction is the technical heart of the fairness debate.
- **Time stamping and clock synchronisation** (PTP/NTP to a national reference) so that order and trade times are comparable across systems — the foundation of any audit trail or surveillance replay.

Connectivity

Leased lines and **MPLS** to the exchange, **VSAT** historically for remote locations, internet-based trading (IBT) and **securities trading using wireless technology (STWT)** for retail, with **two-factor authentication** mandated for client logins.

Exam pointers

1. **Price-time priority** is the matching rule; the pre-open uses a **call auction**.
2. Broker-level **risk checks are mandatory before order entry** to the exchange.
3. Co-location regulation is about **fair and equal access**, not banning speed.
4. **Multicast dissemination is inherently fairer** than sequential unicast push.
5. **Clock synchronisation** underpins audit trails, surveillance and glitch investigation.

Deadlock

Supplementary reference / interview

The four necessary conditions (Coffman)

All four must hold simultaneously:

1. **Mutual exclusion** — at least one resource is non-shareable.
2. **Hold and wait** — a process holding a resource waits for another.
3. **No preemption** — resources cannot be forcibly taken.
4. **Circular wait** — a cycle of processes each waiting for the next.

The four handling strategies

| Strategy | Idea | Cost |

|---|---|---|

| **Prevention** | Structurally negate one of the four conditions | Low resource utilisation |

| **Avoidance** | Grant a request only if the system stays in a **safe state** (Banker's algorithm) | Needs maximum-need declared in advance |

| **Detection and recovery** | Let deadlock happen, detect with a wait-for graph, then recover | Recovery is disruptive |

| **Ignore (Ostrich algorithm)** | Assume it is rare; reboot if it happens | What Unix/Windows actually do |

Prevention in practice

- Kill **hold and wait**: request all resources at once (poor utilisation) or release all before requesting more.
- Kill **no preemption**: preempt resources from a waiting process.
- Kill **circular wait**: impose a **total ordering** on resource types and require requests in increasing order. This is the practical one.

Safe state and Banker's algorithm

A state is **safe** if there is a sequence in which every process can obtain its maximum need and finish. **Safe**
□ no deadlock. Unsafe $\neq$ deadlock, but deadlock is possible.

Data structures: **Available[m]**, **Max[n][m]**, **Allocation[n][m]**, **Need[n][m] = Max - Allocation**.

Safety algorithm: find a process whose **Need $\leq$ Available**; pretend it finishes, add its Allocation back to Available (Work), mark it finished; repeat. If all processes finish, the state is safe and the order found is the **safe sequence**.

Resource-Request algorithm: check Request $\leq$ Need, Request $\leq$ Available, then *pretend* to allocate and run the safety check. If the resulting state is safe, grant it; otherwise the process waits.

Detection

- **Single instance per resource type:** build a **wait-for graph** (collapse the resource nodes); a **cycle** means **deadlock**.
- **Multiple instances:** use a detection algorithm similar to the Banker's safety check.
- In a **resource allocation graph** with multiple instances, a cycle is **necessary but not sufficient** for **deadlock**.

Recovery

- **Process termination** — abort all deadlocked processes, or abort one at a time until the cycle breaks (choose by priority, time run, resources held).
- **Resource preemption** — select a victim, **rollback** to a safe state, and prevent **starvation** by including the number of rollbacks in the victim selection.

Related concepts

Starvation — indefinite postponement without deadlock. **Livelock** — processes keep changing state in response to each other but make no progress. **Priority inversion** — a low-priority process holds a lock needed by a high-priority one; solved by **priority inheritance**.

Exam pointers

1. All **four** Coffman conditions must hold; negating any one prevents deadlock.
2. **Safe** $\square$ no deadlock; **unsafe** does not imply deadlock.
3. Cycle in a single-instance RAG $\square$ deadlock. Cycle in a multi-instance RAG $\square$ **maybe**.
4. Banker's algorithm is **avoidance**, not prevention or detection.
5. Practical prevention = **ordering resources**; practical handling in real OSes = **ignore**.

File Systems and Disk Scheduling

Supplementary reference / interview

File concepts

A **file** is a named collection of related information. Attributes: name, identifier, type, location, size, protection, timestamps.

Access methods: sequential, direct/relative, indexed.

Directory structures: single-level, two-level, tree, **acyclic graph** (sharing via links), general graph (needs garbage collection to handle cycles).

Links: a **hard link** is another directory entry pointing to the same inode (same file, reference counted, cannot cross file systems); a **soft/symbolic link** stores a path (can dangle, can cross file systems).

Allocation methods

| Method | How | Pros | Cons |

|---|---|---|---|

| **Contiguous** | Consecutive blocks | Fast sequential and direct access | External fragmentation; file growth problem |

| **Linked** | Each block points to the next | No external fragmentation | **No direct access**; pointer overhead; reliability risk. **FAT** is a variant with the links in a table |

| **Indexed** | An index block holds all block pointers | Direct access, no external fragmentation | Index block overhead; needs multi-level/linked index for large files |

Unix inode: direct blocks + **single indirect** + **double indirect** + **triple indirect**. A standard numerical asks for the **maximum file size** given block size and pointer size:

> Block size 4 KB, pointer 4 bytes → 1024 pointers per block.

> Max size = 12 direct + 1024 single + 1024² double + 1024³ triple, all × 4 KB.

Free space management: bit vector (bitmap), linked list, grouping, counting.

Disk structure and performance

Access time = seek time + rotational latency + transfer time. Seek time dominates, which is why scheduling exists. Average rotational latency = **half a rotation** (at 7200 rpm, one rotation = 8.33 ms, so ≈ 4.17 ms).

Disk scheduling algorithms

Given a request queue and a current head position:

| Algorithm | Movement |

|---|---|

| **FCFS** | In arrival order — fair, potentially terrible |

| **SSTF** | Shortest seek time first — can **starve** far requests |

| **SCAN (elevator)** | Sweep to one end servicing requests, then reverse |

| **C-SCAN** | Sweep in one direction only, then jump back without servicing — **more uniform waiting time** |

| **LOOK / C-LOOK** | Like SCAN/C-SCAN but reverse at the **last request**, not at the end of the disk |

Always draw the head-movement diagram and **sum the absolute differences** to get total head movement.

RAID

| Level | Technique | Notes |

---|---|---

| **RAID 0** | Striping | Performance, **no redundancy** |

| **RAID 1** | Mirroring | 50% capacity, best read performance |

| **RAID 2/3/4** | Bit/byte/block-level with a dedicated parity disk | Parity disk is a bottleneck |

| **RAID 5** | Block-level striping with **distributed parity** | Survives 1 disk failure; usable capacity $(n-1)/n$ |

| **RAID 6** | Dual distributed parity | Survives **2** failures |

| **RAID 10** | Mirror then stripe | High performance and redundancy; expensive |

File systems to recognise

FAT32, NTFS (journaling, ACLs), ext2/3/4 (ext3+ journaling), XFS, Btrfs, ZFS (copy-on-write, checksums), NFS (network), and **journaling** as the general technique for crash consistency.

Exam pointers

1. **Linked allocation cannot do direct access.**
2. Learn the **inode maximum file size** calculation — it is asked constantly.
3. **C-SCAN gives more uniform wait times** than SCAN.
4. RAID 5 survives **one** failure, RAID 6 **two**. RAID 0 survives none.
5. Average rotational latency is **half** a rotation.

Operating Systems — Fundamentals

Supplementary reference / interview

What an OS is

A program that acts as an **intermediary between the user and the hardware**, managing resources (CPU, memory, I/O, files) and providing an execution environment. Two views: **resource manager** and **extended/virtual machine**.

Types of OS

| Type | Characteristic |

|---|---|

| Batch | Jobs grouped; no interaction; poor CPU utilisation |

| Multiprogramming | Several jobs in memory; CPU switches on I/O wait |

| Multitasking / time-sharing | CPU time sliced among users; response time matters |

| Multiprocessing | More than one CPU; symmetric (SMP) or asymmetric |

| Real-time | Deadline-bound. **Hard RTOS** — missing a deadline is a failure; **soft RTOS** — degraded quality |

| Distributed | Independent machines presented as one system |

| Network | Each machine has its own OS; sharing over a network |

| Embedded | Fixed function, small footprint |

Kernel

The always-resident core. Architectures:

- **Monolithic** — all services in kernel space; fast, but a fault anywhere can crash the system (classic Unix, Linux).

- **Microkernel** — minimal kernel (IPC, scheduling, basic memory); drivers and file systems in user space; more reliable, more context-switch overhead (Minix, QNX).

- **Hybrid** — Windows NT, macOS.

- **Exokernel**, **nanokernel** — research/lightweight variants.

Dual mode operation

The CPU runs in **user mode** or **kernel (supervisor) mode**, distinguished by a **mode bit**. Privileged instructions (I/O, halt, setting timers, modifying page tables) execute **only in kernel mode**. This is the hardware support that makes protection possible.

System calls

The controlled entry point from user mode to kernel mode, triggered by a **software interrupt / trap**.

| Category | Examples (Unix) |

|---|---|

| Process control | fork, exec, exit, wait |

| File management | open, read, write, close, lseek |

| Device management | ioctl, read, write |

| Information | getpid, alarm, sleep |

| Communication | pipe, shmget, mmap, socket |

| Protection | chmod, umask, chown |

fork() returns **0** in the child and the **child's PID** in the parent — a classic output question. `fork(); fork(); fork();` creates $2^3 - 1 = 7$ child processes.

Interrupts vs traps

- **Interrupt** — asynchronous, hardware-generated (I/O completion, timer).
- **Trap / exception** — synchronous, software-generated (system call, divide by zero, page fault).

Both transfer control through the **interrupt vector table** to a handler.

Booting

Power on → **BIOS/UEFI** POST → bootloader (**GRUB**, Windows Boot Manager) → kernel loaded into memory → init/systemd → user space.

Exam pointers

1. Mode bit: **0** = kernel, **1** = user in most textbook conventions.
2. A system call is a **trap**, not a hardware interrupt.
3. Microkernel = more reliable, more overhead; monolithic = faster, less isolated.
4. `fork()` called n times in sequence creates $2^n - 1$ children.
5. Hard real-time is about **guaranteed deadlines**, not about being fast.

I/O Management, Linux Basics and Commands

Supplementary reference / interview

I/O hardware and methods

| Method | How the CPU is involved |

|---|---|

| **Programmed I/O (polling)** | CPU busy-waits on a status bit — wastes cycles |

| **Interrupt-driven I/O** | Device raises an interrupt when ready; CPU handles it |

| **DMA** | A **DMA controller** transfers directly between device and memory; the CPU is interrupted **only at completion** — best for bulk transfers |

Cycle stealing — the DMA controller takes the memory bus for a cycle while the CPU is not using it.

Device categories: block (disks), character (keyboards, serial), network. **Device drivers** present a uniform interface to the kernel. **Spooling** (e.g. printing) and **buffering / caching** smooth speed mismatches.

Linux essentials for the exam

Filesystem hierarchy: `/bin` `/sbin` `/etc` (config) `/var` (logs) `/tmp` `/home` `/usr` `/proc` (kernel/process info as files) `/dev` (device files) `/opt`.

Permissions: `rwX` for user, group, other. Numeric: `r=4`, `w=2`, `x=1`.

- `chmod 755 file` → `rwXr-Xr-X`

- `chmod 644 file` → `rw-r--r--`

- `umask` subtracts from the default (666 for files, 777 for directories).

- Special bits: **SUID (4)**, **SGID (2)**, **sticky bit (1)** — the sticky bit on `/tmp` stops users deleting each other's files.

Commands worth memorising:

| Command | Use |

|---|---|

| `ls -la`, `cd`, `pwd`, `mkdir -p`, `rm -rf`, `cp -r`, `mv` | Navigation and files |

| `cat`, `less`, `head -n`, `tail -f` | Viewing (tail -f for live logs) |

| `grep -r "x" .`, `find / -name f`, `awk`, `sed -i 's/a/b/g'` | Search and transform |

| `ps aux`, `top`, `htop`, `kill -9 PID`, `nice`, `jobs`, `bg`, `fg` | Processes |

| `df -h`, `du -sh`, `free -m`, `mount`, `lsdf` | Storage and memory |

| `chmod`, `chown`, `useradd`, `passwd`, `sudo`, `su` | Permissions and users |

| `tar -czvf a.tar.gz dir`, `gzip`, `zip` | Archiving |

| `ping`, `netstat -tulnp`, `ss`, `ifconfig/ip a`, `traceroute`, `dig`, `curl`, `wget`, `ssh`, `scp` | Networking |

| `systemctl status/start/enable`, `journalctl -u svc`, `crontab -e` | Services and scheduling |

****Cron format:**** `minute hour day-of-month month day-of-week command`. `0 2 \* \* \*` = every day at 02:00.

****Signals:**** SIGHUP(1), SIGINT(2, Ctrl+C), SIGKILL(9, cannot be caught), SIGTERM(15, the polite default), SIGSTOP.

****Pipes and redirection:**** `|`, `>`, `>>`, `2>`, `&>`, `tee`.

Windows equivalents worth recognising

Task Manager, Event Viewer, Registry (HKLM/HKCU), Services (services.msc), PowerShell cmdlets (Get-Process, Get-Service), NTFS permissions and Active Directory / Group Policy.

Exam pointers

1. ****DMA interrupts the CPU only once****, at transfer completion.
2. `chmod 755` = rwxr-xr-x; compute permissions from 4/2/1 quickly.
3. ****SIGKILL (9) cannot be caught or ignored****; SIGTERM (15) can.
4. `/proc` is a ****virtual**** filesystem exposing kernel and process state.
5. The sticky bit matters on shared directories such as `/tmp`.

Memory Management

Supplementary reference / interview

Address binding and the MMU

Logical (virtual) address is generated by the CPU; **physical address** is seen by the memory unit. The **Memory Management Unit** translates between them at run time. Binding can occur at compile time, load time, or **execution time** (required for swapping and virtual memory).

Simplest scheme: **relocation register (base) + limit register**. $\text{Physical} = \text{logical} + \text{base}$, with a check that $\text{logical} < \text{limit}$.

Contiguous allocation

- **Fixed partitions** — internal fragmentation.
- **Variable partitions** — external fragmentation; fixed by **compaction**.

Allocation strategies:

Strategy	Rule	Note
----------	------	------

---	---	---
-----	-----	-----

First fit	First hole large enough	Fastest
------------------	-------------------------	---------

Best fit	Smallest adequate hole	Leaves tiny unusable holes; needs a search
-----------------	------------------------	--

Worst fit	Largest hole	Usually worst in practice
------------------	--------------	---------------------------

Internal fragmentation = space wasted inside an allocated block (paging). **External fragmentation** = free memory exists but is not contiguous (segmentation). **50% rule**: with first fit, about half as many blocks are lost to fragmentation as are allocated.

Paging

Physical memory is divided into **frames**, logical memory into **pages of the same size**. A **page table** maps page number → frame number.

Logical address = page number (p) | page offset (d)
If logical address space = 2^m and page size = 2^n bytes,
then offset takes n bits and page number takes (m - n) bits.

- **No external fragmentation**; internal fragmentation of at most one page in the last frame (average: half a page).
- Page table is per-process and kept in memory; **PTBR** points at it. A naive lookup means **two memory accesses** per reference.
- **TLB** (translation lookaside buffer) — a small associative cache of page-table entries.

Effective Access Time = $h \times (\text{TLB} + m) + (1 - h) \times (\text{TLB} + 2m)$ where h is the hit ratio and m the memory access time (add the TLB access on both paths).

- Page table entry bits: **frame number**, **valid/invalid**, **protection (r/w/x)**, **dirty (modified)**, **reference**, **caching disabled**.

Large page tables are handled by **hierarchical (multi-level) paging**, **hashed page tables**, or an **inverted page table** (one entry per frame — saves space but makes lookup harder).

Segmentation

Memory divided by **logical unit** — code, data, stack, heap — of **variable length**. Address = `**segment number, offset**`; the segment table stores **base and limit**. Matches the programmer's view and supports sharing/protection naturally, but causes **external fragmentation**.

Segmented paging combines both: segments are paged, so there is no external fragmentation and the logical structure is preserved.

Worked numeric pattern

> Logical address space 4 GB (2^{32}), page size 4 KB (2^{12}) → offset **12 bits**, page number **20 bits** → $2^{20} = 1\text{ M entries}$ in the page table. At 4 bytes per entry, the page table alone is **4 MB per process** — which is precisely why multi-level paging exists.

Exam pointers

1. Paging → **internal** fragmentation only. Segmentation → **external** fragmentation.
2. Offset bits = $\log_2(\text{page size})$. Page number bits = address bits – offset bits.
3. Learn the **EAT with TLB** formula; it appears almost every year in some exam.
4. Inverted page table has **one entry per frame**, not per page.
5. Page size is always a **power of two** so that the split is a simple bit division.

| Address space | Own | Shared |

| Creation cost | High | Low |

| Context switch | Expensive (memory map change) | Cheap |

| Communication | IPC required | Shared memory (needs synchronisation) |

| Fault isolation | One crash does not kill others | One thread crashing can kill the process |

Multithreading models: many-to-one (user threads on one kernel thread; one blocking call blocks all), one-to-one (Windows, Linux), many-to-many.

User-level threads — fast, managed by a library, but the kernel cannot schedule them individually.

Kernel-level threads — slower to create but truly parallel on multiple cores.

Inter-process communication

- **Shared memory** — fastest; the processes must synchronise themselves.

- **Message passing** — send/receive; easier in distributed systems; blocking vs non-blocking; direct vs indirect (mailboxes).

- Unix mechanisms: **pipes** (unidirectional, related processes), **named pipes/FIFO**, **message queues**, **shared memory segments**, **sockets**, **signals**.

Special process states

- **Zombie** — the child has terminated but the parent has not yet called `wait()`; the entry stays in the process table.

- **Orphan** — the parent terminated first; the child is adopted by **init/systemd** (PID 1).

Exam pointers

1. Threads share **code, data, files**; they do **not** share stack and registers.
2. **Long-term scheduler** controls the degree of multiprogramming.
3. Context switch time is overhead and depends on hardware register support.
4. Zombie = terminated but not reaped; orphan = parent died first.
5. Shared memory is faster than message passing but needs explicit synchronisation.

CPU Scheduling — Algorithms and Numericals

Supplementary reference / interview

> This is the highest-frequency numerical topic in OS. Practise until Gantt charts are automatic.

Criteria

| Criterion | Direction |

|---|---|

| CPU utilisation | Maximise |

| Throughput (processes/unit time) | Maximise |

| **Turnaround time** = completion – arrival | Minimise |

| **Waiting time** = turnaround – burst | Minimise |

| **Response time** = first CPU allocation – arrival | Minimise |

Preemptive scheduling can take the CPU away from a running process; **non-preemptive** cannot.

Algorithms

| Algorithm | Preemptive? | Key property | Weakness |

|---|---|---|---|

| **FCFS** | No | Simplest, fair in arrival order | **Convoy effect**; bad average waiting time |

| **SJF** | No | **Provably optimal** average waiting time | Needs burst-time knowledge; starves long jobs |

| **SRTF** | Yes | Preemptive SJF | Higher context-switch overhead; starvation |

| **Priority** | Both | Flexible | **Starvation** — fixed by **ageing** |

| **Round Robin** | Yes | Time quantum q ; good **response time** | q too large $\rightarrow$ FCFS; q too small $\rightarrow$ thrashing on context switches |

| **Multilevel Queue** | — | Separate queues by process type, fixed assignment | Inflexible |

| **Multilevel Feedback Queue** | Yes | Processes move between queues; approximates SJF without knowing bursts | Complex tuning |

Worked example

Processes (arrival, burst): P1(0, 5), P2(1, 3), P3(2, 8), P4(3, 6)

FCFS order P1 $\rightarrow$ P2 $\rightarrow$ P3 $\rightarrow$ P4:

| P | AT | BT | CT | TAT = CT-AT | WT = TAT-BT |

|---|---|---|---|---|

| P1 | 0 | 5 | 5 | 5 | 0 |

| P2 | 1 | 3 | 8 | 7 | 4 |

| P3 | 2 | 8 | 16 | 14 | 6 |

| P4 | 3 | 6 | 22 | 19 | 13 |

Average TAT = $(5+7+14+19)/4 = 11.25$; average WT = $(0+4+6+13)/4 = 5.75$.

SRTF on the same set gives a lower average waiting time — always verify by drawing the Gantt chart rather than guessing.

Round Robin notes

With n processes and quantum q , no process waits more than $(n-1)q$ time units. Every context switch costs overhead s , so effective CPU utilisation is $q/(q+s)$.

Multiprocessor scheduling

Load balancing (push and pull migration), **processor affinity** (soft and hard), **gang scheduling**, and **NUMA** awareness.

Real-time scheduling

- **Rate Monotonic (RMS)** — static priority, shorter period gets higher priority. Schedulable if utilisation $\leq n(2^{1/n} - 1)$; for large n this tends to $\ln 2 \approx 0.693$.

- **EDF — Earliest Deadline First** — dynamic priority; schedulable if **total utilisation ≤ 1** ; optimal for uniprocessors.

Exam pointers

1. **SJF is optimal** for average waiting time — but only in theory, since burst times are unknown.
2. Round Robin is chosen for **response time**, not throughput.
3. Starvation is fixed by **ageing**.
4. RMS bound ≈ 0.693 for large n ; EDF bound is **1**.
5. Always compute TAT first, then $WT = TAT - BT$. Most errors come from skipping the Gantt chart.

Process Synchronisation

Supplementary reference / interview

The problem

When multiple processes access shared data concurrently, the outcome depends on the **order of execution** — a **race condition**. Classic example: two threads both doing `counter++` on a shared counter can lose one increment, because `counter++` is really load, increment, store.

Critical section

The section of code accessing shared resources. A correct solution must satisfy:

1. **Mutual exclusion** — at most one process in its critical section.
2. **Progress** — if no process is in the CS, selection of the next entrant cannot be postponed indefinitely by processes not wanting to enter.
3. **Bounded waiting** — a bound exists on how many times others enter before a waiting process gets in.

(Assumptions about relative speed must not be made.)

Software solutions

- **Peterson's solution** — two processes, uses `flag[2]` and `turn`. Satisfies all three conditions. Modern CPUs with instruction reordering can break it without memory barriers.
- **Dekker's algorithm** — the original two-process solution.
- **Bakery algorithm (Lamport)** — n processes, "take a ticket".

Hardware support

TestAndSet, **CompareAndSwap (CAS)**, **Swap/Exchange** — atomic instructions. A **spinlock** built on these busy-waits, which is fine for very short critical sections on multiprocessors and wasteful otherwise.

Semaphores

An integer variable accessed only through two atomic operations:

```
wait(S) / P(S): while (S <= 0) ; S--;  
signal(S) / V(S): S++;
```

- **Binary semaphore (mutex)** — value 0 or 1.
- **Counting semaphore** — counts available instances of a resource.
- A proper implementation uses a **waiting queue** and block/wakeup rather than busy waiting.

Mutex vs semaphore: a mutex has **ownership** — only the locking thread may unlock it. A semaphore is a signalling mechanism and may be signalled by any thread.

Classic problems

| Problem | Point it teaches |

|---|---|

| **Producer–Consumer (bounded buffer)** | Uses ``full``, ``empty`` counting semaphores and a ``mutex``; wrong ordering of `wait()` calls causes deadlock |

| **Readers–Writers** | Multiple readers may share; writers need exclusive access; first/second variants cause reader or writer starvation |

| **Dining Philosophers** | Deadlock and starvation; solutions: allow at most $n-1$ to sit, pick up both forks atomically, or make one philosopher left-handed |

| **Sleeping Barber** | Synchronising a limited waiting room |

Monitors

A high-level construct: shared data plus procedures, with **only one process active inside the monitor at a time**, enforced by the compiler/runtime. **Condition variables** support ``wait()`` and ``signal()``. Signalling disciplines: **signal-and-wait** (Hoare) vs **signal-and-continue** (Mesa — used by Java, which is why you loop on the condition rather than using ``if``).

Java's ``synchronized`` keyword and ``wait()/notify()`` are a monitor implementation.

Exam pointers

1. The three requirements: **mutual exclusion**, **progress**, **bounded waiting**.
2. Peterson's solution works for **two** processes.
3. A mutex has **ownership**; a semaphore does not.
4. In producer-consumer, always ``wait(empty)`` then ``wait(mutex)`` — reversing them deadlocks.
5. Monitors put mutual exclusion in the **compiler/language**, not in the programmer's hands.

Virtual Memory and Page Replacement

Supplementary reference / interview

Idea

Execute a process that is **not entirely in memory**. Benefits: larger logical address space than physical memory, higher degree of multiprogramming, faster process start, and easy sharing via **copy-on-write**.

Implemented by **demand paging**: a page is brought in only when referenced. A reference to a page marked invalid causes a **page fault**.

Page fault handling

1. Trap to the OS
2. Check whether the reference is legal; if not, terminate
3. Find a free frame (or select a victim via the replacement algorithm)
4. If the victim is dirty, write it out
5. Read the required page from disk into the frame
6. Update the page table (valid bit, frame number)
7. Restart the instruction that faulted

Effective Access Time = $(1 - p) \times \text{memory access} + p \times \text{page-fault service time}$, where p is the page fault rate. Because page fault service time is in milliseconds and memory access in nanoseconds, even $p = 0.001$ degrades performance dramatically — a favourite numerical.

Page replacement algorithms

| Algorithm | Rule | Note |

|---|---|---|

| **FIFO** | Replace the oldest page | Suffers **Belady's anomaly** — more frames can mean more faults |

| **Optimal (OPT/MIN)** | Replace the page not used for the longest time in future | Theoretical benchmark only |

| **LRU** | Replace the least recently used | Good approximation of OPT; needs counters or a stack; a **stack algorithm**, so no Belady's anomaly |

| **LRU approximations** | Reference-bit, **second chance (clock)**, enhanced second chance (reference + dirty bit) | What real systems use |

| **LFU / MFU** | By frequency of use | Poor performers in practice |

Belady's anomaly occurs in **FIFO** (and can in second-chance), never in **stack algorithms** like LRU and OPT.

Frame allocation

- **Equal, proportional (by process size), and priority-based** allocation.

- **Global replacement** — a process may take a frame from another; **local replacement** — only from its own set. Global gives better throughput but unpredictable per-process behaviour.

Thrashing

A process spends more time paging than executing. Cause: **too few frames for its active working set**, usually because the degree of multiprogramming is too high. Symptom: CPU utilisation collapses while disk activity saturates — and a naive scheduler responds by admitting **more** processes, making it worse.

Solutions:

- **Working set model** — track the set of pages referenced in the last Δ references; allocate enough frames for it; suspend processes if the sum of working sets exceeds available frames.
- **Page fault frequency (PFF)** — set an upper and lower bound on the fault rate; give frames when above, take frames away when below.

Locality of reference — temporal and spatial — is what makes all of this work.

Related

Copy-on-write in `fork()`; **memory-mapped files**; **page buffering**; **prepaging**; **TLB shutdown** in multiprocessors.

Exam pointers

1. **Belady's anomaly**: FIFO yes, LRU/OPT no.
2. OPT is not implementable — it is the yardstick.
3. Thrashing is a **frame-allocation** problem, not a disk problem.
4. Learn the EAT formula with page faults and practise with $p = 10^3$ style numbers.
5. Second chance = FIFO + reference bit; enhanced second chance also uses the **dirty** bit.

C Programming — Fundamentals

Core preparation lesson

Data types and sizes (typical 64-bit Linux)

| Type | Size | Range / note |

---|---|---

| char | 1 byte | -128 to 127 (signed) |

| int | 4 bytes | -2^{31} to $2^{31}-1$ |

| long | 8 bytes | |

| float | 4 bytes | ~6-7 significant digits |

| double | 8 bytes | ~15-16 digits |

| pointer | 8 bytes | |

Sizes are **implementation-defined**; only `sizeof(char) == 1` is guaranteed. Use `<stdint.h>` types (`int32_t`, `uint64_t`) when size matters.

Operators and precedence traps

- **Precedence (high → low):** `() [] -> ' ' .` → unary `! ~ ++ -- + - * & sizeof` → `* / %` → `+ -` → `<< >>` → relational → `== !=` → `&` → `^` → `|` → `&&` → `||` → `?:` → assignment → `,`

- `++i` vs `i++`: **pre-increment** yields the new value, **post-increment** the old one.

- Expressions like `i = i++ + ++i` are **undefined behaviour** — the standard answer in an exam is "undefined", not a number.

- Integer division truncates: `5/2 == 2`, `-5/2` is implementation-defined before C99 and truncates toward zero from C99.

- `%` works only on integers.

- **Short-circuit:** `&&` and `||` do not evaluate the right side if the result is already determined — so `if (p != NULL && p->x)` is safe and idiomatic.

Control flow

`if/else`, `switch` (integral types only; **fall-through without break** is a classic exam trick; `default` can be anywhere), `for`, `while`, `do-while` (**body executes at least once**), `break`, `continue`, `goto`.

Storage classes

| Class | Scope | Lifetime | Default value |

---|---|---|---

| **auto** | Block | Block | Garbage |

| **register** | Block | Block | Garbage (hint to keep in a register; `&` not allowed) |

| **static** (local) | Block | **Whole program** | 0 |

| **static** (global/function) | **File only** (internal linkage) | Whole program | 0 |

| **extern** | Global | Whole program | 0 |

A `static` local variable **retains its value between calls** — a guaranteed output question.

Functions and parameter passing

C is **strictly call by value**. "Call by reference" is simulated by passing a **pointer**:

```
void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }
swap(&x, &y);
```

Arrays "decay" to a pointer to the first element when passed, which is why `sizeof(arr)` inside a function gives the pointer size, not the array size.

Recursion

Every recursive function needs a **base case** and progress toward it. Each call consumes a stack frame; too deep → **stack overflow**.

```
int fact(int n) { return (n <= 1) ? 1 : n * fact(n - 1); }
```

Tail recursion can be optimised into a loop by the compiler. Classic problems: factorial, Fibonacci (exponential without memoisation), **Towers of Hanoi** ($2^n - 1$ moves), GCD by Euclid, Ackermann.

Preprocessor

`#include`, `#define` (textual substitution — always parenthesise macro arguments: `#define SQR(x) ((x)*(x))`), `#ifdef`/`#ifndef`/`#endif` include guards, `#pragma`.

Macro vs function: macros are expanded inline with no type checking and can evaluate arguments more than once (`SQR(i++)` is a bug); functions are type-checked and evaluated once.

Compilation stages

Preprocessing (.i) -> Compilation (.s assembly) -> Assembly (.o object) -> Linking (executable)

Exam pointers

1. **static locals** keep their value between calls and are initialised to 0.
2. `do-while` runs the body **at least once**.
3. `i = i++ + ++i` is **undefined behaviour**.
4. C is always **call by value**; pointers only simulate reference semantics.
5. `switch` without `break` **falls through** — check every option in output questions.

C++ fundamentals and object behaviour

Core preparation lesson

C++ supports value semantics, references, classes and deterministic destruction. A reference aliases an existing object; a pointer holds an address and can be null. `std::vector` manages a dynamic array, but growth may invalidate pointers and iterators into its storage.

Worked trace

```
int a = 4;
int& b = a;
b += 3; // a is now 7
int c = a;
c += 1; // c is 8; a remains 7
```

Constructors establish an object's state; destructors release resources owned by it. RAII binds resource lifetime to object lifetime. A virtual function can dispatch according to the dynamic object type through a base reference or pointer. Overloading chooses among signatures; overriding supplies derived behaviour for an inherited virtual function.

Pitfalls and drills

Do not infer a numeric result from undefined behaviour such as reading out of bounds or using an object after its lifetime ends. Trace one constructor/destructor sequence, one virtual dispatch example and one exception path. Compare pass-by-value and const-reference parameters. Understand strings, loops, arrays and functions before memorising library details.

Java Essentials

Core preparation lesson

Platform

JDK (development kit: compiler + tools + JRE) □ **JRE** (runtime: libraries + JVM) □ **JVM** (executes bytecode).

`javac` compiles `.java` → **bytecode** `.class` → the JVM interprets and **JIT-compiles** hot paths to native code. This is what "write once, run anywhere" means — the bytecode is portable, the JVM is not.

JVM memory areas: **method area** (class metadata), **heap** (objects; shared), **stack** (per thread, frames), PC register, native method stack.

Garbage collection: **generational** — **young** (eden + survivor) and **old** generations; minor vs major/full GC; **mark-and-sweep** with compaction. Collectors: Serial, Parallel, **G1**, ZGC. `finalize()` is deprecated; use `try-with-resources`.

Language features

- Everything lives in a class; entry point `public static void main(String[] args)`.
- **Primitives** (byte, short, int, long, float, double, char, boolean) vs **wrapper classes** with **autoboxing**.
- **Strings are immutable**; the **string pool** interns literals. Use **StringBuilder** (not thread safe, faster) or **StringBuffer** (synchronised) for repeated concatenation.
- `==` compares **references** for objects; `equals()` compares content. Override `equals()` and `hashCode()` **together** — unequal hashCodes for equal objects break HashMap.
- `final` (constant / no override / no subclass), `finally` (always executes), `finalize()` (deprecated GC hook) — a classic three-way question.
- **Pass by value always** — object references are themselves passed by value.

Collections framework

Collection

■ List : ArrayList, LinkedList, Vector (synchronised), Stack

■ Set : HashSet, LinkedHashSet, TreeSet (sorted)

■ Queue : PriorityQueue, ArrayDeque, LinkedList

Map (separate): HashMap, LinkedHashMap, TreeMap, Hashtable, ConcurrentHashMap

| | ArrayList | LinkedList |

|---|---|---|

| Access | O(1) | O(n) |

| Insert/delete middle | O(n) | O(1) given the node |

| Memory | Compact | Node overhead |

| | HashMap | Hashtable | ConcurrentHashMap |

|---|---|---|

Thread safe	No	Yes (whole-map lock)	Yes (bucket-level)
Null key/value	One null key allowed	None	None
Performance	Best single-threaded	Poor	Best concurrent

HashMap internals: array of buckets, each a linked list that **converts to a red-black tree** when it exceeds a threshold (Java 8+); default capacity 16, load factor 0.75, resize doubles capacity and rehashes.

Exceptions

Throwable
■■■ Error (OutOfMemoryError, StackOverflowError – do not catch)
■■■ Exception
 ■■■ Checked (IOException, SQLException – must be handled or declared)
 ■■■ RuntimeException (unchecked: NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException, ClassCastException, NumberFormatException)

`try / catch / finally / throw / throws`, multi-catch, **try-with-resources** for AutoCloseable. `finally` runs even when the try block returns (except on `System.exit()`).

Multithreading

- Create by **extending Thread** or, preferably, **implementing Runnable / Callable**.
- States: NEW → RUNNABLE → RUNNING → BLOCKED/WAITING/TIMED_WAITING → TERMINATED.
- `synchronized` methods and blocks; **volatile** guarantees visibility (not atomicity); `wait()`, `notify()`, `notifyAll()` must be called inside a synchronized block.
- **ExecutorService** and thread pools; `Future`; `CompletableFuture`; `java.util.concurrent` — `AtomicInteger`, `ConcurrentHashMap`, `CountDownLatch`, `BlockingQueue`.
- **sleep()** does not release the lock; `wait()` does. A guaranteed exam question.

Modern Java

Lambdas `(a, b) -> a + b`, functional interfaces (`Function`, `Predicate`, `Supplier`, `Consumer`), **Streams** (`list.stream().filter(...).map(...).collect(...)` — lazy, with intermediate and terminal operations), `Optional`, `var`, records, sealed classes, virtual threads.

Exam pointers

1. **JDK □ JRE □ JVM.**
2. **sleep()** holds the lock, `wait()` releases it.
3. Strings are **immutable**; `StringBuffer` is synchronised, `StringBuilder` is not.
4. Checked exceptions must be handled or declared; **RuntimeExceptions** need not be.
5. Override `equals()` and `hashCode()` **together**.

Object-Oriented Programming

Core preparation lesson

The four pillars

| Pillar | Meaning | Mechanism |

|---|---|---|

| **Encapsulation** | Bundle data with the methods that act on it, and hide internals | private fields + public methods |

| **Abstraction** | Expose what, hide how | abstract classes, interfaces |

| **Inheritance** | Reuse and specialise | `extends`, base/derived classes |

| **Polymorphism** | One interface, many implementations | overloading and overriding |

Encapsulation hides **data**; abstraction hides **implementation**. Examiners like that distinction.

Class concepts

- **Class** = blueprint; **object** = instance.

- **Constructor** — same name as the class, no return type; **default**, **parameterised**, **copy**. Constructors can be overloaded; **constructors are not inherited**.

- **Destructor** (C++) / **finalizer / garbage collection** (Java) — cleanup.

- **this** — reference to the current object.

- **static** members belong to the class, not an instance; static methods cannot use `this` or access instance members directly.

Inheritance

Types: single, multiple (C++ yes, **Java no for classes** — interfaces instead), multilevel, hierarchical, hybrid.

Diamond problem — with multiple inheritance, an ambiguous base copy. C++ resolves it with **virtual inheritance**; Java sidesteps it by disallowing multiple class inheritance, and resolves interface default-method conflicts by requiring an explicit override.

Access specifiers:

| Specifier | Same class | Same package | Subclass | Anywhere |

|---|---|---|---|

| private | Yes | No | No | No |

| default (package-private) | Yes | Yes | No | No |

| protected | Yes | Yes | **Yes** | No |

| public | Yes | Yes | Yes | Yes |

Polymorphism

| | Overloading (compile-time) | Overriding (run-time) |

|---|---|---|

| Also called | Static / early binding | **Dynamic / late binding** |

| Signature | **Must differ** (parameters) | **Must be the same** |

| Return type | Can differ | Must be same or covariant |

| Scope | Same class | Base and derived class |

| Access | — | Cannot **reduce** visibility |

| Mechanism | Compiler resolves | **vtable / virtual dispatch** |

Return type alone cannot distinguish overloads. A C++ `virtual` function (or any non-static Java method) enables run-time dispatch through the **vtable**; a **pure virtual function** (`= 0`) makes the class **abstract**.

Abstract class vs interface (Java)

| | Abstract class | Interface |

|---|---|---|

| Methods | Abstract + concrete | Abstract; **default and static since Java 8**; private since 9 |

| Fields | Any | **public static final** only |

| Inheritance | One only | **Many** |

| Constructor | Yes | No |

| Use when | Shared state and behaviour | A capability/contract |

SOLID principles

- **S**ingle responsibility — one reason to change.
- **O**pen/closed — open for extension, closed for modification.
- **L**iskov substitution — a subtype must be usable wherever the base type is.
- **I**nterface segregation — many small interfaces beat one fat one.
- **D**ependency inversion — depend on abstractions, not concretions.

Also: **composition over inheritance**, **DRY**, **YAGNI**, **law of Demeter**.

Common design patterns

| Category | Patterns |

|---|---|

| **Creational** | Singleton, Factory, Abstract Factory, Builder, Prototype |

| **Structural** | Adapter, Decorator, Facade, Proxy, Composite, Bridge, Flyweight |

| **Behavioural** | Observer, Strategy, Command, Iterator, State, Template Method, Chain of Responsibility, Mediator |

Know **Singleton** (one instance, global access; thread safety via double-checked locking or an enum in Java), **Factory** (creation without naming the concrete class), **Observer** (publish-subscribe — the model behind market data feeds), and **Strategy** (swap algorithms at run time).

Exam pointers

1. **Overloading** = compile time, **overriding** = run time.
2. Return type alone **cannot** overload a method.
3. Java has no multiple **class** inheritance; interfaces provide it safely.
4. Interface fields are implicitly **public static final**.
5. Learn SOLID by name — it appears in descriptive software-engineering answers.

Output-Based and Code-Tracing Questions

Core preparation lesson

> A large share of the IT objective paper is "what does this print". These are skills you can improve with practice, because the traps repeat.

The recurring traps

1. Pre vs post increment

```
int i = 5;
printf("%d %d", i++, ++i); // undefined behaviour - do not guess a value
int a = 5, b = a++ + ++a;   // b depends on sequencing; also UB in C
```

In a well-formed question: ``i = 5; j = i++;` → j = 5, i = 6. `j = ++i;` → j = 6, i = 6.`

2. Integer division and promotion

```
printf("%d", 5/2);           // 2
printf("%f", 5/2.0);         // 2.500000
printf("%d", 'A' + 1);       // 66
printf("%c", 'A' + 1);       // B
float f = 0.1 + 0.2;         // never exactly 0.3 - floating point
```

3. sizeof

```
char s[] = "hello"; sizeof(s) // 6 (includes '\0')
char *p = "hello"; sizeof(p) // implementation-dependent pointer size; often 8 on 64-bit
strlen(s) // 5
int a[10]; sizeof(a)/sizeof(a[0]) // 10 here; invalid element-count formula for a pointer p
parameter
```

4. static and scope

```
void f() { static int c = 0; c++; printf("%d ", c); }
f(); f(); f(); // 1 2 3
```

5. Loop boundaries and fall-through

```
for (i = 0; i < 5; i++); // note the semicolon - empty body, i ends at 5
switch (2) { case 1: printf("a"); case 2: printf("b"); case 3: printf("c"); }
// prints "bc" - no break
```

6. Pointer arithmetic

```
int a[] = {10, 20, 30, 40};
int *p = a;
printf("%d", *(p + 2)); // 30
printf("%d", *p + 2);   // 12
printf("%d", *(p++));   // 10, then p points to 20
```

7. Java string identity

```
String a = "hi", b = "hi"; // both from the pool
String c = new String("hi");
a == b // true (same interned object)
a == c // false (different object)
a.equals(c) // true
```

8. Java integer caching

```
Integer x = 127, y = 127; // x == y is true (cached -128..127)
Integer p = 128, q = 128; // p == q is FALSE
```

9. Python mutable defaults and closures

```
def f(x, acc=[]):  
    acc.append(x); return acc  
f(1) # [1]  
f(2) # [1, 2] - the same list  
  
fns = [lambda: i for i in range(3)]  
[g() for g in fns] # [2, 2, 2] - late binding of i
```

10. Short-circuit evaluation

```
int i = 0;  
if (0 && i++) ; // i++ never runs, i stays 0  
if (1 || i++) ; // i++ never runs
```

Method

1. **Write down the state** of every variable after each line. Do not trace in your head.
2. Mark the **exact print format** (`%d` vs `%c` vs `%f`) — many "wrong" answers are formatting, not logic.
3. Check for **missing break**, stray semicolon, `=` instead of `==`, and off-by-one loop bounds.
4. If the code is genuinely **undefined behaviour**, the intended answer is usually "undefined / compiler-dependent".
5. Budget 60–90 seconds. If the trace is not converging, mark it and move on — in a 40-minute, 100-question paper, one traced question is worth the same as one one-liner.

Exam pointers

1. `sizeof` an array **inside a function** gives pointer size — the array has decayed.
2. Java `Integer` caching breaks `==` above **127**.
3. Python lambdas in a loop capture the **variable**, not its value.
4. A stray `;` after `for(...)` is the most common planted bug.
5. When in doubt between "undefined" and a specific number, **undefined** is usually the intended answer in C questions.

Pointers, Arrays, Strings and Memory

Core preparation lesson

Pointer basics

```
int x = 10;
int *p = &x;    // p holds the address of x
printf("%d", *p); // 10 - dereference
```

- `&` = address-of, `*` = dereference.

- **Pointer arithmetic scales by the pointed-to type**: if `p` is `int` and `sizeof(int) == 4`, then `p + 1` advances **4 bytes**. This is the single most-tested pointer fact.

- `void` is a generic pointer; it cannot be dereferenced or incremented without a cast.

- **NULL pointer** vs **dangling pointer** (points to freed memory) vs **wild pointer** (uninitialised).

Arrays and pointers

For `int a[5]`:

- `a` decays to `&a[0]`; `a[i]` is exactly `*(a + i)`, which is why `a[i] == i[a]` compiles.

- `a` and `&a` have the same value but different types: `a + 1` moves one int, `&a + 1` moves the whole array.

- 2-D arrays are stored **row-major**: `a[i][j]` is `*(a + i + j)`.

- An array **cannot** be assigned or returned; a pointer can.

Strings

A C string is a `char` array terminated by `'\0'`. `"hello"` occupies **6 bytes**.

```
char s1[] = "hello";    // modifiable array, 6 bytes on the stack
char *s2 = "hello";    // pointer to a string literal - modifying it is UB
```

| Function | Job | Danger |

|---|---|---|

| `strlen` | Length excluding `'\0'` | Reads until it finds a terminator |

| `strcpy` / `strcat` | Copy / concatenate | **Buffer overflow** — use `strncpy` / `strncat` / `snprintf` |

| `strcmp` | Lexicographic compare; 0 if equal | Comparing pointers with `==` compares addresses, not content |

| `sprintf` | Format into a buffer | Overflow — prefer `snprintf` |

`gets()` was **removed from the C standard** because it cannot be used safely — a common security question.

Dynamic memory

| Function | Behaviour |

|---|---|

| `malloc(n)` | n bytes, **uninitialised** |

| `calloc(n, size)` | $n \times \text{size}$ bytes, **zero-initialised** |

| `realloc(p, n)` | Resize; may move the block; returns NULL on failure (do not assign directly to p) |

| `free(p)` | Release; **double free and use-after-free are undefined behaviour** |

Always check the return value for NULL, and free exactly once. Set the pointer to NULL after freeing to avoid dangling use.

Memory layout of a C program (low $\rightarrow$ high):

Text / code segment	(read-only instructions)
Initialised data segment	(globals and statics with values)
BSS	(uninitialised globals/statics $\rightarrow$ zeroed)
Heap	(grows upwards; malloc)
...	
Stack	(grows downwards; locals, call frames)
Command-line args / env	

Stack vs heap:

| | Stack | Heap |

|---|---|---|

| Allocation | Automatic | Manual (malloc/free) |

| Speed | Fast | Slower |

| Size | Limited | Large |

| Lifetime | Scope | Until freed |

| Problem | Stack overflow | **Memory leak**, fragmentation |

Function pointers

```
int add(int a, int b) { return a + b; }
int (*op)(int, int) = add;
printf("%d", op(2, 3)); // 5
```

Used for callbacks, dispatch tables, `qsort`'s comparator — and, conceptually, for how C++ virtual functions work (the vtable).

Structures and unions

- `struct` — members laid out sequentially with **padding** for alignment; `sizeof` is usually more than the sum of members.

- `union` — all members share the same memory; **size = size of the largest member**; only one member is valid at a time.

- **Bit fields** pack members into a specified number of bits.

- `typedef` for readability; `->` to access through a pointer.

Exam pointers

1. **Pointer arithmetic scales by type size.**
2. `sizeof("hello")` is **6**; `strlen("hello")` is **5**.
3. **union size = largest member**; struct size includes **padding**.
4. `calloc` zeroes memory, `malloc` does not.
5. `a` vs `&a`: same address, **different pointer arithmetic**.

Python Essentials

Core preparation lesson

Nature of the language

Interpreted, dynamically typed, garbage collected, multi-paradigm. **CPython** compiles to bytecode (`.pyc`) executed by a virtual machine. The **GIL (Global Interpreter Lock)** allows only one thread to execute Python bytecode at a time — so threads help with **I/O-bound** work and **multiprocessing** is needed for CPU-bound parallelism. That distinction is a common interview question.

Built-in data structures

| Type | Mutable | Ordered | Notes |

---|---|---|---

| **list** | Yes | Yes | Dynamic array; O(1) append, O(n) insert |

| **tuple** | **No** | Yes | Hashable, usable as a dict key |

| **set** | Yes | No | Hash-based; O(1) membership; no duplicates |

| **dict** | Yes | **Insertion-ordered (3.7+)** | Hash map; O(1) average lookup |

| **str** | **No** | Yes | Immutable sequence of characters |

```
squares = [x*x for x in range(10) if x % 2 == 0]      # list comprehension
by_symbol = {t['sym']: t['qty'] for t in trades}      # dict comprehension
lazy     = (x*x for x in range(10**9))              # generator — O(1) memory
```

Functions

- Default arguments are evaluated **once at definition time** — the classic mutable-default bug: `def f(x, acc=[])` shares one list across calls. Use `None` as the default instead.

- `*args` and `**kwargs`; keyword-only arguments.

- **Closures** and **decorators**:

```
def timed(fn):
    def wrapper(*a, **kw):
        t0 = time.time()
        r = fn(*a, **kw)
        print(f"{fn.__name__} took {time.time()-t0:.3f}s")
        return r
    return wrapper

@timed
def slow(): ...
```

- `lambda`, `map`, `filter`, `reduce` (from `functools`), `zip`, `enumerate`, `any`, `all`, `sorted(key=...)`.

OOP in Python

```
class Order:
    exchange = "NSE"                # class attribute
    def __init__(self, sym, qty):   # constructor
        self.sym, self.qty = sym, qty # instance attributes
    def __repr__(self):
        return f"Order({self.sym}, {self.qty})"
```

****Dunder methods:**** ``__init__`, `__str__`, `__repr__`, `__len__`, `__eq__`, `__hash__`, `__iter__`,
`__enter__`/`__exit__`` (context managers). No true access modifiers — ``_x`` is a convention, ``__x`` triggers name mangling.

****MRO**** — method resolution order for multiple inheritance, computed by the ****C3 linearisation**** algorithm; inspect with ``Class.__mro__``.

Mutability trap

```
a = [1, 2, 3]
b = a          # same object
b.append(4)    # a is now [1, 2, 3, 4]
c = a[:]       # shallow copy
import copy; d = copy.deepcopy(a)  # independent
```

Python is ****pass by object reference****: rebinding inside a function does not affect the caller, but mutating does.

Standard library worth knowing

``os`, `sys`, `re`, `json`, `csv`, `datetime`, `collections`` (****Counter, defaultdict, deque, namedtuple, OrderedDict****),
``itertools`, `functools`` (``lru_cache``), ``math`, `random`, `sqlite3`, `logging`, `unittest``/`pytest`, ``threading`,
`multiprocessing`, `asyncio``.

****Data stack:**** NumPy (ndarray, vectorised operations), ****pandas**** (DataFrame, groupby, merge, time series), matplotlib, scikit-learn. For a SEBI IT role, pandas over trade data is a genuinely relevant skill to mention.

Error handling and files

```
try:
    with open("trades.csv") as f:          # context manager closes the file
        data = f.read()
except FileNotFoundError as e:
    log.error("missing file: %s", e)
except (ValueError, TypeError):
    raise
else:
    process(data)
finally:
    cleanup()
```

Exam pointers

1. ****Tuples are immutable and hashable; lists are not.****
2. ****Mutable default arguments**** are evaluated once — a favourite trick.
3. The ****GIL**** limits CPU-bound threading; use multiprocessing.
4. Dicts preserve ****insertion order from Python 3.7****.
5. ``is`` compares identity, ``==`` compares value.

Agile, Scrum, DevOps and CI/CD

Supplementary reference / interview

The Agile Manifesto (2001) — four values

- > **Individuals and interactions** over processes and tools
- > **Working software** over comprehensive documentation
- > **Customer collaboration** over contract negotiation
- > **Responding to change** over following a plan
- >
- > "...while there is value in the items on the right, we value the items on the left more."

That last clause matters — Agile does not mean "no documentation". Twelve supporting principles include early and continuous delivery, welcoming changing requirements, delivering frequently, sustainable pace, technical excellence and self-organising teams.

Scrum

Roles (accountabilities):

- **Product Owner** — owns the Product Backlog and maximises value.
- **Scrum Master** — serves the team, removes impediments; **not a project manager**.
- **Developers** — the cross-functional team that builds the increment.

Events: **Sprint** (a fixed timebox, typically 2–4 weeks) containing **Sprint Planning**, **Daily Scrum** (15 minutes), **Sprint Review** (demonstrate the increment to stakeholders), **Sprint Retrospective** (improve the process).

Artefacts: **Product Backlog**, **Sprint Backlog**, **Increment**, each with a commitment — Product Goal, Sprint Goal, and **Definition of Done**.

Practices: user stories ("As a <role>, I want <goal>, so that <benefit>"), **story points** and relative estimation, **planning poker**, **velocity**, **burndown/burnup charts**, backlog refinement.

Kanban

Visualise the workflow on a board, **limit work in progress (WIP)**, manage flow, make policies explicit, improve continuously. Measured by **lead time**, **cycle time** and **throughput**. Continuous flow rather than fixed sprints; often better than Scrum for support and operations work.

Scrum vs Kanban: timeboxed iterations and fixed roles versus continuous flow and WIP limits.

Scaling frameworks: SAFe, LeSS, Spotify model — recognise the names.

DevOps

A culture and practice set that unites development and operations to shorten the delivery cycle while improving reliability. **CALMS**: Culture, Automation, Lean, Measurement, Sharing.

DORA metrics — the four that matter:

1. **Deployment frequency**
2. **Lead time for changes**
3. **Change failure rate**
4. **Mean time to restore (MTTR)**

High performers are better on all four simultaneously — speed and stability are not a trade-off. That finding is worth quoting.

CI/CD

```
Commit -> CI: build, unit tests, static analysis, security scan (SAST/SCA)
        -> artefact build and signing
        -> CD: deploy to staging, integration and acceptance tests
        -> deploy to production (manual approval = continuous DELIVERY,
                                automatic         = continuous DEPLOYMENT)
        -> monitor, alert, roll back if needed
```

Deployment strategies: blue-green (two environments, switch traffic), **canary** (small percentage first), rolling, feature flags (deploy without releasing).

Infrastructure as Code (Terraform, Ansible), **immutable infrastructure**, **GitOps**. **Observability** = logs + metrics + traces. **SRE concepts:** SLI, SLO, SLA, **error budget**, toil reduction, blameless post-mortems.

DevSecOps — shift security left: SAST, DAST, SCA/dependency scanning, secrets scanning, container image scanning, **SBOM** generation, policy-as-code. For a SEBI-regulated entity this is where **change management and segregation of duties** must be preserved even while automating: the pipeline must enforce approvals, not bypass them.

Where Agile meets regulation

A regulated entity can be agile, but must retain: documented **change approval**, **segregation between who writes and who deploys**, **traceability from requirement to test to release**, and **auditable records**. The practical answer is to encode those controls **into the pipeline** rather than running a manual process alongside it.

Exam pointers

1. Manifesto: **four values**, and the "items on the right still have value" clause.
2. **Scrum Master is not a project manager**; the Product Owner owns the backlog.
3. **Kanban limits WIP**; Scrum timeboxes sprints.
4. **DORA's four metrics** — know them by name.
5. Continuous **delivery** needs approval to release; continuous **deployment** does not.

Software Design

Supplementary reference / interview

Levels

Architectural (high-level) — subsystems and their interactions. **Detailed (low-level)** — module internals, data structures, algorithms.

The two measures that matter

| Coupling | Cohesion |

|---|---|---|

| Meaning | Interdependence **between** modules | Focus **within** a module |

| Goal | **Low** | **High** |

Coupling, worst to best: **content** (one module modifies another's internals) → common (shared global data) → external → control (passing a flag that dictates behaviour) → stamp (passing a whole record when only a field is needed) → **data** (passing only what is needed — best).

Cohesion, worst to best: **coincidental** (unrelated things grouped) → logical → temporal → procedural → communicational → sequential → **functional** (everything contributes to one well-defined task — best).

"High cohesion, low coupling" is the single most quotable line in software design. Be ready to justify it: it localises change, enables independent testing, and limits the blast radius of a defect.

Design principles

Abstraction, modularity, information hiding (Parnas), separation of concerns, hierarchy, refinement, and **SOLID** (see the OOP note). Plus **DRY**, **KISS**, **YAGNI**, and **composition over inheritance**.

Architectural styles

| Style | Characteristic | Fits |

|---|---|---|

| **Layered (n-tier)** | Presentation / business / data layers | Enterprise applications |

| **Client-server** | Requests and responses | Almost everything |

| **Microservices** | Small, independently deployable services with their own data | Large teams, independent scaling |

| **Monolith** | One deployable unit | Small teams, simpler operations — and still the right default for many systems |

| **Event-driven** | Producers, brokers, consumers; asynchronous | **Market data, order flow, streaming surveillance** |

| **Pipe and filter** | Sequential transformations | Data processing, compilers |

| **Microkernel / plugin** | Core plus extensions | Products with varied deployments |

| **CQRS / event sourcing** | Separate read and write models; state as an event log | **Systems needing a complete audit trail** |

| **Serverless** | Functions on demand | Bursty, event-triggered workloads |

> **Event sourcing** deserves a mention in a markets answer: storing the ordered log of events and deriving state from it is exactly how a matching engine recovers deterministically and how a regulator reconstructs what happened at a point in time.

Microservices trade-offs (a favourite discussion question)

For: independent deployment and scaling, technology choice per service, fault isolation, team autonomy.

Against: distributed-system complexity, network latency and partial failure, **distributed transactions** (hence the **saga** pattern instead of two-phase commit), data consistency, harder debugging (needs distributed tracing), operational overhead.

The honest answer: microservices trade in-process complexity for network complexity. Choose them for organisational scaling reasons, and start with a well-structured monolith unless you have those reasons.

Design documentation

Component diagrams, deployment diagrams, sequence diagrams for key flows, **Architecture Decision Records (ADRs)** capturing why a choice was made, and interface contracts. In a regulated environment, design documentation is also **audit evidence** — an undocumented control effectively does not exist.

Exam pointers

1. **Low coupling, high cohesion** — and be able to rank the types of each.
2. **Content coupling is worst; data coupling is best.**
3. **Functional cohesion is best; coincidental is worst.**
4. **Event-driven suits market data; event sourcing suits audit trails.**
5. Microservices buy **independent deployment** at the price of **distributed-systems complexity**.

Estimation and Software Metrics

Supplementary reference / interview

Why estimate

To plan, price and staff — and to know when a project has gone off course. All estimation models are wrong; the useful ones are **calibrated with your own historical data**.

Lines of Code (LOC)

Simple and objective, but language-dependent, measurable only after the fact, and it rewards verbose code. Productivity as **LOC per person-month** is the classic bad metric.

Function Point Analysis

Measures **functionality delivered**, independent of language.

Count five components, each weighted simple/average/complex:

1. **External Inputs (EI)**
2. **External Outputs (EO)**
3. **External Inquiries (EQ)**
4. **Internal Logical Files (ILF)**
5. **External Interface Files (EIF)**

Unadjusted Function Points (UFP) = weighted sum.

Value Adjustment Factor (VAF) = $0.65 + 0.01 \times \Sigma(14 \text{ general system characteristics, each rated } 0-5)$.

FP = $UFP \times VAF$.

FP can be converted to LOC using language-specific multipliers, which is how FP feeds into COCOMO.

COCOMO (Boehm)

Basic COCOMO: **Effort** = $a \times (KLOC)^b$ person-months; **Time** = $c \times (\text{Effort})^d$ months.

| Project mode | a | b | c | d |

|---|---|---|---|

| **Organic** (small team, familiar problem) | 2.4 | 1.05 | 2.5 | 0.38 |

| **Semi-detached** (medium, mixed experience) | 3.0 | 1.12 | 2.5 | 0.35 |

| **Embedded** (tight constraints, complex) | 3.6 | 1.20 | 2.5 | 0.32 |

Average staff = $\text{Effort} / \text{Time}$.

Intermediate COCOMO multiplies by 15 **cost drivers** (product, hardware, personnel, project attributes).

Detailed COCOMO applies them phase by phase. **COCOMO II** modernises the model for reuse, prototyping and off-the-shelf components.

> Note the exponent $b > 1$ in every mode: effort grows **faster than linearly** with size. That is the quantitative form of **Brooks's law** — "adding manpower to a late software project makes it later", because communication paths grow as $n(n-1)/2$.

Other estimation techniques

Expert judgement, **Delphi / wideband Delphi**, **analogy** (similar past projects), **three-point estimation** (PERT: $(O + 4M + P)/6$), **planning poker** and story points in Agile, **bottom-up** work breakdown.

Quality metrics

- **Defect density** = defects / size (KLOC or FP).
- **Defect Removal Efficiency (DRE)** = defects found before release / total defects × 100.
- **MTBF, MTTR, availability** = $MTBF / (MTBF + MTTR)$.
- **Halstead metrics** — derived from operators and operands: program length, vocabulary, volume, difficulty, effort.
- **Maintainability index**, **code churn**, **technical debt ratio**, **test coverage**.

Goodhart's law applies: when a metric becomes a target, it stops being a good measure. Any answer that recommends metrics should also note this.

Exam pointers

1. **COCOMO modes**: organic, semi-detached, embedded — and that the exponent exceeds 1.
2. **Effort** = $a(KLOC)^b$; **Time** = $c(Effort)^d$.
3. **Function points**: EI, EO, EQ, ILF, EIF — five components.
4. **PERT three-point estimate** = $(O + 4M + P)/6$.
5. **Brooks's law** and communication paths $n(n-1)/2$.

Project Management, CPM/PERT and Risk

Supplementary reference / interview

Project management knowledge areas

Scope, schedule, cost, quality, resources, communication, **risk**, procurement, stakeholders, integration. The "iron triangle" of **scope, time and cost**, with quality in the middle — you cannot fix all three.

Work Breakdown Structure

Decompose the project hierarchically into deliverables and then **work packages** small enough to estimate and assign. The WBS is the foundation of every schedule and budget.

Network scheduling — CPM and PERT

Draw the activity network, then compute:

- **Forward pass** → **Earliest Start (ES)** and **Earliest Finish (EF)**: $EF = ES + \text{duration}$; ES of an activity = maximum EF of its predecessors.
- **Backward pass** → **Latest Finish (LF)** and **Latest Start (LS)**: $LS = LF - \text{duration}$; LF = minimum LS of successors.
- **Total float (slack)** = $LS - ES = LF - EF$.
- **Critical path** = the longest path through the network = the chain of activities with **zero float**. It determines the **minimum project duration**; any delay on it delays the project.

A network can have **more than one critical path**. **Free float** is the delay possible without affecting the **next** activity's early start, as opposed to total float which protects the project end.

CPM vs PERT:

| | CPM | PERT |

|---|---|---|

| Duration | **Deterministic** | **Probabilistic** |

| Orientation | Activity | Event |

| Origin | Construction/maintenance | R&D projects |

| Focus | Time-cost trade-off (**crashing**) | Uncertainty |

PERT expected time = $(O + 4M + P)/6$; **variance** = $((P - O)/6)^2$. Project variance = sum of variances **along the critical path**; use the normal distribution to compute the probability of finishing by a date.

Crashing — spending money to shorten critical activities, choosing the lowest **cost slope** (cost increase per unit time saved) first. **Fast tracking** — running activities in parallel, which adds risk.

Gantt charts show the schedule visually; **resource levelling** smooths over-allocation.

Risk management

Process: identification → analysis (probability × impact) → prioritisation (**risk exposure = probability × loss**) → planning → mitigation → monitoring.

Categories: project risks (schedule, resources), product risks (technical, quality), business risks (market, regulatory).

Responses: **avoid, mitigate, transfer, accept** (and, for opportunities: exploit, enhance, share). Maintain a **risk register** with owners and triggers, plus **contingency reserves** for known risks and **management reserves** for unknown ones. **RMMM plan** = Risk Mitigation, Monitoring and Management.

Configuration management

SCM controls change to work products: **configuration items**, **baselines**, **version control** (Git: branch, merge, pull request), **change control board**, **release management**, and **build reproducibility**. In a regulated entity, SCM is where **segregation of duties** and **change approval evidence** live.

Quality management: SQA (process-oriented, preventive) vs SQC (product-oriented, detective); **reviews, walkthroughs and formal inspections** (Fagan). Inspections find defects earlier and more cheaply than testing — an important point.

Exam pointers

- Critical path = longest path = zero float**, and it sets the project duration.
- Float = LS – ES = LF – EF.**
- CPM deterministic, PERT probabilistic**; **PERT variance = $((P-O)/6)^2$** .
- Crash the activity with the lowest cost slope** first.
- Risk exposure = probability × impact**; responses are avoid/mitigate/transfer/accept.

Maintenance, Quality and Reverse Engineering

Supplementary reference / interview

Software maintenance

Typically **60–80% of total lifetime cost** — more than development. Four types:

| Type | Trigger | Share (typical) |

|---|---|---|

| **Corrective** | Fixing defects | ~20% |

| **Adaptive** | Environment changes — new OS, new regulation | ~25% |

| **Perfective** | New or improved functionality, performance | **~50%, the largest** |

| **Preventive** | Improving maintainability before problems occur | ~5% |

The counter-intuitive fact examiners like: **perfective maintenance (enhancement) dominates**, not bug fixing.

Maintainability is improved by modularity, documentation, coding standards, tests, and low coupling. It is degraded by **software ageing** — accumulated patches, lost knowledge, and drift between documentation and code.

Legacy system strategies: scrap, keep maintaining, **re-engineer**, or **replace incrementally** (strangler pattern). A regulated entity rarely gets to do a big-bang replacement of a core system; incremental strangulation with parallel running is the realistic path.

Re-engineering and reverse engineering

- **Reverse engineering** — analysing an existing system to recover its design and specification. Legitimate uses: documentation recovery, interoperability, security analysis.
- **Restructuring** — improving code without changing functionality.
- **Refactoring** — small, behaviour-preserving improvements, backed by tests.
- **Forward engineering** — rebuilding from the recovered design.
- **Re-engineering** = reverse engineering + restructuring + forward engineering.

Technical debt

The implied cost of choosing an easy solution now over a better one later. Deliberate debt (a considered trade-off with a repayment plan) is legitimate; accidental debt is not. It accrues **interest** as slower future changes and more defects. Track it explicitly rather than pretending it does not exist.

Quality models and standards

- **McCall's quality factors** — product operation (correctness, reliability, efficiency, integrity, usability), product revision (maintainability, flexibility, testability), product transition (portability, reusability, interoperability).

- **ISO/IEC 25010 (SQuaRE)** — the modern model: functional suitability, performance efficiency, compatibility, usability, **reliability**, **security**, maintainability, portability.
- **ISO 9001**, **CMMI**, **Six Sigma (DMAIC)**, **TQM**, **Kaizen**.

SQA vs SQC: assurance is **process-oriented and preventive**; control is **product-oriented and detective**. Verification and validation sit inside both.

Reviews

Walkthrough (informal, author-led), **technical review**, **formal inspection** (Fagan: planning, overview, preparation, inspection meeting, rework, follow-up, with defined roles — moderator, author, reader, recorder). Inspections consistently find defects **earlier and more cheaply** than testing, because they catch design and requirement defects before code exists.

Documentation

User documentation (guides, help), **system documentation** (architecture, design, interfaces, ADRs), **process documentation** (plans, standards, procedures). In a SEBI-regulated context add: **system audit reports**, **change records**, **incident and RCA reports**, **BCP/DR plans and drill evidence**, and **access review records** — all of which are audit artefacts, not internal niceties.

Exam pointers

1. **Perfective maintenance is the largest category**, not corrective.
2. Maintenance is **60–80% of lifetime cost**.
3. **Re-engineering = reverse + restructure + forward.**
4. **SQA is preventive and process-focused; SQC is detective and product-focused.**
5. **Formal inspection (Fagan) has defined roles** and outperforms testing on early defect detection.

Requirement Engineering and SRS

Supplementary reference / interview

The process

Elicitation (interviews, workshops, observation, questionnaires, document analysis, prototyping) → **Analysis** (resolve conflicts, prioritise, model) → **Specification** (write the SRS) → **Validation** (reviews, walkthroughs, prototypes) → **Management** (baselines, change control, traceability).

Types of requirement

- **Functional** — what the system does. "The system shall block an order that breaches the client's margin limit."
 - **Non-functional (quality attributes)** — how well it does it: performance, **latency**, throughput, availability, scalability, security, usability, maintainability, portability, **auditability**.
 - **Domain requirements** — imposed by the business area.
 - **Constraints** — technology, regulatory, budget, timeline.
- > In financial systems, non-functional requirements often dominate. "Match orders correctly" is easy; "match them deterministically, in microseconds, with a reproducible audit trail, at peak volumes, with intraday recovery" is the actual engineering problem. Saying that shows you understand the domain.

Characteristics of a good requirement

Correct, unambiguous, complete, consistent, ranked for importance and stability, verifiable, modifiable, traceable. "The system should be fast" fails on unambiguous and verifiable; "99th percentile order acknowledgement under 5 ms at 50,000 messages/second" passes.

SRS structure (IEEE 830 style)

1. **Introduction** — purpose, scope, definitions, references, overview.
2. **Overall description** — product perspective and functions, user characteristics, constraints, assumptions and dependencies.
3. **Specific requirements** — functional requirements, external interfaces, performance, design constraints, **quality attributes**, other requirements.
4. Appendices, index.

Feasibility study

TELOS: **Technical** (can it be built with available technology and skills), **Economic** (cost-benefit, ROI, payback), **Legal** (regulatory and contractual), **Operational** (will it fit the organisation's processes), **Schedule** (can it be delivered in time).

Modelling tools

Use case diagrams (actors, use cases, include/extend), **DFD** (level 0 context diagram → level 1 → level 2; processes, data stores, external entities, data flows — **DFDs show no control flow or timing**), **ER diagrams**, **state diagrams**, **sequence diagrams**, **class diagrams**, **activity diagrams**. **Data dictionary** defines every element precisely.

UML diagram families: structural (class, object, component, deployment, package) vs behavioural (use case, sequence, activity, state, communication).

Requirement management

Baselining, **change control board**, **impact analysis**, **version control of the SRS**, and a **traceability matrix** linking each requirement to its design element, code module and test case — forwards and backwards. Traceability is not bureaucracy in a regulated environment; it is how you demonstrate that a control exists and was tested.

Requirement volatility and **scope creep** are the standard project killers; the defence is a baseline plus a disciplined change process, not refusing change.

Exam pointers

1. **Functional = what; non-functional = how well.**
2. A good requirement is above all **verifiable and unambiguous**.
3. **DFDs show data flow, not control flow or sequence.**
4. **TELOS** for feasibility.
5. The **traceability matrix** links requirement → design → code → test.

SDLC Models

Supplementary reference / interview

Phases

Requirement gathering → analysis → design → implementation → testing → deployment → maintenance. Every model rearranges these; none removes them.

The models

| Model | How it works | Use when | Weakness |

|---|---|---|---|

| **Waterfall** | Strictly sequential; each phase completes before the next | Requirements fixed and well understood; regulatory/fixed-price projects | No feedback; late discovery of errors; working software appears very late |

| **V-model** | Waterfall bent into a V; **each development phase has a matching test phase** (requirements↔UAT, design↔integration testing, code↔unit testing) | Safety- and compliance-critical systems | Same rigidity as waterfall |

| **Incremental** | Deliver in functional increments | Core requirements clear, extras can follow | Needs good architecture up front |

| **Iterative** | Build a rough version, refine repeatedly | Requirements will evolve | Can drift without discipline |

| **Prototype** | Build a throwaway model to clarify requirements | Requirements unclear or UI-heavy | Users mistake the prototype for the product |

| **Spiral** | Iterative with an explicit **risk analysis** quadrant per loop: planning → risk analysis → engineering → evaluation | Large, expensive, high-risk projects | Costly; needs risk expertise |

| **RAD** | Rapid, component-based, heavy user involvement | Short timelines, modular systems | Needs skilled teams and committed users |

| **Big Bang** | No process | Tiny experiments only | Unpredictable |

Boehm's spiral model is the one to name when a question mentions **risk-driven** development.

Cost of fixing a defect rises by roughly an order of magnitude at each later phase — requirements → design → code → testing → production. This single fact justifies early reviews, prototyping and shift-left testing, and is worth quoting.

Choosing a model

Ask four questions: how stable are the requirements; how large and distributed is the team; what is the cost of failure; and how often can you get feedback from real users? A stock exchange's core matching engine and a customer-facing investor app do not deserve the same process — a point worth making explicitly in a descriptive answer.

Process models vs methodologies

CMMI maturity levels: 1 Initial → 2 Managed → 3 Defined → 4 Quantitatively Managed → 5 Optimising. **ISO 9001** for quality management systems. These describe **organisational maturity**, not a development model — a distinction examiners like.

Exam pointers

1. **V-model** pairs each development phase with a test phase.
2. **Spiral = risk-driven**, with four quadrants per loop.
3. Prototype models are for **unclear requirements**.
4. **Defect cost rises ~10× per phase** — the argument for early detection.
5. **CMMI levels 1–5** in order; level 5 is "Optimising".

Software Testing

Supplementary reference / interview

Levels

Unit → Integration → System → Acceptance (UAT)

Unit (a single module, by developers, usually automated) → **Integration** (module interactions; **big bang**, **top-down** with stubs, **bottom-up** with drivers, **sandwich/hybrid**) → **System** (the whole system against requirements, including non-functional) → **Acceptance** (by the customer; **alpha** at the developer's site, **beta** at the user's site).

Regression testing — re-running tests after a change to confirm nothing previously working has broken.
Smoke testing — a quick check that the build is stable enough to test at all. **Sanity testing** — narrow checks after a small change.

Black box vs white box

| | Black box (functional) | White box (structural) |

|---|---|---|

| Knowledge of internals | No | Yes |

| Basis | Requirements | Code paths |

| Techniques | **Equivalence partitioning**, boundary value analysis, decision tables, state transition, error guessing, pairwise | **Statement**, branch, path, condition coverage; basis path testing; cyclomatic complexity |

| Done by | Testers | Developers |

Grey box combines both.

Boundary Value Analysis: defects cluster at boundaries. For a valid range of 1–100, test **0, 1, 2, 99, 100, 101**. **Equivalence partitioning** reduces cases by treating each class as equivalent — one valid class and the invalid classes on either side.

Cyclomatic complexity = $E - N + 2P$ (edges, nodes, connected components), or simply **number of decision points + 1**. It gives the number of independent paths and hence a lower bound on basis-path test cases. A value above roughly 10 suggests a module that should be split.

Coverage hierarchy

Statement coverage (every line executed) < **branch/decision coverage** (every branch taken both ways) < **condition coverage** < **MC/DC** (each condition independently affects the outcome — required in safety-critical standards) < **path coverage** (every path; usually infeasible because loops explode the count).

100% statement coverage does **not** imply 100% branch coverage — a standard exam trap.

Non-functional testing

Performance (response time, throughput), **load** (expected peak), **stress** (beyond peak, to find the breaking point), **soak/endurance** (memory leaks over time), **spike**, **volume**, **scalability**, **security** (VAPT, SAST/DAST), **usability**, **compatibility**, **recovery/failover**, **installation**.

For trading systems, the important ones are **latency percentiles (p99, not average)**, **capacity at peak event-day volumes**, and **failover testing** — because the average case is never what fails.

Test artefacts and process

Test plan, test scenarios, **test cases** (ID, precondition, steps, expected result), test data, **defect report** (severity vs **priority** — severity is technical impact, priority is business urgency; they are independent), **traceability matrix**, test summary report.

Defect life cycle: New → Assigned → Open → Fixed → **Retest** → Closed, with branches to Deferred, Rejected, Duplicate, Reopened.

Verification vs validation: verification asks "are we **building the product right**" (reviews, inspections, static analysis); validation asks "are we **building the right product**" (actual execution against user needs).

Test automation: unit frameworks (JUnit, pytest), API testing, UI automation (Selenium), performance tools (JMeter). The **test pyramid** — many unit tests, fewer integration tests, fewest UI tests — because UI tests are slow and brittle.

Exam pointers

1. **Verification = building it right; validation = building the right thing.**
2. **Cyclomatic complexity = $E - N + 2P$ = decision points + 1.**
3. **Severity $\neq$ priority**, and they are independent.
4. **Top-down integration uses stubs; bottom-up uses drivers.**
5. 100% statement coverage does **not** give 100% branch coverage.

Attacks, Malware and Threats

Core preparation lesson

Malware taxonomy

| Type | Defining behaviour |

|---|---|

| **Virus** | Attaches to a host file; needs user action to spread |

| **Worm** | **Self-replicating across a network**; no host file needed |

| **Trojan** | Disguised as legitimate software |

| **Ransomware** | Encrypts data and demands payment; modern variants also **exfiltrate and threaten publication** (double extortion) |

| **Spyware / Keylogger / Infostealer** | Covert collection of data and credentials |

| **Rootkit** | Hides at OS or kernel level; **bootkit** below the OS |

| **Backdoor** | Persistent unauthorised access |

| **Botnet** | Network of compromised hosts under a C2 server |

| **Logic bomb** | Triggers on a condition or date |

| **Fileless malware** | Lives in memory / uses legitimate tools (**living off the land**: PowerShell, WMI) |

| **Cryptojacking** | Steals compute for mining |

| **Adware, scareware, PUP** | Nuisance to fraud |

Social engineering

Phishing (mass email), **spear phishing** (targeted), **whaling** (executives), **BEC / CEO fraud** (invoice and payment redirection — the most financially damaging category in practice), **vishing** (voice), **smishing** (SMS), **quishing** (QR codes), **pretexting**, **baiting**, **tailgating**, **watering hole**, **deepfake voice/video fraud**.

Defences: awareness training, simulated phishing, DMARC/SPF/DKIM, out-of-band verification for payment changes, and **a culture where verifying an instruction from a senior person is never penalised**.

Network and application attacks

| Attack | Mechanism | Defence |

|---|---|---|

| **DoS / DDoS** | Volumetric (UDP flood, amplification via DNS/NTP/memcached), protocol (SYN flood), application (HTTP flood, slowloris) | Rate limiting, anycast scrubbing, SYN cookies, CDN, upstream filtering |

| **MITM** | Intercepting traffic | TLS with certificate validation, HSTS, certificate pinning |

| **ARP spoofing** | Poisoning the ARP cache on a LAN | Dynamic ARP inspection, static entries, segmentation |

| **DNS spoofing / cache poisoning** | Forged DNS responses | DNSSEC, DoH/DoT |

| **Session hijacking** | Stealing a session token | Secure/HttpOnly/SameSite cookies, rotation on privilege change, short expiry |

| **Replay** | Re-sending a captured valid message | Nonces, timestamps, sequence numbers |

| **Privilege escalation** | Vertical (to admin) / horizontal (to a peer account) | Patching, least privilege, hardening |

| **Supply chain** | Compromise a vendor, library or update channel | SBOM, code signing, vendor due diligence, dependency pinning |

| **Insider threat** | Malicious or negligent employee | Separation of duties, monitoring, DLP, exit process |

| **APT** | Long-dwell, targeted, well-resourced intrusion | Detection engineering, threat hunting, assume-breach |

The attack lifecycle

Cyber Kill Chain (Lockheed Martin): Reconnaissance → Weaponisation → Delivery → Exploitation → Installation → Command & Control → **Actions on Objectives**.

MITRE ATT&CK — a matrix of real-world tactics (initial access, execution, persistence, privilege escalation, defence evasion, credential access, discovery, lateral movement, collection, exfiltration, impact) and the techniques under each. It is the common language of modern SOC work and worth naming explicitly.

Threats specific to securities markets

- **Compromise of a broker's trading terminal or OMS** → unauthorised orders placed in client accounts.
- **Ransomware at an intermediary or MII** → availability failure during market hours, with settlement consequences.
- **Data theft of client KYC/PII** → identity fraud, regulatory breach under DPDP.
- **Market manipulation via hacked social media or fake corporate announcements**.
- **DDoS against a broker's order gateway** at a market-sensitive moment.
- **Insider misuse of UPSI** — a security control problem as much as a legal one, which is why the **Structured Digital Database** requirement exists.
- **Third-party/vendor compromise** — the reason SEBI pushes vendor risk management and SBOM.

Exam pointers

1. **Worm self-replicates without a host file; a virus needs one.**
2. **BEC is the costliest social engineering category**, not mass phishing.
3. Kill chain order — and that **detection should aim left of "actions on objectives"**.
4. **MITRE ATT&CK** = tactics and techniques; know the name and what it is.
5. For any market-specific answer, tie the attack to **availability during market hours** and **settlement risk**.

Authentication, Authorisation and Identity

Core preparation lesson

Authentication factors

| Factor | Examples |

|---|---|

| **Something you know** | Password, PIN, security question |

| **Something you have** | OTP token, smart card, phone, **FIDO2 security key** |

| **Something you are** | Fingerprint, face, iris, voice — **biometrics** |

| Somewhere you are | Geolocation, IP |

| Something you do | Behavioural biometrics, keystroke dynamics |

MFA = two or more **different** factors. A password plus a security question is **not** MFA — both are "something you know". SMS OTP is MFA but the weakest form (SIM swap, SS7 interception); app-based TOTP and **FIDO2/WebAuthn** (phishing-resistant, since the credential is bound to the origin) are stronger.

Biometric metrics: **FAR** (false acceptance — a security failure), **FRR** (false rejection — a usability failure), **CER/EER** (the crossover point where FAR = FRR; **lower CER = better system**). Biometrics cannot be revoked, which is why templates must be stored as irreversible representations, never as raw images.

Password security

Policy: length over complexity, block breached passwords, no forced rotation without cause (current NIST guidance), no password hints, rate limiting and lockout. Storage: **salted, slow KDF** (Argon2/bcrypt/scrypt/PBKDF2). Attacks: brute force, dictionary, **credential stuffing** (reusing leaked pairs), password spraying, rainbow tables, keylogging, shoulder surfing.

Authorisation models

| Model | Basis |

|---|---|

| **DAC** | Owner decides (file permissions) |

| **MAC** | System-enforced labels (classified/secret) — military, SELinux |

| **RBAC** | **Roles** — the standard in enterprises |

| **ABAC** | Attributes of user, resource, environment (time, location, device posture) |

| **Rule-based** | ACL conditions |

Principle of least privilege plus **periodic access review** (recertification) and **privileged access management (PAM)** for admin accounts — including **just-in-time** elevation and session recording.

Federated identity and SSO

- **SSO** — one authentication, many applications.
 - **SAML 2.0** — XML-based; **Identity Provider (IdP)** ↔ **Service Provider (SP)**; common in enterprise web SSO.
 - **OAuth 2.0** — an **authorisation** framework (delegated access via tokens), **not** authentication.
 - **OpenID Connect (OIDC)** — an **authentication** layer on top of OAuth 2.0, adding the **ID token** (a JWT).
 - **JWT** — header.payload.signature, base64url-encoded; signed (JWS) and optionally encrypted (JWE).
Validate the signature, issuer, audience and expiry; never accept `alg: none`.
 - **Kerberos** — ticket-based, symmetric; **KDC = Authentication Server + Ticket Granting Server**; TGT then service tickets; requires **synchronised clocks** (typically within 5 minutes) and is the basis of Active Directory authentication.
 - **LDAP / Active Directory** — directory services; **LDAPS** for encryption.
- > **OAuth vs OIDC** is a favourite interview question: OAuth answers "may this app access that resource on my behalf"; OIDC answers "who is this user".

Indian identity infrastructure

Aadhaar-based e-KYC and e-Sign, **DigiLocker**, **CKYC**, and in the securities market the **KRA** (KYC Registration Agency) system. Relevant constraints: the Supreme Court's Aadhaar judgment restricting mandatory use, **Aadhaar Vault** requirements for storing numbers, **virtual IDs**, and the DPDP Act's consent obligations.

Exam pointers

1. **MFA** needs different factor types, not two of the same.
2. **Lower CER** means a better biometric system.
3. **OAuth** = authorisation; **OIDC** = authentication.
4. Kerberos needs **clock synchronisation** and uses a **TGT**.
5. RBAC is roles; ABAC is attributes; MAC is system-enforced labels.

Cryptography

Core preparation lesson

Classification

Cryptography

■■■ Symmetric (one shared key) : DES, 3DES, AES, RC4, ChaCha20, Blowfish
■■■ Asymmetric (key pair) : RSA, ECC, Diffie-Hellman, ElGamal, DSA
■■■ Hashing (one way, no key) : MD5, SHA-1, SHA-2, SHA-3, bcrypt/scrypt/Argon2

| | Symmetric | Asymmetric |

|---|---|---|

| Keys | One shared secret | Public + private |

| Speed | **Fast** | Slow (100–1000×) |

| Key distribution | **The hard problem** | Solved by publishing the public key |

| Number of keys for n users | $n(n-1)/2$ | $2n$ |

| Provides | Confidentiality | Confidentiality, authentication, **non-repudiation** |

| Use | Bulk data | Key exchange, signatures |

Hybrid cryptosystem — the real-world answer: use asymmetric crypto to exchange a symmetric **session key**, then encrypt the data symmetrically. This is exactly what TLS does.

Symmetric algorithms

| Algorithm | Block | Key | Status |

|---|---|---|---|

| **DES** | 64 bits | **56 bits** effective | **Broken** — brute-forceable |

| **3DES** | 64 bits | 112/168 bits | Deprecated |

| **AES** | **128 bits** | **128 / 192 / 256** bits | **Current standard** (Rijndael); rounds: 10 / 12 / 14 |

| RC4 | Stream | Variable | Broken, removed from TLS |

| ChaCha20 | Stream | 256 bits | Modern, fast in software |

Block cipher modes:

| Mode | Property |

|---|---|

| **ECB** | Each block encrypted independently — **identical plaintext blocks give identical ciphertext**. Never use it; the "ECB penguin" is the standard illustration |

| **CBC** | Chained with an **IV**; needs padding; vulnerable to padding-oracle attacks if done carelessly |

| **CTR** | Turns a block cipher into a stream cipher; parallelisable |

| **GCM** | CTR + authentication tag → **AEAD**: confidentiality *and* integrity. The modern default |

Confusion (relationship between key and ciphertext obscured) and **diffusion** (one plaintext bit affects many ciphertext bits) — Shannon's two design goals. AES uses substitution (S-box) for confusion and permutation/mixing for diffusion; DES uses a **Feistel structure**.

Asymmetric algorithms

- **RSA** — based on the difficulty of **factoring large integers**. Key generation: choose primes p, q ; $n = pq$; $\phi(n) = (p-1)(q-1)$; choose e coprime to $\phi(n)$; $d = e^{-1} \bmod \phi(n)$. Encrypt: $c = m^e \bmod n$. Decrypt: $m = c^d \bmod n$. Typical key size **2048 or 3072 bits**.
- **Diffie-Hellman** — key *exchange*, not encryption; based on the **discrete logarithm** problem; vulnerable to **man-in-the-middle without authentication**. **DHE/ECDHE** provide **forward secrecy**.
- **ECC** — elliptic curve discrete log; **much smaller keys for equivalent strength** (256-bit ECC $\approx$ 3072-bit RSA), which is why it dominates mobile and TLS.
- **DSA / ECDSA / EdDSA** — signature algorithms.

Key management

Generation, distribution, storage, rotation, revocation, destruction. **HSM** (Hardware Security Module) — tamper-resistant hardware that generates and stores keys and performs crypto operations without exporting the key. **KMS** in cloud. **Key escrow**, **split knowledge / dual control**, **forward secrecy**.

Attacks on cryptography

Brute force, **known-plaintext**, **chosen-plaintext**, **chosen-ciphertext**, birthday attack (on hashes), **side-channel** (timing, power, cache, acoustic), replay, **man-in-the-middle**, downgrade attacks, padding oracle, rainbow tables (defeated by **salting**).

Post-quantum

Shor's algorithm would break RSA, DH and ECC on a sufficiently large quantum computer; **Grover's algorithm** halves effective symmetric key strength (so AES-256 remains adequate). NIST has standardised post-quantum algorithms — **ML-KEM (Kyber)** for key encapsulation and **ML-DSA (Dilithium)** / **SLH-DSA (SPHINCS+)** for signatures. **"Harvest now, decrypt later"** is why financial institutions are being asked to plan **crypto-agility** today.

Exam pointers

1. **Symmetric $n(n-1)/2$ keys; asymmetric $2n$.**
2. **AES block size is always 128 bits**, regardless of key length.
3. **Never ECB.** GCM gives confidentiality plus integrity.
4. RSA = factoring; DH/ECC = discrete log; **Shor breaks all three**.
5. TLS uses a **hybrid** scheme — asymmetric for key exchange, symmetric for data.

Defence — SOC, SIEM, Zero Trust and Hardening

Core preparation lesson

Defence in depth

Layered controls so that no single failure is fatal:

Governance & policy
Physical security
Network perimeter (firewall, IPS, DDoS protection)
Segmentation / micro-segmentation / DMZ
Host hardening (patching, EDR, baseline config)
Application security (SDLC, WAF, input validation)
Data (encryption at rest and in transit, DLP, tokenisation)
Identity (MFA, least privilege, PAM)
Monitoring (SIEM, SOC, threat hunting)
Response (IR plan, backups, BCP/DR)

SOC — Security Operations Centre

People: L1 triage, L2 analysis, L3 threat hunting and forensics, incident manager, SOC manager.

Process: use cases, detection rules, **playbooks**, escalation matrix, shift handover, reporting obligations.

Technology: SIEM, **EDR/XDR**, NDR, threat intelligence, **SOAR**, case management.

Metrics: **MTTD** (mean time to detect), **MTTR** (mean time to respond), dwell time, false positive rate, coverage against ATT&CK.

Models: in-house, **MSSP** (managed), hybrid, follow-the-sun. Regulated entities commonly use a **hybrid** model, keeping accountability in-house.

SIEM

Collects logs from firewalls, endpoints, servers, applications, databases, identity systems; **normalises**, **correlates**, **alerts**. Needs **time synchronisation (NTP/PTP)** across sources or correlation is meaningless.

Log sources that matter for a financial entity: authentication and privileged access, database audit logs, application transaction logs, network flow, DNS, email gateway, and change management. **Log retention** must meet regulatory requirements (CERT-In's directions specify retention within India — verify the current period).

SOAR adds automated enrichment and response: isolate a host, disable an account, block an IOC.

Endpoint and network controls

- **EDR/XDR** — behavioural detection, isolation, rollback.
- **Application allow-listing**, disk encryption, **secure baseline (CIS Benchmarks)**, USB control.
- **Patch management** — risk-based, with defined SLAs by severity; **virtual patching** via WAF/IPS where a patch is not yet possible.
- **Network segmentation** — separate the trading core, corporate IT, and internet-facing zones.
- **Micro-segmentation** applies policy per workload.

- **Email security** — SPF/DKIM/DMARC, attachment sandboxing, link rewriting.
- **DLP** — classification-driven prevention of data exfiltration.

Zero trust

Never trust, always verify. Principles (NIST SP 800-207):

1. Verify explicitly — authenticate and authorise on every request, using identity, device posture and context.
2. Least-privilege access — just-enough, just-in-time.
3. **Assume breach** — segment, encrypt everywhere, monitor continuously.

Components: strong identity (MFA/FIDO2), device trust, **micro-segmentation**, **ZTNA** replacing VPN, continuous evaluation. It replaces the idea of a trusted internal network — which matters because most damaging incidents involve **lateral movement** after an ordinary initial compromise.

Secure architecture for a regulated entity

- **Segregate** the trading/matching core from corporate IT, with tightly controlled interfaces.
- **No internet access** from critical servers; jump hosts with session recording for administration.
- **Immutable, WORM audit logs**, shipped off-host in real time.
- **Encrypted backups** with at least one **offline/immutable** copy — the only reliable ransomware control.
- **DR site** with tested RPO/RTO, and **live drills**, not paper plans.
- **Change management** with rollback, and **segregation of duties** between developer, approver and deployer.

Exam pointers

1. **MTTD and MTTR** are the two SOC metrics to quote.
2. SIEM correlation fails without **time synchronisation**.
3. Zero trust = **verify explicitly, least privilege, assume breach** (NIST SP 800-207).
4. The only dependable ransomware control is an **offline/immutable backup that has been restore-tested**.
5. Defence in depth is a structure — in a descriptive answer, draw the layers.

Information Security — Fundamentals

Core preparation lesson

The CIA triad

| Property | Meaning | Threat | Control |

|---|---|---|---|

| **Confidentiality** | Only authorised parties can read | Disclosure, eavesdropping | Encryption, access control, classification |

| **Integrity** | Data is not altered without authorisation | Tampering | Hashing, digital signatures, checksums, WORM storage |

| **Availability** | Systems are usable when needed | DoS, outage, ransomware | Redundancy, DR, backups, DDoS protection |

Extended with **Authenticity**, **Non-repudiation** (you cannot deny having sent it — provided by digital signatures) and **Accountability** (actions traced to an identity, via logging).

> For a market regulator, **integrity and availability** often outrank confidentiality: a trading platform that is down, or an audit trail that can be altered, is a systemic problem.

AAA

Authentication (who are you) → **Authorisation** (what may you do) → **Accounting/Auditing** (what did you do). Protocols: RADIUS, TACACS+, Diameter.

Core vocabulary

| Term | Definition |

|---|---|

| **Asset** | Anything of value — data, system, reputation |

| **Vulnerability** | A weakness |

| **Threat** | A potential cause of harm |

| **Threat actor** | Who might exploit it — insider, criminal, hacktivist, nation state |

| **Exploit** | The means of using a vulnerability |

| **Risk** | **Threat × Vulnerability × Impact** (or likelihood × impact) |

| **Attack surface** | Sum of all exposure points |

| **Attack vector** | The path used |

| **Zero day** | A vulnerability with no patch available |

| **Residual risk** | What remains after controls |

Risk management

Identify assets -> Identify threats & vulnerabilities -> Assess likelihood and impact
-> Treat: AVOID / MITIGATE / TRANSFER (insurance) / ACCEPT
-> Monitor and review

Quantitative: **SLE** (single loss expectancy) = asset value × exposure factor; **ALE** = SLE × **ARO** (annual rate of occurrence). Spend on a control only if it costs less than the ALE reduction.

Qualitative: risk matrices (likelihood × impact), heat maps.

Control types

| By function | By nature |

---|---

| **Preventive** (firewall, MFA) | **Administrative** (policy, training) |

| **Detective** (IDS, SIEM, audit logs) | **Technical/logical** (encryption, ACLs) |

| **Corrective** (backups, patching) | **Physical** (locks, guards, CCTV) |

| **Deterrent**, **Compensating** | |

Security principles

- **Defence in depth** — layered controls; no single point of failure.
- **Least privilege** — the minimum access needed, for the minimum time.
- **Separation of duties** — no one person controls an entire critical process. (In market systems: whoever can place a trade should not be able to alter the trade log.)
- **Need to know**, **fail secure**, **complete mediation**, **open design** (Kerckhoffs's principle — security must rest on the key, not on the secrecy of the algorithm), **economy of mechanism**, **psychological acceptability**.
- **Zero trust** — never trust, always verify; assume breach; verify explicitly; least-privilege access. See the defence note.

Governance

Policy (what and why, mandatory) → **Standard** (mandatory specifics) → **Guideline** (recommended) → **Procedure** (step by step). Supported by **data classification**, **asset inventory**, **acceptable use**, **security awareness training** — and note that awareness training is the control with the best return, because phishing is the dominant initial access vector.

Exam pointers

1. CIA triad, plus **non-repudiation** which comes from **digital signatures**.
2. **Risk = threat × vulnerability × impact**; treatment = avoid / mitigate / transfer / accept.
3. **ALE = SLE × ARO** — a standard numerical.

4. **Kerckhoffs's principle:** the algorithm may be public; only the key is secret.
5. Least privilege + separation of duties is the answer to most "how would you secure X" questions.

Incident Response, Forensics and Threat Intelligence

Core preparation lesson

Incident response lifecycle (NIST SP 800-61)

1. PREPARATION	policy, team, tooling, playbooks, training, contacts
2. DETECTION & ANALYSIS	validate, scope, classify severity, declare an incident
3. CONTAINMENT	short-term (isolate) then long-term (clean path back)
ERADICATION	remove the root cause, close the vulnerability
RECOVERY	restore, monitor closely, confirm normal operation
4. POST-INCIDENT ACTIVITY	lessons learned, RCA, control improvements, reporting

(SANS splits it into six: Preparation, Identification, Containment, Eradication, Recovery, Lessons Learned.)

Severity classification drives escalation and notification. In a regulated entity, **regulatory notification clocks** start at detection, so classification must happen fast and be documented.

Incident response team

CSIRT roles: incident manager, technical lead, forensics, communications/PR, legal and compliance, HR (for insider cases), business owner, and an executive sponsor. **RACI** matrix agreed in advance; **out-of-band communication** (assume corporate email is compromised).

Digital forensics

Principles: preserve the original, work on verified copies, document everything, maintain the **chain of custody**.

- **Order of volatility** (collect most volatile first): CPU registers and cache → **RAM** → network state and running processes → disk → logs on remote systems → archival media.
- **Imaging** with write blockers; **hash the image (SHA-256)** before and after to prove integrity.
- **Artefacts:** memory dump, disk image, event logs, browser history, registry hives, prefetch, **slack space**, deleted file recovery, timeline analysis.
- **Anti-forensics:** log deletion, timestomping, encryption, steganography, fileless techniques.
- **Legal:** in India, electronic evidence admissibility runs through the certificate requirement for electronic records (historically **Section 65B of the Indian Evidence Act**, now carried into the **Bharatiya Sakshya Adhiniyam, 2023**). Know that the certificate requirement exists — it is why chain of custody matters procedurally, not just technically.

Threat intelligence

| Level | Audience | Content |

|---|---|---|

| **Strategic** | Board, management | Trends, actor motivations, risk posture |

| **Tactical** | Security architects | TTPs, ATT&CK techniques |

| **Operational** | SOC, IR | Campaigns, specific actor activity |

| **Technical** | Tooling | **IOCs** — hashes, IPs, domains, URLs |

IOC vs IOA: indicators of **compromise** (artefacts of a past/present breach) vs indicators of **attack** (behaviour in progress). Behaviour-based detection ages better because attackers change infrastructure easily and behaviour slowly — the **pyramid of pain**.

Sharing: **STIX/TAXII** formats; **ISACs** (sector sharing bodies); in India, **CERT-In** advisories and sectoral CERTs (**CERT-Fin** for the financial sector). Sector-wide sharing matters for markets: an attack on one broker is usually a campaign against many.

Threat hunting — proactively searching for compromise on the hypothesis that detection has already failed.

Post-incident

Root cause analysis — five whys, fishbone; distinguish the **technical cause** from the **process cause** (the missing patch vs why patching slipped). **Blameless post-mortems** produce honest facts; blame produces hidden incidents.

Regulatory reporting for a SEBI-regulated entity: incident reporting to **SEBI** under its cyber framework, to **CERT-In** within 6 hours for specified incidents, and to the **Data Protection Board** and affected individuals under DPDP where personal data is breached. Being able to lay out those three parallel obligations cleanly is a strong interview answer.

Exam pointers

1. NIST IR phases: **Preparation** → **Detection & Analysis** → **Containment/Eradication/Recovery** → **Post-incident**.
2. **Order of volatility: RAM before disk.**
3. Hash the forensic image **before and after** to prove integrity; maintain **chain of custody**.
4. **IOC = past artefact, IOA = behaviour in progress.**
5. Know the **three parallel reporting obligations** — SEBI, CERT-In (6 hours), DPDP Board.

IT Act 2000, CERT-In Directions and the DPDP Act

Core preparation lesson

Information Technology Act, 2000

India's primary cyber law, amended substantially in **2008**. It gives legal recognition to **electronic records and digital/electronic signatures**, and creates offences and penalties.

Key sections:

| Section | Content |

|---|---|

| **43** | Penalty and compensation for damage to a computer/system — unauthorised access, downloading, virus introduction, denial of access, **civil liability** |

| **43A** | **Body corporate** handling sensitive personal data must maintain "reasonable security practices"; compensation for negligence (largely superseded in effect by the DPDP Act — verify the current position) |

| **65** | Tampering with computer source documents — 3 years and/or ₹2 lakh |

| **66** | Computer-related offences (dishonestly or fraudulently doing anything in S.43) — **3 years and/or ₹5 lakh** |

| **66B** | Dishonestly receiving a stolen computer resource |

| **66C** | **Identity theft** — 3 years and ₹1 lakh |

| **66D** | **Cheating by personation using a computer resource** (the section used for most online fraud) |

| **66E** | Violation of privacy — capturing/publishing private images |

| **66F** | **Cyber terrorism** — imprisonment up to **life** |

| **67 / 67A / 67B** | Publishing obscene / sexually explicit material / child sexual abuse material electronically |

| **69** | Power to **intercept, monitor or decrypt** in the interest of sovereignty, security, public order etc. |

| **69A** | Power to **block** public access to information |

| **69B** | Monitoring and collecting traffic data for cyber security |

| **70** | **Protected system** — declared critical information infrastructure; **NCIIPC** is the nodal agency |

| **70B** | **CERT-In** as the national nodal agency for incident response |

| **72 / 72A** | Breach of confidentiality and privacy; disclosure of information in breach of a lawful contract |

| **79** | **Safe harbour** for intermediaries, conditional on due diligence |

| **43, 46** | **Adjudicating Officer**; appeals to **TDSAT** (which absorbed the Cyber Appellate Tribunal) |

Institutions: **CCA** (Controller of Certifying Authorities, for digital signatures), **CERT-In** (S.70B), **NCIIPC** (critical infrastructure, S.70), **I4C** and the **National Cyber Crime Reporting Portal** (cybercrime.gov.in).

CERT-In Directions (April 2022, under S.70B(6))

Mandatory for service providers, intermediaries, data centres, body corporates and government organisations:

1. **Report specified cyber incidents within 6 hours** of noticing or being notified.
2. **Synchronise system clocks** to NPL/NIC NTP servers.
3. **Maintain ICT system logs for 180 days**, within Indian jurisdiction.
4. **Data centres, VPS, cloud and VPN providers** must retain subscriber/customer registration data for **five years** after cancellation.
5. **Virtual asset service providers** must maintain KYC and transaction records for five years.
6. Designate a **point of contact** for CERT-In.

> The **six-hour reporting window** is the single most-asked fact from this direction. Know it, and be able to contrast it with the DPDP Act's breach-notification obligation and SEBI's own incident reporting timelines for regulated entities.

Digital Personal Data Protection Act, 2023 (DPDP)

India's data protection law, enacted **11 August 2023**, applying to digital personal data.

Key concepts:

- **Data Principal** — the individual. **Data Fiduciary** — who determines the purpose and means. **Data Processor** — processes on behalf of a fiduciary. **Significant Data Fiduciary (SDF)** — notified on the basis of volume and sensitivity, with extra obligations: a **Data Protection Officer based in India**, an independent **data auditor**, and **Data Protection Impact Assessments**.
- **Lawful bases:** **consent** (free, specific, informed, unconditional, unambiguous, with clear affirmative action) and **certain legitimate uses**.
- **Notice** must accompany consent; consent must be **withdrawable** as easily as it was given. **Consent Managers** are registered with the Board.
- **Rights of the Data Principal:** access, correction and erasure, grievance redressal, nomination. **Duties** are also imposed on principals (a notable feature of this Act).
- **Children:** verifiable parental consent for under-18s; no tracking or targeted advertising at children.
- **Obligations:** purpose limitation, data minimisation, accuracy, **reasonable security safeguards**, **breach notification to the Board and affected principals**, erasure when the purpose is served.
- **Cross-border transfer:** permitted except to countries **restricted** by the Central Government (a negative-list approach).
- **Data Protection Board of India** — adjudicates; appeals lie to **TDSAT**.
- **Penalties:** up to **₹250 crore** for failure to take reasonable security safeguards; up to ₹200 crore for breach-notification failures — no compensation to individuals is provided under the Act itself.

> Implementation is being phased through rules; **verify the current notified status** before quoting timelines in an interview.

How this connects to SEBI

A SEBI-regulated entity is simultaneously: a **body corporate under the IT Act**, a **Data Fiduciary under DPDP**, subject to **CERT-In directions**, and subject to **SEBI's own cyber framework**. An answer that notes this **overlapping compliance burden** — and the need to reconcile different reporting timelines — is exactly the kind of answer an IT-stream panel is looking for.

Exam pointers

1. **CERT-In: 6 hours to report, 180 days of logs, 5 years of subscriber data.**
2. **S.66F cyber terrorism can carry life imprisonment**; S.70 = protected system (NCIIPC).
3. DPDP: **₹250 crore** maximum penalty; appeals to **TDSAT**.
4. **CERT-In is under S.70B**; CCA handles digital signatures.
5. Note the **multi-regulator overlap** — it is the sophisticated point in any answer here.

Hashing, Digital Signatures and PKI

Core preparation lesson

Cryptographic hash functions

A one-way function producing a fixed-size digest. Required properties:

1. **Deterministic** and fast.
2. **Pre-image resistance** — given $h(x)$, you cannot find x .
3. **Second pre-image resistance** — given x , you cannot find $y \neq x$ with $h(x) = h(y)$.
4. **Collision resistance** — you cannot find any pair x, y with the same hash.
5. **Avalanche effect** — one input bit change flips about half the output bits.

| Algorithm | Digest | Status |

|---|---|---|

| MD5 | 128 bits | **Broken** (collisions trivial) |

| SHA-1 | 160 bits | **Broken** (SHAttered, 2017) |

| **SHA-256 / SHA-512** | 256 / 512 bits | **Current standard** |

| SHA-3 (Keccak) | Variable | Different internal design (sponge) |

| **bcrypt, scrypt, Argon2, PBKDF2** | — | **Deliberately slow** — for passwords |

Birthday attack: a collision is expected after about $2^{n/2}$ hashes, which is why a 128-bit digest gives only 64 bits of collision resistance.

Never store passwords as a plain hash. Store `slow_kdf(password + unique salt)` — a **salt** defeats rainbow tables; a **pepper** (a secret stored separately) adds another layer; **work factor** must be tuned upward over time.

MAC and HMAC

A **MAC** proves integrity **and** authenticity using a shared secret key. **HMAC** = $H((K \square \text{opad}) \parallel H((K \square \text{ipad}) \parallel \text{message}))$ — a construction that is secure even with hash functions that have length-extension weaknesses.

MAC vs digital signature: a MAC uses a **shared** key, so either party could have produced it → **no non-repudiation**. A signature uses the sender's **private** key → non-repudiation.

Digital signatures

Signing: hash the message → encrypt the hash with the SENDER'S PRIVATE key → signature

Verification: decrypt the signature with the SENDER'S PUBLIC key → compare to a fresh hash

This provides **integrity, authentication and non-repudiation** — but **not confidentiality**. To get confidentiality as well, encrypt the message with the **recipient's public key** (in practice, with a symmetric session key that is itself encrypted to the recipient).

> **Remember the direction:** encrypt for privacy with the *recipient's public* key; sign for authenticity with the *sender's private* key. Nearly every exam question here tests that one sentence.

Indian legal context: the **IT Act, 2000** gives legal recognition to **digital signatures** (and, after the 2008 amendment, **electronic signatures**). **Certifying Authorities** are licensed by the **Controller of Certifying Authorities (CCA)** under the Act. DSCs are used for filings with MCA, income tax and SEBI systems.

PKI — Public Key Infrastructure

Components:

- **CA (Certifying Authority)** — issues and signs certificates.
- **RA (Registration Authority)** — verifies identity before issuance.
- **Digital certificate (X.509)** — binds a public key to an identity. Fields: version, serial number, signature algorithm, **issuer**, **validity period**, **subject**, subject public key, extensions (SAN, key usage), **CA's signature**.
- **CRL** (certificate revocation list) and **OCSP** (online status check; **OCSP stapling** improves privacy and performance).
- **Root CA → intermediate CA → end-entity certificate** = the **chain of trust**, with the root in the client's **trust store**.

Certificate types: DV, OV, **EV**, wildcard, SAN, code-signing, client certificates. **Self-signed** certificates are fine internally, never for public services.

TLS handshake (simplified, TLS 1.2)

Client Hello (versions, cipher suites, random)
Server Hello (chosen suite, random) + Certificate [+ Server Key Exchange]
Client verifies the certificate chain and validity
Key exchange (ECDHE) -> both derive the same pre-master -> master secret -> session keys
Change Cipher Spec + Finished (both sides)
Encrypted application data

TLS 1.3 removes obsolete algorithms (RSA key transport, static DH, RC4, MD5), mandates **forward secrecy**, and completes the handshake in **1-RTT** (0-RTT for resumption, with replay caveats).

Exam pointers

1. **Sign with the private key, verify with the public key.**
2. **MAC gives no non-repudiation**; a digital signature does.
3. Birthday attack → collisions at about $2^{n/2}$.
4. Passwords need **salt + a deliberately slow KDF**, never plain SHA-256.
5. In India, DSCs are issued by CAs licensed by the **CCA** under the **IT Act, 2000**.

SEBI's Cyber Security & Cyber Resilience Framework (CSCRF)

Core preparation lesson

> If there is one topic where an IT-stream candidate must be stronger than everyone else in the room, it is this one. It is SEBI's own framework, for SEBI's own regulated entities, written for exactly the job you are applying for.

>

> ☐ This is a recent and actively evolving framework, with phased compliance dates and periodic clarifications. Read the **current circular on sebi.gov.in** before your interview and update the specifics below.

Background

Before CSCRF, SEBI's cyber requirements were spread across many circulars issued separately for **MIs**, brokers, depository participants, mutual funds, KRAs, AIFs and others — similar in substance but inconsistent in detail. The **Cybersecurity and Cyber Resilience Framework (CSCRF)**, issued in **August 2024**, consolidated them into one framework for **all SEBI Regulated Entities (REs)**, with a phased implementation through 2025.

Core design

Structured around five (plus one) functions, aligned with the NIST Cybersecurity Framework, moving from "cyber security" to **cyber resilience**:

GOVERNANCE (anchoring function)
IDENTIFY -> PROTECT -> DETECT -> RESPOND -> RECOVER

The addition of **Recover** — and the emphasis on resilience — is the conceptual heart of it: a market regulator's concern is not only preventing incidents but ensuring that critical market functions continue and recover within defined timelines.

Graded, proportionate application

REs are **categorised by size and systemic importance**, with obligations scaled accordingly — broadly:

| Category | Nature |

|---|---|

| **MIs** | Stock exchanges, clearing corporations, depositories — the strictest obligations |

| **Qualified REs** | Large intermediaries above defined thresholds |

| **Mid-size REs** | Moderate obligations |

| **Small-size REs** | Lighter obligations |

| **Self-certification REs** | Smallest entities; simplified compliance |

Thresholds are based on parameters such as number of clients, trading volume, assets under management or custody. **Verify the current thresholds and category names in the live circular.**

Notable requirements to be able to discuss

- **Cyber Capability Index (CCI)** — a scoring mechanism for measuring and reporting cyber maturity, applicable to MIs and Qualified REs, with periodic assessment.
- **SOC requirements** — own SOC, group SOC, or a **market SOC** offered for smaller entities; continuous monitoring expectations.
- **VAPT** — vulnerability assessment and penetration testing at defined intervals, by CERT-In empanelled auditors, with closure timelines for findings.
- **ISO 27001 alignment** and **security audits**.
- **SBOM** — Software Bill of Materials for critical systems: supply-chain transparency, a genuinely modern requirement.
- **Data localisation / data classification**, encryption standards, **API security**.
- **Incident reporting** to SEBI and CERT-In within defined timelines, and **root cause analysis** submission.
- **BCP/DR**: defined **RTO** and **RPO** for critical systems, DR drills, and — for MIs — the long-standing requirement of a **disaster recovery site in a different seismic zone**.
- **Board/management accountability**, a designated **CISO**, and periodic reporting to the board's technology/risk committee.
- **Third-party and vendor risk management**, including cloud service providers.

Related SEBI technology requirements you should know alongside it

- **System audit** requirements for MIs and brokers, with SEBI-prescribed terms of reference.
- **Cyber security and cyber resilience audit** reports submitted periodically.
- **Framework for Adoption of Cloud Services** by REs (2023) — see the cloud note.
- **Algorithmic trading** and **API-based trading** controls.
- **Business continuity for MIs**, including intraday recovery expectations.
- **Structured Digital Database (SDD)** under the PIT Regulations — non-tamperable, time-stamped records of UPSI sharing.

How to talk about it in the interview

A strong answer connects three things:

1. **Why a market regulator cares** — an outage or manipulation at an MI is systemic, not just commercial.
2. **What the framework actually does** — consolidation, graded obligations, resilience and measurable maturity rather than a checklist.
3. **What is hard about it** — smaller intermediaries lack the skills and budget, which is why market SOCs and self-certification tiers exist; and the overlap with CERT-In and DPDP creates a reporting burden that needs rationalising.

That third point — naming a genuine tension rather than reciting the circular — is what distinguishes a candidate who has read it from one who has memorised a summary.

Exam pointers

1. **CSCRF (August 2024)** consolidated SEBI's earlier cyber circulars for **all REs**, phased through 2025.
2. Functions: **Governance + Identify, Protect, Detect, Respond, Recover** — NIST-aligned.
3. Know **CCI, SBOM, VAPT, SOC options, RTO/RPO** by name.
4. MIIIs sit at the top of the graded structure; self-certification REs at the bottom.
5. **Verify the specifics on sebi.gov.in before the interview** — and say so if asked about a detail you are unsure of. Honesty about a moving target reads better than a confidently wrong number.

Standards, Frameworks, Audit and BCP

Core preparation lesson

ISO/IEC 27001

The international standard for an **Information Security Management System (ISMS)**. Certifiable. Built on **Plan-Do-Check-Act**, requiring: scope definition, leadership commitment, **risk assessment and treatment**, a **Statement of Applicability**, controls from **Annex A**, internal audit, management review and continual improvement.

The **2022 revision** reorganised Annex A into **93 controls in four themes** — **Organisational, People, Physical, Technological** — replacing the older 14 domains, and added controls for threat intelligence, cloud security, data leakage prevention and secure coding.

Related: **27002** (implementation guidance), **27005** (risk management), **27017/27018** (cloud, cloud PII), **27701** (privacy), **22301** (business continuity).

NIST

- **Cybersecurity Framework (CSF) 2.0** — **Govern, Identify, Protect, Detect, Respond, Recover**. Voluntary, outcome-based, widely adopted; **SEBI's CSCRF** is aligned to it.
- **SP 800-53** — control catalogue. **SP 800-61** — incident handling. **SP 800-37** — Risk Management Framework. **SP 800-207** — zero trust. **SP 800-145** — cloud definition.
- **NIST is free**, which is why it has become the global default vocabulary.

Other frameworks

| Framework | Focus |

|---|

| **COBIT** | IT **governance** and management — the "are we doing the right things" layer |

| **ITIL** | IT **service management** — incident, problem, change, release |

| **CIS Controls / Benchmarks** | Prioritised, prescriptive hardening |

| **PCI DSS** | Card data security — contractual, not statutory |

| **SOC 1 / SOC 2** | Assurance reports for service organisations (SOC 2 = trust services criteria) |

| **CMMI** | Process maturity, levels 1–5 |

| **COSO** | Enterprise risk management |

| **Basel / RBI IT frameworks** | Financial-sector specific |

COBIT vs ITIL vs ISO 27001 is a standard question: **governance** vs **service management** vs **security management**.

Audit

- **Types:** internal, external, **system audit** (SEBI-mandated for MIs and brokers), **statutory**, **concurrent**.
- **Process:** planning and scoping → control testing (design and operating effectiveness) → evidence gathering → findings and risk rating → management response → **remediation tracking** → follow-up audit.
- **IS audit concepts:** audit trail, **segregation of duties**, change management evidence, access recertification, **sampling**, compensating controls, **materiality**.
- **VAPT** — vulnerability assessment (breadth, automated) vs penetration testing (depth, manual, exploitation). **Red team** (adversary simulation), **blue team** (defence), **purple team** (both, collaboratively). Bug bounty programmes as continuous testing.

Business continuity and disaster recovery

| Term | Meaning |

---|---

| **BIA** | Business Impact Analysis — identifies critical processes and their tolerable downtime |

| **RTO** | **Recovery Time Objective** — how fast you must be back |

| **RPO** | **Recovery Point Objective** — how much data you can afford to lose |

| **MTD / MTPD** | Maximum tolerable downtime |

| **MTTR / MTBF** | Mean time to repair / between failures |

Recovery site options:

| Site | Readiness | Cost | Typical RTO |

---|---|---|---

| **Cold** | Space and power only | Low | Days |

| **Warm** | Hardware and some data | Medium | Hours |

| **Hot** | Fully mirrored, ready | High | Minutes |

| **Active-active** | Both sites live | Highest | Near zero |

Testing: checklist review → **tabletop exercise** → simulation → parallel test → **full interruption test**. A plan that has never been tested is not a plan.

For MIs, SEBI has long required a **DR site in a different seismic zone**, periodic **live DR drills**, and defined recovery timelines for market systems — because the systemic requirement is that the market reopens, not merely that the company survives.

Exam pointers

1. **ISO 27001:2022** → 93 controls, 4 themes.
2. **NIST CSF 2.0** adds "Govern" to the original five functions.
3. **RTO** = time, **RPO** = data. Never mix them up.

4. COBIT = governance, ITIL = service management, ISO 27001 = security management.

5. Hot site = minutes; cold site = days. DR plans must be ****tested****, and the exercise ladder is a quotable structure.

UNIX commands and shell scripting

Core preparation lesson

A shell starts programs, expands variables and connects input/output streams. Know pwd, cd, ls, cp, mv, mkdir, cat, head, tail, wc, sort, uniq, grep, find and chmod. A pipe sends standard output of one command to standard input of the next. Quote expansions to preserve spaces.

Worked script

```
name="study hub"
printf '%s\n' "$name"
for n in 1 2 3; do
    printf '%s\n' "$n"
done
if [ "$#" -gt 0 ]; then
    printf '%s\n' "$1"
fi
```

\$0 is the script name, \$1 the first argument, \$# the argument count and \$? the previous exit status. Zero normally means success. A function's return communicates status; output is written separately. Use grep -E for extended regular expressions and distinguish globbing from regex.

Drill

Trace a loop with a counter; predict the result of a conditional; explain `sort names.txt | uniq -c`. Repeated values must be adjacent for uniq to combine them. Practise only on disposable example files.

[GNU Bash manual](<https://www.gnu.org/software/bash/manual/bash.html>)

Strings, pattern matching, matrices and JSON

Core preparation lesson

Strings are sequences; substring bounds and mutability depend on the language. Trace indices explicitly. Repeated concatenation of immutable strings can repeatedly copy existing content. For pattern search, a simple scan compares the pattern at each possible starting position; prefix-based algorithms can avoid repeating comparisons.

Worked example

Text ABABA contains ABA at zero-based positions 0 and 2. A method that jumps by the entire pattern length after a match may miss the overlap. State whether overlaps are included before counting matches.

Matrices and JSON

For an m by n matrix, check row and column bounds independently. In row-major storage, $\text{offset} = \text{row} * n + \text{column}$ using zero-based indices. JSON has objects, arrays, strings, numbers, booleans and null; it does not support comments or trailing commas. A JSON object key is a string. Parse before accessing nested values and check whether a key exists.

Drill

Trace a two-pointer palindrome check on an empty string, one character and abba. Trace the transpose of a 2 by 3 matrix. Count frequencies using a map, then compare the time complexity with a nested-loop solution.

Data warehousing, ETL and analytical models

Core preparation lesson

An operational database supports individual transactions; a warehouse combines historical data for analysis. Extract obtains source data, transform standardises and validates it, and load writes the target. ELT loads before performing transformations. Neither acronym alone guarantees data quality.

Model example

A trade fact table contains measures such as quantity and value plus keys to date, instrument and venue dimensions. Declare its grain: one row per executed trade. A daily aggregate has a different grain and cannot be mixed without care. A star schema connects facts directly to dimensions; a snowflake normalises dimension structures.

Analytical operations

Roll-up aggregates daily values into months. Drill-down moves to finer detail. Slice selects one dimension value; dice selects a subcube. A data mart focuses on one subject area. Metadata records definitions, provenance and transformations.

Quality drill

Ten source rows include a duplicated transaction, one missing key and two inconsistent date formats. State which rows you would quarantine and why. Reconcile row counts and totals after each stage; preserve raw data so transformations can be audited. Slowly changing dimension type 1 overwrites history; type 2 retains versions with validity intervals.

APIs, REST, Microservices and Integration

Supplementary reference / interview

REST constraints (Fielding)

1. **Client-server** separation
2. **Stateless** — each request carries all context
3. **Cacheable**
4. **Uniform interface** — resources, representations, self-descriptive messages, HATEOAS
5. **Layered system**
6. Code on demand (optional)

Resource-oriented design:

GET	/api/v1/trades?symbol=INFY&from=2026-01-01	list, filtered
GET	/api/v1/trades/12345	one
POST	/api/v1/trades	create
PUT	/api/v1/trades/12345	replace
PATCH	/api/v1/trades/12345	partial update
DELETE	/api/v1/trades/12345	remove

Nouns not verbs, plural collections, versioning in the path or a header, consistent error bodies, pagination ('limit'/'offset' or cursor), filtering and sorting as query parameters.

REST vs SOAP vs GraphQL vs gRPC

| | REST | SOAP | GraphQL | gRPC |

|---|---|---|---|

| Format | Usually JSON | **XML only** | JSON over a query language | **Protocol Buffers (binary)** |

| Transport | HTTP | HTTP, SMTP, others | HTTP | **HTTP/2** |

| Contract | OpenAPI (optional) | **WSDL (strict)** | Schema | `.proto` |

| Strengths | Simple, cacheable, ubiquitous | Formal contracts, **WS-Security**, ACID via WS-AT | Client fetches exactly what it needs; no over/under-fetching | Fast, streaming, strongly typed |

| Weaknesses | Over/under-fetching | Verbose, heavy | Caching is harder; query cost control needed | Not browser-native without a proxy |

SOAP still appears in banking and older regulatory integrations, which is why it is worth knowing rather than dismissing.

API security

- **Authentication:** API keys (weak alone), OAuth 2.0 / OIDC, mTLS for server-to-server.
- **Authorisation:** scopes, RBAC/ABAC — and check it on **every** request, per object (**broken object level authorisation** is the top API vulnerability).
- **Transport:** TLS everywhere; certificate pinning for mobile.

- **Input validation and schema enforcement**; reject unknown fields.
- **Rate limiting and quotas** per client; **429** responses with `Retry-After`.
- **Logging and monitoring** of API usage; anomaly detection on access patterns.
- **API gateway** — a single entry point handling authentication, rate limiting, routing, transformation and observability.
- **OWASP API Security Top 10** — know it exists and that **BOLA/IDOR** heads the list.

> For a market context: broker APIs that let clients place orders programmatically need authentication, per-client throttling, **order-rate limits**, and full audit trails — and SEBI has been tightening expectations around API-based and algorithmic access precisely for these reasons.

Integration patterns

Synchronous request-response (simple, coupled, fails together) vs **asynchronous** messaging (decoupled, resilient, eventually consistent). **Webhooks** for server-to-server callbacks — secure them with signatures and replay protection. **File-based batch integration** (still the backbone of much regulatory reporting), **ETL**, and **ESB** in older enterprise architectures.

Distributed transactions: two-phase commit is usually impractical across services; use the **saga pattern** (a sequence of local transactions with compensating actions) and **idempotency keys**.

Documentation and contracts

OpenAPI/Swagger for REST, **WSDL** for SOAP, **.proto** for gRPC, **AsyncAPI** for event-driven systems. **Contract testing** (consumer-driven) prevents a provider change from silently breaking consumers — important where regulated entities integrate with exchange and depository APIs.

Exam pointers

1. **REST is stateless**; that is the constraint most often violated in practice.
2. **SOAP uses XML and WSDL**; REST is an architectural style, not a protocol.
3. **GraphQL solves over-fetching**, at the cost of caching and query-cost control.
4. **BOLA/IDOR is the top API vulnerability** — authorise per object, every time.
5. **Sagas with compensating transactions** replace 2PC across services.

Web and System Architecture

Supplementary reference / interview

Tiers

| Architecture | Layers |

|---|---|

| **1-tier** | Everything on one machine |

| **2-tier** | Client ↔ database |

| **3-tier** | **Presentation ↔ Application/Business logic ↔ Data** |

| **N-tier** | Adds caching, messaging, integration layers |

The 3-tier split exists so that each layer can be **scaled, secured and changed independently** — the data tier never faces the internet, the presentation tier holds no business rules.

Servers

- **Web server** (Nginx, Apache) — serves static content, terminates TLS, reverse proxies.
- **Application server** (Tomcat, JBoss, Gunicorn) — executes business logic.
- **Database server** — persistence.
- **Reverse proxy / load balancer** in front; **CDN** at the edge.

Scaling

| | Vertical (scale up) | Horizontal (scale out) |

|---|---|---|

| Method | Bigger machine | More machines |

| Limit | Hardware ceiling | Practically unbounded |

| Complexity | Low | Needs statelessness, load balancing, distributed state |

| Failure | Single point | Redundant |

Stateless application servers are the precondition for horizontal scaling — session state moves to a shared store (Redis) or a signed token (JWT).

Load balancing algorithms: round robin, weighted round robin, least connections, IP hash (**sticky sessions**), least response time. **L4** (transport, fast) vs **L7** (application-aware routing, TLS termination, header-based rules).

Caching

Layers: browser cache → CDN → reverse proxy → application cache (Redis/Memcached) → database query cache.

Strategies: cache-aside (lazy loading), read-through, write-through, write-behind. **Eviction:** LRU, LFU, TTL. **Cache invalidation** is genuinely one of the hard problems — stale prices in a market application are worse than slow ones, which is why financial UIs often push rather than cache.

Cache stampede — many requests miss simultaneously and hammer the origin; mitigated by request coalescing and jittered TTLs.

Asynchronous processing

Message queues (RabbitMQ, SQS) and **event streams** (Kafka) decouple producers from consumers, absorb spikes, and allow retries. Patterns: work queue, publish-subscribe, dead-letter queue, **idempotent consumers** (because at-least-once delivery means duplicates will happen).

Delivery guarantees: at-most-once, **at-least-once** (the usual practical choice), exactly-once (hard, and usually achieved as at-least-once plus idempotency).

Reliability patterns

Timeout, **retry with exponential backoff and jitter**, **circuit breaker** (stop calling a failing dependency and fail fast), **bulkhead** (isolate resource pools), **rate limiting**, **graceful degradation**, **idempotency keys**.

These matter in trading and payment systems specifically: a blind retry of an order submission without an idempotency key can place the order twice. That is a concrete, memorable example to use in an answer.

Observability

Logs (discrete events), **metrics** (aggregated numbers, time series), **traces** (a request's path across services). Correlation IDs to stitch a request together across components. Alerting on **symptoms users feel** (error rate, latency) rather than on every internal anomaly.

Exam pointers

- 3-tier** = presentation, application, data — and why each is separated.
- Horizontal scaling requires **stateless** application servers.
- L4 vs L7 load balancing** — transport vs application aware.
- Circuit breaker, retry with backoff, idempotency** — name these three in any resilience answer.
- At-least-once delivery + idempotent consumers** is the practical messaging design.

HTML, CSS, JavaScript and the DOM

Supplementary reference / interview

HTML

Structure and semantics. Key elements: `<html>`, `<head>` (meta, title, link, script), `<body>`, semantic tags (`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`), `<form>` with `<input>`/`<select>`/`<textarea>`/`<button>`, `<table>` with `<thead>`/`<tbody>`/`<tr>`/`<td>`, `<img>` with **alt text**.

HTML5 additions: semantic elements, `<canvas>`, `<video>`/`<audio>`, **localStorage / sessionStorage**, **WebSockets**, Web Workers, geolocation, form validation attributes (`required`, `pattern`, `type="email"`).

Accessibility: alt text, label-input association, ARIA roles, keyboard navigation, colour contrast. Accessibility is a legal and inclusion issue for any investor-facing government or regulated platform — worth mentioning.

CSS

Selectors: element, `.class`, `#id`, attribute, descendant, child (`>`), pseudo-class (`:hover`, `:nth-child`), pseudo-element (`::before`).

Specificity (highest first): inline style → id → class/attribute/pseudo-class → element. `!important` overrides everything and should be avoided.

Box model: content → padding → border → margin. `box-sizing: border-box` makes width include padding and border — the sane default.

Layout: `display` (block, inline, inline-block, flex, grid, none), **Flexbox** (one-dimensional), **Grid** (two-dimensional), `position` (static, relative, absolute, fixed, sticky), float (legacy).

Responsive design: media queries, relative units (% , em, rem, vw, vh), mobile-first, `viewport` meta tag.

JavaScript

Types: primitives (string, number, boolean, null, undefined, symbol, bigint) and objects. Dynamically and weakly typed, which is why `==` (coerces) differs from `===` (strict) — always use `===`.

Scope and hoisting: `var` is function-scoped and hoisted (initialised as undefined); `let` and `const` are block-scoped with a **temporal dead zone**.

Closures — a function retaining access to its defining scope. The basis of module patterns, private state and callbacks.

this — depends on the call site: method call (the object), plain call (undefined in strict mode / global otherwise), `new` (the new object), explicit `call`/`apply`/`bind`. **Arrow functions** do not bind their own `this` — they inherit it lexically. This is the single most-asked JavaScript question.

Asynchrony: the **event loop** with a call stack, task queue and microtask queue. **Callbacks** → **Promises** → **async/await**. **Promises:** pending, fulfilled, rejected; `.then`, `.catch`, `Promise.all` / `allSettled` / `race`.

```
async function loadTrades() {
  try {
```

```

const res = await fetch('/api/trades');
if (!res.ok) throw new Error(res.status);
const data = await res.json();
render(data);
} catch (e) {
  showError(e.message);
}
}

```

DOM: `querySelector`, `createElement`, `append`, `addEventListener`, **event bubbling and capturing**, `event.preventDefault()`, **event delegation** (one listener on a parent instead of many on children).

ES6+: arrow functions, destructuring, spread/rest, template literals, modules (`import`/`export`), classes, optional chaining `?.`, nullish coalescing `??`.

Frameworks (recognise, do not memorise)

React (component-based, virtual DOM, hooks), **Angular** (full framework, TypeScript, two-way binding), **Vue**. **SPA vs SSR vs static** rendering, and why SSR helps first-paint and SEO.

Exam pointers

- CSS specificity:** `inline` > `id` > `class` > `element`.
- `===` does not coerce; `==` does.
- Arrow functions inherit `this` lexically.
- `let`/`const` are **block-scoped**; `var` is function-scoped and hoisted.
- Event delegation** relies on **bubbling**.

High Availability, Disaster Recovery and Reliability

Supplementary reference / interview

Availability arithmetic

| Availability | Downtime per year |

|---|---|

| 99% ("two nines") | ~3.65 days |

| 99.9% | ~8.77 hours |

| 99.99% | ~52.6 minutes |

| 99.999% ("five nines") | ~5.26 minutes |

****Availability = MTBF / (MTBF + MTTR)**** — so you can improve availability by failing less often **or** by recovering faster. In distributed systems, ****reducing MTTR is usually the cheaper lever****, which is why automated failover and good runbooks matter more than heroic hardware.

****Components in series**** multiply availability (each dependency drags the total down); ****components in parallel**** with redundancy improve it: combined failure probability = product of individual failure probabilities.

Redundancy patterns

- ****Active-passive**** — a standby takes over on failure; simpler, wasted capacity, failover time.
- ****Active-active**** — all nodes serve traffic; better utilisation and instant failover, but needs state coordination.
- ****N+1 / N+M**** redundancy; ****hot / warm / cold**** standby.
- ****Multi-AZ**** (within a region) and ****multi-region**** (across geography). ****Multi-cloud**** for provider-level risk.

****Single points of failure**** to hunt for: one load balancer, one database primary, one DNS provider, one identity provider, one power feed, one network path, ****one person who knows how it works****.

Database availability

****Replication:**** synchronous (zero RPO, latency-bound distance) vs asynchronous (distance-tolerant, potential data loss). ****Primary-replica**** vs ****multi-primary****. ****Automatic failover**** with a quorum to avoid ****split brain****. ****Read replicas**** for scaling reads. ****Sharding**** for write scaling.

RTO, RPO and the plan

- ****RTO**** — how quickly service must be restored.
- ****RPO**** — how much data loss is tolerable.
- ****BIA**** determines both, per system, from business impact rather than engineering preference.

****Backup discipline:**** the ****3-2-1 rule**** — three copies, two media types, one off-site — extended today to ****3-2-1-1-0****: one ****immutable/offline**** copy and ****zero errors on a verified restore test****. An untested backup is

a hypothesis, not a backup.

Reliability engineering

SLI (a measured indicator: latency, error rate) → **SLO** (an internal target) → **SLA** (an external commitment with consequences). **Error budget** = $1 - \text{SLO}$; if you have budget left, ship faster; if you have burned it, stop and stabilise. This reframes reliability as a resource rather than an absolute.

Chaos engineering — deliberately injecting failure to verify resilience assumptions. **Game days** and **DR drills** — the organisational equivalent.

Graceful degradation — shed non-essential features to keep the core running. For a trading platform: keep order entry and position view alive even if charting, research and analytics are dropped. Deciding that hierarchy **in advance** is the design work.

In the market context

MLIs must meet **SEBI's continuity expectations**: DR in a different seismic zone, defined recovery timelines, live drills with member participation, and reporting of technical glitches with root cause analysis. The engineering vocabulary above is exactly what those obligations translate into — being able to speak both languages in one answer is the point of this note.

Exam pointers

1. Memorise the **nines table** — $99.9\% \approx 8.8$ hours a year.
2. **Availability** = $\text{MTBF} / (\text{MTBF} + \text{MTTR})$; cutting MTTR is usually cheaper.
3. **Synchronous replication** = RPO 0 but distance-limited.
4. **3-2-1-1-0 backup rule**, with a **tested restore**.
5. **Error budget** = $1 - \text{SLO}$ — the cleanest way to express the speed/stability trade-off.

HTTP, Sessions and Web Protocols

Supplementary reference / interview

Request and response

Request: method, path, HTTP version, headers, optional body.

Response: status line, headers, body.

Common headers: `Host`, `User-Agent`, `Accept`, `Content-Type`, `Content-Length`, `Authorization`, `Cookie`/`Set-Cookie`, `Cache-Control`, `ETag`, `Origin`, `Referer`, and the security headers below.

Methods and properties

Method	Safe	Idempotent	Body
GET	Yes	Yes	No
HEAD	Yes	Yes	No
POST	No	No	Yes
PUT	No	Yes	Yes
PATCH	No	No	Yes
DELETE	No	Yes	No
OPTIONS	Yes	Yes	No

Safe = no server state change. **Idempotent** = repeating it has the same effect as doing it once. POST is neither, which is exactly why double-submitting a form can create two orders.

Status codes

1xx informational · **2xx** success (200 OK, 201 Created, 204 No Content) · **3xx** redirection (301 permanent, 302 found, **304 Not Modified**) · **4xx** client error (400, **401 unauthenticated**, **403 forbidden**, 404, 409 conflict, **429 too many requests**) · **5xx** server error (500, 502, 503, 504).

State management

HTTP is **stateless**. State is added by:

- **Cookies** — `Set-Cookie` with attributes: **HttpOnly** (not readable by JavaScript — blunts XSS theft), **Secure** (HTTPS only), **SameSite** (Strict/Lax/None — the main CSRF defence), `Domain`, `Path`, `Expires`/`Max-Age`.
- **Server-side sessions** — a session ID in a cookie, data on the server (or in Redis for horizontal scaling).
- **Tokens (JWT)** — stateless, self-contained; easy to scale, harder to revoke. Store carefully; a JWT in `localStorage` is readable by any injected script.

- **Hidden form fields, URL parameters** — legacy and leaky.

Security headers

| Header | Effect |

|---|---|

| **HSTS** (`Strict-Transport-Security`) | Force HTTPS for a period |

| **CSP** (`Content-Security-Policy`) | Restrict which sources may load scripts, styles and frames — the strongest XSS mitigation |

| `X-Content-Type-Options: nosniff` | Stop MIME sniffing |

| `X-Frame-Options` / CSP `frame-ancestors` | Prevent **clickjacking** |

| `Referrer-Policy` | Limit referrer leakage |

| `Permissions-Policy` | Restrict browser features |

CORS

The **same-origin policy** blocks cross-origin reads by default. **CORS** relaxes it explicitly: the server sends `Access-Control-Allow-Origin` and related headers; non-simple requests are preceded by an **OPTIONS** preflight. CORS is a **browser-enforced relaxation**, not a server security control — a common misunderstanding worth stating correctly.

HTTP versions

- **HTTP/1.1** — persistent connections, pipelining (rarely used), head-of-line blocking.

- **HTTP/2** — binary framing, **multiplexing** many streams over one TCP connection, header compression (HPACK), server push. Still suffers TCP-level head-of-line blocking.

- **HTTP/3** — over **QUIC (UDP)**, independent streams, faster connection setup, better on lossy mobile networks.

WebSocket

Full-duplex over a single TCP connection, established via an HTTP **Upgrade** handshake (`ws://`, `wss://`). The right choice for **live price tickers, order updates and dashboards** — the alternatives being long polling and Server-Sent Events (one-way, simpler, auto-reconnecting).

Exam pointers

1. **PUT and DELETE are idempotent; POST and PATCH are not.**
2. **401 = not authenticated, 403 = authenticated but forbidden.**
3. **HttpOnly** blunts XSS cookie theft; **SameSite** is the CSRF defence.

4. **CORS is browser-enforced**, and relaxes the same-origin policy rather than tightening security.
5. **HTTP/3 runs on QUIC over UDP.**

Containers, Orchestration and Delivery Infrastructure

Supplementary reference / interview

Containers

A container packages an application with its dependencies, sharing the host kernel through **namespaces** (isolation of PID, network, mount, user) and **cgroups** (resource limits).

Docker concepts: `Dockerfile` → **image** (layered, immutable) → **container** (a running instance) → **registry** (Docker Hub, ECR, private). Layer caching, multi-stage builds (compile in one stage, copy only the artefact into a minimal runtime image), `.dockerignore`, volumes for persistence, bridge/host/overlay networking.

Container security: run as a **non-root user**, minimal base images (distroless/alpine), scan images for CVEs, sign images, read-only root filesystem, drop Linux capabilities, never bake secrets into images, and remember that **a container is not a security boundary as strong as a VM**.

Kubernetes

| Object | Purpose |

|---|

| **Pod** | Smallest deployable unit; one or more containers sharing network and storage |

| **ReplicaSet** | Maintains a desired number of pod replicas |

| **Deployment** | Declarative updates and **rolling deployments/rollbacks** |

| **Service** | Stable virtual IP and DNS name for a set of pods (ClusterIP, NodePort, LoadBalancer) |

| **Ingress** | HTTP routing into the cluster |

| **ConfigMap / Secret** | Configuration and sensitive values |

| **StatefulSet** | Stable identities and storage for stateful workloads |

| **DaemonSet** | One pod per node (log shippers, agents) |

| **Job / CronJob** | Batch and scheduled work |

| **Namespace** | Logical partitioning |

| **PV / PVC** | Persistent storage |

Control plane: API server, **etcd** (the cluster's state store — back it up, secure it, it is the crown jewel), scheduler, controller manager. **Node components:** kubelet, kube-proxy, container runtime.

Kubernetes security: RBAC, network policies (default-deny between namespaces), pod security standards, admission controllers (OPA/Gatekeeper), secrets encryption at rest, image provenance.

Delivery infrastructure

- **Load balancers and reverse proxies** — Nginx, HAProxy, cloud LBs.

- **CDN** — caches static assets at the edge; also absorbs DDoS.

- **Service mesh** (Istio, Linkerd) — sidecar proxies providing **mTLS between services**, traffic shaping, retries and observability without changing application code.
- **API gateway** — authentication, rate limiting, routing at the edge of the service estate.
- **Message brokers** — Kafka, RabbitMQ for asynchronous decoupling.

Deployment patterns

Blue-green (two full environments, instant switch and rollback), **canary** (a small share of traffic first, monitored), **rolling** (replace instances gradually), **feature flags** (deploy dark, release later). Each has a different rollback story — which is the criterion that matters in a regulated environment where a bad release during market hours is a reportable event.

Exam pointers

1. Containers isolate with **namespaces + cgroups**, sharing the **host kernel**.
2. **Pod** is the smallest deployable unit in Kubernetes; Deployment manages ReplicaSets.
3. **etcd** holds cluster state — securing and backing it up is essential.
4. **Service mesh** gives mTLS and observability without code changes.
5. **Blue-green** switches instantly; canary exposes a small percentage first.

Web Application Security — OWASP

Supplementary reference / interview

OWASP Top 10 themes (know these by name and fix)

| Risk | What it is | Fix |

|---|---|---|

| **Broken access control** | Users acting outside intended permissions; **IDOR** | Deny by default; **server-side** authorisation checks on every object |

| **Cryptographic failures** | Weak or missing encryption, secrets in code | TLS everywhere, strong algorithms, proper key management |

| **Injection (SQLi, command, LDAP)** | Untrusted input interpreted as code | **Parameterised queries**, ORMs, input validation, least-privilege DB accounts |

| **Insecure design** | Missing controls by design | Threat modelling, secure design patterns |

| **Security misconfiguration** | Defaults, verbose errors, open cloud storage | Hardening baselines, automated configuration checks |

| **Vulnerable components** | Outdated libraries | **SCA/dependency scanning**, **SBOM**, patching |

| **Identification and authentication failures** | Weak passwords, session flaws | MFA, secure session management, lockouts |

| **Software and data integrity failures** | Unsigned updates, insecure CI/CD | Code signing, pipeline hardening, verify dependencies |

| **Security logging and monitoring failures** | No detection | Centralised logging, alerting, tested response |

| **SSRF** | Server coerced into requesting internal resources | Allow-lists, block link-local/metadata addresses, network egress control |

The attacks you must be able to explain

SQL injection

```
-- vulnerable
"SELECT * FROM users WHERE name = '" + input + "'"
-- input: ' OR '1'='1
```

Fix: parameterised/prepared statements. Input escaping and WAFs are secondary. Also limit DB account privileges and avoid dynamic SQL entirely where possible. **Blind SQLi** (boolean and time-based) extracts data without visible output.

Cross-Site Scripting (XSS)

Injecting script that runs in another user's browser.

- **Stored** — persisted on the server (a comment field) and served to everyone.
- **Reflected** — echoed back from the request (a search term in a URL).
- **DOM-based** — entirely client-side, via unsafe `innerHTML` or URL fragment handling.

Fixes: context-aware output encoding, Content-Security-Policy, HttpOnly cookies, safe DOM APIs (textContent, not innerHTML), framework auto-escaping, input validation.

Cross-Site Request Forgery (CSRF)

The victim's browser is tricked into sending an authenticated request (transfer funds, change email) to a site where they are logged in.

Fixes: anti-CSRF tokens (synchroniser pattern), SameSite cookies, re-authentication for sensitive actions, and checking Origin/Referer.

XSS vs CSRF: XSS exploits the user's trust in the site; CSRF exploits the site's trust in the user's browser. Memorise that line.

Others

Clickjacking (invisible iframe — fixed by frame-ancestors), session fixation (regenerate the session ID on login), directory traversal (../etc/passwd — canonicalise and allow-list paths), file upload attacks (validate type by content, store outside the web root, never execute), insecure deserialisation, XXE (disable external entities in XML parsers), open redirect, business logic flaws (negative quantities, price manipulation in the request body — a genuinely relevant class for trading applications).

Secure development

Threat modelling with STRIDE: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege. DREAD for rating. Secure SDLC: requirements → threat model → secure coding standards → SAST (static, in the pipeline) → DAST (running application) → IAST/RASP → penetration test → monitoring.

Secrets management: never in source control; use a vault, rotate regularly, scan repositories for leaked keys.

Exam pointers

- Parameterised queries are the SQL injection fix — say it first.
- XSS = trust in the site; CSRF = trust in the browser.
- CSP is the strongest XSS mitigation; SameSite is the CSRF one.
- STRIDE for threat modelling — know all six letters.
- SAST is static, DAST is dynamic; both belong in the pipeline.

Arithmetic — Percentage, Ratio, Profit, Interest, Time and Work

Core preparation lesson

> This note is your formula sheet. Revise it weekly; the exam rewards recall speed, not derivation.

Percentage

- **Fraction-percentage table (memorise, it saves minutes):**

$1/2 = 50\%$ · $1/3 = 33.33\%$ · $1/4 = 25\%$ · $1/5 = 20\%$ · $1/6 = 16.67\%$ · $1/7 = 14.28\%$ · $1/8 = 12.5\%$ · $1/9 = 11.11\%$ · $1/11 = 9.09\%$ · $1/12 = 8.33\%$ · $1/16 = 6.25\%$ · $1/20 = 5\%$

- **Successive changes of a% and b%: net = $a + b + \frac{ab}{100}$** (use signs).

- If a value increases by x%, to bring it back the decrease needed is $\frac{x}{100+x} \times 100$.

- **If A is x% more than B, then B is $\frac{x}{100+x} \times 100$ % less than A.**

Ratio and proportion

- $a : b = c : d \Rightarrow ad = bc$. Componendo-dividendo: $\frac{a+b}{a-b} = \frac{c+d}{c-d}$.

- If $a : b = 2 : 3$ and $b : c = 4 : 5$, then $a : b : c = 8 : 12 : 15$ (multiply to match b).

- **Partnership:** profit share $\propto$ (capital × time).

Averages

- Average = sum / count. **Weighted average = $\frac{\sum(w_i x_i)}{\sum w_i}$**

- **Alligation:** (Cheaper quantity)/(Dearer quantity) = (Dearer price – Mean)/(Mean – Cheaper price). Use it for mixtures, average speeds and blended interest rates — it is the fastest tool in the section.

- Average of the first n naturals = $\frac{n+1}{2}$.

Profit and loss

- Profit% = $\frac{SP - CP}{CP} \times 100$. **Always on cost price** unless stated otherwise.

- **$SP = CP \times \frac{100 + \text{profit}\%}{100}$**

- Successive discounts of a% and b%: net discount = $a + b - \frac{ab}{100}$.

- If an article is sold at two prices with equal profit% and loss%, there is always a **net loss of $\frac{x^2}{100}\%$** .

- **Marked price, discount, cost price** chains: work backwards from SP.

Simple and compound interest

$SI = P \times R \times T / 100$

$CI = P[(1 + R/100)^T - 1]$

Amount (CI) = $P(1 + R/100)^T$

Half-yearly: rate R/2, time 2T. Quarterly: rate R/4, time 4T.

- **Difference between CI and SI for 2 years = $P(R/100)^2$**
- For 3 years: **$P(R/100)^2(3 + R/100)$**
- **Rule of 72:** money doubles in approximately $72/R$ years at compound interest.
- SI doubles a sum in $100/R$ years.

Time, speed and distance

- Speed = distance / time. **$1 \text{ km/h} = 5/18 \text{ m/s}$; $1 \text{ m/s} = 18/5 \text{ km/h}$.**
- **Average speed for equal distances at speeds x and $y = 2xy/(x+y)$** (harmonic mean). For equal **times**, it is the arithmetic mean.
- **Relative speed:** opposite directions $\rightarrow$ add; same direction $\rightarrow$ subtract.
- **Trains:** a train crossing a pole covers its own length; crossing a platform covers length + platform.
- **Boats:** downstream = $b + s$; upstream = $b - s$; **$b = (\text{down} + \text{up})/2$** , **$s = (\text{down} - \text{up})/2$** .

Time and work

- If A does a job in n days, A's one-day work = **$1/n$** .
- A and B together: **$1/a + 1/b$** ; combined time = **$ab/(a+b)$** .
- **Efficiency is inversely proportional to time.** If A is twice as efficient as B, A takes half the time.
- **Pipes and cisterns:** inlet positive, outlet negative; same arithmetic as work.
- **$M_1D_1H_1/W_1 = M_2D_2H_2/W_2$** — the universal work-equivalence formula.

Exam pointers

1. **Successive percentage change: $a + b + ab/100$** . Signs matter.
2. **CI – SI for 2 years = $P(R/100)^2$** .
3. **Average speed for equal distances = $2xy/(x+y)$** — never the plain average.
4. **Alligation** is the fastest route for every mixture-type question.
5. Equal profit% and loss% on two sales always gives a **net loss of $x^2/100$ %**.

Quant Foundations — Number System and Algebra

Core preparation lesson

Number classification

Natural $\rightarrow$ Whole $\rightarrow$ Integers $\rightarrow$ Rational $\rightarrow$ Real. **Prime numbers** have exactly two factors (**1** is not prime; **2** is the only even prime). Composite, co-prime, perfect numbers.

Divisibility rules

Divisor Rule
--- ---
2 Last digit even
3 Digit sum divisible by 3
4 Last two digits divisible by 4
5 Ends in 0 or 5
6 Divisible by 2 and 3
8 Last three digits divisible by 8
9 Digit sum divisible by 9
11 Difference of alternate digit sums divisible by 11

Factors, HCF and LCM

For $N = a^p \times b^q \times c^r$ (prime factorisation):

- **Number of factors** = $(p+1)(q+1)(r+1)$
- Sum of factors = $[(a^{p+1}-1)/(a-1)] \times \dots$ for each prime
- **HCF $\times$ LCM** = product of the two numbers (for two numbers only)
- HCF = product of **common primes with the lowest** powers; LCM = **all primes with the highest** powers

Remainder theorems

- **Remainder of a product** = product of remainders (then reduce).
- **Fermat's little theorem:** if p is prime and a is not divisible by p , then $a^{(p-1)} \equiv 1 \pmod{p}$.
- **Cyclicity of unit digits:** most digits repeat with a cycle of 4 (2: 2,4,8,6; 3: 3,9,7,1; 7: 7,9,3,1; 8: 8,4,2,6), while 0, 1, 5, 6 are constant and 4, 9 have cycles of 2. **To find the unit digit of a^b , reduce $b \pmod{4}$.**

Essential algebra identities

$$\begin{aligned}(a + b)^2 &= a^2 + 2ab + b^2 \\(a - b)^2 &= a^2 - 2ab + b^2 \\a^2 - b^2 &= (a + b)(a - b) \\(a + b)^3 &= a^3 + b^3 + 3ab(a + b)\end{aligned}$$

$$a^3 + b^3 = (a + b)(a^2 - ab + b^2)$$

$$a^3 - b^3 = (a - b)(a^2 + ab + b^2)$$

$$a^3 + b^3 + c^3 - 3abc = (a + b + c)(a^2 + b^2 + c^2 - ab - bc - ca)$$

If $a + b + c = 0$, then $a^3 + b^3 + c^3 = 3abc$

Quadratic equations

For $ax^2 + bx + c = 0$:

- Roots = $\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$

- Sum of roots = $-b/a$, product = c/a

- Discriminant $D = b^2 - 4ac$: $D > 0$ real and distinct; $D = 0$ real and equal; $D < 0$ complex.

Progressions

- AP: nth term = $a + (n-1)d$; Sum = $\frac{n}{2} [2a + (n-1)d] = \frac{n}{2} (\text{first} + \text{last})$

- GP: nth term = $ar^{(n-1)}$; Sum = $a(r^n - 1)/(r - 1)$; infinite GP sum = $a/(1 - r)$ when $|r| < 1$

- Sum of first n naturals = $\frac{n(n+1)}{2}$; squares = $\frac{n(n+1)(2n+1)}{6}$; cubes = $\frac{[n(n+1)/2]^2}{1}$

Mensuration essentials

| Shape | Formula |

---|---

| Circle | Area πr^2 , circumference $2\pi r$ |

| Triangle | Area $\frac{1}{2} \times \text{base} \times \text{height}$; Heron's $\sqrt{s(s-a)(s-b)(s-c)}$ |

| Equilateral triangle | $(\sqrt{3}/4)a^2$ |

| Cube | Volume a^3 , surface area $6a^2$ |

| Cuboid | lbh ; surface $2(lb + bh + hl)$ |

| Cylinder | $\pi r^2 h$; curved surface $2\pi r h$ |

| Cone | $(1/3)\pi r^2 h$; curved surface $\pi r l$, where $l = \sqrt{r^2 + h^2}$ |

| Sphere | $(4/3)\pi r^3$; surface $4\pi r^2$ |

Exam pointers

1. Number of factors = product of (power + 1).
2. HCF $\times$ LCM = product of the two numbers — only for two numbers.
3. Unit digit: reduce the exponent mod 4.
4. Sum of roots = $-b/a$; product = c/a .
5. AP sum = $\frac{n}{2} \times (\text{first} + \text{last})$ is faster than the standard form when you know both ends.

Data Interpretation

Core preparation lesson

Why DI decides the section

DI typically carries the largest block of marks in Quant, and it rewards **calculation discipline** far more than mathematical insight. Most candidates lose marks to arithmetic slips and to attempting the wrong set, not to conceptual gaps.

Types

Tables · Bar graphs (simple, stacked, grouped) · Line graphs · Pie charts (degrees: $100\% = 360^\circ$, so $1\% = 3.6^\circ$) · Caselets (text-based) · Mixed/combination sets · Missing-data DI · Data sufficiency

The method

1. **Spend 30 seconds reading the chart before any question** — axis labels, units (lakhs? crores? percentages?), the time period, and any footnote. Most DI errors come from misreading units.
2. **Scan all questions in the set**, and attempt the direct-read ones first.
3. **Approximate aggressively** where options are far apart; calculate exactly only when they are close.
4. **Leave a set entirely** if it requires heavy computation for every question and other sets are available. Set selection is a skill.

Approximation techniques

- **Percentage as fractions:** $12.5\% = 1/8$, $16.67\% = 1/6$, $33.33\% = 1/3$, $14.28\% = 1/7$. Converting before multiplying is much faster.
- **Ratio comparison without dividing:** to compare a/b and c/d , cross-multiply — compare ad with cb .
- **Percentage change** = $(\text{new} - \text{old})/\text{old} \times 100$. The denominator is **always the old value** — the most common DI mistake.
- For "which year had the highest growth", compare **ratios**, not absolute differences.
- Round to two significant figures when options differ by more than 5%.

Speed arithmetic worth drilling

- **Tables up to 25**, squares to 30, cubes to 15.
- **Multiplication by 5** = $\times 10/2$; **by 25** = $\times 100/4$; **by 11** = shift and add.
- **Percentage of a number:** 10% then scale. $35\% \text{ of } 480 = 48 + 48 \times 2.5 = 48 + 120 = 168$.
- Reciprocals: $1/7 = 0.1428$, $1/9 = 0.111$, $1/11 = 0.0909$, $1/13 = 0.0769$.

Common question forms

| Form | Approach |

|---|---|

| Direct read | Locate and answer — do these first |

| Ratio between two categories | Cross-multiply, do not divide |

| Percentage change over years | $(\text{New} - \text{Old})/\text{Old}$ |

| Average across categories | Sum then divide; watch for weighted cases |

| "Approximately what percentage" | Estimate; options will be far apart |

| Combination with missing data | Solve the missing value first using a total |

Data sufficiency

Do **not** solve. Determine only whether the information is **enough**.

1. Check statement I alone.
2. Check statement II alone.
3. Only then both together.

Never carry an assumption from statement I while testing statement II — that is the single biggest source of error.

Exam pointers

1. **Read the units first.** Lakhs versus crores has ended more attempts than any concept gap.
2. **Percentage change denominator is always the old value.**
3. **Cross-multiply to compare ratios** rather than dividing.
4. **Set selection is a scoring skill** — abandon calculation-heavy sets.
5. In data sufficiency, evaluate each statement **independently first**.

Permutation, Combination and Probability

Core preparation lesson

Fundamental principles

- **Multiplication (AND):** if one task can be done in m ways and another in n ways, both together can be done in $m \times n$ ways.
- **Addition (OR):** either one or the other $\rightarrow m + n$ ways.

Permutation vs combination

| | Permutation | Combination |

|---|---|

| Order | **Matters** | Does **not** matter |

| Formula | $nPr = \frac{n!}{(n-r)!}$ | $nCr = \frac{n!}{r!(n-r)!}$ |

| Example | Arranging, ranking, seating, forming numbers | Selecting a team, choosing items |

$nPr = nCr \times r!$ — the relationship to remember.

Useful identities: $nC_0 = nC_n = 1$ · $nC_1 = n$ · $nCr = nC(n-r)$ · $nCr + nC(r-1) = (n+1)Cr$ · $\sum nCr$ from $r=0$ to $n = 2^n$ (the number of subsets).

Standard patterns

| Situation | Formula |

|---|---|

| Arrangements of n distinct objects | $n!$ |

| With repetitions (p alike, q alike) | $\frac{n!}{(p! q!)}$ |

| Circular arrangement of n | $(n-1)!$ |

| Circular where clockwise = anticlockwise (necklace) | $\frac{(n-1)!}{2}$ |

| r objects from n , with repetition allowed | n^r |

| Selecting at least one from n | $2^n - 1$ |

| Two particular people always together | Treat as one unit: $(n-1)! \times 2!$ |

| Two particular people never together | Total – together |

| Distributing n identical items into r groups (each ≥ 1) | $(n-1)C(r-1)$ |

Probability

$P(E)$ = favourable outcomes / total outcomes, between 0 and 1.

$$\begin{aligned} P(A \text{ or } B) &= P(A) + P(B) - P(A \text{ and } B) \\ &= P(A) + P(B) \quad \text{if mutually exclusive} \end{aligned}$$

$$\begin{aligned}
 P(A \text{ and } B) &= P(A) \times P(B) && \text{if independent} \\
 &= P(A) \times P(B|A) && \text{in general} \\
 P(\text{not } A) &= 1 - P(A)
 \end{aligned}$$

Conditional probability: $P(A|B) = P(A \cap B)/P(B)$.

Bayes' theorem: $P(A|B) = P(B|A) \cdot P(A) / P(B)$. Worth knowing conceptually — it is also the basis of the false-positive problem in fraud detection, which makes it a nice cross-link to your IT paper.

Standard setups:

- A pack of cards: 52 cards, 4 suits of 13, **26 red / 26 black**, 12 face cards, 4 aces.
- Two dice: 36 outcomes; sum 7 is the most likely (6 ways).
- Coins: n coins $\rightarrow 2^n$ outcomes.
- **"Without replacement"** changes the denominator each draw — the most common error.

Complement technique: "at least one" problems are almost always faster as $1 - P(\text{none})$.

Exam pointers

1. **Order matters $\rightarrow$ permutation; selection only $\rightarrow$ combination.**
2. **Circular arrangement = $(n-1)!$, necklace = $(n-1)!/2$.**
3. **Repetitions divide by the factorial of each repeated group.**
4. **"At least one" = $1 - P(\text{none})$.**
5. In "without replacement" problems, **update both numerator and denominator**.

Number Series, Quadratic Comparison and Simplification

Core preparation lesson

Number series

Find the pattern, then the missing or wrong term. Check patterns **in this order**:

1. **Difference** — constant, or an arithmetic pattern in the differences (+2, +4, +6...).
2. **Second-level difference** — differences of differences form a pattern.
3. **Ratio / multiplication** — $\times 2$, $\times 3$, $\times 1.5$, or multiplier changing ($\times 2$, $\times 3$, $\times 4$...).
4. **Squares and cubes** — near n^2 , $n^2 \pm 1$, n^3 , $n^3 \pm n$.
5. **Prime numbers** — 2, 3, 5, 7, 11, 13 embedded in the pattern.
6. **Alternating series** — two interleaved patterns; check odd and even positions separately.
7. **Combination** — $\times 2$ then +3, alternately.
8. **Factorials or cumulative sums** — each term adds the previous term.

Worked examples:

- 7, 10, 16, 28, 52, ? $\rightarrow$ differences 3, 6, 12, 24, **48** $\rightarrow$ answer **100**.

- 4, 9, 25, 49, 121, ? $\rightarrow$ squares of primes 2, 3, 5, 7, 11, **13** $\rightarrow$ **169**.

- 2, 6, 12, 20, 30, ? $\rightarrow n(n+1) \rightarrow$ **42**.

- 5, 11, 23, 47, ? $\rightarrow \times 2 + 1 \rightarrow$ **95**.

Wrong-term series: find the pattern from three consecutive terms that fit, then test the rest.

Quadratic equation comparison

You are given two equations and must compare x and y .

Method: factorise both, obtain both roots, and compare **every root of x with every root of y** .

Conclusion options: $x > y$, $x < y$, $x \geq y$, $x \leq y$, or **relationship cannot be established**.

The key trap: if the ranges of x and y **overlap** even partially, the answer is "cannot be established".

Candidates lose these marks by comparing only the largest roots.

Quick factorisation: for $x^2 + bx + c$, find two numbers whose product is c and sum is b . Signs: if c is positive, both roots share the sign of b (with opposite sign to b); if c is negative, roots have opposite signs.

Simplification and approximation

BODMAS order: Brackets $\rightarrow$ Order (powers/roots) $\rightarrow$ Division $\rightarrow$ Multiplication $\rightarrow$ Addition $\rightarrow$ Subtraction.

Approximation questions are solved by rounding, not by exact computation:

- $\sqrt{2} = 1.414$, $\sqrt{3} = 1.732$, $\sqrt{5} = 2.236$, $\sqrt{7} = 2.646$, $\pi = 3.14$

- Squares to 30 and cubes to 15 should be instant recall.
- Convert percentages to fractions before multiplying.

Speed technique — "approximate then verify the last digit" works well when options are close: estimate the magnitude, then compute only the unit digit to discriminate.

Exam pointers

1. Check **differences first, then ratios, then squares/cubes** — in that order.
2. Alternating series: **check odd and even positions separately.**
3. In quadratic comparison, **overlapping ranges mean "cannot be established"**.
4. Compare **all four root pairs**, not just the largest.
5. Squares to 30, cubes to 15, and the fraction-percentage table must be **recall, not calculation**.

Speed Maths and the Formula Sheet

Core preparation lesson

> Print this page. Revise it every Sunday. In a 60-minute paper, recall speed is worth more than technique.

Must-be-instant recall

Tables to 25.

Squares to 30: ... $21^2=441$, $22^2=484$, $23^2=529$, $24^2=576$, $25^2=625$, $26^2=676$, $27^2=729$, $28^2=784$, $29^2=841$, $30^2=900$.

Cubes to 15: $11^3=1331$, $12^3=1728$, $13^3=2197$, $14^3=2744$, $15^3=3375$.

Powers of 2 to 2^{15} : 1024, 2048, 4096, 8192, 16384, 32768 — useful in both Quant and your IT paper.

Fraction $\leftrightarrow$ percentage:

$1/2=50$ · $1/3=33.33$ · $1/4=25$ · $1/5=20$ · $1/6=16.67$ · $1/7=14.28$ · $1/8=12.5$ · $1/9=11.11$ · $1/10=10$ · $1/11=9.09$ · $1/12=8.33$ · $1/13=7.69$ · $1/14=7.14$ · $1/15=6.67$ · $1/16=6.25$ · $1/20=5$ · $1/25=4$

Multiplication shortcuts

- **$\times 5$** $\rightarrow \times 10 \div 2$ · **$\times 25$** $\rightarrow \times 100 \div 4$ · **$\times 50$** $\rightarrow \times 100 \div 2$ · **$\times 125$** $\rightarrow \times 1000 \div 8$

- **$\times 11$** $\rightarrow$ write the digits apart and insert the sums: $43 \times 11 \rightarrow 4 \ (4+3) \ 3 = 473$

- **$\times 9$, $\times 99$** $\rightarrow \times 10 - x$, $\times 100 - x$

- **Squares ending in 5:** $65^2 \rightarrow 6 \times 7 = 42$, append 25 $\rightarrow 4225$

- **$(a+b)(a-b) = a^2 - b^2$:** $47 \times 53 = 50^2 - 3^2 = 2500 - 9 = 2491$

- **Numbers near 100:** $96 \times 97 \rightarrow (96-3) \mid (4 \times 3) = 93 \mid 12 = 9312$

The formula sheet

PERCENTAGE	successive: $a + b + ab/100$ increase x% then restore: $[x/(100+x)] \times 100$
PROFIT	profit% on CP; successive discount $a + b - ab/100$ equal profit% and loss% $\rightarrow$ net loss $x^2/100$ %
SI/CI	$SI = PRT/100$; $CI = P[(1+R/100)^T - 1]$ $CI - SI$ (2 yr) $= P(R/100)^2$; rule of 72 for doubling
TSD	$1 \text{ km/h} = 5/18 \text{ m/s}$ avg speed (equal distance) $= 2xy/(x+y)$ boats: $b = (d+u)/2$, $s = (d-u)/2$
WORK	together $= ab/(a+b)$; $M1D1H1/W1 = M2D2H2/W2$
AVERAGE	alligation: $(\text{cheap qty})/(\text{dear qty}) = (\text{dear} - \text{mean})/(\text{mean} - \text{cheap})$
AP/GP	AP sum $= n/2(\text{first}+\text{last})$; GP inf $= a/(1-r)$ $\Sigma n = n(n+1)/2$; $\Sigma n^2 = n(n+1)(2n+1)/6$; $\Sigma n^3 = [n(n+1)/2]^2$
P&C	$nPr = n!/(n-r)!$; $nCr = n!/[(r!(n-r)!)]$; circular $= (n-1)!$
PROBABILITY	$P(A \cup B) = P(A)+P(B)-P(A \cap B)$; "at least one" $= 1 - P(\text{none})$
MENSURATION	sphere $(4/3)\pi r^3$, $4\pi r^2$; cone $(1/3)\pi r^2 h$, $\pi r l$; cylinder $\pi r^2 h$, $2\pi r h$

Exam-hall discipline

1. **Two passes.** First pass: only questions you can finish in under 45 seconds. Second pass: the rest. Never let one question consume three minutes.

2. **Attempt rate beats accuracy obsession**, up to a point — but with negative marking, never guess blindly on a question you have not read properly.
3. **Write the intermediate step.** Mental arithmetic beyond two steps is where silly errors live.
4. **Track time by block**, not by question: if 20 minutes of your Quant allocation are gone and you have done fewer than 8 questions, move to DI or switch sections.

Exam pointers

1. Squares to 30 and the fraction table are **non-negotiable recall**.
2. **Two-pass strategy** is the single biggest score improver in Paper 1.
3. **Successive change and CI-SI** formulas appear nearly every year.
4. Use $a^2 - b^2$ for products symmetric about a round number.
5. Rewrite this formula sheet **from memory** once a week — that is the actual revision method.

Data sufficiency and critical reasoning

Core preparation lesson

For data sufficiency test statement I alone, then II alone, then both only if necessary. Determine whether there is exactly one answer, not whether you can calculate a convenient example. In assumptions distinguish a necessary premise from a merely helpful suggestion.

Worked example

Question: is integer x positive? I: $x^2 = 9$. Insufficient, because x can be -3 or 3 . II: $x > 0$. Sufficient by itself. A common error is combining I and II before testing independence.

Argument drill

Claim: a training programme improved scores because its participants scored higher. An alternative explanation is that stronger candidates volunteered. An appropriate study compares equivalent groups or controls prior scores. To test an alleged necessary assumption, negate it and ask whether the argument can still stand. For a course of action prefer a feasible response to the stated problem; do not invent additional facts.

Direct and coded inequalities

Core preparation lesson

Join chains only when their direction supports a comparison. $A > B \geq C$ implies $A > C$. $A > B < C$ does not compare A and C. Equality can connect either side of a chain. Decode each symbol into its ordinary relation before combining it.

Worked example

$P \geq Q$, $Q > R$ and $R = S$. Then $P > S$ follows. $S \geq Q$ does not follow. If $P > Q$ and $Q \leq R$, P and R cannot be ordered without more information.

Drill

Create ten chains of four variables, mixing strict and non-strict signs. For every undetermined conclusion, supply two numerical assignments showing opposite possible orders. For either/or conclusions, check that the alternatives exhaust the possibilities and cannot both be true. Do not decide merely because both individual conclusions are unproved.

Directions, relations, ranking and coding

Core preparation lesson

Use coordinates for direction: east adds x, north adds y. For family relations draw a generation chart and do not infer gender from a name. A rank from the top and a rank from the bottom for the same person give total = top + bottom - 1.

Worked examples

A person walks 3 km east and 4 km north. Displacement is 5 km northeast; distance travelled is 7 km. A candidate is 8th from the top and 13th from the bottom: there are 20 candidates. If CAT becomes DBU by shifting each letter forward once, DOG becomes EPH.

Input-output

Compare consecutive steps and identify what is moved, where it goes, and what remains fixed. Test your proposed rule on every visible step. For mixed alphabet-number strings count only the requested adjacent pattern, including boundary positions. Practise with a 60-second cap per standalone question.

Seating, floor and scheduling puzzles

Core preparation lesson

Draw slots before reading clues. Record fixed positions first, then relative blocks, then exclusions. In circular seating fix one person to remove rotational duplicates. Facing the centre, left is clockwise; facing outward reverses it. In floor puzzles label bottom as 1 unless the question says otherwise.

Worked example

A, B, C and D sit left to right facing north. B is immediately right of A; D is at the right end; C is not adjacent to D. The only arrangement is C A B D. Test it against every clue rather than accepting the first arrangement that fits two clues.

Practice method

Solve one linear, one floor and one scheduling set. Branch when an ambiguous clue has two placements; cross out a branch as soon as it contradicts another clue. If a set is still open after four minutes during a speed drill, move on and return later. Record the clue you misread. Never add assumptions such as one person per month unless stated.

Syllogisms and logical conclusions

Core preparation lesson

Translate All A are B as containment, No A is B as disjoint sets, and Some A are B as at least one shared member. Only accept a conclusion that holds in every diagram consistent with the statements. All does not by itself establish existence under formal logic; follow explicit examination conventions if the handout specifies them.

Worked example

All coders are readers. Some readers are musicians. Conclusion: some coders are musicians. This does not follow: the musician-readers could all lie outside the coders set. Conversely, some musicians are readers follows directly from the second statement.

Possibility questions

Distinguish must be true from can be true. A possibility is valid if at least one arrangement satisfies both the premises and the proposed possibility. Test universal claims with counterexamples. Practise five questions each of containment, exclusion and possibility; draw the counterexample for each rejected conclusion.

Free Reference Books & Official Sources

Supplementary reference / interview

> Everything on this page is **legally free** — either open-licensed, published free by the author, or an official government/regulator publication. No pirated PDFs: apart from the ethics, a SEBI candidate downloading copyrighted books from shady sites is a bad look.

>

> **How to use this page:** open a link, download the PDF, then drop it into the **Book Reader** in this app. It will render there and remember your page across sessions.

A. Securities market, SEBI & economy (official — start here)

These are free, authoritative, and the actual source that every coaching PDF copies from.

| Resource | Why | Where |

|---|---|---|

| **SEBI — Bare Acts, Rules & Regulations** | SEBI Act, SCRA, Depositories Act, LODR, ICDR, PIT, SAST in full text | [sebi.gov.in/legal](https://www.sebi.gov.in/legal.html) |

| **SEBI Annual Report** | The single best document for the interview — read at least the overview and the IT/surveillance chapters | [sebi.gov.in → Reports](https://www.sebi.gov.in/sebiweb/other/OtherAction.do?doValue=yes&type=1) |

| **SEBI Handbook of Statistics on the Indian Securities Market** | All the market data you might be asked to quote | sebi.gov.in → Reports and Statistics |

| **SEBI Investor Education material & booklets** | Plain-language explanations of every market concept; free PDFs and comics | investor.sebi.gov.in |

| **SEBI Circulars & Press Releases** | Your daily current affairs source | [sebi.gov.in](https://www.sebi.gov.in/) |

| **NSE — Indian Securities Market: A Review (ISMR)** | Annual review of the market, free PDF | [nseindia.com](https://www.nseindia.com/) |

| **RBI — Financial Stability Report, Annual Report, Trend & Progress of Banking** | Banking and systemic risk | [rbi.org.in/Publications](https://www.rbi.org.in/Scripts/AnnualPublications.aspx) |

| **Economic Survey + Union Budget** | Economy section, in the government's own words | [indiabudget.gov.in](https://www.indiabudget.gov.in/) |

| **NCERT Class 11 — Indian Economic Development; Class 12 — Macroeconomics** | Still the cleanest foundation for the economy portion | ncert.nic.in/textbook.php |

| **India Code — IT Act 2000, Companies Act 2013** | Bare statutory text | [indiacode.nic.in](https://www.indiacode.nic.in/) |

| **CERT-In advisories & directions** | Cyber current affairs with an Indian regulatory angle | [cert-in.org.in](https://www.cert-in.org.in/) |

| **MeitY — DPDP Act 2023 and rules** | Data protection, directly relevant to the IT stream | [meity.gov.in](https://www.meity.gov.in/) |

B. Operating Systems

- **Operating Systems: Three Easy Pieces (OSTEP)** — Arpaci-Dusseau. Free chapter-wise PDFs, and genuinely the best OS book written. →

pages.cs.wisc.edu/~remzi/OSTEP

- **NPTEL — Operating Systems** (IIT faculty, full video course + notes) → nptel.ac.in

- **Linux Documentation Project / man pages** for the command-line portion.

Read for the exam: processes, scheduling, synchronisation, deadlock, memory, virtual memory, file systems. Skip the research chapters.

C. Computer Networks

- **Computer Networks: A Systems Approach** — Peterson & Davie, now **open source and free online** → book.systemsapproach.org

- **Beej's Guide to Network Programming** — free, excellent for sockets →

beej.us/guide/bgnet

- **NPTEL — Computer Networks**; **Kurose & Ross companion site** has free slides and interactive problems.

- **RFCs** for anything you want the authoritative answer on → [rfc-editor.org](https://www.rfc-editor.org/)

D. DBMS

- **Database Design (2nd ed.)** — BCcampus open textbook, CC-licensed →

opentextbc.ca/dbdesign01

- **Database System Concepts (Silberschatz)** — the book is paid, but the **author's site** gives away all slides and practice exercises free → [db-book.com](https://www.db-book.com/)

- **Readings in Database Systems ("the Red Book")** — free, for depth → [redbook.io](http://www.redbook.io/)

- **SQL practice:** sqlzoo.net, pgexercises.com, LeetCode SQL (free tier), HackerRank SQL.

E. Data Structures & Algorithms

- **Algorithms** — Jeff Erickson (free, superb, rigorous) →

jeffe.cs.illinois.edu/teaching/algorithms

- **Algorithms** — Dasgupta, Papadimitriou & Vazirani (free PDF from the authors) →

people.eecs.berkeley.edu/~vazirani/algorithms.html

- **Open Data Structures** — Pat Morin, free in Java/C++/Python →

opendatastructures.org

- **Practice:** LeetCode, GeeksforGeeks, Codeforces — free tiers are more than enough for this exam.

F. Programming (C / Java / Python)

- **Modern C** — Jens Gustedt, free PDF →
gustedt.gitlabpages.inria.fr/modern-c
- **Think Python** — Allen Downey, free →
greenteapress.com/wp/think-python-2e
- **Automate the Boring Stuff with Python** — free online →
automatetheboringstuff.com
- **Java:** Oracle's official **Java Tutorials** are free and complete →
docs.oracle.com/javase/tutorial

G. Computer Organisation & Digital Logic

- **nand2tetris** — build a computer from NAND gates up; free course and materials →
[nand2tetris.org](https://www.nand2tetris.org/)
- **NPTEL** — Computer Organization and Architecture (IIT Kharagpur/Madras versions both good).
- **MIT OpenCourseWare 6.004** → ocw.mit.edu

H. Software Engineering

- **Software Engineering at Google** — free to read online →
abseil.io/resources/swe-book
- **Google SRE Book** and **SRE Workbook** — free online, excellent on reliability, error budgets, incident response → sre.google/books
- **SWEBOK Guide** (IEEE) — free download after a form; the canonical vocabulary for descriptive answers.
- **Agile Manifesto + Scrum Guide** — both free, both short, both quotable →
scrumguides.org

I. Cyber Security (highest-value block for you)

- **NIST Cybersecurity Framework 2.0** and the **SP 800 series** — all free, and the vocabulary SEBI's own framework borrows → [csrc.nist.gov](https://csrc.nist.gov/publications/sp800)
- Start with **SP 800-53** (controls), **SP 800-61** (incident handling), **SP 800-145** (cloud definition), **SP 800-207** (zero trust).
- **OWASP Top 10** and the **OWASP ASVS / Cheat Sheet Series** — free →
[owasp.org](https://owasp.org/www-project-top-ten/)
- **Crypto 101** — free introductory cryptography book → [crypto101.io](https://www.crypto101.io/)
- **Handbook of Applied Cryptography** — Menezes et al., free chapter PDFs →
cacr.uwaterloo.ca/hac
- **SEBI CSCRf** and earlier cyber circulars — free on sebi.gov.in, and **non-negotiable reading for this stream**.
- **MITRE ATT&CK** — free knowledge base of attacker techniques →
attack.mitre.org

J. Cloud, Big Data, AI & Blockchain

- **NIST SP 800-145** — the definition of cloud computing everyone cites.
- **AWS / Azure / GCP official documentation and free-tier training** — free, and more current than any book.
- **Google Cloud Architecture Framework**, **AWS Well-Architected Framework** — free whitepapers.
- **Bitcoin whitepaper** (Nakamoto, 9 pages) and the **Ethereum whitepaper** — read the originals, not summaries.
- **RBI CBDC concept note** — free, and India-specific.
- **Dive into Deep Learning (d2l.ai)** — free, interactive → [\[d2l.ai\]\(https://d2l.ai/\)](https://d2l.ai/)

K. English, Quant & Reasoning

- **The Elements of Style** — Strunk, public domain → [\[gutenberg.org/ebooks/37134\]\(https://www.gutenberg.org/ebooks/37134\)](https://www.gutenberg.org/ebooks/37134)
- **Purdue OWL** — free, thorough grammar and formal-writing reference → [\[owl.purdue.edu\]\(https://owl.purdue.edu/\)](https://owl.purdue.edu/)
- **Editorials from The Hindu / Indian Express / Mint** — free daily, and the best essay and précis practice material there is.
- **NCERT Mathematics (Class 8–10)** — free, and enough for the arithmetic base → [\[ncert.nic.in\]\(https://ncert.nic.in/textbook.php\)](https://ncert.nic.in/textbook.php)
- **Free bank-exam practice portals** (Oliveboard, Testbook, Adda247, Gradeup free sections) for daily quizzes on Quant, Reasoning and English. Use the free daily quizzes; you do not need a paid course for Phase I Paper 1.

L. Video courses when a topic will not click

- **NPTEL / SWAYAM** — free, certified, IIT-taught, and covers every IT paper subject → [\[nptel.ac.in\]\(https://nptel.ac.in/\)](https://nptel.ac.in/) · [\[swayam.gov.in\]\(https://swayam.gov.in/\)](https://swayam.gov.in/)
- **MIT OpenCourseWare**, **Stanford CS free lectures**, **freeCodeCamp** on YouTube.
- **NSE Academy / BSE Institute** free webinars for market topics.

What to actually buy (the short list)

Free material covers ~90% of this exam. The three things genuinely worth paying for:

1. **A good mock test series** for Phase I and Phase II — this is the one purchase with a clear return.
2. **One standard book for Quant/Reasoning** if you are weak there and want structured drills (R.S. Aggarwal or the Arihant/Disha bank-exam series).
3. **NISM/NCFM workbooks** only if you want certification value alongside preparation.

Everything else on this page is free.

Exam pointers

1. **Official sources beat coaching PDFs** — and they are what the interview panel has read.
2. Download the PDFs **once**, keep them in this app's Book Reader, and you can study offline on your phone.
3. Never let collecting resources become a substitute for finishing the roadmap. Two books finished beats twenty downloaded.